Spring Handbook — Vol 1

Spring core, configuration, AOP, MVC and REST, REST clients, data access and transactions, Spring Data JPA, caching (Ch 1–9)

9 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — The Spring Ecosystem and Your First App**
   - 1.1 What Spring is
   - 1.2 What Spring Boot adds
   - 1.3 The Spring ecosystem
   - 1.4 History and versions
   - 1.5 Creating a project with start.spring.io
   - 1.6 Project layout and the build file
   - 1.7 Starters
   - 1.8 The main class and `@SpringBootApplication`
   - 1.9 Running the application
   - 1.10 DevTools
   - 1.11 Your first REST endpoint
   - 1.12 Testing the first app
   - 1.13 In the interview
   - Chapter TL;DR
2. **Chapter 2 — IoC and Dependency Injection**
   - 2.1 Inversion of Control and Dependency Injection
   - 2.2 The container: `BeanFactory` vs `ApplicationContext`
   - 2.3 Defining beans
   - 2.4 Injection types
   - 2.5 How Spring picks a bean
   - 2.6 Bean scopes
   - 2.7 The bean lifecycle
   - 2.8 Post-processors: the container's extension points
   - 2.9 Circular dependencies
   - 2.10 Lazy initialization
   - 2.11 `@Conditional` and profiles
   - 2.12 Application events
   - 2.13 Null-safety with JSpecify (Framework 7)
   - 2.14 Programmatic registration with `BeanRegistrar` (Framework 7)
   - 2.15 In the interview — quick round
   - Chapter TL;DR
3. **Chapter 3 — Configuration and Auto-configuration**
   - 3.1 Externalized configuration and the `Environment`
   - 3.2 `application.properties` vs `application.yaml`
   - 3.3 Precedence: which value wins?
   - 3.4 Environment variables and relaxed binding
   - 3.5 `@Value`
   - 3.6 `@ConfigurationProperties` — type-safe configuration
   - 3.7 Profiles, profile-specific files and profile groups
   - 3.8 Importing configuration: `spring.config.import`
   - 3.9 Placeholders, random values and other sources
   - 3.10 How auto-configuration works
   - 3.11 Seeing what Boot decided
   - 3.12 Overriding and excluding auto-configuration
   - 3.13 Writing your own auto-configuration and starter
   - 3.14 Boot 4 modularization
   - 3.15 In the interview — quick round
   - Chapter TL;DR
4. **Chapter 4 — AOP and Proxies**
   - 4.1 Cross-cutting concerns and the AOP vocabulary
   - 4.2 Writing aspects with `@Aspect`
   - 4.3 Pointcut expressions
   - 4.4 How Spring AOP works: proxies
   - 4.5 Proxy limitations — proved
   - 4.6 Spring features built on proxies
   - 4.7 Framework 7 core resilience: `@Retryable` and `@ConcurrencyLimit`
   - 4.8 Spring AOP vs AspectJ
   - 4.9 Common mistakes (whole chapter)
   - 4.10 In the interview
   - 4.11 Chapter TL;DR
5. **Chapter 5 — Spring MVC and REST APIs**
   - 5.1 REST in one page
   - 5.2 How a request flows: the `DispatcherServlet`
   - 5.3 Controllers and request mappings
   - 5.4 Reading input and writing output
   - 5.5 JSON with Jackson 3
   - 5.6 Validation
   - 5.7 Error handling and Problem Details (RFC 9457)
   - 5.8 Content negotiation
   - 5.9 API versioning (Framework 7)
   - 5.10 CORS and file upload
   - 5.11 Filters vs interceptors (vs AOP)
   - 5.12 Documenting the API: OpenAPI with springdoc
   - 5.13 Putting it together: design checklist and mistakes
   - 5.14 In the interview
   - 5.15 Chapter TL;DR
6. **Chapter 6 — REST Clients and HTTP Interfaces**
   - 6.1 Choosing a client
   - 6.2 `RestClient` basics
   - 6.3 Error handling
   - 6.4 Timeouts and the underlying HTTP library
   - 6.5 Interceptors
   - 6.6 HTTP service interfaces (`@HttpExchange`)
   - 6.7 Client-side API versioning and retries
   - 6.8 Testing clients
   - 6.9 `RestTemplate` and `WebClient` in brief
   - 6.10 Common mistakes, real-world examples, interview
   - 6.11 Chapter TL;DR
7. **Chapter 7 — Data Access with JDBC and Transactions**
   - 7.1 The data-access stack
   - 7.2 DataSource and HikariCP
   - 7.3 JdbcClient and JdbcTemplate
   - 7.4 Schema and data initialization
   - 7.5 Exception translation: DataAccessException
   - 7.6 Testing JDBC code with @JdbcTest
   - 7.7 How a Spring transaction works
   - 7.8 @Transactional attributes

- 7.9 Propagation
- 7.10 Rollback rules
- 7.11 Isolation levels
- 7.12 readOnly and timeout
- 7.13 Transaction pitfalls
- 7.14 Programmatic transactions: TransactionTemplate
- 7.15 @TransactionalEventListener
- 7.16 Interview questions
- Chapter TL;DR

8. **Chapter 8 — Spring Data JPA and Hibernate**
   - 8.1 JPA, Hibernate and Spring Data — who does what
   - 8.2 Setting up
   - 8.3 Entities and ids
   - 8.4 The persistence context and dirty checking
   - 8.5 Relationships
   - 8.6 Fetch types and LazyInitializationException
   - 8.7 The N+1 problem
   - 8.8 Repositories
   - 8.9 Derived queries
   - 8.10 @Query: JPQL, native SQL and @Modifying
   - 8.11 Pagination and sorting
   - 8.12 Projections
   - 8.13 Auditing
   - 8.14 Locking: optimistic and pessimistic
   - 8.15 Open Session in View
   - 8.16 Schema migrations with Flyway
   - 8.17 Testing JPA code
   - 8.18 Performance checklist
   - 8.19 Interview questions
   - Chapter TL;DR
9. **Chapter 9 — Spring Data beyond JPA, and caching**
   - 9.1 The Spring Data family
   - 9.2 Spring Data JDBC and aggregates
   - 9.3 Spring Data REST
   - 9.4 Schema migrations: Flyway beyond the basics, and Flyway vs Liquibase
   - 9.5 Spring Data MongoDB (not run — needs Docker)
   - 9.6 Spring Data Redis (not run — needs Docker)
   - 9.7 Testing MongoDB and Redis with Testcontainers (not run)
   - 9.8 The caching abstraction
   - 9.9 Choosing a data technology
   - 9.10 Interview questions
   - Chapter TL;DR

Chapter 1 — The Spring Ecosystem and Your First App

Spring is the most widely used framework for server-side Java applications (programs that run on a server and answer other programs): web APIs, back-office systems, batch jobs (programs that process a large pile of data in one run, often at night) and message consumers (programs that react to messages arriving on a queue).

> **Key term — API / REST API:** An **API** (Application Programming Interface) is the set of operations one program offers to others. A **web API** offers them over HTTP. A **REST API** is a web API in a common style: each thing (an order, a user) has a URL, and you act on it with standard HTTP methods such as `GET` (read) and `POST` (create), usually exchanging JSON. HTTP basics: System Design Fundamentals.

This chapter explains what Spring and Spring Boot are, their history and current versions, then builds, runs, packages and tests a first REST API with **Spring Boot 4.1.1** on **Java 25**. You only need basic Java; Java features (records, annotations, lambdas, Maven) are linked to the Java Handbook rather than re-taught.

All runnable code is in `code/hello-boot/`: a Maven project whose tests were run with `mvn -q -B -ntp verify` on JDK 25. Outputs below are real (long log lines trimmed with `...`).

1.1 What Spring is

What it is

A **library** is code you call (e.g. a JSON parser). A **framework** is code that calls *you*: it owns the program's structure and plugs your classes in at the right moments. This reversal is **Inversion of Control (IoC)** — "don't call us, we'll call you".

Spring Framework is an open-source Java framework whose core is an **IoC container**: an object that creates your application's objects, connects them, configures them and manages their whole life. Objects managed by the container are **beans**. Around that core Spring offers modules for web (Spring MVC — the classic web framework, one thread per request; WebFlux — a non-blocking alternative), data access (**JDBC** — Java Database Connectivity, Java's standard low-level SQL API; and **transactions** — groups of database changes that succeed or fail as one unit), messaging, testing, scheduling and more.

The problem it solves

Without a container, every class builds its own dependencies with `new`:

```
// Hard-wired: OrderService decides WHICH repository and WHICH mailer it uses.
class OrderService {
    private final OrderRepository repo = new JdbcOrderRepository(new HikariDataSource(...));
    private final Mailer mailer = new SmtpMailer("smtp.example.com", 25);
}
```

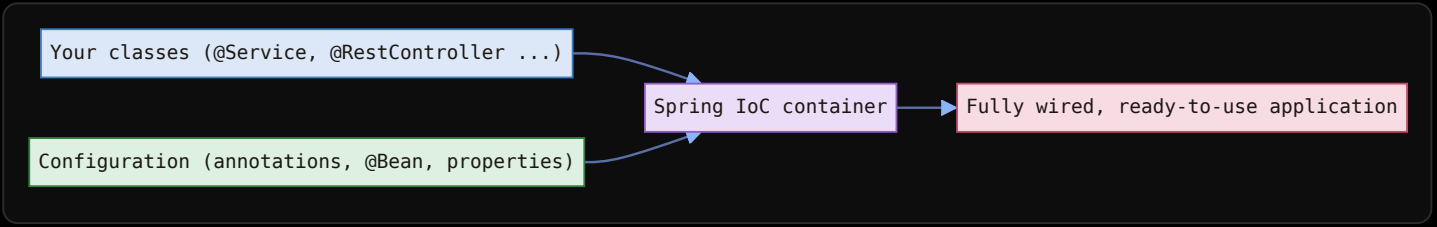
Problem	Why it hurts
Tight coupling	Swapping `JdbcOrderRepository` for another implementation means editing `OrderService`.
Hard to test	A unit test of `OrderService` now needs a real database and a real mail server.
Scattered configuration	Host names, pool sizes and credentials are buried in constructors all over the code.

Spring's answer is **Dependency Injection (DI)**: a class *declares* what it needs (usually as constructor parameters) and the container *supplies* it. The class depends on an interface; the container picks the implementation. Chapter 2 covers DI in depth; the design principle behind it is the Dependency Inversion Principle — see LLD §4.10 and the Hollywood Principle in LLD Chapter 5.

Beyond DI, Spring gives battle-tested answers for the plumbing every server needs: HTTP handling, JSON conversion (**JSON** is a plain-text data format like `{"name":"Ada"}`), transactions, security, scheduling, caching and consistent exception types.

How it works (bird's-eye view)

You write ordinary Java classes (POJOs — "Plain Old Java Objects") and mark those Spring should manage, e.g. with `@Service` (an **annotation**: a `@`-label frameworks read at runtime). At startup the container reads this **configuration metadata** (annotations, `@Bean` methods, or XML in old projects), creates each bean injecting what it needs, and may wrap beans in **proxies** (stand-ins that intercept calls) to add transactions or security (Chapter 4). It then calls your code when requests, messages or timers arrive, and at shutdown runs clean-up callbacks.



Real-world examples

Banks, retailers, airlines, streaming and public-sector back ends; Netflix has publicly described moving its Java services to Spring Boot. It is the most common Java stack in back-end interviews.

1.2 What Spring Boot adds

What it is

Spring Boot is built *on top of* Spring Framework to make Spring applications quick to create and production-ready. It does not replace Spring — every Boot application *is* a Spring application.

Feature	What it means	Where in this book
Starters	One dependency per capability (`spring-boot-starter-webmvc`) bringing a tested set of libraries.	§1.7
Dependency management	Boot picks compatible versions of hundreds of libraries, so your `pom.xml` has no `<version>` tags.	§1.6
Auto-configuration	Boot inspects your classpath (the jars your app includes) and settings and creates sensible beans (web server, JSON mapper, a `DataSource` — an object that hands out database connections). Your own beans always win.	Chapter 3
Embedded server	Tomcat (or Jetty) runs *inside* your app; you run `java -jar app.jar`.	§1.9
Externalized configuration	Settings from `application.properties` / `.yaml`, environment variables (e.g. `SERVER_PORT=9090`), command-line arguments, with a clear precedence order.	Chapter 3
Production features	Health checks, metrics, tracing (following one request across services), structured logging, graceful shutdown (Actuator).	Chapter 15
Testing support	Slices (tests that start only one layer) like `@WebMvcTest`, `@SpringBootTest`, test starters.	Chapter 12

Why it matters

Before Boot (2014), a Spring web app needed a WAR file (Web Application Archive — a zip you copy into a separately installed server), an external Tomcat, configuration for the `DispatcherServlet` (Spring MVC's central request dispatcher, Chapter 5), a view resolver, a JSON converter, a data source, and a careful choice of mutually compatible library versions. Boot reduced that to a generated project and a `main` method, keeping every default overridable.

Spring vs Spring Boot

Question	Spring Framework	Spring Boot
What is it?	The core: IoC container, MVC, transactions...	An opinionated layer (sensible defaults) that configures Spring for you
Use one without the other?	Yes — plain Spring works without Boot	No — Boot always uses Spring Framework
Config / server / versions	You declare infrastructure beans, deploy a WAR, pick each library version	Auto-configured, embedded server, versions from Boot's BOM
Typical use today	Libraries, special setups	Almost every new application

Common mistakes

- Thinking Boot is a different framework with different annotations. `@Service`, `@Autowired`, `@Transactional`, `@RestController` are Spring *Framework* annotations; Boot mostly adds `@SpringBootApplication`, `@ConfigurationProperties` and test annotations.
- Believing auto-configuration is uninspectable magic. It is ordinary `@Configuration` classes guarded by conditions, and Boot can print a report of every decision (Chapter 3).

In the interview

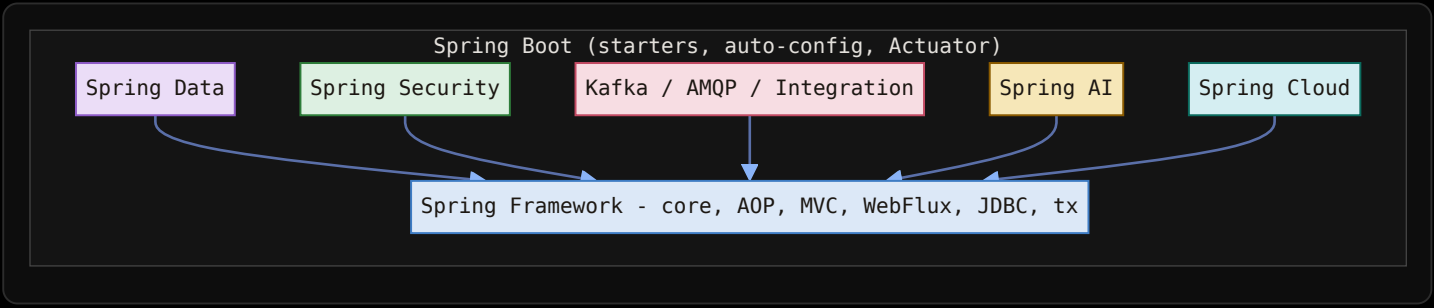
Q: What is the difference between Spring and Spring Boot? *30-second answer:* Spring Framework provides the IoC container and modules like MVC and transactions. Boot sits on top: starters for dependency sets, a managed version BOM, auto-configuration that creates beans based on classpath and properties, an embedded server so the app runs as a plain jar, externalized configuration and production features like Actuator. Boot removes boilerplate; it does not replace Spring.

- *Follow-up: Can you override what Boot configures?* Yes. Auto-configured beans are usually guarded by `@ConditionalOnMissingBean`, so defining your own bean of that type makes Boot back off; many settings are just properties.
- *Follow-up: Does Boot make the app slower?* Startup does extra condition evaluation, but the runtime path is the same Spring code. Boot also offers AOT (ahead-of-time: work done at build time instead of startup) processing and native images (the app compiled to a stand-alone executable needing no JVM) for faster startup (Chapter 15).

1.3 The Spring ecosystem

"Spring" is also an umbrella name for a family of projects maintained by the Spring team (part of Broadcom since its acquisition of VMware). All are version-managed by Spring Boot, so you rarely pick their versions yourself.

Project	What it gives you	Chapter
Spring Framework	IoC container, AOP (Aspect-Oriented Programming — cross-cutting behaviour such as logging around methods), MVC, WebFlux, JDBC, transactions, testing	2, 4, 5, 7, 13
Spring Boot	Starters, auto-configuration, embedded servers, Actuator	1, 3, 12, 15
Spring Data	Repository interfaces (you declare data-access methods; Spring writes the code) for JPA, JDBC, MongoDB, Redis, R2DBC...	8, 9
Spring Security	Authentication (who are you?), authorization (what may you do?), OAuth2 (delegated login/access), JWT (a signed token carrying user claims)	10, 11
Spring Cloud	Config server, service discovery, gateway (single entry point in front of many services), load balancing	16
Spring for Apache Kafka / Spring AMQP	Messaging with Kafka and RabbitMQ (message brokers)	14
Spring Batch	Large, restartable batch jobs	— (mentioned in 14)
Spring Integration	Enterprise integration patterns (channels, adapters)	—
Spring Session	Store HTTP sessions (per-user server-side state) in Redis or a database	10
Spring for GraphQL	GraphQL (query language where the client picks the fields) servers	—
Spring Modulith	Structure a monolith (one app deployed as a single unit) into verified modules	16
Spring AI	LLM chat clients, embeddings (text as number vectors), RAG (retrieval-augmented generation), tool calling	17



Real-world stack. A typical online-shop order service: Spring Boot + Spring MVC (REST API) + Spring Data JPA (PostgreSQL) + Spring Security (JWT) + Spring for Apache Kafka ("order placed" events) + Actuator/Micrometer (Spring's metrics library). This book builds each layer.

1.4 History and versions

A short timeline

Year	Release	What changed
2002	Rod Johnson's book *Expert One-on-One J2EE Design and Development*	A lighter alternative to EJB (Enterprise JavaBeans, J2EE's heavyweight component model); its sample code became Spring
2004	Spring Framework 1.0	IoC container, AOP, transaction abstraction, XML configuration
2007	Spring 2.5	Annotation-driven config: `@Autowired`, `@Component`, `@Controller`
2009	Spring 3.0	Java 5 baseline, `@Configuration` / `@Bean`, SpEL (Spring Expression Language, `#{...}`), REST support in MVC
2013	Spring 4.0	Java 8 support, `@Conditional` (foundation of auto-configuration)
2014	Spring Boot 1.0	Starters, auto-configuration, embedded servers
2017–18	Spring 5.0 / Boot 2.0	Java 8 baseline, reactive stack (WebFlux)
2022	Spring 6.0 / Boot 3.0	**Java 17 baseline**, `javax.*` → `jakarta.*`, AOT and GraalVM native images
Nov 2025	**Spring Framework 7.0** (13 Nov) / **Spring Boot 4.0** (20 Nov)	Jakarta EE 11, JSpecify null-safety (annotations marking what may be `null`), modular Boot jars, Jackson 3, API versioning, HTTP service clients, JUnit 6
Jun 2026	Spring Boot 4.1.0 (10 Jun)	gRPC (fast binary remote-call protocol) support, Jackson config updates, observability updates
Aug 2026	Spring Boot 4.1.1 (20 Aug)	Bug fixes and dependency upgrades — **the version used in this book**

The current generation — what Boot 4.1.1 gives you

From Boot 4.1.1's dependency BOM (`spring-boot-dependencies-4.1.1.pom`):

Component	Version in Boot 4.1.1	Notes
Java	**17 minimum**, 25 recommended	This book uses Java 25 (LTS) — see Java Ch. 17
Spring Framework	7.0.9	
Jakarta EE	11 (Servlet 6.1; JPA 3.2 — object/database mapping; Bean Validation 3.1 — rules like `@NotNull`)	
Tomcat / Jetty	11.0.24 / 12.1.12	Undertow no longer supported (lacks Servlet 6.1)
Jackson	3.1.5 (`tools.jackson` packages)	Jackson 2 still available via a separate, deprecated module
Hibernate ORM / Validator	7.4.5 / 9.1.3	ORM = object-relational mapper
Spring Security	7.1.1	
Spring Data	2026.0.1 release train (modules released together)	
JUnit Jupiter	6.0.3	JUnit 4 support deprecated in Framework 7
Mockito / AssertJ	5.23.0 / 3.27.7	Test fakes / readable assertions
Kotlin	2.3.21 (2.2 minimum)	
GraalVM (native images)	25+	

`javax` → `jakarta`

Java EE was handed to the Eclipse Foundation in 2017 and renamed **Jakarta EE**; for trademark reasons its packages moved from `javax.*` to `jakarta.*` (Jakarta EE 9, 2020). Since Spring 6 / Boot 3 Spring uses only `jakarta` names (`jakarta.servlet.http.HttpServletRequest`, `jakarta.persistence.Entity`, `jakarta.annotation.PostConstruct`, `jakarta.validation.constraints.NotNull`). Framework 7 removed the last support for `javax.annotation` and `javax.inject`. A tutorial importing `javax.persistence` is for Boot 2 or older.

How versions relate

- **Boot pins everything.** You choose the Boot version; its BOM chooses compatible Spring Framework, Data, Security, Tomcat, Jackson, Hibernate, JUnit, etc.
- **Cadence.** A new minor (4.0, 4.1 — features, no breaking changes) roughly every six months; patch releases (4.1.1, 4.1.2 — bug fixes only) in between.
- **Support.** Each minor line gets roughly 13 months of free open-source support, then commercial support. Check spring.io/projects/spring-boot "Support" before planning an upgrade.
- **Version names.** `-M1`, `-M2` milestones, `-RC1` release candidates, plain `4.1.1` is GA (generally available). `-SNAPSHOT` = nightly builds — never in production.

Common mistakes

- Adding `<version>` to a Spring dependency in a Boot project — overrides the tested version. Let the parent manage it.
- Mixing `javax.*` and `jakarta.*` imports after copying old code.
- Following a Boot 2/3 tutorial for Boot 4 without checking renamed starters (§1.7) and Jackson 3 package names.

1.5 Creating a project with start.spring.io

What it is

Spring Initializr (**start.spring.io**) is a web service that generates a ready-to-build Boot project as a zip. IntelliJ IDEA, VS Code (Spring Boot Extension Pack) and Eclipse (Spring Tools) call the same service from their "New project" wizards.

How it works

Choose: **Project** — *Gradle - Groovy* (default), *Gradle - Kotlin* or *Maven* (build tools; this book uses Maven); **Language** — Java (Kotlin, Groovy offered); **Spring Boot** — latest GA without suffix (**4.1.1**; avoid `SNAPSHOT` / `M` / `RC`); **Metadata** — *Group* `com.example` (reversed domain), *Artifact* `hello-boot` (also the jar name), *Package* `com.example.hello` (main class goes here), *Packaging* `Jar` (`War` only for an external server), *Java* 17/21/25/27 (this book: 25); **Dependencies** — *Spring Web* (id `web`) and *Spring Boot DevTools* (id `devtools`). Then Generate, unzip, open. From a terminal:

```
curl https://start.spring.io/starter.zip \
  -d type=maven-project -d bootVersion=4.1.1 -d javaVersion=25 \
  -d groupId=com.example -d artifactId=hello-boot -d packageName=com.example.hello \
  -d dependencies=web,devtools -o hello-boot.zip
```

For Boot 4.1.1, *Spring Web* produces `spring-boot-starter-webmvc` and the test starter `spring-boot-starter-webmvc-test` — exactly the `pom.xml` in §1.6 (checked against the generated file).

Common mistakes

- Choosing a Java version newer than your installed JDK (`release version 25 not supported`). Check `java -version`, see Java §1.4.
- Adding every dependency "just in case". Each starter can switch on auto-configuration — e.g. *Spring Data JPA* without a database fails with "Failed to configure a DataSource".
- Putting classes in a package *outside* the main class's package (§1.8).

1.6 Project layout and the build file

The layout

```
hello-boot/
├── pom.xml                          ← build file (Maven)
├── mvnw, mvnw.cmd, .mvn/            ← Maven Wrapper: builds without installing Maven
└── src/
    ├── main/
    │   ├── java/com/example/hello/
    │   │   ├── HelloApplication.java   ← the main class (§1.8)
    │   │   ├── HelloController.java    ← REST endpoint (§1.11)
    │   │   ├── GreetingService.java
    │   │   └── Greeting.java
    │   └── resources/
    │       ├── application.properties  ← configuration (Chapter 3)
    │       ├── static/                 ← optional: files served as-is (HTML, CSS, JS)
    │       └── templates/              ← optional: server-side templates (Thymeleaf)
    └── test/java/com/example/hello/
        ├── HelloControllerTest.java
        └── HelloApplicationTests.java
```

Standard Maven layout — `src/main/java`, `src/main/resources`, `src/test/java` (Java §18.1–18.2). Everything under `src/main/resources` ends up on the **classpath** (where the JVM loads classes and files from), which is how Boot finds `application.properties`.

The `pom.xml`

```xml
<!-- code/hello-boot/pom.xml -->
<parent>                                            <!-- 1 -->
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.1</version>
  <relativePath/>
</parent>

<groupId>com.example</groupId>
<artifactId>hello-boot</artifactId>
<version>0.0.1-SNAPSHOT</version>

<properties>
  <java.version>25</java.version>                   <!-- 2 -->
</properties>

<dependencies>
  <dependency>                                      <!-- 3 -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
  <dependency>                                      <!-- 4 -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
  <dependency>                                      <!-- 5 -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>                                        <!-- 6 -->
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

1. **`spring-boot-starter-parent`** — a parent POM (Project Object Model, Maven's `pom.xml`; children inherit its settings) that (a) imports the dependency BOM (**Bill of Materials**: tested versions for hundreds of libraries), (b) sets UTF-8, (c) compiles with `-parameters` (so Spring can read constructor parameter names), (d) pre-configures plugins (surefire for tests, resources filtering, the Boot plugin).
2. **`java.version`** — becomes `maven.compiler.release`, so `javac --release 25`.
3. **Web starter** — no `<version>`: the parent manages it.
4. **DevTools** — `runtime` + `optional` keeps it out of projects depending on yours; the Boot plugin also leaves it out of the packaged jar by default.
5. **Test starter** — test classpath only.
6. **`spring-boot-maven-plugin`** — adds goals such as `spring-boot:run`, and during `package` **repackages** the jar into an executable "fat jar" containing all dependency jars (§1.9).

If you cannot use the parent (company parent POM), import the BOM and configure `-parameters` and the plugin's `repackage` goal yourself:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>4.1.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Gradle equivalent (what Initializr generates):

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1'
    id 'io.spring.dependency-management' version '1.1.7'
}
java { toolchain { languageVersion = JavaLanguageVersion.of(25) } }
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
tasks.named('test') { useJUnitPlatform() }
```

Commands: `./gradlew bootRun`, `./gradlew bootJar`, `./gradlew test`.

1.7 Starters

What it is

A **starter** is a dependency with (almost) no code; it pulls in a curated, version-aligned set of libraries for one capability. `spring-boot-starter-webmvc` gives you Spring MVC, embedded Tomcat, Jackson, logging, and the Boot auto-configuration that wires them.

How it works

Real runtime dependency tree of `hello-boot` (`mvn dependency:tree -Dscope=runtime`):

```
com.example:hello-boot:jar:0.0.1-SNAPSHOT
+- org.springframework.boot:spring-boot-starter-webmvc:jar:4.1.1:compile
|  +- org.springframework.boot:spring-boot-starter:jar:4.1.1:compile
|  |  +- org.springframework.boot:spring-boot-starter-logging:jar:4.1.1:compile   (Logback 1.5.38, SLF4J bridges)
|  |  ...
|  +- org.springframework.boot:spring-boot-starter-jackson:jar:4.1.1:compile
|  |  \- org.springframework.boot:spring-boot-jackson:jar:4.1.1:compile
|  |     \- tools.jackson.core:jackson-databind:jar:3.1.5:compile
|  +- org.springframework.boot:spring-boot-starter-tomcat:jar:4.1.1:compile
|  |  +- org.springframework.boot:spring-boot-starter-tomcat-runtime:jar:4.1.1:compile
|  |  |  +- org.apache.tomcat.embed:tomcat-embed-core:jar:11.0.24:compile
|  |  |  ...
|  |  \- org.springframework.boot:spring-boot-tomcat:jar:4.1.1:compile
|  +- org.springframework.boot:spring-boot-http-converter:jar:4.1.1:compile
|  |  \- org.springframework:spring-web:jar:7.0.9:compile
|  \- org.springframework.boot:spring-boot-webmvc:jar:4.1.1:compile
|     +- org.springframework.boot:spring-boot-servlet:jar:4.1.1:compile
|     \- org.springframework:spring-webmvc:jar:7.0.9:compile
\- org.springframework.boot:spring-boot-devtools:jar:4.1.1:runtime (optional)
   +- org.springframework.boot:spring-boot:jar:4.1.1:compile   (spring-core / spring-context 7.0.9, jspecify 1.0.1)
   \- org.springframework.boot:spring-boot-autoconfigure:jar:4.1.1:compile
```

The Boot 4 pattern: each technology has a **Boot module** holding its auto-configuration (`spring-boot-webmvc`, `spring-boot-tomcat`, `spring-boot-jackson`) and a **starter** combining that module with the library.

Boot 4 modular starters

Boot 3 shipped all auto-configuration in one `spring-boot-autoconfigure` jar. **Boot 4 split it into per-technology modules** and renamed some starters. In 4.1.1 the core `spring-boot-autoconfigure` jar lists only 12 auto-configurations; MVC's live in `spring-boot-webmvc`, Tomcat's in `spring-boot-tomcat`, etc. (Chapter 3).

Capability	Boot 4 starter	Boot 3 name (if different)
Servlet web / REST (Spring MVC + Tomcat)	`spring-boot-starter-webmvc`	`spring-boot-starter-web`
Reactive web (WebFlux + Netty)	`spring-boot-starter-webflux`	
Blocking HTTP client (`RestClient`)	`spring-boot-starter-restclient`	part of `-web`
Reactive HTTP client (`WebClient`)	`spring-boot-starter-webclient`	part of `-webflux`
JSON (Jackson 3)	`spring-boot-starter-jackson`	`spring-boot-starter-json` (still available in 4.x)
Bean Validation	`spring-boot-starter-validation`	
JDBC / JPA	`spring-boot-starter-jdbc` / `spring-boot-starter-data-jpa`	
Security	`spring-boot-starter-security`	
OAuth2 resource server	`spring-boot-starter-security-oauth2-resource-server`	`spring-boot-starter-oauth2-resource-server`
AspectJ-based AOP	`spring-boot-starter-aspectj`	`spring-boot-starter-aop`
Kafka	`spring-boot-starter-kafka`	(previously just `spring-kafka`)
Actuator	`spring-boot-starter-actuator`	
Core only (no web)	`spring-boot-starter`	
Testing, generic	`spring-boot-starter-test`	
Testing, per technology	`spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-security-test`, ...	— (new in Boot 4)

Rules of thumb for Boot 4:

- **Every technology starter has a `-test` companion** bringing its test slice (`@WebMvcTest` lives in `spring-boot-webmvc-test`). Each test starter includes `spring-boot-starter-test`.
- Some old names remain in 4.1 as **deprecated** aliases (`spring-boot-starter-web`, `spring-boot-starter-oauth2-resource-server`); others are gone (`spring-boot-starter-aop` → `spring-boot-starter-aspectj`). Prefer the new names.
- For a quick Boot 3 upgrade, `spring-boot-starter-classic` and `spring-boot-starter-test-classic` restore the old "everything on the classpath" behaviour while you fix imports.
- Third-party starters are named `<name>-spring-boot-starter` (e.g. `mybatis-spring-boot-starter`); only official ones start with `spring-boot-starter-` (Chapter 3 builds one).

Common mistakes

- Adding both `-webmvc` and `-webflux` starters and being surprised it's a *servlet* app: with both present Boot chooses Spring MVC.
- Using `spring-boot-starter-test` alone and finding `@WebMvcTest` missing in Boot 4 — add `spring-boot-starter-webmvc-test`.
- Excluding a transitive library (one you get indirectly) by hand to "slim down" and breaking auto-configuration. Use the intended starter (e.g. exclude `spring-boot-starter-tomcat`, add `spring-boot-starter-jetty`).

1.8 The main class and `@SpringBootApplication`

```java
// code/hello-boot/src/main/java/com/example/hello/HelloApplication.java
package com.example.hello;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication            // = @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan
public class HelloApplication {

    public static void main(String[] args) {
        SpringApplication.run(HelloApplication.class, args);   // creates and starts the ApplicationContext
    }
}
```

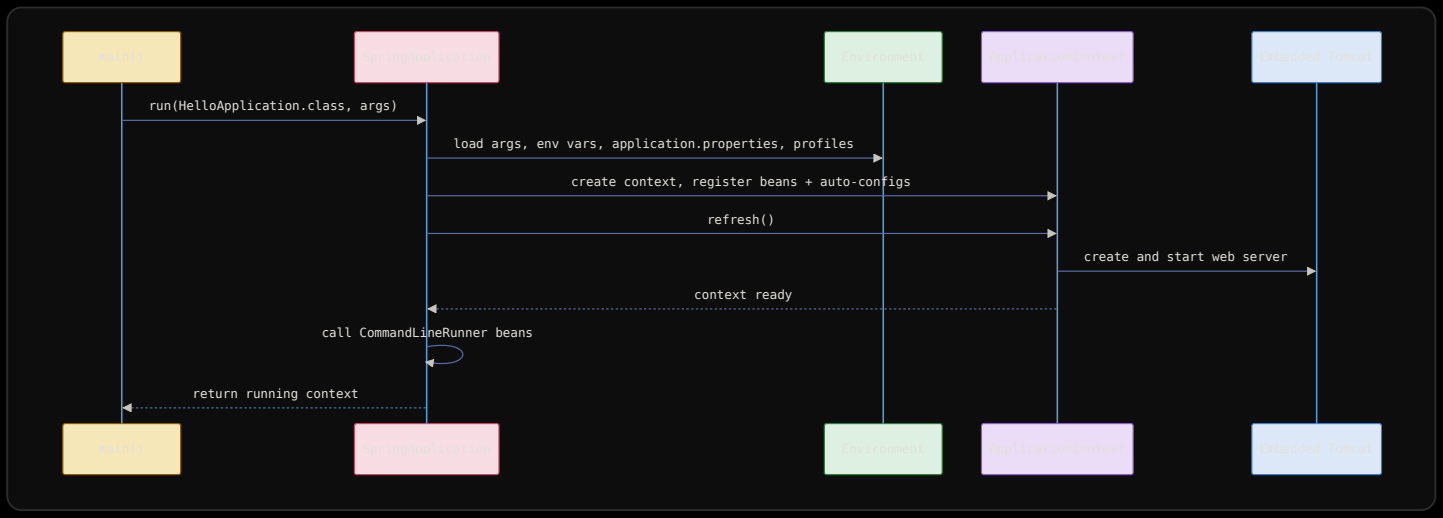
What `@SpringBootApplication` means

Annotation	What it does
`@SpringBootConfiguration`	Marks this class as a configuration source (a specialised `@Configuration`). Tests use it to find your app's configuration.
`@EnableAutoConfiguration`	Loads candidate auto-configuration classes and applies those whose conditions match (Chapter 3).
`@ComponentScan`	Scans **this class's package and all sub-packages** for `@Component`, `@Service`, `@Repository`, `@Controller`, `@RestController`, `@Configuration` classes and registers them as beans.

What `SpringApplication.run` does

1. Decides the application type from the classpath: **servlet** (Spring MVC present), **reactive** (only WebFlux) or **none**.
2. Prepares the `Environment`: command-line args, environment variables, `application.properties`/`.yaml`, active profiles (Chapter 3).
3. Prints the banner.
4. Creates the right `ApplicationContext` (the IoC container; web-server-aware for a servlet app).
5. Registers bean definitions: scanned classes plus auto-configurations whose conditions pass.
6. **Refreshes** the context: creates all singleton beans (one shared instance — the default) and, for a web app, starts embedded Tomcat.
7. Runs `CommandLineRunner`/`ApplicationRunner` beans (run-once-at-startup code).

8. Returns the running context; the JVM keeps running while Tomcat's threads are alive.



Where to put the main class

In the **root package** (`com.example.hello`), with every other class in that package or below (`com.example.hello.orders`, `...web`). Scanning starts from the main class's package, so a class in `com.example.other` is silently **not** a bean.

Common mistakes

- Main class in a sub-package (`com.example.hello.app`) while controllers live in `com.example.hello.web` → endpoints return 404 because the controllers were never scanned.
- Main class in the *default* (unnamed) package — Boot warns, and scanning every jar on the classpath breaks startup.
- Writing logic in `main` after `run(...)` expecting it to run "before the server starts". Use a `CommandLineRunner` or an event listener (Chapter 2).

In the interview

Q: What does `@SpringBootApplication` do? It combines `@SpringBootConfiguration` (this class is configuration), `@EnableAutoConfiguration` (apply Boot's conditional auto-configurations) and `@ComponentScan` (register annotated classes from this package downwards).

- *Follow-up: Scan a package outside the main package?* Move the code under the root package (best), or `@SpringBootApplication(scanBasePackages = "...")` / `@Import` for specific classes.
- *Follow-up: Exclude one auto-configuration?* `@SpringBootApplication(exclude = SomeAutoConfiguration.class)` or the property `spring.autoconfigure.exclude`.

1.9 Running the application

Three ways to run

Way	Command	When
IDE	Run `HelloApplication.main`	Daily development, breakpoints
Maven plugin	`./mvnw spring-boot:run` (or `mvn spring-boot:run`)	Terminal development; reads `src/main/resources` directly
Executable jar	`./mvnw package` then `java -jar target/hello-boot-0.0.1-SNAPSHOT.jar`	Production, containers, CI

Settings on the command line:

```
mvn spring-boot:run -Dspring-boot.run.arguments="--server.port=9090"
java -jar target/hello-boot-0.0.1-SNAPSHOT.jar --server.port=9090 --spring.profiles.active=dev
```

The startup log

Running the packaged jar on JDK 25 (`java -jar target/hello-boot-0.0.1-SNAPSHOT.jar --server.port=18081`):

```
 :: Spring Boot ::                (v4.1.1)      ← after the ASCII-art banner

INFO 91361 --- [hello-boot] [main] c.e.hello.HelloApplication : Starting HelloApplication v0.0.1-SNAPSHOT using Java
25.0.2 with PID 91361 ...
INFO 91361 --- [hello-boot] [main] c.e.hello.HelloApplication : No active profile set, falling back to 1 default
profile: "default"
INFO 91361 --- [hello-boot] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 18081 (http)
INFO 91361 --- [hello-boot] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache
Tomcat/11.0.24]
INFO 91361 --- [hello-boot] [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization
completed in 376 ms
INFO 91361 --- [hello-boot] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 18081 (http) with context
path '/'
INFO 91361 --- [hello-boot] [main] c.e.hello.HelloApplication : Started HelloApplication in 0.728 seconds (process
running for 0.929)
```

(Timestamps removed, logger names shortened.)

Log part	Meaning
`91361`	Process ID
`[hello-boot]`	`spring.application.name` from `application.properties`
`[main]`	Thread name
`No active profile set...`	Only `application.properties` applies (profiles: Chapter 3)
`Tomcat initialized/started on port ...`	The embedded server is listening
`Started HelloApplication in 0.728 seconds`	The context is fully refreshed and ready

On the first request you also see `Initializing Spring DispatcherServlet 'dispatcherServlet'` — Spring MVC's front controller (one object that receives and routes every request) is created lazily (Chapter 5). Ctrl+C (or `kill <pid>`) shows:

```
INFO ... [ionShutdownHook] o.s.boot.tomcat.GracefulShutdown : Commencing graceful shutdown. Waiting for active requests
to complete
INFO ... [tomcat-shutdown] o.s.boot.tomcat.GracefulShutdown : Graceful shutdown complete
```

Graceful shutdown is on by default: the server stops accepting new requests and lets in-flight ones finish (up to `spring.lifecycle.timeout-per-shutdown-phase`, default 30 s) before the context closes (Chapter 15).

Inside the executable jar

`mvn package` produced a 19.9 MB jar. Layout (`unzip -l`) and manifest (`MANIFEST.MF`, tells Java which class to start):

```
META-INF/MANIFEST.MF
org/springframework/boot/loader/...        ← Boot's launcher classes
BOOT-INF/classes/com/example/hello/*.class ← your code
BOOT-INF/classes/application.properties
BOOT-INF/lib/*.jar                         ← 34 dependency jars, stored unchanged
BOOT-INF/classpath.idx, BOOT-INF/layers.idx
```

```
Main-Class: org.springframework.boot.loader.launch.JarLauncher
Start-Class: com.example.hello.HelloApplication
Spring-Boot-Version: 4.1.1
Spring-Boot-Classes: BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/
```

A plain JVM cannot load classes from jars *nested* inside a jar. So `java -jar` starts Boot's **`JarLauncher`** (`Main-Class`), which sets up a class loader able to read `BOOT-INF/lib/*.jar`, then calls your **`Start-Class`**. `layers.idx` groups files into layers (dependencies change rarely, your classes often) so Docker images can cache them (Chapter 15). General jar packaging: Java §18.12.

Common mistakes

- `Web server failed to start. Port 8080 was already in use.` — another process (often a previous run) holds the port. Stop it or set `server.port`; `server.port=0` picks a random free port. Without editing files: `--server.port=9090` or env var `SERVER_PORT`.
- Running `java -jar` on the *original* jar (`*.jar.original`) — no launcher, fails with "no main manifest attribute".
- Expecting DevTools to work with `java -jar`: packaged apps disable it (§1.10).

1.10 DevTools

What it is

Spring Boot DevTools (`spring-boot-devtools`) is a development-only module for a faster edit–run loop:

Feature	What it does
Automatic restart	When compiled classes on the classpath change, the app restarts in about a second.
Development property defaults	Turns off template and static-resource caching, enables the H2 console (web UI for the in-memory H2 database), etc.
Global settings	`~/.config/spring-boot/spring-boot-devtools.properties` applies to all your apps.
Remote DevTools	Opt-in restart of a remote app over HTTP; a security risk — trusted networks only.
LiveReload	Browser auto-refresh; **disabled by default since Boot 4.0, deprecated in 4.1** (no replacement).

How restart works

1. **Two class loaders**: a *base* loader for jars that don't change (Spring, Tomcat, Jackson...) and a *restart* loader for your classes.
2. It watches compiled output folders (e.g. `target/classes`).
3. When something changes there — IDE compile-on-save, IntelliJ *Build Project*, or `mvn compile` — it discards only the restart loader and starts the context again.
4. The warm base loader makes this much faster than a cold start.

Changes under `static/`, `public/`, `templates/` and `META-INF/resources` do **not** trigger a restart. Tune with `spring.devtools.restart.exclude` / `additional-exclude`, or disable with `spring.devtools.restart.enabled=false`.

When it is off

DevTools treats an app launched with `java -jar` (or from a special class loader) as production and disables itself. It is also off in tests: this chapter's test log shows `Restart disabled due to context in which it is running`.

Common mistakes

- Expecting a restart after *editing* a `.java` file without compiling — DevTools watches compiled classes. Enable "build project automatically".
- Shipping DevTools to production by removing `<optional>true</optional>` or configuring the plugin to include it.
- Confusing restart with hot-swapping (replacing code inside the running JVM): restart re-creates the whole context, so in-memory state (sessions, caches) is lost. Tools like JRebel do true class reloading.

1.11 Your first REST endpoint

What we build

`GET /hello?name=Ada` → `{"message":"Hello, Ada!","length":11}`, with the typical layering: a **controller** (HTTP), a **service** (logic) and a **record** (the JSON shape).

```java
// Greeting.java — the response body
public record Greeting(String message, int length) {}
```

A **record** is a compact immutable data class (Java 16 — Java §6.8). Jackson 3 serializes its components as JSON fields.

```java
// GreetingService.java
@Service                                  // a bean: Spring creates one instance and manages it
public class GreetingService {
    public Greeting greet(String name) {
        String message = "Hello, " + name + "!";
        return new Greeting(message, message.length());
    }
}
```

```java
// HelloController.java
@RestController                           // a bean whose method results are written to the HTTP response body
public class HelloController {

    private final GreetingService service;

    HelloController(GreetingService service) { // constructor injection: Spring passes the GreetingService bean
        this.service = service;
    }

    @GetMapping("/hello")                 // handles GET /hello and GET /hello?name=Ada
    Greeting hello(@RequestParam(defaultValue = "World") String name) {
        return service.greet(name);
    }
}
```

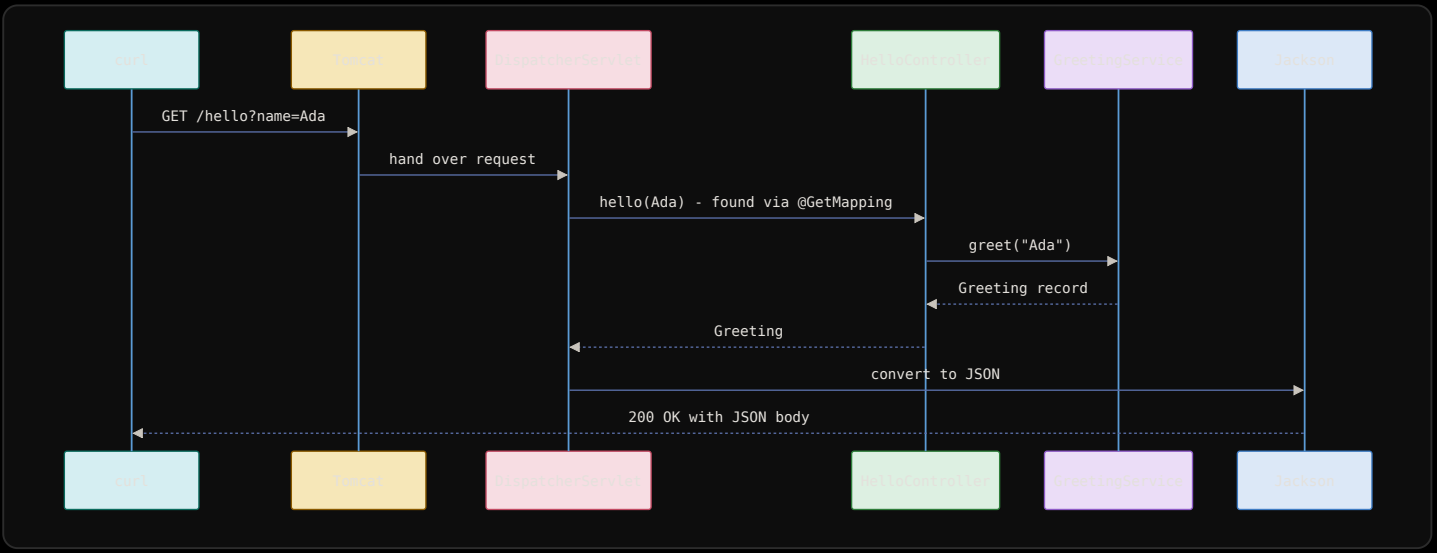
Piece	Role
`@RestController`	`@Controller` + `@ResponseBody`: the return value becomes the response body (JSON), not a view name.
Constructor parameter	Spring passes the singleton `GreetingService`. No `@Autowired` needed with a single constructor (Chapter 2).
`@GetMapping("/hello")`	Maps HTTP GET `/hello` to this method.
`@RequestParam(defaultValue = "World")`	Reads query parameter `name`; if absent, `"World"`.
Return type `Greeting`	Jackson (auto-configured) writes JSON with `Content-Type: application/json`.

Calling it

```
$ curl -s 'localhost:18081/hello?name=Ada'
{"message":"Hello, Ada!","length":11}
$ curl -s localhost:18081/hello
{"message":"Hello, World!","length":13}
$ curl -s -i localhost:18081/hello | head -3
HTTP/1.1 200
Content-Type: application/json
Content-Length: 39
$ curl -s localhost:18081/nope
{"timestamp":"2026-09-28T23:24:50.396Z","status":404,"error":"Not Found","path":"/nope"}
```

The last response is Boot's default error handling (`/error` endpoint). Chapter 5 replaces it with RFC 9457 `ProblemDetail` responses (a standard JSON shape for errors).

What happened during a request



Common mistakes

- `@Controller` instead of `@RestController` for a JSON API → `"Hello"` is treated as a *view name* and fails with a template error.
- Business logic in the controller. Keep controllers thin (parse input, call a service, return output).
- Returning mutable database entities directly — prefer small response records (Chapter 8).

1.12 Testing the first app

Spring Boot tests are ordinary JUnit tests with annotations that start (part of) the container. Two styles cover most needs; Chapter 12 goes deeper. JUnit basics: Java §18.4.

Test 1 — a web *slice* with `MockMvcTester`

```
// HelloControllerTest.java
@WebMvcTest(HelloController.class)        // only the web layer; no real server
@Import(GreetingService.class)             // the service is not part of the web slice, so add it
class HelloControllerTest {

    @Autowired
    MockMvcTester mvc;                     // AssertJ-flavoured MockMvc (Framework 6.2+)

    @Test
    void greetsWorldByDefault() {
        var result = mvc.get().uri("/hello").exchange();
        System.out.println("GET /hello          -> " + result.getResponse().getStatus() + " " + contentOf(result));
        assertThat(result).hasStatusOk()
                .bodyJson().extractingPath("$.message").isEqualTo("Hello, World!");
    }

    @Test
    void greetsByName() {
        var result = mvc.get().uri("/hello?name=Ada").exchange();
        System.out.println("GET /hello?name=Ada -> " + result.getResponse().getStatus() + " " + contentOf(result));
        assertThat(result).hasStatusOk()
                .bodyJson().extractingPath("$.length").isEqualTo(11);
    }
    // contentOf(...) reads the response body as a String
}
```

- `@WebMvcTest` starts a *slice*: MVC infrastructure, controllers, JSON and filters only — no services, repositories or server. Hence `@Import(GreetingService.class)` (the common alternative is mocking it with `@MockitoBean`, Chapter 12).
- `MockMvcTester` sends requests straight to the `DispatcherServlet` in memory, with AssertJ assertions like `hasStatusOk()` and `bodyJson().extractingPath(...)` (JSONPath; `$` is the root).

Test 2 — the whole app on a real port with `RestTestClient`

```
// HelloApplicationTests.java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
class HelloApplicationTests {

    @LocalServerPort int port;
    @Autowired RestTestClient client;      // Framework 7 test client, pre-bound to the random port

    @Test
    void realHttpCall() {
        String body = client.get().uri("/hello?name=Spring")
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentType("application/json")
                .expectBody(String.class).returnResult().getResponseBody();
        System.out.println("Tomcat on random port " + (port > 0 ? "OK" : "?") + ", body = " + body);
        assertThat(body).isEqualTo("{\"message\":\"Hello, Spring!\",\"length\":14}");
    }
}
```

- **`@SpringBootTest`** starts the full context like `main`; with `RANDOM_PORT` it also starts Tomcat on a free port, so the test makes real HTTP calls.
- **`RestTestClient`** (new in Framework 7) is a fluent test client for blocking apps; `@AutoConfigureRestTestClient` (module `spring-boot-resttestclient`, included by the webmvc test starter) binds it to the running server.
- Surefire (the unit-test plugin) runs only classes ending in `Test`, `Tests` or `TestCase` by default — a class like `HelloApplicationIT` never runs with `mvn verify` here; `*IT` classes need the Failsafe plugin, which this project does not configure.

Output

`mvn -q -B -ntp verify` (banners and log lines removed):

```
Tomcat on random port OK, body = {"message":"Hello, Spring!","length":14}
GET /hello?name=Ada -> 200 {"message":"Hello, Ada!","length":11}
GET /hello          -> 200 {"message":"Hello, World!","length":13}
```

(JUnit does not guarantee method order, so the two `GET` lines can swap.) Exit code 0: all three tests passed.

Trade-offs

	`@WebMvcTest` + `MockMvcTester`	**`@SpringBootTest(RANDOM_PORT)` + `RestTestClient`**
Starts	Web layer only, no server	Whole app + real Tomcat
Speed	Fast	Slower
Catches	Mapping, JSON, validation, controller logic	Everything incl. filters, server config, real HTTP
Use for	Most controller tests	A few end-to-end "smoke" tests

1.13 In the interview

Q: Walk me through what happens when you run a Spring Boot jar. `java -jar` starts `JarLauncher` from the manifest; it builds a class loader for `BOOT-INF/lib` jars and calls the `Start-Class` main method. `SpringApplication.run` detects a servlet app, prepares the `Environment` (args, env vars, config files, profiles), creates the `ApplicationContext`, registers scanned beans and matching auto-configurations, refreshes the context — creating singletons and starting embedded Tomcat — then runs `CommandLineRunner`s.

- *Follow-up: Why an embedded server?* One self-contained artifact per service: same jar in dev, CI and production, easy to containerise, no server-version drift.
- *Follow-up: Can you still build a WAR?* Yes: packaging `war`, extend `SpringBootServletInitializer`, mark the server starter `provided`.

Q: What is a starter? A code-less dependency bringing a curated, version-aligned library set plus the Boot module holding its auto-configuration; in Boot 4, one per technology with a `-test` companion (§1.7).

Q: What changed in Spring Boot 4 / Framework 7? Java 17 baseline (25 recommended), Jakarta EE 11 (Servlet 6.1, Tomcat 11), modularized Boot jars and renamed starters, Jackson 3 (`tools.jackson`), JUnit 6, JSpecify null-safety, API versioning in MVC/WebFlux, HTTP service client auto-configuration, `RestTestClient`, core retry/concurrency-limit annotations, Undertow removed. Details in Chapter 17.

Q: `@WebMvcTest` vs `@SpringBootTest`? Fast MVC-only slice vs the whole app (optionally on a real server) — see the §1.12 trade-offs table.

Chapter TL;DR

- **Spring Framework** = IoC container that creates and wires your objects (beans) + modules for web, data, transactions, messaging.
- **Spring Boot** = Spring + starters + managed versions (BOM) + auto-configuration + embedded server + externalized config + Actuator.
- **Current generation**: Framework 7 and Boot 4 (Nov 2025); this book uses **Boot 4.1.1 → Framework 7.0.9**, Jakarta EE 11, Tomcat 11, Jackson 3, JUnit 6, on **Java 25** (Java 17 minimum).
- Generate projects at **start.spring.io**; the `pom.xml` inherits `spring-boot-starter-parent`, has version-less starters and the `spring-boot-maven-plugin`.
- **Boot 4 starters are modular**: `spring-boot-starter-webmvc` (was `-web`), a `-test` starter per technology.
- `@SpringBootApplication` = `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan` (from the main class's package down).
- Run with the IDE, `mvn spring-boot:run`, or `java -jar` on the fat jar (`JarLauncher` → your `Start-Class`). Graceful shutdown is on by default.
- **DevTools** restarts fast on recompilation and disables itself for packaged jars and tests.
- A REST endpoint is a `@RestController` method with `@GetMapping`; return a record and Jackson writes JSON.
- Test controllers with `@WebMvcTest` + `MockMvcTester`; smoke-test the whole app with `@SpringBootTest(RANDOM_PORT)` + `RestTestClient`.

Next: Chapter 2 — IoC and Dependency Injection opens up the container you just used.

Chapter 2 — IoC and Dependency Injection

Everything in Spring — web controllers, transactions, security, Spring Data repositories — sits on one foundation: the **IoC container** that creates your objects and connects them. Understand the container and the rest of Spring stops feeling like magic. This chapter covers bean definition, dependency resolution and injection, scopes, the full lifecycle, post-processors, circular dependencies, lazy beans, conditions and profiles, events, and two Framework 7 additions — JSpecify null-safety and `BeanRegistrar`.

Runnable code: `code/ioc-container/` (plain Spring containers inside JUnit tests, so each demo gets its own isolated context) and `code/web-scopes/` (request and session scopes). Both built with `mvn -q -B -ntp verify` on Spring Boot 4.1.1 / Framework 7.0.9 / JDK 25. Each demo writes to a small `Trace` log that the tests print, so the outputs show the *order* in which Spring did things.

2.1 Inversion of Control and Dependency Injection

What it is

- **Dependency**: an object another object needs to do its job. A `PriceCalculator` needs a `TaxPolicy`.
- **Inversion of Control (IoC)**: an outside party (the framework), not your code, creates and controls collaborators and calls your code.
- **Dependency Injection (DI)**: the form of IoC where an object receives its dependencies from outside — constructor, setter or field — instead of creating them with `new` or looking them up.
- **IoC container**: the Spring object that performs DI for a whole application. Objects it manages are **beans**.

The problem it solves

```java
// code/ioc-container/src/main/java/com/example/ioc/basics/PriceCalculator.java
public class PriceCalculator {
    private final TaxPolicy taxPolicy;

    public PriceCalculator(TaxPolicy taxPolicy) {       // the dependency is handed in, not created here
        this.taxPolicy = taxPolicy;
        Trace.add("PriceCalculator created with " + taxPolicy.getClass().getSimpleName());
    }

    public long totalCents(long netCents) { return netCents + taxPolicy.taxCents(netCents); }
}
```

`PriceCalculator` depends on the *interface* `TaxPolicy`, not on `FlatTaxPolicy`. That already gives testability (pass a fake `TaxPolicy` in a unit test) and flexibility (swap implementations without editing the class). DI is first a *design style*; the container automates the wiring:

```java
// Without Spring — you wire by hand (fine for 3 classes, painful for 300):
PriceCalculator calc = new PriceCalculator(new FlatTaxPolicy());

// With Spring — describe the wiring once:
@Configuration
public class ShopConfig {
    @Bean TaxPolicy taxPolicy() { return new FlatTaxPolicy(); }

    @Bean PriceCalculator priceCalculator(TaxPolicy taxPolicy) {   // parameter = dependency
        return new PriceCalculator(taxPolicy);
    }
}
// ...and ask the container for the finished object:
try (var ctx = new AnnotationConfigApplicationContext(ShopConfig.class)) {
    PriceCalculator calc = ctx.getBean(PriceCalculator.class);
}
```

Why the container is worth it

Hand wiring	Spring container
You decide creation order	Spring derives the order from the dependency graph
One object per `new`; sharing needs care	Singletons shared automatically; other scopes available
No central place for cross-cutting behaviour (logging, security touching many classes)	Can wrap beans in proxies (stand-ins that intercept calls): transactions, security, caching (Chapter 4)
Configuration values passed by hand	Values injected from properties/environment (Chapter 3)
Start-up/shut-down hooks by hand	Lifecycle callbacks and graceful shutdown

When not to use it

Beans are the long-lived "service" objects — services, repositories, controllers, clients, configuration. Data objects (a `Greeting`, an `Order` entity, a DTO — Data Transfer Object, a plain data carrier between layers) are created with `new`, many times per request; making them beans adds nothing.

The design ideas behind DI — Dependency Inversion and "program to an interface" — are in LLD §4.10 and LLD Chapter 5.

2.2 The container: `BeanFactory` vs `ApplicationContext`

What it is

- `BeanFactory` — the basic container: holds **bean definitions** (recipes: class, scope, dependencies, init method...) and creates beans from them on request.
- `ApplicationContext` — extends `BeanFactory` with everything a real application needs. It is what you always use; Spring Boot creates one for you.

Feature	`BeanFactory`	`ApplicationContext`
Create and inject beans	Yes	Yes
Singletons created	**Lazily**, on first `getBean`	**Eagerly**, at startup (`refresh()`) — errors show up immediately
Registers `BeanPostProcessor`s automatically	No (manual)	Yes — so `@Autowired`, `@PostConstruct`, `@Transactional` work
Application events (`publishEvent`, `@EventListener`)	No	Yes
`Environment`, profiles, property sources	No	Yes
i18n (internationalization) messages (`MessageSource`)	No	Yes
Resource loading (`classpath:`, `file:`)	No	Yes
Typical implementations	`DefaultListableBeanFactory`	`AnnotationConfigApplicationContext`; in Boot: servlet/reactive web-server contexts

Eager vs lazy — proved

```
// ContainerBasicsTest.java (excerpt)
try (var ctx = new AnnotationConfigApplicationContext(ShopConfig.class)) {
    Trace.add("-- context refreshed, now calling getBean");
    PriceCalculator calc = ctx.getBean(PriceCalculator.class);
    Trace.add("total for 1000 = " + calc.totalCents(1000));
    assertThat(ctx.getBean(PriceCalculator.class)).isSameAs(calc);   // singleton
}

var bf = new DefaultListableBeanFactory();
bf.registerBeanDefinition("taxPolicy", new RootBeanDefinition(FlatTaxPolicy.class));
var calcDef = new RootBeanDefinition(PriceCalculator.class);
calcDef.setAutowireMode(RootBeanDefinition.AUTOWIRE_CONSTRUCTOR);
bf.registerBeanDefinition("priceCalculator", calcDef);
Trace.add("-- definitions registered, nothing created yet");
bf.getBean(PriceCalculator.class);
```

Output:

```
--- ApplicationContext (eager) ---
 1. FlatTaxPolicy created
 2. PriceCalculator created with FlatTaxPolicy
 3. -- context refreshed, now calling getBean
 4. total for 1000 = 1200
--- Bare BeanFactory (lazy) ---
 1. -- definitions registered, nothing created yet
 2. FlatTaxPolicy created
 3. PriceCalculator created with FlatTaxPolicy
```

The context created both beans *before* anyone asked, created `FlatTaxPolicy` first because `PriceCalculator` needs it, and `getBean` twice returned the same instance (the default **singleton** scope).

How startup works (`refresh()`)

1. **Load bean definitions** — `@Configuration` classes, component scanning, `@Import`, registrars.
2. **Run `BeanFactoryPostProcessor`s** — may read or change *definitions* (e.g. resolve `${...}` placeholders) before any bean exists (§2.8).
3. **Register `BeanPostProcessor`s** — objects that get a chance to modify every bean.
4. **Instantiate all non-lazy singletons** — construct, inject, init callbacks, post-processors (§2.7).
5. **Finish** — start `SmartLifecycle` beans, publish `ContextRefreshedEvent`.

In Spring Boot, `SpringApplication.run` does all this (§1.8).

Common mistakes

- Calling `ctx.getBean(...)` throughout your code ("service locator" style). Use injection; `getBean` belongs in tests, `main` methods and framework code.
- Creating a second `ApplicationContext` inside a bean — a duplicate set of beans that are not the ones the rest of the app uses.

In the interview

Q: BeanFactory vs ApplicationContext? `BeanFactory` is the minimal container: definitions + lazy creation. `ApplicationContext` adds eager singleton creation, automatic post-processor registration (needed for annotations), events, `Environment`/profiles, i18n and resource loading. Applications use `ApplicationContext`.

- *Follow-up: Why is eager creation good?* Configuration errors (missing bean, bad property) fail at startup instead of on the first request in production.

2.3 Defining beans

Four ways; most apps use the first two.

1. Component scanning with stereotype annotations

Stereotype annotations say what *role* a class plays. Spring finds annotated classes while scanning packages (`@ComponentScan`, included in `@SpringBootApplication`).

Annotation	Meaning	Extra behaviour
`@Component`	Generic bean	—
`@Service`	Business logic	None (documents intent)
`@Repository`	Data access	Persistence exceptions translated to Spring's `DataAccessException` (when a translation post-processor is present — Boot/Spring Data set it up)
`@Controller`	Spring MVC controller (returns views)	Detected as a request handler
`@RestController`	`@Controller` + `@ResponseBody`	Return values written as the body
`@Configuration`	Holds `@Bean` methods	Proxied in full mode (below)

All are meta-annotated with `@Component` (their declaration carries `@Component`), which is why scanning finds them.

```
@Repository public class InvoiceRepository {}

@Service public class InvoiceService {
    private final InvoiceRepository repository;
    public InvoiceService(InvoiceRepository repository) { this.repository = repository; }
}

@Component("renderer") public class PdfRenderer {}   // explicit bean name

public class NotABean {}                              // no annotation -> ignored
```

```
scanned beans: invoiceRepository, invoiceService, renderer
```

Bean names. Default: simple class name with lower-case first letter (`InvoiceService` → `invoiceService`); `@Component("renderer")` overrides it. A *nested* class includes the outer class: `Ambiguous.DiskStorage` → `ambiguous.DiskStorage` (see §2.5's output).

2. `@Bean` methods

A method that builds the object; Spring calls it once (for a singleton) and registers the result. Use `@Bean` for **classes you can't annotate** (third-party: `ObjectMapper` — Jackson's JSON converter, `DataSource`, a cloud SDK client) and when construction needs logic.

```
@Configuration
public class ShopConfig {
    @Bean TaxPolicy taxPolicy() { return new FlatTaxPolicy(); }                  // bean name = method name
    @Bean PriceCalculator priceCalculator(TaxPolicy taxPolicy) { return new PriceCalculator(taxPolicy); }
}
```

`@Bean` attributes: `name`/`value` (names and aliases), `initMethod`, `destroyMethod`, `autowireCandidate`, `defaultCandidate` (6.2). Combine with `@Scope`, `@Lazy`, `@Primary`, `@Profile`, `@Conditional...` on the same method.

Full vs lite `@Configuration` — proved

A `@Bean` method can call *another* `@Bean` method. What that returns depends on the mode:

```java
@Configuration                      // "full" mode: proxyBeanMethods = true (the default)
public class FullConfig {
    @Bean AuditLog auditLog() { return new AuditLog(); }
    @Bean OrderService orderService() { return new OrderService(auditLog()); }       // inter-bean call
    @Bean PaymentService paymentService() { return new PaymentService(auditLog()); } // inter-bean call
}

@Configuration(proxyBeanMethods = false)   // "lite" mode: no CGLIB subclass
public class LiteConfig { /* identical methods */ }

@Configuration(proxyBeanMethods = false)   // lite mode done right: ask for dependencies as parameters
public class LiteSafeConfig {
    @Bean AuditLog auditLog() { return new AuditLog(); }
    @Bean OrderService orderService(AuditLog log) { return new OrderService(log); }
    @Bean PaymentService paymentService(AuditLog log) { return new PaymentService(log); }
}
```

Output (`AuditLog` numbers each instance it creates):

```
full: config class = FullConfig$$SpringCGLIB$$0
full: order.log=#1 payment.log=#1, AuditLogs created=1
lite: config class = LiteConfig
lite: order.log=#2 payment.log=#3, AuditLogs created=3
lite+params: order.log=#1 payment.log=#1, AuditLogs created=1
```

How it works:

1. In **full mode** Spring generates a **CGLIB subclass** of your configuration class at startup (`FullConfig$$SpringCGLIB$$0`). CGLIB is a bytecode library that creates subclasses at runtime.
2. The subclass overrides each `@Bean` method: "if the bean exists in the container, return it; otherwise call the real method once" — so `auditLog()` called twice still yields one singleton.
3. In **lite mode** (`proxyBeanMethods = false`, or `@Bean` methods in a plain `@Component`) there is no subclass: `auditLog()` is a plain Java call creating a new, *unmanaged* object each time — 3 `AuditLog`s, two unknown to Spring (no lifecycle callbacks, no proxies).
4. **Parameters** avoid the problem in both modes: Spring injects the container's bean.

	Full (`@Configuration`)	**Lite (`proxyBeanMethods = false` or `@Component`)**
Inter-bean method calls	Return the managed singleton	Create new unmanaged objects
Startup cost	Generates a CGLIB subclass	None — slightly faster, friendlier to AOT/native images
Class may be `final` / methods `private`?	No (must be subclassable)	Yes
Recommended style	Works either way if you use parameters	Spring Boot's own auto-configurations all use lite mode

Rule of thumb: declare dependencies as `@Bean` method parameters and never call `@Bean` methods directly — then the mode doesn't matter.

3. `@Import`

`@Import(SomeConfig.class)` pulls in another configuration class (or a plain class as a bean, or an `ImportSelector` / `ImportBeanDefinitionRegistrar` / `BeanRegistrar`) without scanning. Auto-configuration and `@Enable...` annotations are built on it (Chapter 3).

4. Programmatic registration — `BeanRegistrar` (Framework 7)

For beans whose number or type depends on runtime logic. See §2.14.

Older projects also define beans in **XML** (`<bean id="..." class="...">`); it still works, but new code uses annotations.

Common mistakes

- `@Service` on an interface — scanning registers only concrete classes. Annotate the implementation.

- Calling a `@Bean` method from a lite configuration and getting a second instance (above).
- Two beans with the same name (e.g. two `@Bean` methods named `dataSource`). Boot disables bean-definition overriding by default, so startup fails with `BeanDefinitionOverrideException` — rename one.

In the interview

Q: `@Component` vs `@Bean`? `@Component` goes on *your* class and is found by scanning; `@Bean` goes on a factory *method* for third-party classes or when creation needs code. The resulting beans behave the same.

- *Follow-up: What does `proxyBeanMethods = false` change?* No CGLIB subclass; inter-bean calls become plain Java calls creating new objects. Safe with parameters, cheaper at startup.
- *Follow-up: `@Service` vs `@Component`?* No technical difference; `@Service` documents intent. `@Repository` adds exception translation.

2.4 Injection types

What it is

```java
// Constructor injection (preferred)
@Service
public class AlertService {
    private final Notifier notifier;
    public AlertService(Notifier notifier) { this.notifier = notifier; }   // no @Autowired needed
}

// Setter (method) injection and field injection
@Component
public class OtherStyles {
    @Autowired @Qualifier("push")
    Notifier fieldInjected;                  // field injection: needs reflection, can't be final

    Notifier setterInjected;

    @Autowired
    void setNotifier(@Qualifier("email") Notifier n) {   // setter/method injection
        this.setterInjected = n;
    }
}
```

`@Autowired` means "inject here". Since Spring 4.3 a class with **exactly one constructor** doesn't need it; with several constructors, mark the one to use. The standard `jakarta.inject.Inject` works the same if `jakarta.inject-api` is on the classpath (Framework 7 dropped `javax.inject`).

Trade-offs

	Constructor	Setter	Field
Fields can be `final` (immutable)	Yes	No	No
Object complete after `new`	Yes	No — may be half-initialised	No
Unit test without Spring	Just call the constructor	Call setters	Needs reflection or Spring
Required dependency obvious	Yes (in the signature)	No	No
Circular dependency possible	No (fails fast, §2.9)	Yes	Yes
Too many dependencies visible	Yes — a 9-parameter constructor is a visible design smell	Hidden	Hidden
Good for	**Almost everything**	Optional dependencies, re-configuration	Test classes (`@Autowired MockMvcTester mvc`)

Common mistakes

- Field injection in production code — hides dependencies and makes plain unit tests awkward.
- `@Autowired` on a single constructor "just in case" — harmless noise.
- Using a dependency *inside the constructor* of a field-injected class → `NullPointerException`, because fields are injected after construction.

In the interview

Q: Why is constructor injection recommended? Dependencies are `final`, the object is complete when constructed, required dependencies are explicit, unit tests just call `new`, and circular dependencies fail fast. The Spring team recommends it.

- *Follow-up: When setter injection?* Truly optional dependencies or values that can change after startup.

2.5 How Spring picks a bean

What it is

To inject a `Notifier`, Spring looks for **candidate beans assignable to that type** (the type itself, implementations, subclasses):

1. **Exactly one candidate** → inject it.
2. **Several candidates** → narrow down in order:
    1. `@Qualifier("name")` on the injection point selects that bean name (or qualifier).
    2. Otherwise a `@Primary` bean wins.
    3. Otherwise, if all but one are `@Fallback` (Framework 6.2), the remaining one wins.
    4. Otherwise a bean whose **name equals the parameter name** wins (needs the `-parameters` compiler flag, which Boot's parent sets).
    5. Otherwise → `NoUniqueBeanDefinitionException`. (`@jakarta.annotation.Priority` on candidate classes is also consulted — highest wins — but is rare in application code.)
3. **No candidate** → `NoSuchBeanDefinitionException` — unless the point is optional (`Optional<T>`, `ObjectProvider<T>`, `@Nullable`, `@Autowired(required = false)`).
4. **Collection/array/Map of the type** → *all* candidates: `List<Notifier>` ordered by `@Order`/`Ordered` (lowest first, unordered last), `Map<String, Notifier>` keyed by bean name.

Example

```
public final class Notifiers {
    @Component("sms") @Primary @Order(1)
    public static class SmsNotifier implements Notifier { public String send(String m) { return "SMS: " + m; } }

    @Component("email") @Order(2)
    public static class EmailNotifier implements Notifier { public String send(String m) { return "EMAIL: " + m; } }

    @Component("push") @Order(3)
    public static class PushNotifier implements Notifier { public String send(String m) { return "PUSH: " + m; } }
}

@Service
public class AlertService {
    public AlertService(Notifier defaultNotifier,                    // by type -> the @Primary one
                        @Qualifier("email") Notifier emailNotifier,  // by name
                        List<Notifier> all,                          // all, sorted by @Order
                        Map<String, Notifier> byName,                // bean name -> bean
                        Optional<AuditTrail> audit,                  // absent -> Optional.empty()
                        ObjectProvider<AuditTrail> auditProvider) {  // lazy, optional lookup
        ...
    }
}
```

Output:

```
by type (@Primary): SMS: hi
@Qualifier(email):  EMAIL: hi
List (by @Order):   [SMS: hi, EMAIL: hi, PUSH: hi]
Map keys:           [email, push, sms]
optional AuditTrail present? false
```

The qualifier beats `@Primary` because it is more specific. The failure case — two `Storage` beans, no tie-breaker:

```java
@Component public static class DiskStorage implements Storage {}
@Component public static class CloudStorage implements Storage {}
@Component public static class Backup { public Backup(Storage storage) {} }
```

```
startup failed: No qualifying bean of type 'com.example.ioc.ambiguous.Ambiguous$Storage' available:
expected single matching bean but found 2: ambiguous.CloudStorage,ambiguous.DiskStorage
```

The exception is `UnsatisfiedDependencyException` wrapping `NoUniqueBeanDefinitionException` — the context never starts.

The tie-breakers compared

Tool	Where it goes	Meaning	Use when
`@Primary`	On a bean	"Default choice for this type"	One implementation is the normal one
`@Fallback` (6.2+)	On a bean	"Use me only if nothing else qualifies"	A default that others should override
`@Qualifier("x")`	Injection point (optionally the bean)	"I want exactly this one"	A consumer needs a specific implementation
Custom qualifier annotation	e.g. `@Qualifier @interface Fast {}`	Type-safe qualifier	Avoid string names spread across code
`List<T>` / `Map<String,T>`	Injection point	"Give me all of them"	Strategy/plugin patterns, broadcast
`ObjectProvider<T>`	Injection point	Lazy/optional/multiple lookup: `getIfAvailable()`, `getIfUnique()`, `orderedStream()`	Optional beans, prototypes (§2.6)

`List<Notifier>` + `Map` is the Spring way to implement the **Strategy pattern**: each strategy is a bean, the consumer receives all and picks by key — LLD §11.3.

Common mistakes

- Adding a second implementation later (e.g. a test stub) and breaking every injection point that relied on "only one". Decide on `@Primary` or qualifiers early.
- Qualifier string typos — they fail at startup, but a custom qualifier annotation avoids them entirely.
- Expecting `@Order` to choose the single injected bean. It only orders collections.

In the interview

Q: Two beans implement the same interface. How does Spring decide? By type; with several candidates, `@Qualifier`, then `@Primary`, then `@Fallback`, then a bean named like the parameter; otherwise `NoUniqueBeanDefinitionException` at startup. Or inject `List<T>` / `Map<String,T>` to get all.

- *Follow-up: Optional dependency?* `Optional<T>`, `ObjectProvider<T>.getIfAvailable()`, a `@Nullable` parameter (§2.13), or `@Autowired(required = false)` on a setter.

2.6 Bean scopes

What it is

A **scope** decides *how many instances* of a bean exist and *how long* each lives.

Scope	Instances	Lives until	Available in
`singleton` (default)	One per container	Context closes	All apps
`prototype`	New one **per injection/lookup**	Garbage collected — Spring forgets it after handing it out	All apps
`request`	One per HTTP request	Request ends	Web apps
`session`	One per HTTP session (one user's requests, tied by a cookie)	Session expires	Web apps
`application`	One per `ServletContext` (the web app as a whole)	App stops	Web apps
`websocket`	One per WebSocket session	Socket closes	WebSocket apps

Declare with `@Scope("prototype")` (or `ConfigurableBeanFactory.SCOPE_PROTOTYPE`), `@RequestScope`, `@SessionScope`, `@ApplicationScope`.

> **Spring singleton ≠ GoF Singleton.** The "Gang of Four" Singleton pattern enforces one instance *per class loader* via a private constructor. A Spring singleton is one instance *per container and bean definition* — you could define two beans of the same class. See LLD §8.8.

Singleton vs prototype — and the classic trap

```
@Component @Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class ShoppingCart { public final int id = SEQ.incrementAndGet(); @PreDestroy void close() {...} }

@Component
public class PriceList { public final int id = SEQ.incrementAndGet(); @PreDestroy void close() {...} }

@Service                                          // singleton that depends on a prototype
public class CheckoutService {
    private final ShoppingCart injectedOnce;               // injected ONE time, at creation
    private final ObjectProvider<ShoppingCart> carts;       // asks the container each time
    public int cartIdFromField() { return injectedOnce.id; }
    public int cartIdFromProvider() { return carts.getObject().id; }
}
```

Output:

```
singleton: PriceList #1, #1
prototype: ShoppingCart #2, #3
prototype injected into singleton: #1, #1 (stuck!)
via ObjectProvider:                #4, #5 (fresh)
--- destroy callbacks on close ---
 1. PriceList#1 @PreDestroy
```

1. `getBean(PriceList.class)` always returns #1; `getBean(ShoppingCart.class)` returns a new one each time.
2. **A prototype injected into a singleton is created once** — when the singleton is built — and reused forever.
3. Fix: *look up* a new instance when needed — `ObjectProvider<T>.getObject()` (shown), `@Lookup` method injection, or a scoped proxy.
4. **Spring does not call destroy callbacks on prototypes** — only `PriceList`'s `@PreDestroy` ran. The code that asked for a resource-holding prototype must clean up.

Request and session scopes — proved

A singleton controller can hold "the current request's object" only through a **scoped proxy**: a stand-in that, on every call, finds the real instance for the *current* request or session.

```
@Component @RequestScope          // one instance per HTTP request; a scoped proxy is injected into singletons
public class RequestInfo { private final int id = SEQ.incrementAndGet(); public int id() { return id; } }

@Component @SessionScope          // one instance per HTTP session
public class Cart { private final List<String> items = new ArrayList<>(); ... }

@RestController                   // a singleton...
public class CartController {
    private final Cart cart;              // ...holding proxies that find the right
    private final RequestInfo request;    //    session/request instance on every call

    @PostMapping("/cart/{item}")
    String add(@PathVariable String item) {
        cart.add(item);
        return "request#" + request.id() + " cart=" + cart.items() + " proxy=" +
cart.getClass().getSimpleName().contains("CGLIB");
    }
}
```

Two posts with Alice's session and one with Bob's (`MockHttpSession`):

```
alice: request#1 cart=[book] proxy=true
alice: request#2 cart=[book, pen] proxy=true
bob:   request#3 cart=[mug] proxy=true
```

Every request got a new `RequestInfo`; Alice's requests shared one `Cart`, Bob got his own — through the same singleton controller. `@RequestScope` / `@SessionScope` create CGLIB proxies by default (`proxyMode = TARGET_CLASS`); with plain `@Scope("request")` you must set `proxyMode` yourself.

Thread safety

A singleton is shared by **all request threads at once**. Keep singletons **stateless** (only `final` references to other beans and configuration); per-request data goes in parameters/locals or request-scoped beans. Concurrency basics: Java Chapter 14.

Common mistakes

- A mutable field (`private int counter`, a `HashMap` cache) in a singleton → **race conditions** under load (the result depends on thread timing: two threads read `counter = 5`, both write `6`, one increment is lost; it shows only under concurrency, so tests often miss it).
- Expecting a new prototype per method call when it was injected once.
- Injecting a request-scoped bean into a singleton without a proxy → `Scope 'request' is not active for the current thread` at startup.
- Session scope in a horizontally scaled app (several copies behind a load balancer) without session replication (Spring Session, Chapter 10; System Design Fundamentals).

In the interview

Q: What are Spring's bean scopes? singleton (default, one per container), prototype (new per lookup, not destroyed by Spring), and web scopes request, session, application, websocket.

- *Follow-up: Prototype inside singleton?* Injected once, so effectively a singleton for that consumer. Use `ObjectProvider`, `@Lookup`, or a scoped proxy.
- *Follow-up: Are singletons thread-safe?* Spring doesn't make them so; they're shared, so design them stateless.

2.7 The bean lifecycle

What it is

From creation to destruction Spring calls a fixed sequence of hooks. You rarely need more than `@PostConstruct` / `@PreDestroy`, but the order explains many "why is my field null?" questions. This bean implements *every* hook:

```
public class CacheWarmer implements BeanNameAware, InitializingBean, DisposableBean {
    public CacheWarmer()                          { Trace.add("constructor"); }
    @Autowired void setDependency(Dependency d)   { Trace.add("setter injection"); }
    @Override public void setBeanName(String n)   { Trace.add("BeanNameAware.setBeanName(" + n + ")"); }
    @PostConstruct void postConstruct()           { Trace.add("@PostConstruct"); }
    @Override public void afterPropertiesSet()    { Trace.add("InitializingBean.afterPropertiesSet"); }
    void customInit()                             { Trace.add("@Bean(initMethod) customInit"); }

    @PreDestroy void preDestroy()                 { Trace.add("@PreDestroy"); }
    @Override public void destroy()               { Trace.add("DisposableBean.destroy"); }
    void customDestroy()                          { Trace.add("@Bean(destroyMethod) customDestroy"); }
}

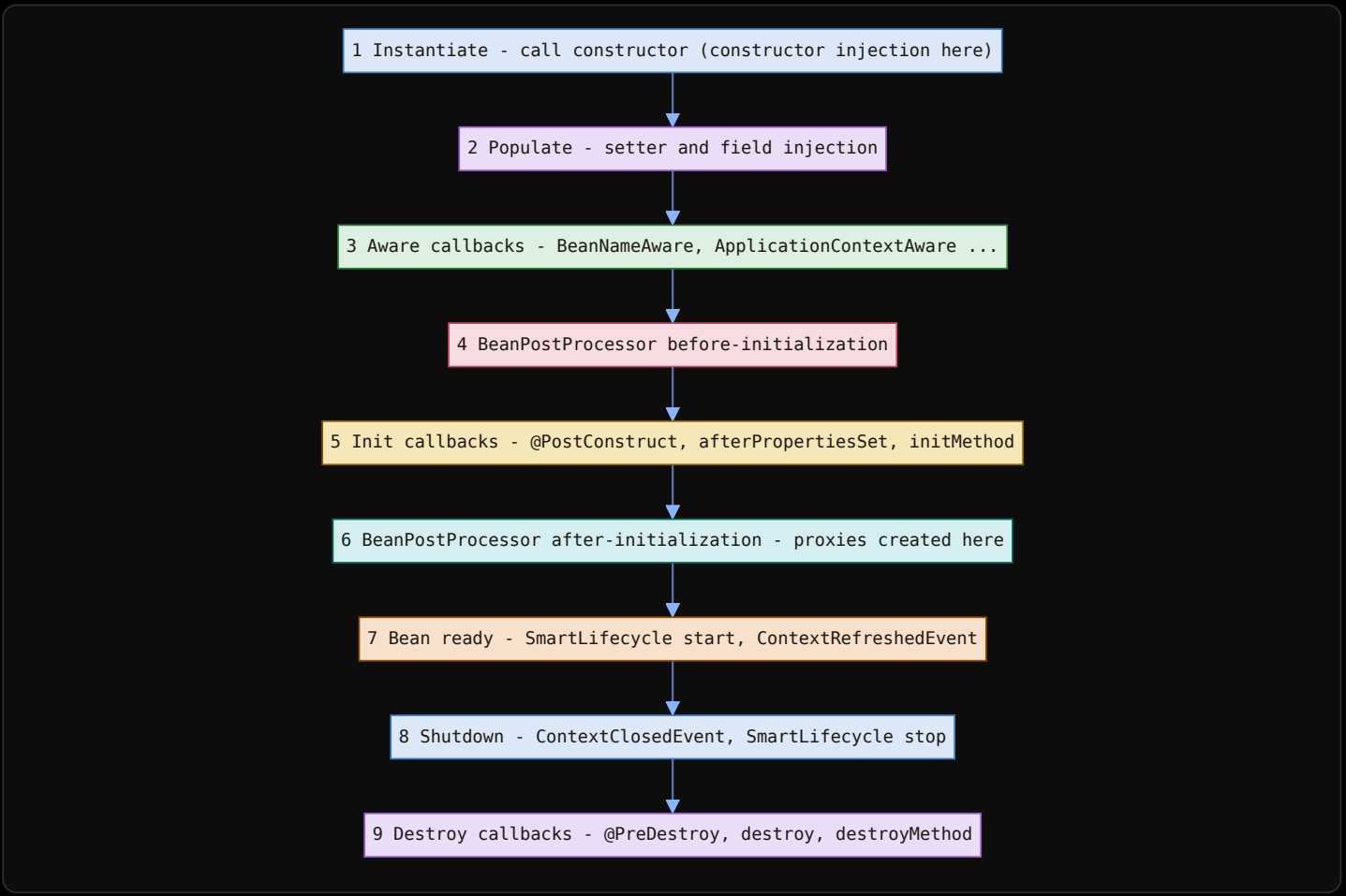
@Configuration
public class LifecycleConfig {
    @Bean static TracingPostProcessor tracingPostProcessor() { return new TracingPostProcessor(); } // static!
    @Bean Dependency dependency() { return new Dependency(); }
    @Bean(initMethod = "customInit", destroyMethod = "customDestroy")
    CacheWarmer cacheWarmer() { return new CacheWarmer(); }
    @Bean Poller poller() { return new Poller(); }                // a SmartLifecycle bean

    @EventListener void onRefresh(ContextRefreshedEvent e) { Trace.add("ContextRefreshedEvent"); }
    @EventListener void onClose(ContextClosedEvent e)      { Trace.add("ContextClosedEvent"); }
}
```

Output (context created, then closed):

```
--- bean lifecycle ---
 1. constructor
 2. setter injection
 3. BeanNameAware.setBeanName(cacheWarmer)
 4. BeanPostProcessor.before(cacheWarmer)
 5. @PostConstruct
 6. InitializingBean.afterPropertiesSet
 7. @Bean(initMethod) customInit
 8. BeanPostProcessor.after(cacheWarmer)
 9. SmartLifecycle.start (Poller)
10. ContextRefreshedEvent
11. ----- application running -----
12. ContextClosedEvent
13. SmartLifecycle.stop (Poller)
14. @PreDestroy
15. DisposableBean.destroy
16. @Bean(destroyMethod) customDestroy
```

How it works



Notes on the steps:

- **3** `Aware` — the bean can receive its name, the `BeanFactory`, `ApplicationContext`, `Environment`. Prefer normal injection; `Aware` ties code to Spring.
- **4** — `BeanPostProcessor.postProcessBeforeInitialization` runs for every BPP. `@PostConstruct` itself is implemented by a BPP (`CommonAnnotationBeanPostProcessor`), which is why our custom "before" ran just before it.
- **5** — init order: `@PostConstruct` → `InitializingBean.afterPropertiesSet()` → `@Bean(initMethod)`.
- **6** — `postProcessAfterInitialization`: the bean may be *replaced* here, typically by a proxy (transactions, AOP, §2.8).
- **7** — after **all** singletons are ready: `SmartLifecycle.start()` (by phase), then `ContextRefreshedEvent` (in Boot then `ApplicationStartedEvent` and `ApplicationReadyEvent`).
- **8** — on `close()` or the JVM shutdown hook (which Boot registers): `ContextClosedEvent`, `SmartLifecycle.stop()` in reverse phase order.
- **9** — `@PreDestroy` → `DisposableBean.destroy()` → `@Bean(destroyMethod)`; beans are destroyed in reverse dependency order.

Which hook to use

Hook	Use for	Notes
`@PostConstruct` / `@PreDestroy` (`jakarta.annotation`)	Validate state, warm a cache, open/close a resource	**Default choice** — standard, no Spring interface
`InitializingBean` / `DisposableBean`	Framework or library code	Couples your class to Spring
`@Bean(initMethod, destroyMethod)`	Third-party classes you can't annotate	`destroyMethod` is inferred: a public `close()` / `shutdown()` is called automatically
`SmartLifecycle`	Things that *run* while the app is up: message listeners, pollers, schedulers	`start` / `stop`, `getPhase()` orders them (lower starts first, stops last), `isAutoStartup()`
`ApplicationReadyEvent` listener (Boot)	"Do X once the app is fully started"	Runs after runners; the server already accepts traffic
`CommandLineRunner` / `ApplicationRunner` (Boot)	One-off startup tasks with access to args	Run before `ApplicationReadyEvent`

Common mistakes

- Heavy work (remote calls, long cache loads) in constructors or `@PostConstruct` — slows startup and a failure kills the app. Consider lazy loading or `ApplicationReadyEvent`.
- Calling `@Transactional` methods from `@PostConstruct` expecting a transaction: the proxy is applied in step 6, *after* `@PostConstruct` (step 5), and you are calling `this` anyway (self-invocation, Chapter 4).
- Expecting `@PreDestroy` on prototypes (§2.6).
- `kill -9` (a forced kill) runs no shutdown hook, so no destroy callbacks.

In the interview

Q: Describe the Spring bean lifecycle. Instantiate → inject → `Aware` → BPP before-init → `@PostConstruct` → `afterPropertiesSet` → custom init → BPP after-init (proxies) → ready → `SmartLifecycle.start` → `ContextRefreshedEvent`; on shutdown `@PreDestroy` → `destroy()` → custom destroy.

- *Follow-up: Where does a `@Transactional` proxy come from?* A `BeanPostProcessor`'s after-initialization step wraps the bean.
- *Follow-up: `@PostConstruct` vs constructor?* In the constructor, setter/field injection and post-processing haven't happened yet; `@PostConstruct` runs after all injection.

2.8 Post-processors: the container's extension points

What it is

	`BeanFactoryPostProcessor` (BFPP)	`BeanPostProcessor` (BPP)
Works on	Bean **definitions** (recipes)	Bean **instances**
Runs	Once, after definitions load, **before any normal bean is created**	For every bean, before and after its init callbacks
Can	Change scopes, class names, property values; register more definitions	Modify the bean, or **return a different object** (proxy/wrapper)
Spring's own examples	`PropertySourcesPlaceholderConfigurer` (resolves `${...}`), `ConfigurationClassPostProcessor` (processes `@Configuration`, `@ComponentScan`, `@Import`)	`AutowiredAnnotationBeanPostProcessor` (`@Autowired`, `@Value`), `CommonAnnotationBeanPostProcessor` (`@PostConstruct`, `@PreDestroy`), the AOP auto-proxy creator (`@Transactional`, `@Cacheable`, `@Async`)

Most of Spring's annotation "magic" is ordinary post-processors.

Example — a BPP that swaps beans for logging proxies

```
@Target(ElementType.TYPE) @Retention(RetentionPolicy.RUNTIME)
public @interface LogCalls {}

@Component @LogCalls
public class FriendlyGreeter implements Greeter { public String greet(String name) { return "Hi " + name; } }

@Component
public class LogCallsPostProcessor implements BeanPostProcessor {
    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName) {
        if (!bean.getClass().isAnnotationPresent(LogCalls.class)) return bean;
        return Proxy.newProxyInstance(                          // JDK dynamic proxy over the bean's interfaces
                bean.getClass().getClassLoader(), bean.getClass().getInterfaces(),
                (proxy, method, args) -> {
                    Trace.add("-> " + beanName + "." + method.getName() + "(" + args[0] + ")");
                    Object result = method.invoke(bean, args);  // call the real object
                    Trace.add("<- " + result);
                    return result;
                });
    }
}

@Component
public class CountingFactoryPostProcessor implements BeanFactoryPostProcessor {
    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory bf) {
        Trace.add("BeanFactoryPostProcessor: friendlyGreeter defined=" + bf.containsBeanDefinition("friendlyGreeter")
                + ", created=" + bf.containsSingleton("friendlyGreeter"));
    }
}
```

Output (test gets the `Greeter` bean and calls `greet("Ada")`):

```
--- post-processors ---
 1. BeanFactoryPostProcessor: friendlyGreeter defined=true, created=false
 2. bean class = jdk.proxy2.$Proxy45
 3. -> friendlyGreeter.greet(Ada)
 4. <- Hi Ada
```

1. The BFPP ran when the `friendlyGreeter` **definition** existed but no **instance** did.
2. The BPP returned a proxy, so the container stored and injected the *proxy* (`jdk.proxy2.$Proxy45`). Every caller goes through the logging code — exactly how Spring AOP adds transactions and caching (Chapter 4). Proxy pattern: LLD §9.7.
3. A **JDK dynamic proxy** (Java's built-in runtime object implementing given interfaces, routing every call to one handler) implements only the *interfaces*, so `ctx.getBean(FriendlyGreeter.class)` would now fail — one reason to inject interfaces, and one reason Spring often uses class-based CGLIB proxies.

Rules

- Declare BFPP and BPP `@Bean` methods **`static`** (as in `LifecycleConfig`). They are created very early; a non-static method forces the whole configuration class to be created early too, so its own `@Autowired` / `@Value` may not be processed. Spring logs a message when this happens.
- A BPP should not depend on ordinary beans: anything it injects is created before all BPPs are registered and is "not eligible for getting processed by all BeanPostProcessors" (e.g. no transactional proxy) — Spring logs exactly that phrase.
- Order several post-processors with `Ordered` / `PriorityOrdered` or `@Order`.

2.9 Circular dependencies

What it is

A **circular dependency** is a cycle in the dependency graph: `OrderService` needs `CustomerService`, which needs `OrderService` (or a longer loop A → B → C → A).

What happens — proved

```
// Constructor cycle
@Component class OrderService    { OrderService(CustomerService customers) {} }
@Component class CustomerService { CustomerService(OrderService orders) {} }

// Field cycle
@Component class OrderService    { @Autowired CustomerService customers; }
@Component class CustomerService { @Autowired OrderService orders; }

// Cycle broken with @Lazy
@Component class OrderService    { OrderService(@Lazy CustomerService customers) {...} }
@Component class CustomerService { CustomerService(OrderService orders) {...} }
```

Output:

```
constructor cycle -> BeanCurrentlyInCreationException: Error creating bean with name 'ctorCycle.CustomerService':
Requested bean is currently in creat...
field cycle, plain Spring -> OK, orders.customers.orders == orders? true
field cycle, Spring Boot -> BeanCurrentlyInCreationException
@Lazy cycle -> OK, injected class = LazyCycle$CustomerService$$SpringCGLIB$$0, call = customers
```

How it works

1. **Constructor cycles cannot be satisfied.** `new OrderService(customers)` needs a finished `CustomerService`, whose constructor needs a finished `OrderService`... Spring detects the loop and throws `BeanCurrentlyInCreationException`.
2. **Field/setter cycles work in plain Spring**: Spring constructs `OrderService` (empty), registers an *early reference* in an internal cache, creates `CustomerService` injecting that reference, then finishes `OrderService`. (The internal "singleton caches" are often called the three-level cache in interviews.)
3. **Spring Boot forbids this by default** (`spring.main.allow-circular-references=false`, since Boot 2.6): a cycle usually means entangled classes, and an early reference is the raw object, not the transactional proxy. Boot's startup report prints the cycle as an ASCII diagram.
4. **`@Lazy` on an injection point** injects a proxy; the real `CustomerService` is looked up on first call, breaking the creation-time cycle — note the `$$SpringCGLIB$$` class name.

How to fix a cycle (best first)

1. **Redesign.** Usually one class does two jobs. Extract shared logic into a third bean, or move a method to where its data lives.
2. **Use events** (§2.12): `OrderService` publishes `OrderPlaced`; `CustomerService` listens. No compile-time dependency.
3. **Inject lazily**: `@Lazy` on the parameter, or `ObjectProvider<CustomerService>` with `getObject()` when needed.
4. **Last resort**: `spring.main.allow-circular-references=true` — hides the design problem.

In the interview

Q: How does Spring handle circular dependencies? Constructor cycles fail with `BeanCurrentlyInCreationException`. Field/setter cycles can be resolved via early singleton references, but Boot disallows them by default since 2.6. Fix by redesigning (extract a third class), decoupling with events, or `@Lazy` / `ObjectProvider`.

- *Follow-up: Why is constructor injection "good" here?* Cycles fail fast at startup instead of silently producing half-initialised or unproxied objects.

2.10 Lazy initialization

What it is

By default the `ApplicationContext` creates every singleton at startup. A **lazy** bean is created on first use.

```java
@Component @Lazy                              // not created at startup; created on first use
public class ReportGenerator { public ReportGenerator() { Trace.add("ReportGenerator created (lazy)"); } }

@Component                                    // eager by default
public class RateCache { public RateCache() { Trace.add("RateCache created (eager)"); } }
```

```
--- @Lazy ---
 1. RateCache created (eager)
 2. -- context started
 3. ReportGenerator created (lazy)
--- spring.main.lazy-initialization=true ---
 1. -- context started
 2. RateCache created (eager)
```

The second run used Boot with `spring.main.lazy-initialization=true`, which makes **every** bean lazy — even `RateCache`. `@Lazy(false)` opts a bean back out.

Trade-offs

Eager (default)	Lazy
Slower startup	Faster startup
Misconfiguration fails **at startup**	Misconfiguration fails **on first request** — maybe in production
First request is fast	First request pays the creation cost
Memory used up front	Memory only for used beans

When to use it: individual rarely used, expensive beans; global lazy init only in development, tests or CLI tools where startup dominates. Keep production eager so broken configuration can't get past startup. (For fast production startup, Chapter 15 covers AOT and CDS — Class Data Sharing, a JVM cache of pre-loaded classes.)

2.11 `@Conditional` and profiles

What it is

- `@Conditional(SomeCondition.class)` — register a bean *only if* a `Condition` returns true. A `Condition` sees the environment (properties, profiles), the registry (definitions so far) and the classpath. This is the building block of Boot's auto-configuration (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty` — Chapter 3).
- `@Profile("dev")` — a ready-made condition: register only when that **profile** (a named set of beans/settings for one environment: `dev`, `test`, `prod`) is active. Expressions allowed: `"!dev"`, `"prod & eu"`, `"dev | test"`.

```java
public class OnFeatureFlag implements Condition {
    @Override
    public boolean matches(ConditionContext context, AnnotatedTypeMetadata metadata) {
        return context.getEnvironment().getProperty("features.fraud-check", Boolean.class, false);
    }
}

@Configuration
public class PaymentBeans {
    @Bean @Profile("dev")                     // only when profile "dev" is active
    PaymentGateway fakeGateway() { return () -> "fake gateway (dev)"; }

    @Bean @Profile("!dev")                    // any time "dev" is NOT active
    PaymentGateway httpGateway() { return () -> "HTTP gateway (real)"; }

    @Bean @Conditional(OnFeatureFlag.class)   // custom condition
    FraudChecker fraudChecker() { return new FraudChecker(); }
}
```

Two contexts — one with profile `dev` and `features.fraud-check=true`, one with neither:

```
profile dev : fake gateway (dev), fraudChecker beans = 1
no profile  : HTTP gateway (real), fraudChecker beans = 0
```

How to activate profiles

`spring.profiles.active=dev` in a property file, `--spring.profiles.active=dev`, env var `SPRING_PROFILES_ACTIVE=dev`, `@ActiveProfiles("test")` in tests, or `context.getEnvironment().setActiveProfiles("dev")` in plain Spring. With none active, the `default` profile is active. Profile-specific *property files* (`application-dev.yaml`) and profile groups: Chapter 3.

Common mistakes

- Profiles for things that are just values (URLs, timeouts). Keep one bean and vary the *property*; reserve profiles for genuinely different beans (a fake payment gateway in dev).
- `@Profile("prod")` on a bean tests need → "no bean". Provide a `"!prod"` alternative.
- A custom `Condition` in a Boot app when `@ConditionalOnProperty` already does it.

2.12 Application events

What it is

The container includes an in-process **publish/subscribe** mechanism: any bean publishes an event object through `ApplicationEventPublisher`; any `@EventListener` method whose parameter type matches receives it. Publishers don't know who listens — the **Observer pattern** (LLD §11.1). Since Spring 4.2 an event can be any object (no need to extend `ApplicationEvent`); records are ideal.

```
public record OrderPlaced(String orderId, long cents) {}
public record StockReserved(String orderId) {}

@Service
public class OrderService {
    private final ApplicationEventPublisher events;
    public OrderService(ApplicationEventPublisher events) { this.events = events; }

    public void place(String id, long cents) {
        Trace.add("OrderService: saving " + id + " on " + Thread.currentThread().getName());
        events.publishEvent(new OrderPlaced(id, cents));   // listeners run NOW, same thread
        Trace.add("OrderService: publishEvent returned");
    }
}

@Component
public class OrderListeners {
    @EventListener @Order(1)
    void sendEmail(OrderPlaced e) { Trace.add("  email listener for " + e.orderId() + " on " +
Thread.currentThread().getName()); }

    @EventListener @Order(2)
    StockReserved reserveStock(OrderPlaced e) {          // a non-void return value is published as a new event
        Trace.add("  stock listener for " + e.orderId());
        return new StockReserved(e.orderId());
    }

    @EventListener(condition = "#event.cents > 10000")    // SpEL filter: only big orders
    void alertBigOrder(OrderPlaced event) { Trace.add("  BIG order alert for " + event.orderId()); }

    @EventListener
    void onStock(StockReserved e) { Trace.add("  shipping listener sees StockReserved(" + e.orderId() + ")"); }
}
```

Output for `place("A-1", 2_500)` then `place("B-2", 50_000)`:

```
--- events ---
 1. OrderService: saving A-1 on main
 2.   email listener for A-1 on main
 3.   stock listener for A-1
 4.   shipping listener sees StockReserved(A-1)
 5. OrderService: publishEvent returned
 6. OrderService: saving B-2 on main
 7.   email listener for B-2 on main
 8.   stock listener for B-2
 9.   shipping listener sees StockReserved(B-2)
10.   BIG order alert for B-2
11. OrderService: publishEvent returned
```

How it works

1. `publishEvent` finds all listeners whose parameter type matches (generics respected).
2. It calls them **synchronously, on the publisher's thread** (`main` above), in `@Order` sequence; unordered listeners run after ordered ones.
3. A non-`void`/non-`null` return value is published as a new event — `StockReserved` was handled *before* the remaining `OrderPlaced` listener ran.
4. `condition` is a SpEL expression over `#event` (or the parameter name); `false` skips the listener.
5. `publishEvent` returns only after all listeners finish. **An exception in a listener propagates to the publisher.**

Variations and built-in events

Need	Use	Chapter
Listener on another thread	`@Async` + `@EventListener` (needs `@EnableAsync`)	14
Run only after the publisher's DB transaction commits	`@TransactionalEventListener(phase = AFTER_COMMIT)`	7
React to container stages	`ContextRefreshedEvent`, `ContextClosedEvent` (Framework); `ApplicationStartingEvent`, `ApplicationEnvironmentPreparedEvent`, `ApplicationPreparedEvent`, `ApplicationStartedEvent`, `ApplicationReadyEvent`, `ApplicationFailedEvent`, `WebServerInitializedEvent` (Boot)	2.7, 15
Events between services	A message broker (e.g. Kafka) — application events are in-memory, single JVM	14

Trade-offs

Direct method call	Application event
Explicit, easy to follow in the IDE	Decoupled — publisher doesn't know listeners
Compile-time dependency	No dependency; breaks cycles (§2.9)
Obvious error handling	Synchronous by default: slow/failing listeners slow/fail the publisher
—	Not durable: lost if the JVM crashes

Common mistakes

- Assuming events are asynchronous. Not without `@Async`.
- Using in-memory events for "must not be lost" integration; use the transactional outbox (save the event in the same DB transaction as the data, publish from there) + a broker (Chapter 14, microservices notes).
- Sending emails in a plain listener inside a transaction that later rolls back — use `@TransactionalEventListener`.

2.13 Null-safety with JSpecify (Framework 7)

What it is

`NullPointerException` is Java's most common runtime error, and Java can't express "this may be null". **JSpecify** (`org.jspecify.annotations`) is a standard annotation set agreed by several vendors (Google, JetBrains, Uber, Spring, ...):

Annotation	Meaning
`@NullMarked` (package, class or module)	"In here, every type is non-null unless marked otherwise"
`@Nullable`	This field/parameter/return value may be `null`
`@NonNull`	Explicitly non-null (rarely needed inside `@NullMarked`)
`@NullUnmarked`	Opt a part back out

Spring Framework 7 and Boot 4 annotate their whole codebase with JSpecify, replacing `org.springframework.lang.Nullable` / `NonNull` (deprecated since 7.0). IDEs and build-time checkers such as **NullAway** (an Error Prone plugin — Google's extra compiler checks) warn when you might dereference `null`; Kotlin sees Spring APIs with correct nullable/non-null types. `jspecify` comes transitively with `spring-core`.

Example

```
// package-info.java
@NullMarked
package com.example.ioc.nullsafety;
import org.jspecify.annotations.NullMarked;

// ReportService.java
@Service
public class ReportService {
    private final @Nullable Translator translator;   // may be absent

    // Spring treats a parameter annotated with any @Nullable as optional: it injects null if no bean.
    public ReportService(@Nullable Translator translator) { this.translator = translator; }

    public String title(String raw) {
        return translator == null ? raw : translator.translate(raw);   // the null check the annotation asks for
    }

    public @Nullable String findTitle(int id) { return id == 1 ? "Q3 sales" : null; }  // callers see null is possible
}
```

With no `Translator` bean in the context:

```
no Translator bean -> title = Q3 sales, findTitle(2) = null
```

How it works

1. `@NullMarked` makes every unannotated type in the package non-null by *contract*; `@Nullable` marks exceptions. Tools enforce it — at runtime the annotations do nothing.
2. **Spring's container reads them for injection**: an injection point annotated with any annotation named `Nullable` is optional, so a missing bean gives `null` instead of a startup failure (like `Optional<T>`).
3. Add NullAway to your build to turn the contract into compile errors (optional; Spring itself does this).

Common mistakes

- Thinking `@Nullable` adds runtime checks — it only documents and enables tooling.
- Mixing old `org.springframework.lang.Nullable` with JSpecify in new code.
- Misplacing `@Nullable`: JSpecify annotations are *type-use* annotations (attached to a type wherever it is written), so arrays and nested types are written `String @Nullable []` or `Outer.@Nullable Inner`.

2.14 Programmatic registration with `BeanRegistrar` (Framework 7)

What it is

`@Bean` methods register a *fixed* set of beans. When the set depends on runtime data — "one client bean per configured channel" — Framework 7's `BeanRegistrar` registers beans with ordinary code (loops, `if`s), in a form that also works with AOT/native images.

```
public class ChannelRegistrar implements BeanRegistrar {
    public record Channel(String name) {}
    public record Router(java.util.List<Channel> channels) {}

    @Override
    public void register(BeanRegistry registry, Environment env) {
        String[] names = env.getProperty("channels", String[].class, new String[] {"email"});
        for (String n : names) {                                    // one bean per configured channel
            registry.registerBean(n + "Channel", Channel.class, spec -> spec.supplier(ctx -> new Channel(n)));
        }
        registry.registerBean("router", Router.class, spec -> spec
                .lazyInit()
                .supplier(ctx -> new Router(ctx.beanProvider(Channel.class).orderedStream().toList())));
    }
}

@Configuration
@Import(ChannelRegistrar.class)   // a BeanRegistrar is activated with @Import
public class RegistrarConfig {}
```

With `channels=email,sms,push`:

```
registered: emailChannel, smsChannel, pushChannel, router
router -> Router[channels=[Channel[name=email], Channel[name=sms], Channel[name=push]]]
```

The `spec` offers the same options as annotations: `primary()`, `fallback()`, `lazyInit()`, `prototype()`, `scope(...)`, `order(...)`, `description(...)`, `supplier(...)` (whose context fetches other beans with `bean(...)`/`beanProvider(...)`). Use it for libraries and dynamic setups; in application code `@Component` and `@Bean` remain simpler.

2.15 In the interview — quick round

Question	Short answer
What is IoC? DI?	IoC: the framework controls object creation and calls your code. DI: objects receive dependencies instead of building them; Spring's container does it.
BeanFactory vs ApplicationContext?	Minimal lazy container vs full container: eager singletons, post-processors, events, environment, i18n. (§2.2)
`@Component` vs `@Bean`?	Class annotation found by scanning vs factory method for third-party classes/complex creation. (§2.3)
Full vs lite configuration?	Full = CGLIB subclass makes inter-bean calls return singletons; lite = plain calls. Pass dependencies as parameters. (§2.3)
Why constructor injection?	Immutable, complete objects, explicit, testable, fails fast on cycles. (§2.4)
Two beans of one type?	`@Qualifier`, `@Primary`, `@Fallback`, parameter name, or inject `List`/`Map`. (§2.5)
Default scope? Thread-safe?	Singleton; shared across threads, so keep it stateless. (§2.6)
Prototype in singleton?	Injected once; use `ObjectProvider`/`@Lookup`/scoped proxy. (§2.6)
Lifecycle order?	Construct → inject → aware → BPP before → `@PostConstruct` → `afterPropertiesSet` → init-method → BPP after → ... → `@PreDestroy` → `destroy` → destroy-method. (§2.7)
BPP vs BFPP?	Instances vs definitions. `@Autowired`/`@Transactional` are BPPs; `${}` resolution is a BFPP. (§2.8)
Circular dependency?	Constructor cycles fail; Boot rejects field cycles by default; refactor, events, or `@Lazy`. (§2.9)
`@Profile` vs `@Conditional`?	`@Profile` is a specific condition on active profiles; `@Conditional` takes any `Condition`. (§2.11)
Are events async?	No — synchronous on the publisher's thread unless `@Async`. (§2.12)
What's new in the Framework 7 container?	JSpecify null-safety, `BeanRegistrar`, `javax.inject`/`javax.annotation` support removed. (§2.13–2.14)

Chapter TL;DR

(The §2.15 table is the question-by-question recap.)

- **DI**: classes declare dependencies as constructor parameters (`final` fields); the **`ApplicationContext`** creates beans eagerly, in dependency order, at startup.
- **Define beans** by scanning stereotypes, `@Bean` methods (third-party classes; full `@Configuration` = CGLIB subclass, lite = plain calls — pass dependencies as parameters), `@Import`, or Framework 7's `BeanRegistrar`.
- **Picking a bean**: type → `@Qualifier` → `@Primary` → `@Fallback` → parameter name; `List`/`Map` for all; `Optional`/`ObjectProvider`/`@Nullable` for optional.
- **Scopes**: singleton (shared, keep stateless), prototype (new per lookup, no destroy callbacks), request/session via scoped proxies.
- **Lifecycle & post-processors**: BPP-after-init is where proxies appear; declare post-processors `static`. Constructor cycles fail, Boot rejects field cycles.
- **Lazy init**, **`@Conditional`/`@Profile`**, synchronous **events**, and Framework 7's **JSpecify** null-safety round out the container.

Next: Chapter 3 — Configuration and Auto-configuration shows where property values come from and how Boot uses conditions to configure beans for you.

Chapter 3 — Configuration and Auto-configuration

A real application runs in many places — your laptop, CI (the continuous-integration server), staging (a production-like test environment), production — with different database URLs, passwords, timeouts and feature flags (on/off switches). Spring Boot's answer is **externalized configuration**: the same jar reads its settings from outside, from sources with a well-defined priority. Boot then uses those settings, plus what is on the classpath, to **auto-configure** beans. This chapter covers both halves and ends by writing your own auto-configuration and starter.

Runnable code: `code/config-demo/` (a non-web Boot app started in tests with different arguments, fake environment variables and profiles) and `code/greeter/` (a three-module build: auto-configuration module, starter, and an app using it). Built with `mvn -q -B -ntp verify` on Spring Boot 4.1.1 / JDK 25; all outputs are real.

Background: the container, `@Conditional` and profiles in Chapter 2 (§2.11). The architecture idea — keep config out of the artifact — is the *externalized configuration* pattern in the microservices notes.

3.1 Externalized configuration and the `Environment`

What it is

Externalized configuration means settings live *outside* the compiled code, so one artifact runs everywhere and only its configuration changes (factor III, "Config", of the Twelve-Factor App methodology for cloud-friendly services).

In Spring all configuration is reached through the `Environment`:

- a **property source** (`PropertySource`) is one named set of key/value pairs — a file, the OS environment, the command line, a map in a test;
- the `Environment` holds an **ordered list** of property sources and the active **profiles**;
- `environment.getProperty("shop.name")` asks each source *in order* and returns the **first** value found — a higher source hides the same key in lower ones.

Why it matters

- **No rebuilds per environment** — the same jar goes from test to production.
- **Secrets stay out of Git** — passwords arrive through environment variables or mounted files (files a platform places into the app's file system at startup).
- **Operations can change behaviour** (timeouts, connection-pool sizes, feature flags) without a code change.

How it works in Boot

1. `SpringApplication.run` builds the `Environment` *before* creating any bean.
2. It adds sources: command-line arguments, `SPRING_APPLICATION_JSON`, Java system properties (`-Dkey=value`), OS environment variables, `random.*`, then the **config data** files (`application.properties`/`.yaml`, profile-specific variants, imports).
3. Beans read values through `@Value`, `@ConfigurationProperties` or `Environment` injection.
4. Auto-configuration conditions (`@ConditionalOnProperty`) read the same `Environment`.

3.2 `application.properties` vs `application.yaml`

Syntax

Both hold the same data; pick one per project. The demo uses YAML (indentation shows nesting):

```yaml
# config-demo/src/main/resources/application.yaml (first part)
spring:
  application:
    name: config-demo
shop:
  name: Corner Shop
  checkout-timeout: 45s              # kebab-case in files; binds to checkoutTimeout
  max-upload: 10MB
  currencies: [USD, EUR]
  discounts:
    gold: 15
    silver: 5
  payment:
    retries: 2
```

The same in `.properties`:

```properties
spring.application.name=config-demo
shop.name=Corner Shop
shop.checkout-timeout=45s
shop.max-upload=10MB
shop.currencies=USD,EUR          # or shop.currencies[0]=USD / shop.currencies[1]=EUR
shop.discounts.gold=15
shop.discounts.silver=5
shop.payment.retries=2
```

	`.properties`	**`.yaml` / `.yml`**
Structure	Flat `a.b.c=value` lines	Indentation-based tree
Lists	`x=a,b` or `x[0]=a`	`[a, b]` or `- a` lines
Multiple documents in one file	`#---` separator line	`---` separator line
Pitfalls	Long repeated prefixes	Indentation errors; unquoted `on`, `no`, `010` may be read as booleans/octal — quote them
Loadable by `@PropertySource`	Yes	**No**
Both in the same folder	`.properties` wins over `.yaml`	

Use **kebab-case** keys in files (`checkout-timeout`), the recommended canonical form.

Where Boot looks

Boot loads `application.properties`/`.yaml` from these locations; **later ones override earlier ones**:

1. The classpath root (`src/main/resources/application.yaml` ends up here)
2. The classpath `/config` package
3. The current directory
4. The `config/` subdirectory of the current directory
5. Immediate child directories of `config/` (e.g. `config/db/application.yaml`)

So an operator can drop a `config/application.yaml` next to the jar to override the packaged one. Change the file name with `spring.config.name`, replace the search locations with `spring.config.location`, or add locations with `spring.config.additional-location` (these three must be set early: command-line args, system properties or env vars).

Multi-document files

A file can contain several documents, each **activated** conditionally:

```yaml
# ... first document above ...
---
# a second document inside the same file, used only when "prod" is active
spring:
  config:
    activate:
      on-profile: prod
shop:
  payment:
    gateway-url: https://pay.prod.example.test
    retries: 5
```

`spring.config.activate.on-profile` takes a profile expression (`prod | staging`); `spring.config.activate.on-cloud-platform` (e.g. `kubernetes`) is the other activation property. Later documents override earlier ones.

3.3 Precedence: which value wins?

The full order

From **lowest** to **highest** priority, per the Boot 4.1 reference:

#	Source	Example
1	Default properties	`SpringApplication.setDefaultProperties(...)`
2	`@PropertySource` on `@Configuration` classes	`@PropertySource("classpath:legacy.properties")`
3	**Config data** files (below)	`application.yaml`, `application-prod.yaml`
4	`RandomValuePropertySource`	`${random.uuid}`
5	**OS environment variables**	`SHOP_NAME=...`
6	**Java system properties**	`java -Dshop.name=... -jar app.jar`
7	JNDI (Java Naming and Directory Interface, classic app-server lookup) attributes	(app servers)
8–9	`ServletContext` / `ServletConfig` init parameters	(WAR deployments to an external server)
10	`SPRING_APPLICATION_JSON`	`SPRING_APPLICATION_JSON='{"shop":{"name":"X"}}'`
11	**Command-line arguments**	`java -jar app.jar --shop.name=...`
12	`properties` attribute of tests	`@SpringBootTest(properties = "shop.name=T")`
13	`@DynamicPropertySource`	Testcontainers (real databases in Docker for tests) URLs (Chapter 12)
14	`@TestPropertySource`	
15	DevTools global settings	`~/.config/spring-boot/`

Within **config data** (row 3), lowest → highest:

1. `application.*` packaged **inside** the jar
2. `application-{profile}.*` inside the jar
3. `application.*` **outside** the jar (current dir, `config/`)
4. `application-{profile}.*` outside the jar

Memory aid: *specific beats general* (profile file > plain file; outside > inside; command line > everything the app ships with), and tests override everything.

Proved

The test starts the real app five times, each time adding one higher source (fake env vars injected via a custom `StandardEnvironment`; see `Apps.java`):

```
// PrecedenceTest.java (simplified)
shopName(Map.of());                                                    // 1
shopName(Map.of(), "--spring.profiles.active=dev");                    // 2
shopName(Map.of("SHOP_NAME", "Env Shop"), "--spring.profiles.active=dev");   // 3
System.setProperty("shop.name", "SysProp Shop");
shopName(Map.of("SHOP_NAME", "Env Shop"));                             // 4
shopName(Map.of("SHOP_NAME", "Env Shop"), "--shop.name=CLI Shop");     // 5 (system property still set)
```

```
1 application.yaml only          -> Corner Shop
2 + application-dev.yaml         -> Corner Shop (DEV)
3 + env SHOP_NAME                -> Env Shop
4 + -Dshop.name (system prop)    -> SysProp Shop
5 + --shop.name (command line)   -> CLI Shop
```

The `Environment`'s actual search order with `--spring.profiles.active=dev` (first wins):

```
Environment property sources (first wins):
  configurationProperties
  commandLineArgs
  systemProperties
  systemEnvironment
  random
  Config resource 'class path resource [application-dev.yaml]' via location 'optional:classpath:/'
  Config resource 'class path resource [owner.properties]' via location 'classpath:owner.properties'
  Config resource 'class path resource [application.yaml]' via location 'optional:classpath:/' (document #0)
  applicationInfo
  defaultProperties
```

- `configurationProperties` is not a real source; it's Boot's adapter exposing all the others to the binder (the component that copies property values into Java objects) with relaxed names.
- `owner.properties` sits *above* `application.yaml` because `application.yaml` imports it (§3.8): **an imported file overrides the file that imports it**.
- The profile file `application-dev.yaml` is above both.

Common mistakes

- A value in `application.yaml` "ignored" — an environment variable with the same relaxed name (from a Kubernetes manifest or `.env` file) overrides it.
- Secrets in `application.yaml` in Git. Use env vars, mounted files (`configtree:`, §3.8), or a secret manager.
- Expecting `@PropertySource` to beat `application.yaml` — it is near the bottom (row 2).

In the interview

Q: A property is set in `application.yaml`, an environment variable and a command-line argument. Which wins?
The command-line argument; then system properties, env vars, profile-specific files, `application.yaml`. Test properties beat all.

- *Follow-up: Where did a value come from?* Actuator's (Chapter 15) `env` endpoint (`/actuator/env/shop.name`) lists every source defining it, highest first; binding failure messages also show the *origin* (e.g. `"shop.name" from property source "commandLineArgs"`, §3.6).

3.4 Environment variables and relaxed binding

What it is

Environment variable names can't contain dots or dashes on most systems, so Boot maps them by rule. **Relaxed binding** means Boot accepts several spellings of a property when binding to `@ConfigurationProperties`:

Form	Example	Use in
Kebab case (canonical, recommended)	`shop.checkout-timeout`	`.properties` / `.yaml`
Camel case	`shop.checkoutTimeout`	Also accepted in files
Underscore	`shop.checkout_timeout`	Also accepted in files
Upper case with underscores	`SHOP_CHECKOUTTIMEOUT`	**Environment variables**

Environment-variable rule: replace `.` with `_`, **remove** `-`, upper-case. List index: `SHOP_CURRENCIES_0=USD`. So `shop.payment.retries` → `SHOP_PAYMENT_RETRIES`, `spring.profiles.active` → `SPRING_PROFILES_ACTIVE`.

```
Apps.start(Map.of("SHOP_PAYMENT_RETRIES", "4", "SHOP_CHECKOUTTIMEOUT", "2m"))
```

```
SHOP_PAYMENT_RETRIES=4, SHOP_CHECKOUTTIMEOUT=2m -> retries=4, checkoutTimeout=PT2M
```

Common mistakes

- Thinking `SHOP_CHECKOUT_TIMEOUT` (underscore where the dash was) is ignored. For backward compatibility Boot also binds this **legacy** spelling to `shop.checkout-timeout` (proved in ch19 — `SHOP_CHECKOUT_TIMEOUT=3m -> PT3M`). The real mistake is using two spellings for one property: they are separate variables, so searching for `SHOP_CHECKOUTTIMEOUT` misses the other. Stick to the documented form (drop the dash).
- camelCase in `@Value("${shop.checkoutTimeout}")`. `@Value` gets relaxed matching only with the canonical kebab-case placeholder: `@Value("${shop.checkout-timeout}")`.

3.5 `@Value`

What it is

`@Value` injects one value into a field, constructor parameter or `@Bean` method parameter.

```
// config-demo/src/main/java/com/example/config/ValueStyle.java
@Component
public class ValueStyle {
    ValueStyle(@Value("${shop.name}") String name,                                  // required: startup fails if missing
               @Value("${shop.support-email:help@example.test}") String email, // default after the colon
               @Value("#{${shop.payment.retries:3} + 1}") int maxAttempts,      // SpEL: #{...}
               @Value("${shop.owner}") String owner) {                          // comes from an imported file
        ...
    }
}
```

```
@Value                  -> name=Corner Shop, supportEmail=help@example.test, maxAttempts=3, owner=Ada
```

Syntax	Meaning
`${key}`	Property placeholder; startup fails with `Could not resolve placeholder` if missing
`${key:default}`	Default when missing (`${key:}` = empty string)
`#{expression}`	SpEL (Spring Expression Language) — arithmetic, method calls, other beans: `#{@clock.millis()}`
`#{${key} * 2}`	Placeholder resolved first, then SpEL evaluated

Conversion to `int`, `boolean`, `Duration`, enums etc. is automatic; comma-separated strings convert to arrays/lists.

When not to use it

For more than two or three related settings use `@ConfigurationProperties`: `@Value` strings are scattered, untyped until runtime, unvalidated, and get no IDE auto-completion.

3.6 `@ConfigurationProperties` — type-safe configuration

What it is

`@ConfigurationProperties("shop")` binds **every property under a prefix** into one Java object. With a **record** the object is immutable and bound through its constructor ("constructor binding").

```
// config-demo/src/main/java/com/example/config/ShopProperties.java
@ConfigurationProperties("shop")
@Validated
public record ShopProperties(
        @NotBlank String name,
        @DefaultValue("30s") Duration checkoutTimeout,
        @DefaultValue("5MB") DataSize maxUpload,
        @DefaultValue("USD") List<String> currencies,
        Map<String, Integer> discounts,
        @Valid @DefaultValue Payment payment) {

    public record Payment(
            @NotBlank @DefaultValue("https://pay.example.test") String gatewayUrl,
            @Min(0) @Max(5) @DefaultValue("3") int retries) {
    }
}

@SpringBootApplication
@ConfigurationPropertiesScan          // registers every @ConfigurationProperties class in this package tree
public class ConfigDemoApplication { ... }
```

Bound from the YAML in §3.2:

```
@ConfigurationProperties -> ShopProperties[name=Corner Shop, checkoutTimeout=PT45S, maxUpload=10485760B, currencies=
[USD, EUR], discounts={gold=15, silver=5}, payment=Payment[gatewayUrl=https://pay.example.test, retries=2]]
```

Any bean can take `ShopProperties` as a constructor parameter and read `props.payment().retries()`.

How it works

1. **Register** the class: `@ConfigurationPropertiesScan` on the application, or `@EnableConfigurationProperties(ShopProperties.class)` on a configuration class (typical in auto-configurations).
2. At startup Boot's **`Binder`** walks the constructor parameters (or setters, for JavaBean-style classes), builds each name from prefix + relaxed parameter name, and looks it up in the `Environment`.
3. **Conversion**: numbers, booleans, enums, `List`, `Map`, nested records, and:

Type	Accepted values	Notes
`java.time.Duration`	`45s`, `5m`, `2h`, `1d`, `500ms`, `PT45S` (ISO-8601), plain `45`	Plain numbers default to milliseconds; change with `@DurationUnit(ChronoUnit.SECONDS)`
`org.springframework.util.unit.DataSize`	`10MB`, `512KB`, `1GB`, plain `1024`	Plain numbers are bytes; `@DataSizeUnit` changes it. `10MB` = 10 × 1024 × 1024 = `10485760B`
`java.time.Period`	`1y3d`, `P1Y3D`	

4. **Defaults**: `@DefaultValue("...")` supplies a value when the property is absent. A bare `@DefaultValue` on a nested record (as on `payment`) means "create it even if no `shop.payment.*` exists" — otherwise it would be `null`. `gatewayUrl` came from its default because only `retries` was configured.
5. **Validation**: with `@Validated` and a Bean Validation provider on the classpath (`spring-boot-starter-validation` → Hibernate Validator), constraints are checked at startup. `@Valid` on nested objects makes their constraints run too.

Validation failure — proved

```
Apps.start("--shop.payment.retries=9", "--shop.name= ")
```

Boot's failure analyzer (turns a startup exception into a readable explanation) prints this and the app does not start:

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Binding to target com.example.config.ShopProperties failed:

    Property: shop.name
    Value: " "
    Origin: "shop.name" from property source "commandLineArgs"
    Reason: must not be blank

    Property: shop.payment.retries
    Value: "9"
    Reason: must be less than or equal to 5


Action:

Update your application's configuration
```

The root exception is `BindValidationException`. **Failing at startup is the point**: a bad production setting is caught during deployment, not at 3 a.m. when the first payment retries.

IDE auto-completion — configuration metadata

`spring-boot-configuration-processor`, as an annotation processor (a compiler plug-in that can generate files), generates `META-INF/spring-configuration-metadata.json` at compile time; IDEs use it for auto-completion and documentation of `shop.*` keys. Descriptions come from the record's Javadoc `@param` tags:

```
{"name": "shop.checkout-timeout", "type": "java.time.Duration", "description": "how long a checkout may take",
"sourceType": "com.example.config.ShopProperties", "defaultValue": "30s"}
{"name": "shop.currencies", "type": "java.util.List<java.lang.String>", "description": "accepted currency codes",
"sourceType": "com.example.config.ShopProperties", "defaultValue": "USD"}
```

```
<!-- pom.xml: since JDK 23, javac no longer runs processors found on the classpath by default -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>
...
<plugin>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <annotationProcessorPaths>
      <path>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-configuration-processor</artifactId>
      </path>
    </annotationProcessorPaths>
  </configuration>
</plugin>
```

Other shapes

- **JavaBean style** (no-arg constructor, getters/setters) is still supported, needed for mutable properties; records are simpler.
- With several constructors, mark the binding one with `@ConstructorBinding`.
- **Third-party classes**: `@ConfigurationProperties("pool")` on a `@Bean` method; Boot binds `pool.*` onto the returned object's setters.

`@Value` vs `@ConfigurationProperties`

	`@ConfigurationProperties`	`@Value`
Granularity	A whole group under a prefix	One value
Type safety	Typed object, checked at startup	Converted per injection point
Relaxed binding	Yes	Limited (use kebab-case placeholders)
Validation	`@Validated` + Jakarta constraints	No
IDE metadata / auto-complete	Yes (processor)	No
SpEL	No	Yes
Default values	`@DefaultValue` / field initialisers	`${key:default}`
Best for	Any real settings of your app or library	A single value, or when you need SpEL

Common mistakes

- Forgetting to register the class → "No qualifying bean of type ShopProperties". `@Component` does **not** work for a record: constructor binding only happens for classes registered by scanning or `@EnableConfigurationProperties`; as a plain component Spring would try to inject beans of type `String`, `Duration`... and fail.
- Expecting `@DefaultValue`/field defaults to be visible via `@Value` or `Environment.getProperty` — they exist only in the bound object.
- A typo in a key (`shop.chekout-timeout`) is silently ignored — metadata-based IDE warnings and required-field validation help.
- Leaving out `@Valid` on nested objects, so their constraints never run.

In the interview

Q: `@Value` vs `@ConfigurationProperties`? `@ConfigurationProperties` binds a prefix into a typed, validated object with relaxed binding and IDE metadata — use it for groups. `@Value` injects single values and supports SpEL.

- *Follow-up: Immutable configuration?* A record (constructor binding) — no setters.
- *Follow-up: Fail fast on bad config?* `@Validated` + constraints; Boot refuses to start and prints each invalid property with its origin.

3.7 Profiles, profile-specific files and profile groups

What it is

A **profile** is a named environment ("dev", "test", "prod"). Activating it (§2.11) registers `@Profile("...")` beans *and* loads **profile-specific configuration**: `application-{profile}.properties`/`.yaml` files and documents with `spring.config.activate.on-profile`.

```yaml
# application-dev.yaml — loaded only when "dev" is active; overrides application.yaml
shop:
  name: Corner Shop (DEV)
  checkout-timeout: 5m
```

Profile groups

A **profile group** gives one name to several profiles:

```yaml
spring:
  profiles:
    group:
      production: [prod, metrics]     # --spring.profiles.active=production turns on both
```

```
Apps.start("--spring.profiles.active=production")
```

```
active profiles = [production, prod, metrics]
payment         = Payment[gatewayUrl=https://pay.prod.example.test, retries=5]
```

Activating `production` also activated `prod`, so the `on-profile: prod` document applied.

Rules worth knowing

Property	Meaning
`spring.profiles.active`	Profiles to activate (comma-separated). Last listed wins on conflicts between their files.
`spring.profiles.default`	Profiles active when none are set (default: `default`)
`spring.profiles.include`	Profiles always added on top of the active ones
`spring.profiles.group.<name>`	Expand one profile into several

- `spring.profiles.active`, `.default` and `.include` may only appear in **non-profile-specific** documents — activating a profile from inside `application-dev.yaml` fails with `InvalidConfigDataPropertyException`. Define `spring.profiles.group` there too.
- Profile-specific files always override the non-specific file in the same location.
- In tests: `@ActiveProfiles("test")`.

Common mistakes

- Per-developer profiles (`application-john.yaml`) committed to Git. Use an ignored `config/application.yaml` or env vars.
- Different jars per environment — the point is one jar plus a profile.
- Deeply layered profiles (`dev,eu,feature-x,local`) nobody can reason about. Keep a small, documented set.

3.8 Importing configuration: `spring.config.import`

What it is

`spring.config.import` pulls more configuration into the `Environment` from a config file (or the command line):

```
spring:
  config:
    import:
      - classpath:owner.properties              # must exist
      - optional:file:./local-overrides.yaml    # ignored if missing
```

```
shop.owner from owner.properties = Ada
```

The optional `./local-overrides.yaml` doesn't exist and the app starts anyway.

Location form	Example	Use
`classpath:`	`classpath:owner.properties`	Split a large config into files
`file:`	`file:/etc/myapp/extra.yaml`	Files on the host/volume
`optional:` prefix	`optional:file:./local.yaml`	Don't fail if missing (without it: `ConfigDataLocationNotFoundException`)
`configtree:`	`optional:configtree:/etc/secrets/`	**Kubernetes secrets/ConfigMaps (objects holding secret or plain settings) mounted as files**: each file name becomes a key, its content the value (`/etc/secrets/db/password` → `db.password`)
Extension hint	`file:/etc/app/conf[.yaml]`	Location without a recognisable extension
Other schemes (via libraries)	`configserver:http://...` (Spring Cloud Config), `vault://` (Spring Cloud Vault), `aws-secretsmanager:` / `aws-parameterstore:` (Spring Cloud AWS)	Central config / secret stores (Chapter 16)

Rules: relative locations resolve against the importing file; an imported file **overrides** its importer (§3.3); imports are processed once even if listed twice.

3.9 Placeholders, random values and other sources

- **Placeholders in property files**: `app.description=${spring.application.name} runs on port ${server.port:8080}`. Refer to keys in canonical kebab-case.
- **Random values** (row 4 of the precedence table), for test secrets or instance ids (`uuid` = random 128-bit universally unique identifier):

  ```
  my.secret=${random.value}
  my.id=${random.uuid}
  my.number=${random.int}
  my.port=${random.int[1024,65536]}
  my.small=${random.int(10)}
  ```

  Each placeholder is evaluated independently at startup; they are not secure random tokens — never use them for production secrets.
- `SPRING_APPLICATION_JSON`: a JSON blob in one env var or system property, flattened into properties — handy where only one variable can be set.
- `@PropertySource("classpath:legacy.properties")` on a `@Configuration` class: plain Spring's way to add a file. Low priority, `.properties` only (no YAML, no profiles). In Boot prefer `spring.config.import`.
- **Injecting** `Environment`: `env.getProperty("shop.name")`, `env.getProperty("shop.payment.retries", Integer.class, 3)`, `env.matchesProfiles("prod")`. Useful in conditions and registrars (§2.14).

3.10 How auto-configuration works

What it is

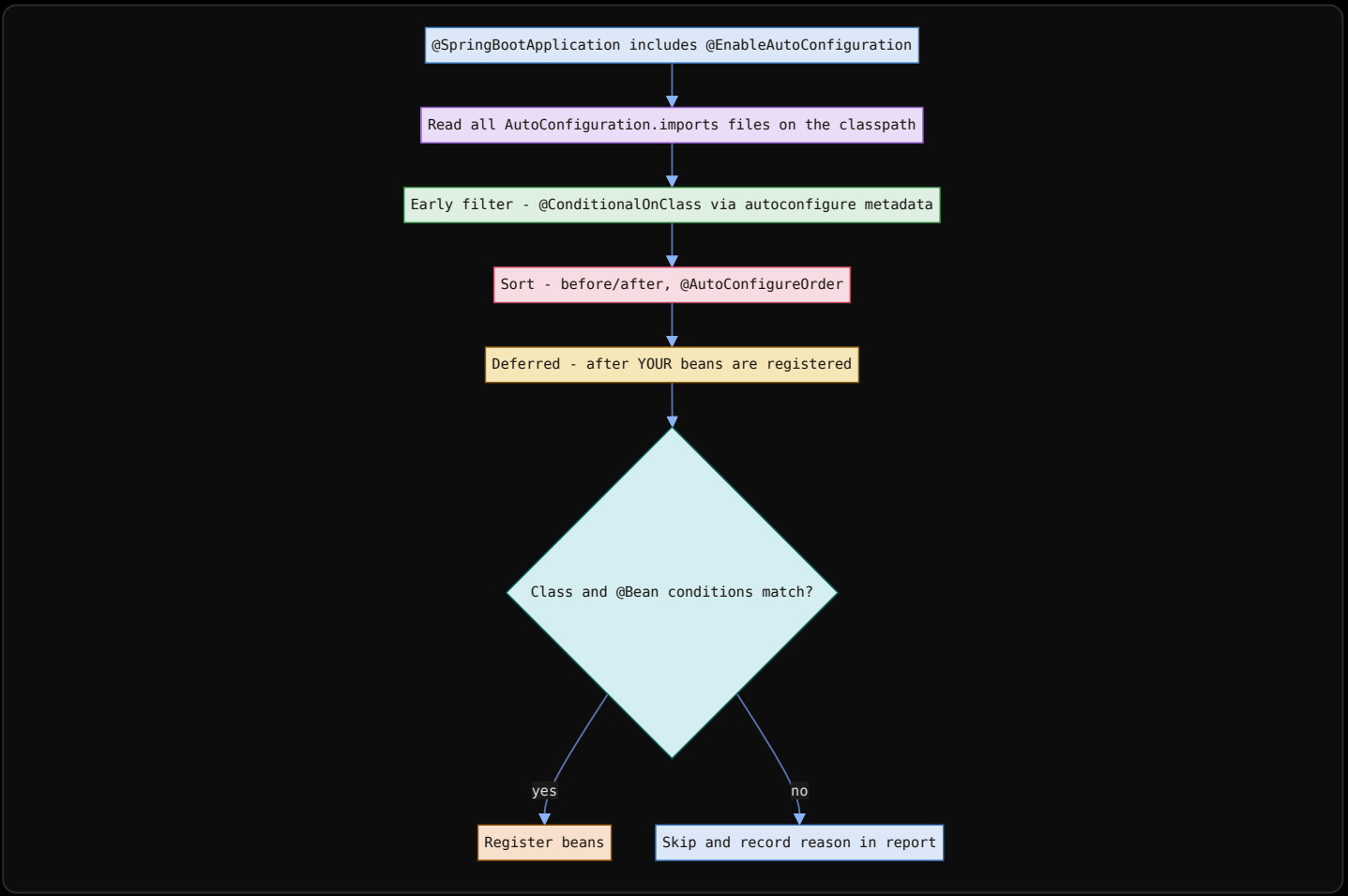
Auto-configuration registers beans you'd otherwise write yourself — a `DispatcherServlet`, embedded Tomcat, a Jackson `JsonMapper`, a `DataSource`, a transaction manager (starts, commits and rolls back transactions) — *only when it makes sense*: the library is on the classpath, you haven't defined your own bean, and the relevant property allows it.

Each auto-configuration is a `@Configuration` class (marked `@AutoConfiguration`) whose class and `@Bean` methods carry **conditions** — nothing beyond Chapter 2's `@Conditional` and `@Import`.

How it works

1. `@SpringBootApplication` includes **`@EnableAutoConfiguration`**, which imports `AutoConfigurationImportSelector`.

2. The selector reads every `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` file on the classpath (one class name per line; each auto-configuring jar ships one).
3. **Early filtering**: using pre-computed `META-INF/spring-autoconfigure-metadata.properties` (generated by `spring-boot-autoconfigure-processor`), Boot drops candidates whose `@ConditionalOnClass` can't match — without loading those classes. Keeps startup fast.
4. Candidates are **sorted** by `@AutoConfiguration(before/after = ...)` and `@AutoConfigureOrder`.
5. It is a **deferred** import: auto-configurations are processed **after all your own configuration and scanned beans** are registered — which makes `@ConditionalOnMissingBean` reliable.
6. Remaining conditions are evaluated; matches register beans. Every decision goes into the **condition evaluation report** (§3.11).



Boot 4.1.1's web MVC module ships (from `spring-boot-webmvc-4.1.1.jar`):

```
org.springframework.boot.webmvc.autoconfigure.DispatcherServletAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.WebMvcAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.WebMvcObservationAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.actuate.endpoint.web.WebMvcHealthEndpointExtensionAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.actuate.web.mappings.WebMvcMappingsAutoConfiguration
org.springframework.boot.webmvc.autoconfigure.error.ErrorMvcAutoConfiguration
```

The condition annotations

All in `org.springframework.boot.autoconfigure.condition`:

Annotation	Matches when	Typical use
`@ConditionalOnClass` / `@ConditionalOnMissingClass`	A class is / isn't on the classpath	"Configure Jackson only if Jackson is present"
`@ConditionalOnBean` / `@ConditionalOnMissingBean`	A bean of a type/name exists / doesn't exist	**Back off** if the user defined their own
`@ConditionalOnSingleCandidate`	Exactly one (or one primary) bean of a type	"Create a `JdbcTemplate` if there's one `DataSource`"
`@ConditionalOnProperty`	A property has a value (`havingValue`), or exists; `matchIfMissing`	Feature switches
`@ConditionalOnBooleanProperty` (Boot 3.5+)	A boolean property is `true` (by default); `matchIfMissing`	`spring.jmx.enabled` (JMX = Java Management Extensions, built-in JVM monitoring)
`@ConditionalOnResource`	A resource exists	`classpath:META-INF/build-info.properties`
`@ConditionalOnWebApplication` / `@ConditionalOnNotWebApplication`	Servlet or reactive web app / not	Web-only beans
`@ConditionalOnWarDeployment` / `@ConditionalOnNotWarDeployment`	Deployed as WAR / not	
`@ConditionalOnThreading`	Platform (classic OS) or virtual (lightweight, JVM-managed) threads selected (`spring.threads.virtual.enabled`)	Executors (Chapter 14)
`@ConditionalOnCloudPlatform`	Running on Kubernetes, Cloud Foundry, ...	Platform tweaks
`@ConditionalOnJava`	JVM version range	
`@ConditionalOnExpression`	A SpEL expression is true	Last resort — can trigger early bean creation

Common mistakes

- `@ConditionalOnBean` / `@ConditionalOnMissingBean` in *your own* (non-auto) configuration. They only see beans registered *so far*, so the result depends on processing order; they're reliable in auto-configurations because those run last.
- `@ConditionalOnClass(SomeLib.class)` on a `@Bean` *method* returning a type from that library — if the class is missing, loading the configuration class itself can fail. Put the condition on a nested `@Configuration` class (the Boot docs' pattern).

In the interview

Q: How does Spring Boot auto-configuration work? `@EnableAutoConfiguration` imports a deferred selector that loads class names from every `META-INF/spring/...AutoConfiguration.imports`, filters by `@ConditionalOnClass` early via generated metadata, sorts them, and — after user configuration is registered — evaluates their conditions. Matching configurations register beans; most are `@ConditionalOnMissingBean`, so user beans take priority.

- *Follow-up: Where were they listed before?* In `META-INF/spring.factories` under `EnableAutoConfiguration`; the `.imports` file arrived in Boot 2.7 and `spring.factories` stopped working for auto-configuration in Boot 3.0.
- *Follow-up: Why deferred?* So `@ConditionalOnMissingBean` can see the user's beans and back off.

3.11 Seeing what Boot decided

The `--debug` report

Start any Boot app with `--debug` (or `debug=true`) and Boot prints the **CONDITIONS EVALUATION REPORT**. Excerpt from `java -jar config-demo.jar --debug`:

```
============================
CONDITIONS EVALUATION REPORT
============================

Positive matches:
-----------------

   AopAutoConfiguration matched:
      - @ConditionalOnBooleanProperty (spring.aop.auto=true) matched (OnPropertyCondition)

   ValidationAutoConfiguration matched:
      - @ConditionalOnClass found required class 'jakarta.validation.executable.ExecutableValidator' (OnClassCondition)
      - @ConditionalOnResource found location classpath:META-INF/services/jakarta.validation.spi.ValidationProvider
(OnResourceCondition)

   ValidationAutoConfiguration#defaultValidator matched:
      - @ConditionalOnMissingBean (types: jakarta.validation.Validator; SearchStrategy: all) did not find any beans
(OnBeanCondition)
   ...

Negative matches:
-----------------

   AopAutoConfiguration.AspectJAutoProxyingConfiguration:
      Did not match:
         - @ConditionalOnClass did not find required class 'org.aspectj.weaver.Advice' (OnClassCondition)

   JmxAutoConfiguration:
      Did not match:
         - @ConditionalOnBooleanProperty (spring.jmx.enabled=true) did not find property 'spring.jmx.enabled'
(OnPropertyCondition)
      Matched:
         - @ConditionalOnClass found required class 'org.springframework.jmx.export.MBeanExporter' (OnClassCondition)

   TaskExecutorConfigurations.TaskExecutorConfiguration#applicationTaskExecutorVirtualThreads:
      Did not match:
         - @ConditionalOnThreading did not find VIRTUAL (OnThreadingCondition)
   ...

Exclusions:
-----------

    None

Unconditional classes:
----------------------

    org.springframework.boot.autoconfigure.context.ConfigurationPropertiesAutoConfiguration
    org.springframework.boot.autoconfigure.context.PropertyPlaceholderAutoConfiguration
    ...
```

- **Positive matches** — applied. `Class#method` entries are individual `@Bean` methods.
- **Negative matches** — skipped, with the failing condition (and any that matched). Look here when "Boot didn't create X": the missing class, bean or property is named.
- **Exclusions** — auto-configurations you excluded explicitly.
- **Unconditional classes** — no class-level condition.

`ValidationAutoConfiguration` matched only because `spring-boot-starter-validation` put Hibernate Validator on the classpath — auto-configuration reacting to a dependency.

The same report in code and over HTTP

The report is a queryable bean-factory object:

```
var report = ConditionEvaluationReport.get(ctx.getBeanFactory());
var outcomes = report.getConditionAndOutcomesBySource().get("...JmxAutoConfiguration");
```

```
TaskExecutionAutoConfiguration  MATCHED
   - @ConditionalOnClass found required class 'org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor'
JmxAutoConfiguration  DID NOT MATCH
   - @ConditionalOnClass found required class 'org.springframework.jmx.export.MBeanExporter'
   - @ConditionalOnBooleanProperty (spring.jmx.enabled=true) did not find property 'spring.jmx.enabled'
AopAutoConfiguration  MATCHED
   - @ConditionalOnBooleanProperty (spring.aop.auto=true) matched
auto-configuration classes evaluated: 7
```

With **Actuator** (Chapter 15), `GET /actuator/conditions` returns the same report as JSON (expose with `management.endpoints.web.exposure.include=conditions`; keep it off public networks). `/actuator/configprops` shows all bound `@ConfigurationProperties` beans and `/actuator/env` the property sources.

3.12 Overriding and excluding auto-configuration

In order of preference:

1. **Set a property** (`server.port`, `spring.jackson.*`, `spring.datasource.*`). The reference's "Common Application Properties" appendix lists them; your IDE completes them from metadata.
2. **Define your own bean.** Auto-configured beans are `@ConditionalOnMissingBean`, so yours replaces Boot's (proved in §3.13).
3. **Use a customizer** to *adjust* rather than replace (e.g. `JsonMapperBuilderCustomizer` for Jackson 3, `WebServerFactoryCustomizer` for Tomcat).
4. **Exclude an auto-configuration:**

```
@SpringBootApplication(exclude = JmxAutoConfiguration.class)
```

```
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jmx.JmxAutoConfiguration
```

5. **Remove the dependency** if you don't use it.

Common mistakes

- Excluding by a class name copied from a Boot 3 tutorial — in Boot 4 most auto-configuration classes moved to per-module packages (e.g. `org.springframework.boot.webmvc.autoconfigure.WebMvcAutoConfiguration`, previously `org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration`). Check the report for the current name.
- `@EnableWebMvc` in a Boot app "to configure MVC" — it signals full manual control and switches off Boot's MVC auto-configuration (Chapter 5).

3.13 Writing your own auto-configuration and starter

Why

Companies write starters so every service gets the same client library, tracing setup or security defaults with one dependency (e.g. an internal `payments-client-spring-boot-starter`). Open-source examples: `mybatis-spring-boot-starter`, `springdoc-openapi-starter-webmvc-ui` (generates an OpenAPI description and docs UI).

The structure

The Boot 4.1 reference recommends two modules (merge into one `acme-spring-boot-starter` if simple):

Module	Name pattern	Contains
Auto-configure module	`acme-spring-boot`	`@AutoConfiguration` classes, `@ConfigurationProperties`, the `.imports` file
Starter	`acme-spring-boot-starter`	No code; depends on the auto-configure module, the library, and `spring-boot-starter`
(Optional) test starter	`acme-spring-boot-starter-test`	Test support

Never start your module names with `spring-boot` (reserved for official modules), and use your own property prefix (`acme.*`), not `spring.*`/`server.*`.

Our example wires a tiny `GreetingService` library:

```
greeter/
├── pom.xml                        ← aggregator: builds the modules together
└── modules/
    ├── autoconfig/                ← auto-configure module (artifactId greeting-spring-boot)
    │   ├── src/main/java/com/example/greeting/core/GreetingService.java
    │   ├── src/main/java/com/example/greeting/autoconfigure/GreetingProperties.java
    │   ├── src/main/java/com/example/greeting/autoconfigure/GreetingAutoConfiguration.java
    │   └── src/main/resources/META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
    ├── starter/pom.xml            ← dependencies only (artifactId greeting-spring-boot-starter)
    └── demo/                      ← an application that uses the starter
```

(Normally `GreetingService` would be a separate library jar.)

The code

```java
/** The "library" the starter wires up. It knows nothing about Spring. */
public class GreetingService {
    private final String template;
    public GreetingService(String template) { this.template = template; }
    public String greet(String name) { return template.formatted(name); }
}

/**
 * @param enabled  whether to auto-configure a GreetingService
 * @param template message template; %s is replaced by the name
 */
@ConfigurationProperties("greeting")
public record GreetingProperties(
        @DefaultValue("true") boolean enabled,
        @DefaultValue("Hello, %s!") String template) {
}

@AutoConfiguration                                          // a @Configuration(proxyBeanMethods = false) for Boot
@ConditionalOnClass(GreetingService.class)                  // 1. only if the library is on the classpath
@ConditionalOnBooleanProperty(name = "greeting.enabled", matchIfMissing = true) // 2. unless switched off
@EnableConfigurationProperties(GreetingProperties.class)    // 3. bind greeting.* into GreetingProperties
public class GreetingAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean                               // 4. back off if the user defined their own
    GreetingService greetingService(GreetingProperties props) {
        return new GreetingService(props.template());
    }
}
```

```
# src/main/resources/META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
com.example.greeting.autoconfigure.GreetingAutoConfiguration
```

The auto-configure module's `pom.xml` depends on `spring-boot-autoconfigure` and declares two annotation processors: `spring-boot-configuration-processor` (property metadata) and `spring-boot-autoconfigure-processor`, which generated this file for early class filtering (§3.10 step 3):

```
# target/classes/META-INF/spring-autoconfigure-metadata.properties (generated)
com.example.greeting.autoconfigure.GreetingAutoConfiguration=
com.example.greeting.autoconfigure.GreetingAutoConfiguration.ConditionalOnClass=com.example.greeting.core.GreetingService
```

The starter's `pom.xml` is just:

```
<artifactId>greeting-spring-boot-starter</artifactId>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>
  <dependency>
    <groupId>com.example.greeting</groupId>
    <artifactId>greeting-spring-boot</artifactId>
    <version>${project.version}</version>
  </dependency>
</dependencies>
```

Testing auto-configuration with `ApplicationContextRunner`

`ApplicationContextRunner` builds a tiny throw-away context per test with exactly the configuration, properties and classpath you specify — faster and more precise than `@SpringBootTest`, with AssertJ assertions (`hasSingleBean`, `doesNotHaveBean`, `hasBean`).

```
private final ApplicationContextRunner runner = new ApplicationContextRunner()
        .withConfiguration(AutoConfigurations.of(GreetingAutoConfiguration.class));

@Test void createsTheBeanByDefault() {
    runner.run(ctx -> assertThat(ctx).hasSingleBean(GreetingService.class));
}
@Test void propertiesCustomiseIt() {
    runner.withPropertyValues("greeting.template=Welcome back, %s.").run(ctx -> ...);
}
@Test void canBeSwitchedOff() {
    runner.withPropertyValues("greeting.enabled=false").run(ctx ->
assertThat(ctx).doesNotHaveBean(GreetingService.class));
}
@Configuration(proxyBeanMethods = false)
static class UserConfig { @Bean GreetingService myGreeting() { return new GreetingService("Yo %s"); } }

@Test void backsOffWhenTheUserDefinesOne() {
    runner.withUserConfiguration(UserConfig.class).run(ctx ->
            assertThat(ctx).hasSingleBean(GreetingService.class).hasBean("myGreeting"));
}
@Test void skippedWhenLibraryIsMissing() {
    runner.withClassLoader(new FilteredClassLoader(GreetingService.class))   // pretend the jar is absent
          .run(ctx -> assertThat(ctx).doesNotHaveBean("greetingService"));
}
```

Output (each test prints what it saw) — default bean, property binding, on/off switch, back-off, and skip-when-absent all proved:

```
default         -> Hello, Ada!
with property   -> Welcome back, Ada.
enabled=false   -> GreetingService beans: 0
user bean       -> Yo Ada (bean name: myGreeting)
class missing   -> has greetingService? false
```

Using the starter

The demo app has one dependency on `greeting-spring-boot-starter`, one property, and **no** `@Bean` or `@Import` for `GreetingService`:

```
# modules/demo/src/main/resources/application.properties
greeting.template=Good morning, %s!
```

```
@SpringBootTest
class DemoApplicationTests {
    @Autowired GreetingService greetings;          // provided by the starter's auto-configuration
    ...
}
```

```
app says: Good morning, Ada!
GreetingAutoConfiguration matched? true
```

Checklist for a good starter

1. Guard the class with `@ConditionalOnClass` of the library's key type; guard beans with `@ConditionalOnMissingBean`.
2. Put `@ConditionalOnClass` for *optional* integrations on nested `@Configuration(proxyBeanMethods = false)` classes.
3. `@ConfigurationProperties` with your own prefix and Javadoc on every property (it becomes IDE help).
4. Order relative to Boot's auto-configurations with `@AutoConfiguration(after = ...)` when you depend on their beans.
5. Test with `ApplicationContextRunner` (`WebApplicationContextRunner` for web parts).
6. Keep the starter free of code and of optional dependencies.

In the interview

Q: How would you create a custom Spring Boot starter? An `@AutoConfiguration` class with `@ConditionalOnClass` / `@ConditionalOnMissingBean` /property conditions and a `@ConfigurationProperties` record, listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, packaged as `acme-spring-boot`; publish an `acme-spring-boot-starter` pom depending on it and the library. Test with `ApplicationContextRunner`.

- *Follow-up: How does a user override your bean?* Define a bean of the same type; `@ConditionalOnMissingBean` makes yours back off.

3.14 Boot 4 modularization

What changed

Up to Boot 3.5, one jar — `spring-boot-autoconfigure` — held auto-configuration for *every* technology (156 auto-configuration classes in 3.5.6). **Boot 4 split it into one module per technology.** In 4.1.1, `spring-boot-autoconfigure` lists only 12 core auto-configurations (AOP, task execution/scheduling, JMX, SSL, lifecycle, message source, property placeholders...). The rest moved to modules such as:

Module jar	Auto-configurations (4.1.1)	Example
`spring-boot-webmvc`	6	`org.springframework.boot.webmvc.autoconfigure.WebMvcAutoConfiguration`
`spring-boot-tomcat`	5	`...tomcat.autoconfigure.servlet.TomcatServletWebServerAutoConfiguration`
`spring-boot-jackson`	1	`...jackson.autoconfigure.JacksonAutoConfiguration`
`spring-boot-jdbc`	9	DataSource, JdbcTemplate/JdbcClient, transactions
`spring-boot-actuator-autoconfigure`	24	Actuator endpoints
`spring-boot-micrometer-metrics`	28	Metrics

Why it matters to you

- **Smaller classpath, faster and more predictable startup**: only modules you use are present — fewer candidates, no accidental auto-configuration from a stray transitive library.
- **Clearer ownership**: each module has its own packages, starter and `-test` starter (`@WebMvcTest` in `spring-boot-webmvc-test`, `@DataJpaTest` in `spring-boot-data-jpa-test`).

- **Migration impact**: imports of auto-configuration classes (`exclude = ...`, `@ImportAutoConfiguration`, `@AutoConfiguration(after = ...)`) and test-slice annotations change package. The quick path is `spring-boot-starter-classic` / `spring-boot-starter-test-classic` (Boot 3-style "everything on the classpath") while you move to modular starters. The mechanism (imports file + conditions) is unchanged. Full migration notes: Chapter 17.

3.15 In the interview — quick round

Question	Short answer
Name the main configuration sources in priority order.	Command line > `SPRING_APPLICATION_JSON` > system properties > env vars > `random.*` > profile files (outside > inside jar) > `application.*` > `@PropertySource` > defaults. Tests on top. (§3.3)
How does an env var map to a property?	`.` → `_`, drop `-`, upper-case: `SHOP_CHECKOUTTIMEOUT` → `shop.checkout-timeout`. (§3.4)
`@Value` vs `@ConfigurationProperties`?	Single value + SpEL vs typed, validated, relaxed, documented group. (§3.6)
How do you validate configuration?	`@Validated` + Jakarta constraints (+ `@Valid` for nested); app fails to start with each bad property and origin. (§3.6)
Profile-specific config?	`application-{profile}.yaml`, `on-profile` documents; groups activate several profiles. (§3.7)
Kubernetes secrets as config?	`spring.config.import=optional:configtree:/etc/secrets/` (or env vars). (§3.8)
How does auto-configuration work?	`.imports` files → early class filter → sort → deferred → conditions → beans; `@ConditionalOnMissingBean` lets you override. (§3.10)
Why didn't Boot create bean X?	Run with `--debug` or `/actuator/conditions`; read the negative match. (§3.11)
How do you turn off one auto-configuration?	`exclude =` on `@SpringBootApplication` or `spring.autoconfigure.exclude`. (§3.12)
How do you write a starter?	`@AutoConfiguration` + conditions + properties in `acme-spring-boot`, listed in `.imports`; `acme-spring-boot-starter` pom; test with `ApplicationContextRunner`. (§3.13)
What did Boot 4 modularize?	Auto-configuration split into per-technology modules with their own starters and test starters. (§3.14)

Chapter TL;DR

- All configuration flows through the **`Environment`**: an ordered list of property sources; the first source with a key wins.
- **Precedence** (high → low): tests > command line > `SPRING_APPLICATION_JSON` > system properties > env vars > profile-specific files > `application.*` (outside jar > inside; imported file > importer) > `@PropertySource` > defaults.
- **Env vars** use relaxed binding: `SHOP_PAYMENT_RETRIES` → `shop.payment.retries`; drop dashes.
- Prefer **`@ConfigurationProperties`** **records** (`@ConfigurationPropertiesScan`, `@DefaultValue`, `Duration` / `DataSize`, `@Validated`) over scattered `@Value`s; add the configuration processor for IDE metadata.
- **Profiles** select beans and config files; **profile groups** activate several at once; `spring.config.activate.on-profile` makes multi-document files conditional.
- **`spring.config.import`** adds files, optional files, and `configtree:` directories (Kubernetes secrets).
- **Auto-configuration** = `@AutoConfiguration` classes listed in `AutoConfiguration.imports`, processed after your beans, guarded by `@ConditionalOnClass` / `OnMissingBean` / `OnProperty` / `OnBooleanProperty`...; debug with `--debug` or `/actuator/conditions`; override with properties, your own bean, customizers, or `exclude`.
- A **custom starter** = auto-configure module + code-free starter pom; test with `ApplicationContextRunner`.

- **Boot 4** split auto-configuration into per-technology modules (`spring-boot-webmvc`, `spring-boot-jdbc`, ...), each with a starter and a `-test` starter.

Next: Chapter 4 — AOP and Proxies explains the proxies you met in Chapter 2 and how `@Transactional`, `@Cacheable` and `@Async` are built on them.

Chapter 4 — AOP and Proxies

Many things an application must do are not part of any single business rule: logging, timing, transactions, caching, security checks, retries. Written by hand inside every service method, they drown the business logic, and one forgotten line becomes a bug. **Aspect-Oriented Programming (AOP)** lets you write such a concern once and declare *where* it applies. Spring implements AOP with **proxies** — stand-in objects that wrap your beans — and the same mechanism makes `@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize` and Framework 7's `@Retryable` work.

Understanding proxies is the most useful piece of "Spring internals" knowledge: it explains the most common Spring bug (an annotation that silently does nothing) and comes up in almost every Spring interview.

All code lives in `code/aop-proxies/` — Spring Boot 4.1.1 / Framework 7.0 on Java 25. Every claim marked "proved" is checked by a JUnit test; outputs are real `mvn -q verify` output (log lines trimmed as `...`).

> **Prerequisites.** Beans, the `ApplicationContext` and `BeanPostProcessor`: Chapter 2. The Proxy *design pattern* without Spring: LLD Proxy Pattern.

4.1 Cross-cutting concerns and the AOP vocabulary

The problem it solves

An `OrderService` with ten methods must log each call, time it and run it in a transaction. Without AOP every method looks like this:

```
public String placeOrder(String item, int qty) {
    log.info("placeOrder {} {}", item, qty);          // logging
    long start = System.nanoTime();                    // timing
    TransactionStatus tx = txManager.getTransaction(def);  // transaction
    try {
        String id = /* ... the 3 lines that actually matter ... */;
        txManager.commit(tx);
        return id;
    } catch (RuntimeException e) {
        txManager.rollback(tx);
        throw e;
    } finally {
        metrics.record("placeOrder", System.nanoTime() - start);
    }
}
```

Such code is a **cross-cutting concern**: a requirement that *cuts across* many classes instead of living in one. Two symptoms:

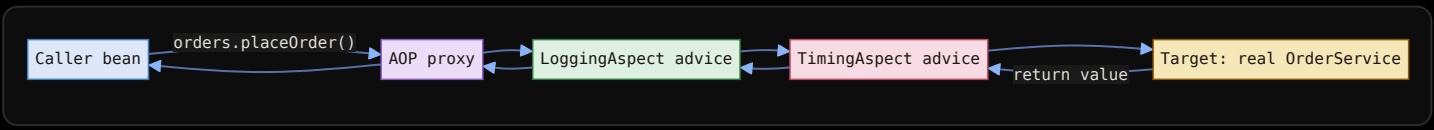
- **Scattering** — the same code spread over many classes.
- **Tangling** — each method mixes several unrelated concerns.

What it is

AOP modularises cross-cutting concerns: write the concern once in an **aspect**, describe with a **pointcut** which methods it applies to, and the framework applies it.

The vocabulary

Term	Meaning	In this chapter's code
Join point	A point during execution where behaviour *could* be added. In Spring AOP always **a method execution on a bean** (not field access, not constructors).	the call `orders.placeOrder("book", 2)`
Pointcut	A predicate selecting join points — *where*.	`execution(public * ch04.aop.OrderService.*(..))`
Advice	The code to run there, plus *when* (before, after, around...) — *what* and *when*.	`LoggingAspect.before(...)`
Aspect	A class bundling pointcuts and advice for one concern.	`LoggingAspect`, `TimingAspect`
Target object	The real object being advised.	the plain `OrderService` instance
AOP proxy	The object Spring creates that wraps the target and runs the advice. Other beans receive the proxy.	`OrderService$$SpringCGLIB$$0`
Weaving	Linking aspects with targets. Spring does it **at runtime by creating proxies**; AspectJ (the original, full AOP tool for Java) can do it at compile time or class-load time.	done by a `BeanPostProcessor` at startup
Advisor	Spring's internal pairing of one advice with one pointcut. Rarely written by hand.	—
Introduction	Adding a new interface to an advised object. Rarely used.	—



The caller *thinks* it holds an `OrderService`; it holds the proxy, which runs the advice chain and eventually calls the real object.

4.2 Writing aspects with `@Aspect`

How to set it up

1. Add the starter. In **Boot 4** it is `spring-boot-starter-aspectj` (called `spring-boot-starter-aop` up to Boot 3.x; renamed to make clear it brings AspectJ's annotation support):

   ```
   <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-aspectj</artifactId>
   </dependency>
   ```

2. Boot's `AopAutoConfiguration` sees AspectJ on the classpath and switches on `@EnableAspectJAutoProxy` (property `spring.aop.auto`, default `true`). In plain Spring add `@EnableAspectJAutoProxy` to a `@Configuration` class yourself.
3. Annotate the class **both** `@Aspect` and `@Component` (or register it with `@Bean`). `@Aspect` alone is *not* a stereotype — scanning ignores it, and an aspect that is not a bean never runs.

The five kinds of advice

Annotation	Runs	Can it change the outcome?	Typical use
`@Before`	before the method	only by throwing (the method then never runs)	argument checks, audit "who called"
`@AfterReturning`	after a normal return	reads the return value (`returning=`), cannot replace it	log results, publish an event
`@AfterThrowing`	after the method throws	reads the exception (`throwing=`); it still propagates unless the advice throws a different one	translate/log errors
`@After`	always, like `finally`	no	release resources, clear context
`@Around`	wraps the call; *you* call `proceed()`	yes: skip the call, change arguments or return value, catch exceptions, retry	timing, caching, transactions, retries

Spring docs' rule: use the **least powerful** advice that does the job. A `@Before` cannot forget to call `proceed()`; an `@Around` can.

Example: logging aspect

The business class knows nothing about logging (`OrderService.java`):

```
@Service
public class OrderService {
    public String placeOrder(String item, int quantity) {
        return "order-" + item + "-x" + quantity;
    }
    public void cancel(String orderId) {
        if (orderId.startsWith("unknown")) {
            throw new IllegalArgumentException("no such order: " + orderId);
        }
    }
    @Timed("monthly-report")
    public String monthlyReport() throws InterruptedException {
        Thread.sleep(20);                  // pretend to do slow work
        return "report ready";
    }
}
```

The aspect (`LoggingAspect.java`) writes into a small `CallLog` bean so a test can check the exact order:

```java
@Aspect                 // "this class contains advice + pointcuts"
@Component              // ...and it must also be a bean, or Spring never sees it
@Order(1)               // lower number = outer layer = runs first on the way in
public class LoggingAspect {
    private final CallLog log;
    public LoggingAspect(CallLog log) { this.log = log; }

    // A named, reusable pointcut: every public method of OrderService
    @Pointcut("execution(public * ch04.aop.OrderService.*(..))")
    void orderServiceMethods() {}

    @Before("orderServiceMethods()")
    public void before(JoinPoint jp) {
        log.add("before " + jp.getSignature().getName() + " args=" + Arrays.toString(jp.getArgs()));
    }

    @AfterReturning(pointcut = "orderServiceMethods()", returning = "result")
    public void afterReturning(JoinPoint jp, Object result) {
        log.add("afterReturning " + jp.getSignature().getName() + " -> " + result);
    }

    @AfterThrowing(pointcut = "orderServiceMethods()", throwing = "ex")
    public void afterThrowing(JoinPoint jp, Exception ex) {
        log.add("afterThrowing " + jp.getSignature().getName() + " -> " + ex.getMessage());
    }

    @After("orderServiceMethods()")   // like 'finally': runs on success AND on exception
    public void after(JoinPoint jp) {
        log.add("after " + jp.getSignature().getName());
    }
}
```

- `@Pointcut` on an empty method gives the expression a reusable **name** (`orderServiceMethods()`); the body never runs.
- `JoinPoint` exposes the method signature, the arguments (`getArgs()`), the target (`getTarget()`) and the proxy (`getThis()`).
- `returning = "result"` binds the return value to the parameter named `result`; declaring it `String` instead of `Object` restricts matching to methods returning `String`.
- `throwing = "ex"` does the same for the exception; the parameter type also filters.

The test (`AspectsTest.java`):

```java
@Test
void adviceRunsAroundASuccessfulCall() {
    String result = orders.placeOrder("book", 2);
    System.out.println("result = " + result);
    log.lines().forEach(l -> System.out.println("  " + l));
    assertThat(log.lines()).containsExactly(
            "before placeOrder args=[book, 2]",
            "afterReturning placeOrder -> order-book-x2",
            "after placeOrder");
}

@Test
void adviceRunsWhenTheMethodThrows() {
    assertThatThrownBy(() -> orders.cancel("unknown-7"))
            .isInstanceOf(IllegalArgumentException.class);   // the aspect does not swallow it
    log.lines().forEach(l -> System.out.println("  " + l));
    assertThat(log.lines()).containsExactly(
            "before cancel args=[unknown-7]",
            "afterThrowing cancel -> no such order: unknown-7",
            "after cancel");
}
```

Output:

```
result = order-book-x2
  before placeOrder args=[book, 2]
  afterReturning placeOrder -> order-book-x2
  after placeOrder
  before cancel args=[unknown-7]
  afterThrowing cancel -> no such order: unknown-7
  after cancel
```

`@After` ran **after** `@AfterReturning` / `@AfterThrowing`. Since Framework 5.2.7, advice of the *same* aspect on the *same* join point has a fixed precedence — `@Around`, `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing` from highest to lowest — and `@After` is effectively invoked after that aspect's returning/throwing advice, like a `finally` block.

`@Around` advice and custom annotations

`@Around` receives a `ProceedingJoinPoint`; `proceed()` continues to the next advice or the real method, and not calling it skips the method. A common pattern triggers an aspect with your own annotation:

```
@Retention(RetentionPolicy.RUNTIME)  // must be visible at runtime for the pointcut to see it
@Target(ElementType.METHOD)
public @interface Timed {
    String value() default "";       // optional label
}
```

`RUNTIME` retention is required because Spring inspects annotations by reflection at runtime; with `CLASS` / `SOURCE` retention the pointcut never matches.

```
@Aspect
@Component
@Order(2)               // inner layer: runs after LoggingAspect's @Before
public class TimingAspect {
    private final CallLog log;
    public TimingAspect(CallLog log) { this.log = log; }

    // Matches any bean method annotated with @Timed and binds the annotation to 'timed'
    @Around("@annotation(timed)")
    public Object time(ProceedingJoinPoint pjp, Timed timed) throws Throwable {
        long start = System.nanoTime();
        log.add("timer start [" + timed.value() + "]");
        try {
            return pjp.proceed();          // call the next advice / the real method
        } finally {
            long ms = (System.nanoTime() - start) / 1_000_000;
            log.add("timer stop [" + timed.value() + "] took>=20ms? " + (ms >= 20));
        }
    }
}
```

- `@annotation(timed)` uses the *parameter name*; its type `Timed` tells AspectJ which annotation to look for, and the instance is passed in (`timed.value()`).
- The advice **must return** what `proceed()` returned. An `@Around` method declared `void`, or one that forgets `return`, makes every advised method return `null` — a classic bug.
- Declare `throws Throwable` and let exceptions pass unless you intend to handle them.

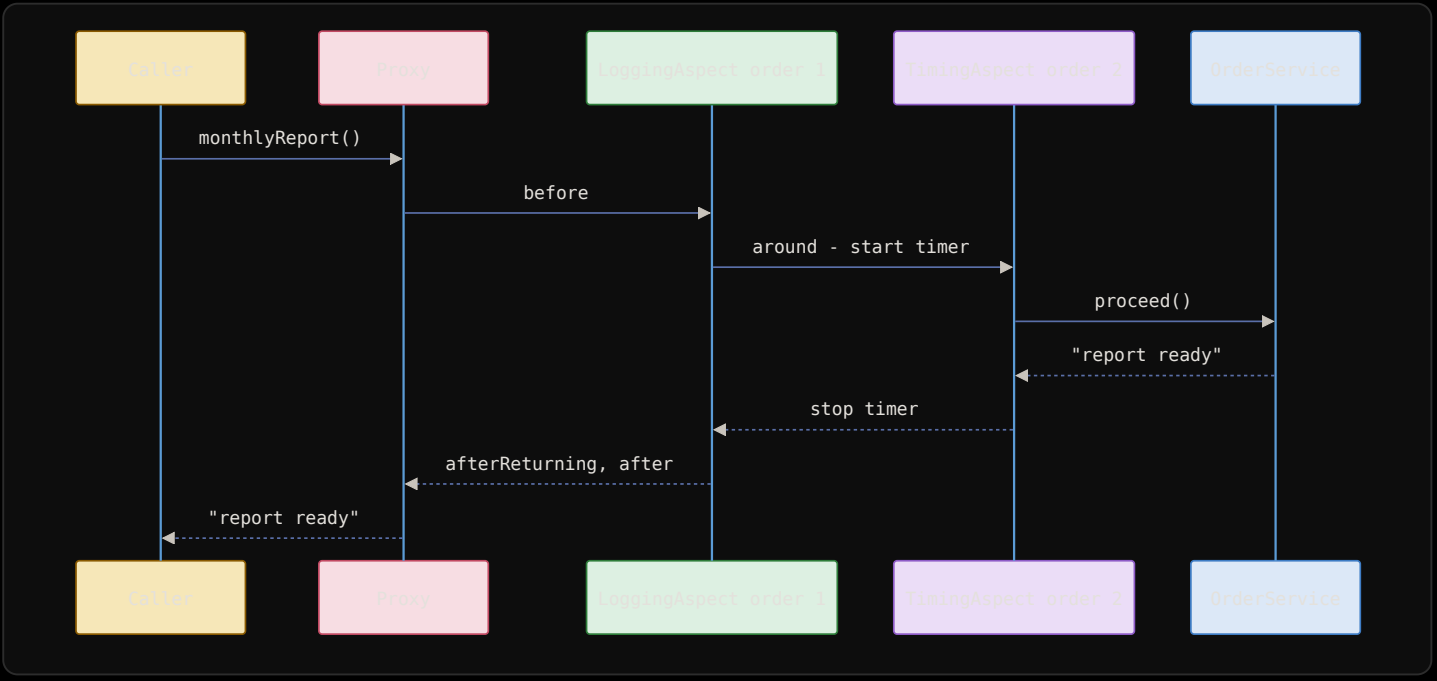
Ordering several aspects

Spring orders aspects on the same method with `@Order` (or `Ordered`). Lower value = higher precedence = **outer** layer: first in, last out. Without an explicit order the sequence is undefined.

```java
@Test
void twoAspectsNestByOrder() throws Exception {
    orders.monthlyReport();
    log.lines().forEach(l -> System.out.println("  " + l));
    assertThat(log.lines()).containsExactly(
            "before monthlyReport args=[]",        // @Order(1) LoggingAspect: outer
            "timer start [monthly-report]",        // @Order(2) TimingAspect: inner
            "timer stop [monthly-report] took>=20ms? true",
            "afterReturning monthlyReport -> report ready",
            "after monthlyReport");
}
```

Output:

```
before monthlyReport args=[]
timer start [monthly-report]
timer stop [monthly-report] took>=20ms? true
afterReturning monthlyReport -> report ready
after monthlyReport
```



Spring's infrastructure advisors (transactions, caching, security) have orders too; a retry aspect must be *outside* the transaction aspect so each retry gets a fresh transaction.

Common mistakes

- **Forgetting** `@Component` on the aspect → nothing happens, no error.
- `@Around` **without** `return pjp.proceed()` → methods return `null`, or the real method never runs.
- **Swallowing exceptions** in `@Around` → callers think a failure succeeded, and an outer `@Transactional` doesn't roll back.
- **Pointcut too broad** (`execution(* *(..))`) → Spring tries to proxy every bean, including infrastructure; startup slows or fails oddly. Restrict to your packages.
- **Relying on unordered aspects** → inconsistent behaviour between runs or environments.

4.3 Pointcut expressions

What it is

A pointcut expression is a tiny query language (from AspectJ) that selects methods. Spring AOP supports a subset of AspectJ's **pointcut designators** (PCDs):

Designator	Matches method executions ...	Example
`execution(...)`	whose signature matches a pattern (the main one)	`execution(* com.shop.service.*.*(..))`
`within(type-pattern)`	declared in matching types/packages	`within(com.shop..*)`
`this(Type)`	where the **proxy** is an instance of `Type`	`this(com.shop.Auditable)`
`target(Type)`	where the **target** is an instance of `Type`	`target(com.shop.Repository)`
`args(...)`	whose runtime arguments match (and binds them)	`args(id, ..)`
`@annotation(A)`	on methods annotated with `A`	`@annotation(com.shop.Timed)`
`@within(A)`	in classes annotated with `A`	`@within(org.springframework.stereotype.Service)`
`@target(A)`	where the target's runtime class has `A`	`@target(com.shop.Audited)`
`@args(A)`	whose argument types carry annotation `A`	`@args(com.shop.Sensitive)`
`bean(name)`	on beans with a matching name — **Spring-only**, not AspectJ	`bean(*Service)`

The `execution` pattern, piece by piece

```
execution( modifiers?  return-type  declaring-type?.method-name(params)  throws? )
execution( public      *            com.shop.service..*Service.find*(Long, ..) )
```

Part	Meaning in the example
`public`	optional; only public methods (with CGLIB proxies, protected/package-private methods can also be matched)
`*`	any return type
`com.shop.service..`	the package `com.shop.service` **or any sub-package** (`..` in a type pattern)
`*Service`	any type whose name ends in `Service`
`find*`	method names starting with `find`
`(Long, ..)`	first parameter `Long`, then zero or more of anything (`..` in a parameter list). `()` = no params, `(*)` = exactly one param of any type

Combining and reusing

Pointcuts combine with `&&`, `||`, `!` and can reference named pointcuts from other classes:

```
public class CommonPointcuts {
    @Pointcut("within(com.shop..web..*)")     public void webLayer() {}
    @Pointcut("within(com.shop..service..*)") public void serviceLayer() {}
    @Pointcut("execution(* com.shop..*Repository.*(..))") public void dataAccess() {}
}

@Before("com.shop.aop.CommonPointcuts.serviceLayer() && args(id, ..)")
public void checkId(Long id) { /* id is bound from the first argument */ }
```

The chapter's `TracingAspect` uses a `within` pointcut that excludes the aspect's own class:

```
@Before("within(ch04.proxy..*) && !within(ch04.proxy.TracingAspect)")
public void trace(JoinPoint jp) { log.add("trace " + jp.getSignature().toShortString()); }
```

Performance tip

Matching is done once per bean method at startup and cached, so runtime cost is small — but **kinded** (`execution`) and **scoping** (`within`) designators are cheapest; contextual ones (`this`, `target`, `args`, `@annotation`) are more expensive alone. Include a `within(...)` scope whenever possible.

Common mistakes

- `com.shop.*` matches only types directly in `com.shop` (one package level — `execution(* com.acme.*.*(..))` misses `com.acme.billing.InvoiceService`); use `com.shop..*` for sub-packages.
- `execution(* save(*))` matches exactly one parameter, not "any parameters" — that is `(..)`.
- A custom trigger annotation only on an **interface** method: `@annotation(...)` matches the method actually executed on the bean class, so annotate the implementation. (Spring's own annotations such as `@Transactional` use a more forgiving lookup.)

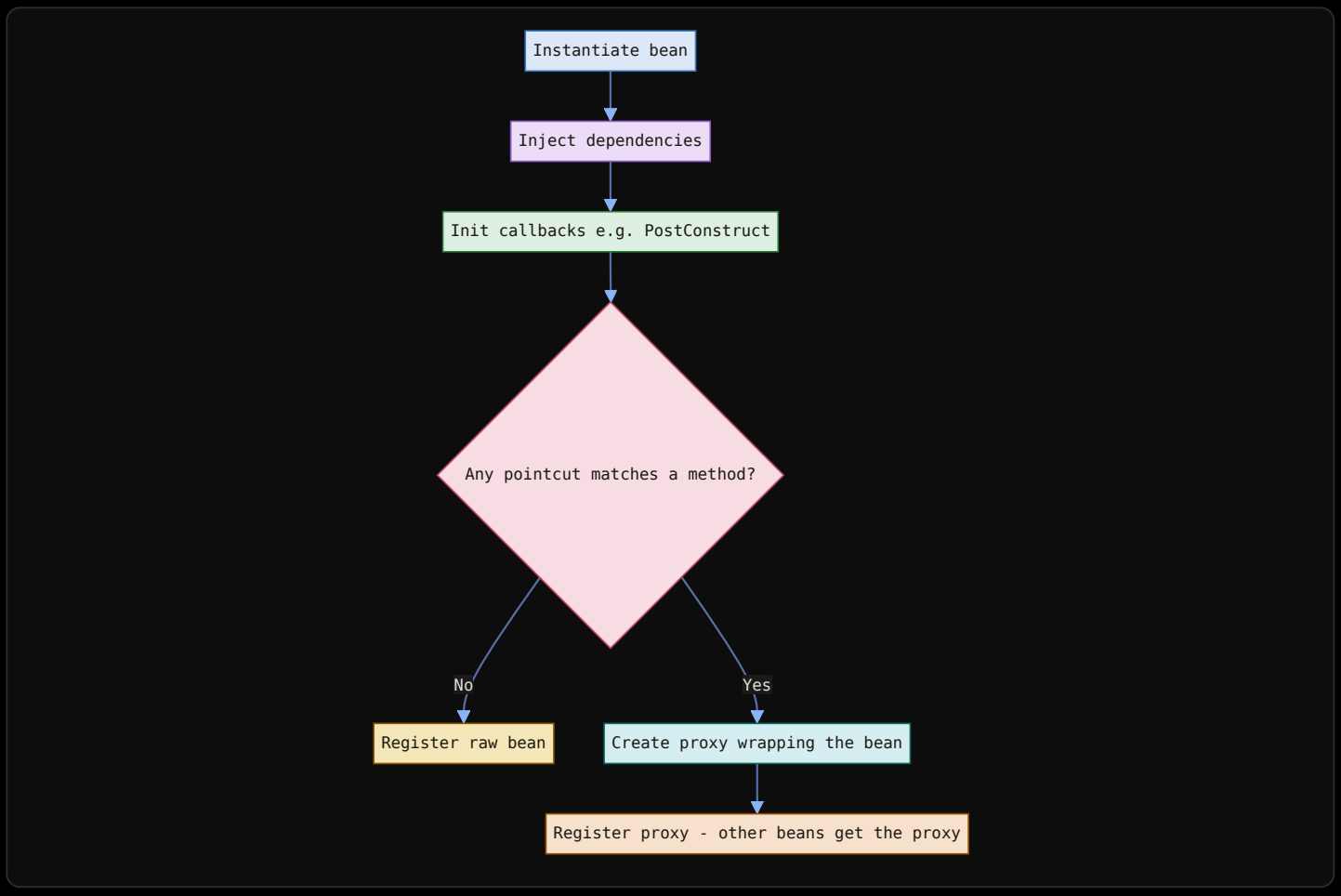
4.4 How Spring AOP works: proxies

What it is

A **proxy** has the same type as your bean (callers can't tell the difference) but forwards each call through the advice chain to the real object. Spring creates proxies **at startup** in a `BeanPostProcessor` (§2.8); for `@Aspect` support that is `AnnotationAwareAspectJAutoProxyCreator`.

How it works

1. The container creates the real bean (constructor, injection, `@PostConstruct`).
2. In `postProcessAfterInitialization` the auto-proxy creator asks: *does any advisor's pointcut match any method of this bean's class?*
3. No: the raw bean is registered. Yes: it builds a proxy with the matching advisors, and **the proxy** is registered and injected everywhere.
4. At call time the proxy walks a chain of interceptors (advice objects that act before and after passing the call on); the last link calls the real method.



Two kinds of proxy

	JDK dynamic proxy	CGLIB proxy
Built with	`java.lang.reflect.Proxy` (part of the JDK)	bytecode generation of a **subclass** (Spring ships a repackaged CGLIB)
Requires	the bean implements at least one interface	the class is not `final` and has a usable constructor
The proxy *is a*	implementation of the interfaces only	subclass of the bean's class
Can be injected as the concrete class?	**No** — `BeanNotOfRequiredTypeException`	Yes
Methods it can intercept	interface methods (public)	non-final, non-private methods (public, protected, package-private within the same package)
Class name you'll see	`jdk.proxy2.$Proxy83`	`OrderService$$SpringCGLIB$$0`

Which one does Spring pick?

- **Plain Spring Framework:** JDK proxy if the bean implements an interface, otherwise CGLIB (unless `proxyTargetClass = true`).
- **Spring Boot:** `spring.aop.proxy-target-class` defaults to `true`, so Boot uses **CGLIB for everything**, even beans with interfaces — so injecting by concrete class never fails. Set it to `false` for plain-Spring behaviour.
- **Framework 7:** `@Proxyable` on a bean class or `@Bean` method overrides the global default for that bean: `@Proxyable(ProxyType.INTERFACES)`, `@Proxyable(ProxyType.TARGET_CLASS)`, or `@Proxyable(interfaces = ...)` to expose only some interfaces. Framework 7 also merges the proxy settings of the various `@Enable...` annotations into one global default.

Proved in `ProxyTypesTest.java`:

```
static String describe(Object bean) {
    String kind = AopUtils.isJdkDynamicProxy(bean) ? "JDK dynamic proxy"
                : AopUtils.isCglibProxy(bean) ? "CGLIB proxy" : "not a proxy";
    return kind + " -> " + bean.getClass().getName();
}

@Test
void whatKindOfProxyDidSpringCreate() {
    System.out.println("OrderService (no interface):       " + describe(orders));
    System.out.println("DefaultGreetingService (interface): " + describe(greeting));
    System.out.println("@Proxyable(INTERFACES) bean:        " + describe(interfaceOnly));
    assertThat(AopUtils.isCglibProxy(orders)).isTrue();
    assertThat(AopUtils.isCglibProxy(greeting)).isTrue();     // Boot default: proxy-target-class=true
    assertThat(AopUtils.isJdkDynamicProxy(interfaceOnly)).isTrue();
    assertThat(interfaceOnly).isNotInstanceOf(DefaultGreetingService.class);
}
```

where the third bean is:

```
@Service
@Proxyable(ProxyType.INTERFACES)       // ask for a JDK dynamic proxy
public class InterfaceOnlyGreetingService implements GreetingService { ... }
```

Output:

```
OrderService (no interface):       CGLIB proxy -> ch04.aop.OrderService$$SpringCGLIB$$0
DefaultGreetingService (interface): CGLIB proxy -> ch04.proxy.DefaultGreetingService$$SpringCGLIB$$0
@Proxyable(INTERFACES) bean:        JDK dynamic proxy -> jdk.proxy2.$Proxy83
```

A proxy by hand: `java.lang.reflect.Proxy`

A JDK dynamic proxy without Spring: give the JDK interfaces and an `InvocationHandler`; every call lands in `invoke(proxy, method, args)`:

```
GreetingService target = new DefaultGreetingService();
InvocationHandler handler = (proxy, method, args) -> {
    System.out.println("  [handler] before " + method.getName());
    Object result = method.invoke(target, args);      // delegate to the real object
    System.out.println("  [handler] after  " + method.getName());
    return result;
};
GreetingService proxy = (GreetingService) Proxy.newProxyInstance(
        GreetingService.class.getClassLoader(), new Class<?>[] {GreetingService.class}, handler);
String out = proxy.greet("Ada");
System.out.println("result = " + out + ", proxy class = " + proxy.getClass().getSimpleName());
```

```
  [handler] before greet
  [handler] after  greet
result = Hello, Ada, proxy class = $Proxy137
```

Spring's JDK proxies are exactly this, with an `InvocationHandler` that runs the advisor chain.

Spring's `ProxyFactory`

Spring exposes its proxy machinery as `ProxyFactory` (handy in tests or libraries). The advice is an AOP Alliance (a shared set of AOP interfaces) `MethodInterceptor` — the low-level interface all Spring advice is converted into:

```
MethodInterceptor upper = inv -> ((String) inv.proceed()).toUpperCase();

ProxyFactory jdk = new ProxyFactory(new DefaultGreetingService());
jdk.addInterface(GreetingService.class);   // interfaces known + proxyTargetClass=false -> JDK
jdk.addAdvice(upper);
GreetingService p1 = (GreetingService) jdk.getProxy();

ProxyFactory cglib = new ProxyFactory(new DefaultGreetingService());
cglib.setProxyTargetClass(true);           // force a subclass
cglib.addAdvice(upper);
DefaultGreetingService p2 = (DefaultGreetingService) cglib.getProxy();
```

```
factory 1: JDK dynamic proxy -> jdk.proxy2.$Proxy83 says HELLO, BOB
factory 2: CGLIB proxy -> ch04.proxy.DefaultGreetingService$$SpringCGLIB$$1 says HELLO, BOB
```

(The JDK reuses one proxy class per interface set, hence `$Proxy83` twice. The numbers depend on how many proxy classes were generated before, so your run may differ.)

Trade-offs

Concern	JDK proxy	CGLIB proxy
Inject by concrete class	fails	works
`final` classes / methods	irrelevant (interface-based)	class cannot be proxied / method silently not advised
Startup cost	tiny	generates a class per bean (small, cached)
Constructor	not involved	Spring uses Objenesis (creates objects without calling a constructor), so the target's constructor is **not** run again for the proxy
Recommended	when you program to interfaces and want only interface methods exposed	Boot's default; works for classes without interfaces

E.g. in a Boot app a `@Transactional` `PaymentService implements Payments` gets a CGLIB proxy; in plain Spring it gets a JDK proxy, and injecting `PaymentServiceImpl` fails because the proxy only implements `Payments`.

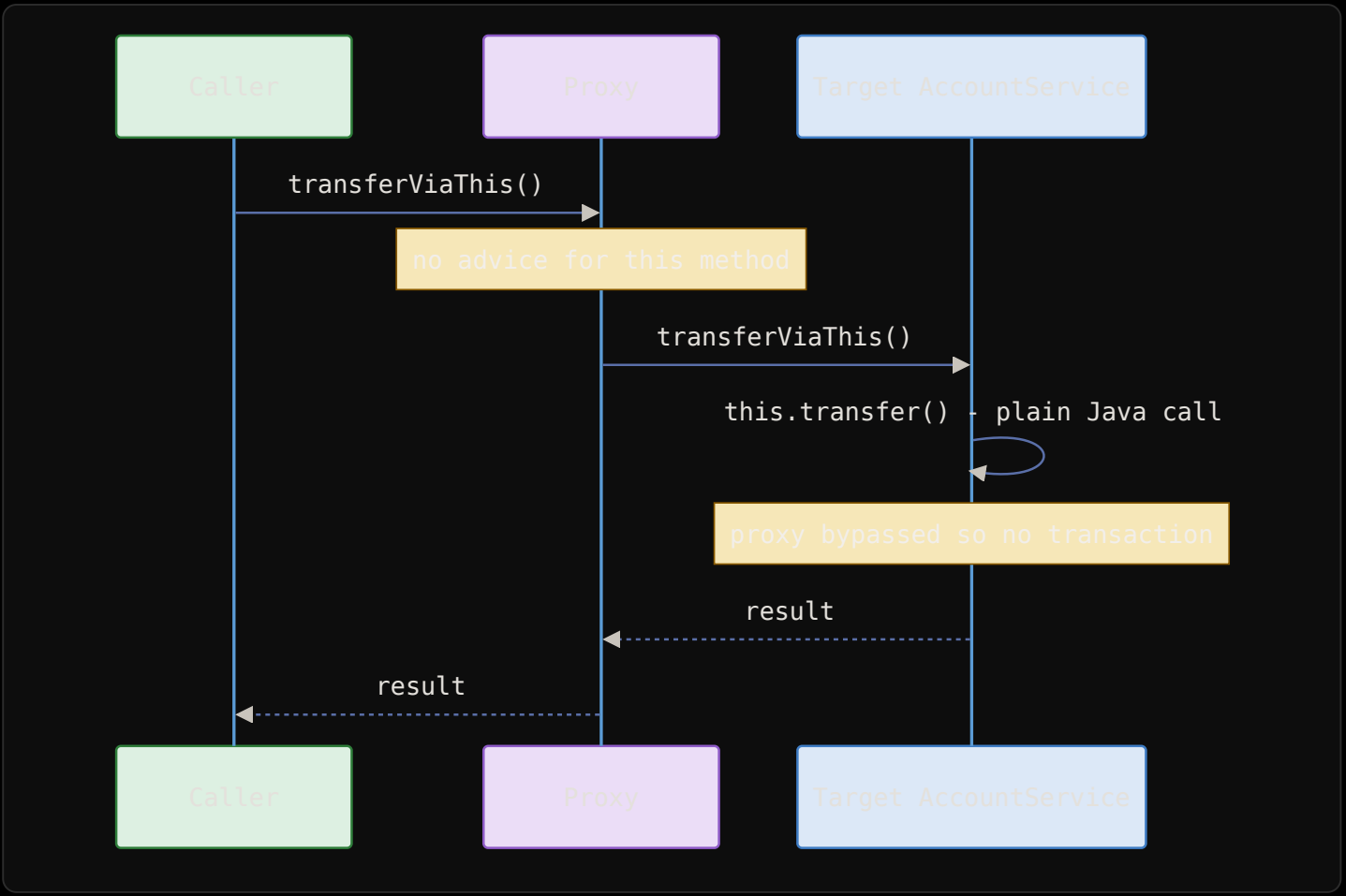
4.5 Proxy limitations — proved

Because the advice lives in the proxy, **advice runs only when a call passes through the proxy**. Every limitation below follows from that.

Self-invocation

What it is

Inside a bean, `this` is the **target object**, not the proxy. `this.someMethod()` (or just `someMethod()`) is a plain Java call that never touches the proxy: no transaction, caching, async, retry or security check.



Proof 1 — `@Transactional`

```
@Service
public class AccountService {
    private final ObjectProvider<AccountService> self;   // lazy handle to *our own proxy*

    public AccountService(ObjectProvider<AccountService> self) { this.self = self; }

    @Transactional
    public String transfer() {
        // true only if a transaction proxy started a transaction before we got here
        return "transaction active = " + TransactionSynchronizationManager.isActualTransactionActive();
    }
    public String transferViaThis()      { return this.transfer(); }             // skips the proxy
    public String transferViaSelfProxy() { return self.getObject().transfer(); }  // goes through it
}
```

`TransactionSynchronizationManager.isActualTransactionActive()` reports whether a real transaction is bound to the current thread (Spring tracks transactions per thread). The project has H2 (in-memory database) + `spring-boot-starter-jdbc`, so Boot configured a `DataSourceTransactionManager`.

Proof 2 — `@Cacheable`

```
@Cacheable("prices")                    // proxy checks the "prices" cache before calling us
public int price(String sku) {
    realCalls.incrementAndGet();         // counts how often the real method body runs
    return sku.length() * 100;
}
public int priceTwiceViaThis(String sku) {
    return this.price(sku) + this.price(sku);   // self-invocation: cache never consulted
}
```

Proof 3 — `@Async`

```
@Async                                  // proxy hands the call to a TaskExecutor thread
public CompletableFuture<String> build() {
    return CompletableFuture.completedFuture(Thread.currentThread().getName());
}
public CompletableFuture<String> buildViaThis() {
    return this.build();                // runs on the CALLER's thread
}
```

The test (`SelfInvocationTest.java`) asserts all three:

```
caller -> proxy -> transfer():        transaction active = true
caller -> proxy -> this.transfer():   transaction active = false
caller -> proxy -> self.transfer():   transaction active = true
2 external calls -> real method ran 1 time(s)
2 internal calls -> real method ran 2 time(s)
caller thread:          main
@Async through proxy:   task-1
@Async via this.build(): main
```

`task-1` is a thread of Boot's auto-configured `applicationTaskExecutor` (a thread pool of reusable workers). Via `this`, the "async" method ran synchronously on `main` — no error, no warning.

Fixes, best first

1. **Move the method to another bean.** If `transfer()` must be transactional on its own, it probably belongs in a separate service anyway.
2. **Call through the proxy by self-injection** — `ObjectProvider<T>` or `@Autowired @Lazy MyService self` (shown above). Works but surprises readers; comment it.
3. **`AopContext.currentProxy()`** — needs `@EnableAspectJAutoProxy(exposeProxy = true)` and couples code to Spring AOP; the docs discourage it.
4. **Annotate the outer (entry-point) method** instead, if that's what you meant.
5. **AspectJ weaving** (§4.8) — advice woven into bytecode, so `this` calls are advised. Rarely worth the build complexity.

`final` methods on CGLIB proxies

A CGLIB proxy is a subclass, which **cannot override a `final` method** — so a `final` method runs **on the proxy object itself**. Worse, Spring creates proxy instances without running your constructor, so the proxy's fields are `null`.

```
@Service
public class PricingService {
    private final List<String> currencies;
    public PricingService() { this.currencies = List.of("EUR", "USD"); }

    public String currencies()             { return "currencies=" + currencies; }
    public final String currenciesFinal()  { return "currencies=" + currencies; }
}
```

`TracingAspect` advises every method of this class, so it is proxied:

```
String normal = pricing.currencies();
String viaFinal = pricing.currenciesFinal();
System.out.println("normal method: " + normal);
System.out.println("final method:  " + viaFinal);
System.out.println("advice log:    " + log.lines());
assertThat(viaFinal).isEqualTo("currencies=null");           // proxy's own, never-initialised field
```

```
normal method: currencies=[EUR, USD]
final method:  currencies=null
advice log:    [trace PricingService.currencies()]
```

The final method was not advised **and** read `null`. In real code: a mysterious `NullPointerException` on a field that "definitely was injected". Kotlin classes are `final` by default, which is why Kotlin Spring projects use the `kotlin-spring` (all-open) compiler plugin to make annotated classes non-final.

A `final` **class** cannot be subclassed at all; if it needs advice and has no interface, startup fails saying CGLIB cannot subclass it.

Other things proxies cannot advise

Case	Why	What happens
`private` methods	not visible to a subclass, not on an interface	advice silently ignored
`static` methods	not called on an instance	ignored
Calls inside the **constructor** or `@PostConstruct`	proxy does not exist yet / you are on the target	ignored (e.g. no transaction, no async for `@Transactional` / `@Async` called from `@PostConstruct`)
Objects created with `new`	not Spring beans, never proxied	ignored
Field access	Spring AOP supports only method execution join points	not possible (needs AspectJ)
Methods only on the class, with a JDK proxy	the proxy implements only interfaces	not reachable through the proxy

For `@Transactional`: since Framework 6.0, `protected` and package-private methods can be transactional with CGLIB proxies; with interface-based proxies they must be `public`, and `private` never works.

In the interview

30-second answer: "Spring AOP is proxy-based. At startup a BeanPostProcessor wraps any bean matching a pointcut in a proxy — CGLIB by default in Boot, JDK dynamic proxy for interface-only proxying — and other beans get the proxy. Advice runs only when a call goes through the proxy, so `this.method()` calls, private or final methods, and calls from constructors skip `@Transactional`, `@Cacheable`, `@Async` and custom aspects. The fix is usually to move the method to another bean, or call through an injected reference to the proxy."

- *"My `@Transactional` method doesn't roll back — why?"* → Self-invocation; private/final method; the exception is checked (declared with `throws`, e.g. `IOException` — rollback defaults to unchecked `RuntimeException`s only, see Chapter 7); exception caught inside the method; bean created with `new`.
- *"How can you see if a bean is proxied?"* → `AopUtils.isAopProxy(bean)` / `isCglibProxy` / `isJdkDynamicProxy`, or print `bean.getClass()` and look for `$$SpringCGLIB$$` or `$Proxy`.

4.6 Spring features built on proxies

Many annotations in this book are "AOP you did not write". They use infrastructure advisors, not `@Aspect` classes, but obey **all** the rules of §4.5 (e.g. `@PreAuthorize` does not protect a `private` method).

Annotation	Switched on by	Interceptor / processor	Chapter
`@Transactional`	Boot auto-config (`@EnableTransactionManagement` in plain Spring)	`TransactionInterceptor`	7
`@Cacheable`, `@CachePut`, `@CacheEvict`	`@EnableCaching`	`CacheInterceptor`	9
`@Async`	`@EnableAsync`	`AsyncExecutionInterceptor`	14
`@Retryable`, `@ConcurrencyLimit` (Framework 7)	`@EnableResilientMethods`	`RetryAnnotationBeanPostProcessor`, `ConcurrencyLimitBeanPostProcessor`	4.7 below
`@PreAuthorize`, `@PostAuthorize`, `@Secured`	`@EnableMethodSecurity`	`AuthorizationManagerBeforeMethodInterceptor`, ...	10
`@Validated` on a class + constraint annotations on method parameters	Boot auto-config (`MethodValidationPostProcessor`)	`MethodValidationInterceptor`	5
`@Observed` (Micrometer, Spring's metrics library)	an `ObservedAspect` bean	an `@Aspect`	15
Spring Data repository interfaces	`@EnableJpaRepositories` / Boot	JDK proxy generated for the interface	8
`@Configuration` classes (full mode)	always	CGLIB subclass intercepting `@Bean` method calls (not AOP advice, same technique)	2

Order matters when several apply. With `@Retryable` + `@Transactional` on one method, if the transaction interceptor is outside the retry interceptor, all attempts share one transaction already marked rollback-only (doomed to be undone) after the first failure. Put retries one layer up, or set orders explicitly (`@EnableTransactionManagement(order = ...)`, `@EnableResilientMethods(order = ...)`).

4.7 Framework 7 core resilience: `@Retryable` and `@ConcurrencyLimit`

What it is

Spring Framework **7.0** (November 2025) added a small resilience toolkit (keeping an app working when its dependencies fail) to the core, in `org.springframework.resilience.annotation` and `org.springframework.core.retry`:

- **`@Retryable`** — call a method again when it throws, with back-off delay.
- **`@ConcurrencyLimit`** — at most N threads inside a method at a time (a *bulkhead*: a limit that stops one overloaded part from sinking the whole app, named after a ship's watertight walls).
- **`RetryTemplate`** / **`RetryPolicy`** — the same retry logic, programmatically.

They replace most uses of the separate **Spring Retry** project; Boot 4 removed dependency management for Spring Retry. They are **not** a full resilience library: no circuit breaker (stops calling a failing service for a while), rate limiter (caps calls per second) or time limiter (gives up after a deadline) — use Resilience4j or Spring Cloud Circuit Breaker (Chapter 16, circuit breaker notes).

Why it matters

Remote calls fail *transiently*: a timeout, a 503 ("service unavailable") during a deploy, a dropped connection. Retrying once or twice with a short, growing delay turns many such failures into successes. And with **virtual threads** (very cheap JVM-managed threads — Java book 15.4) there is no thread-pool size naturally capping how many callers hit a slow downstream service — `@ConcurrencyLimit` puts that cap back.

How to switch it on

Add `@EnableResilientMethods` to a configuration class. As of Boot 4.1 there is no auto-configuration for it, so without it `@Retryable` is silently ignored.

```
@SpringBootApplication
@EnableCaching          // turns on the @Cacheable proxy
@EnableAsync            // turns on the @Async proxy
@EnableResilientMethods // Framework 7: turns on @Retryable and @ConcurrencyLimit
public class AopApplication { ... }
```

(Alternatively register `RetryAnnotationBeanPostProcessor` and/or `ConcurrencyLimitBeanPostProcessor` beans.)

`@Retryable` attributes (Framework 7.0)

Attribute	Default	Meaning
`includes` / `value`	empty = all exceptions	exception types to retry (also matches **nested causes**)
`excludes`	empty	exception types never to retry
`predicate`	none	a `MethodRetryPredicate` class for custom decisions, applied after includes/excludes
`maxRetries`	`3`	retries **after** the first call → total attempts = 1 + `maxRetries`
`delay`	`1000` ms	wait before the first retry
`multiplier`	`1.0`	each delay = previous × multiplier (exponential back-off when > 1)
`maxDelay`	`Long.MAX_VALUE`	cap for the growing delay
`jitter`	`0`	random variation per delay, so many clients don't retry in lock-step
`timeout`	`0` = none	total time budget for all attempts including delays (since 7.0.2)
`timeUnit`	`MILLISECONDS`	unit for the numeric delay attributes
`...String` variants	—	`maxRetriesString`, `delayString`, ... accept `${...}` placeholders and SpEL

It also works on methods returning `Mono` / `Flux` (Project Reactor's types for 0–1 and 0–N future values; the retry is applied to the reactive pipeline), and publishes a `MethodRetryEvent` per failed attempt, observable with `@EventListener`.

Example: a flaky client

```
@Service
public class InventoryClient {
    // up to 3 retries (4 attempts in total), waiting 10 ms, then 20 ms, then 40 ms
    @Retryable(includes = TransientException.class, maxRetries = 3, delay = 10, multiplier = 2)
    public int stockFor(String sku) {
        attempts.add("attempt " + (attempts.size() + 1));
        if (failuresLeft-- > 0) {
            throw new TransientException("inventory timed out");
        }
        return 42;
    }

    // IllegalArgumentException is not in 'includes', so it is never retried
    @Retryable(includes = TransientException.class, maxRetries = 3, delay = 10)
    public int stockForBadSku(String sku) {
        attempts.add("attempt " + (attempts.size() + 1));
        throw new IllegalArgumentException("bad sku " + sku);
    }
}
```

The tests (`ResilienceTest.java`) make it fail twice (recovers), ten times (gives up), and with a non-retryable exception:

```
inventory.failNextCalls(2);
int stock = inventory.stockFor("SKU-1");           // succeeds on attempt 3

inventory.failNextCalls(10);
assertThatThrownBy(() -> inventory.stockFor("SKU-1"))
        .isInstanceOf(TransientException.class);    // the LAST original exception
assertThat(inventory.attempts()).hasSize(4);        // 1 call + 3 retries
```

```
stock = 42 after [attempt 1, attempt 2, attempt 3]
gave up after [attempt 1, attempt 2, attempt 3, attempt 4]
IllegalArgumentException -> [attempt 1]
```

When all attempts fail, the caller gets the **last original exception**, not a wrapper — existing `catch` blocks keep working.

`@ConcurrencyLimit`

```
@ConcurrencyLimit(2)                        // default policy BLOCK: extra callers wait
public void print(String doc) throws InterruptedException {
    int now = inside.incrementAndGet();
    maxInside.accumulateAndGet(now, Math::max);
    Thread.sleep(50);                        // hold the "printer" for a while
    inside.decrementAndGet();
}

@ConcurrencyLimit(limit = 1, policy = ConcurrencyLimit.ThrottlePolicy.REJECT)  // 7.0.3+
public void printUrgent(String doc) throws InterruptedException {
    Thread.sleep(100);
}
```

- `value` / `limit`: maximum concurrent invocations **per bean instance**; `1` makes the method behave like a lock on the bean.
- `limitString`: placeholder/SpEL variant, e.g. `"${printer.max-concurrent}"`.
- `policy` (since 7.0.3): `BLOCK` (default) makes extra callers wait; `REJECT` throws `InvocationRejectedException` (a `RejectedExecutionException` subclass) immediately.

6 virtual threads call `print`; a second caller hits `printUrgent` while the first is inside:

```
6 callers, max inside print() at once = 2
second caller -> InvocationRejectedException
```

Programmatic: `RetryTemplate`

For a retry scope of a few lines, or policies chosen at runtime:

```
RetryPolicy policy = RetryPolicy.builder()
        .maxRetries(4)
        .delay(Duration.ofMillis(5))
        .multiplier(2)
        .includes(TransientException.class)
        .build();
RetryTemplate retry = new RetryTemplate(policy);

String result = retry.execute(() -> {
    if (calls.incrementAndGet() < 3) throw new TransientException("busy");
    return "done on call " + calls.get();
});
```

```
RetryTemplate -> done on call 3
```

`execute(...)` throws a checked `RetryException` on exhaustion holding every failure (`getExceptions()`, `getLastException()`, `getRetryCount()`); `invoke(...)` rethrows the last original exception unchecked. A `RetryListener` can observe each attempt (logging, metrics).

Trade-offs: which retry tool?

	Framework 7 `@Retryable` / `RetryTemplate`	Spring Retry (`org.springframework.retry`)	Resilience4j
Where	built into `spring-context` / `spring-core`	separate library, no longer managed by Boot 4	separate library + Spring Boot integration
Retry + back-off + jitter	yes	yes	yes
Recovery method (`@Recover`)	no — catch the exception at the caller	yes	fallback methods
Circuit breaker, rate limiter, time limiter	no	basic circuit breaker only	yes, all
Bulkhead	`@ConcurrencyLimit`	no	semaphore and thread-pool bulkheads
Metrics	via `RetryListener` / `MethodRetryEvent`	listeners	built-in Micrometer metrics
Choose when	simple retries / limits, no extra dependency	legacy code	circuit breaking and rich resilience policies

When not to use it

- **Non-idempotent operations** (charging a card, sending an email) — a retry after a timeout may do the work twice.

  > **Key term — idempotent:** doing it twice has the same effect as doing it once. "Set the price to 10" is idempotent; "add 10 to the balance" is not. An **idempotency key** is a unique id the caller sends so the server can recognise and skip a repeat. See System Design Fundamentals and the Retry pattern notes. Retry only operations safe to repeat, or make them idempotent first.

- **Errors that won't go away** — validation errors, 4xx responses (the *request* was wrong), `NullPointerException`. Set `includes` narrowly.
- **Retry at every layer** — gateway, client library and service each retrying 3 times turns one request into 64 downstream calls ("retry storm"). Retry at one layer.
- **Inside a transaction** — see §4.6.

Common mistakes

- Forgetting `@EnableResilientMethods` → no retries, no error.
- Self-invocation → `this.stockFor()` is not retried.
- Reading `maxRetries = 3` as "3 attempts" → it is 4.
- No jitter across a fleet of instances → synchronised retry waves after an outage.

4.8 Spring AOP vs AspectJ

What it is

AspectJ is the original, full AOP language for Java. Spring borrowed its annotations and pointcut syntax (hence `org.aspectj.lang.annotation.*` imports), but by default does **not** use the AspectJ weaver — it uses its own proxies and only AspectJ's library to *parse and match* pointcuts.

	Spring AOP (default)	AspectJ weaving
How advice is applied	runtime proxy around a bean	bytecode modified at compile time (`ajc`), post-compile, or load time (LTW — load-time weaving — with a Java agent that changes classes as the JVM loads them)
Join points	method execution on Spring beans only	method call and execution, constructor, field get/set, static initialisers, ... on **any** object
Self-invocation, `final`, `private`	not advised	advised
Objects created with `new`	not advised	advised
Setup	add the starter	AspectJ compiler plugin or `-javaagent`, extra build complexity
Performance	a few extra method calls per advised invocation	practically zero overhead (code inlined into the method)
Good for	95% of enterprise needs: transactions, logging, security	fine-grained or domain-object advice, very hot paths

`@Transactional`, `@Cacheable` and `@Async` can switch to AspectJ mode (`@EnableTransactionManagement(mode = AdviceMode.ASPECTJ)` plus the weaver), removing the self-invocation limitation. Few teams do; extracting a bean is simpler.

When not to use AOP at all

- **Business logic** — a domain rule ("orders over 1000 need approval") belongs explicitly in the service; hidden advice is hard to follow and test.
- **When a decorator (a wrapper object adding behaviour), filter or interceptor fits** — HTTP concerns belong in servlet filters / `HandlerInterceptor` (Chapter 5); client concerns in `ClientHttpRequestInterceptor` (Chapter 6).
- **Very broad pointcuts** — "log every method" is noise and cost; use tracing/metrics tools (Chapter 15).

Real-world examples

- **Spring itself**: transactions, caching, security, async, retries.
- **Micrometer** `@Timed` / `@Observed` — aspects recording metrics/traces.
- **Audit logging** in banking/insurance: an `@Audited` annotation + `@AfterReturning` advice writing who-did-what records.
- **Multi-tenant routing** (one app serving many customer organisations with separate data): an `@Around` aspect sets a tenant id in a `ThreadLocal`/scoped value before repository calls and clears it after.
- **Feature flags / rate limits** on specific methods via custom annotations.

4.9 Common mistakes (whole chapter)

Symptom	Likely cause	Fix
Aspect never runs	aspect is not a bean; starter missing; pointcut doesn't match	add `@Component`; add `spring-boot-starter-aspectj`; test the pointcut
`@Transactional` / `@Cacheable` / `@Async` / `@Retryable` ignored	self-invocation; private/final method; bean made with `new`; missing `@Enable...`	extract to another bean; make method public and non-final; inject the bean; enable the feature
`NullPointerException` on an injected field inside a method	`final` method on a CGLIB proxy runs on the proxy	remove `final`
`BeanNotOfRequiredTypeException`	JDK proxy injected where the concrete class is expected	inject the interface, or use CGLIB (Boot default)
Advised method returns `null`	`@Around` advice doesn't return `proceed()`'s result	`return pjp.proceed();`
Slow startup / weird errors after adding an aspect	pointcut matches infrastructure beans	scope with `within(com.yourapp..*)`
Retry happens inside a failing transaction	transaction interceptor wraps the retry interceptor	retry at a higher layer or set advisor orders

---

4.10 In the interview

Q: What is AOP and how does Spring implement it? AOP modularises cross-cutting concerns such as logging, transactions and security. You write an aspect containing advice and a pointcut; Spring applies it at runtime by wrapping matching beans in proxies created by a `BeanPostProcessor`. Callers get the proxy, which runs the advice chain and then the real method.

- *Follow-up: advice types?* → `@Before`, `@AfterReturning`, `@AfterThrowing`, `@After`, `@Around`. Only `@Around` controls whether the method runs and what it returns; `@AfterThrowing` doesn't stop the exception.
- *Follow-up: order two aspects?* → `@Order` / `Ordered`; lower value is the outer layer.
- *Follow-up: join point in Spring AOP?* → Only method execution on a Spring bean.

Q: JDK dynamic proxy vs CGLIB — which does Spring Boot use? CGLIB for all beans by default (`spring.aop.proxy-target-class=true`), which allows injecting by class. Plain Spring uses JDK proxies when interfaces exist. Framework 7 adds `@Proxyable` to override per bean.

- *Follow-up: limits of CGLIB?* → No `final` classes; `final`/`private` methods are not advised; `final` methods even run on the uninitialised proxy instance.

Q: Why might `@Async` / `@Transactional` not work? → Self-invocation (most common), not a Spring bean, private/final method, missing `@EnableAsync`/transaction manager, or for transactions: checked exception / exception swallowed. (Details and proofs in §4.5.)

Q: What's new for resilience in Spring Framework 7? → `@Retryable` (delay, multiplier, jitter, maxDelay, includes/excludes, reactive support; attempts = 1 + `maxRetries`, last exception rethrown), `@ConcurrencyLimit` (bulkhead, BLOCK or REJECT), enabled with `@EnableResilientMethods`; plus `RetryTemplate`/`RetryPolicy` in `spring-core`. No circuit breaker — use Resilience4j.

- *Follow-up: danger of retries?* → Duplicate side effects on non-idempotent operations and retry storms; retry only transient errors, add jitter, retry in one layer.

Q: Spring AOP vs AspectJ? → Spring AOP: proxies, method execution on beans only, easy. AspectJ: bytecode weaving, any join point, no self-invocation problem, more build complexity.

4.11 Chapter TL;DR

- **AOP** = write a cross-cutting concern once (an **aspect**: **advice** + **pointcut**) and apply it to many **join points** (in Spring: bean method executions).
- `@Aspect` + `@Component`, `spring-boot-starter-aspectj` (Boot 4 name). Five advice types; `@Around` must `return pjp.proceed()`. Order aspects with `@Order` (lower = outer).
- Pointcuts: `execution(...)`, `within(...)`, `@annotation(...)`, `args(...)`, `bean(...)`; `..` means "any sub-package" or "any parameters".
- Spring AOP is **proxy-based**. Boot uses **CGLIB** subclasses by default; JDK proxies implement interfaces only; Framework 7's `@Proxyable` overrides per bean.
- **Advice runs only through the proxy**: self-invocation, `private`, `final`, `static`, constructor/`@PostConstruct` calls and `new` objects are not advised — proved for `@Transactional`, `@Cacheable`, `@Async` and a `final` method.
- `@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize`, `@Validated`, `@Retryable` are all proxy features with the same rules.
- Framework 7 **resilience**: `@EnableResilientMethods`, `@Retryable` (attempts = 1 + `maxRetries`, last exception rethrown), `@ConcurrencyLimit` (BLOCK/REJECT), `RetryTemplate`. No circuit breaker in core.
- Next: Chapter 5 — Spring MVC and REST.

Chapter 5 — Spring MVC and REST APIs

Spring MVC is Spring's web framework for the classic, thread-per-request Servlet stack (one thread handles a request from start to end, waiting during I/O; the Servlet API is the standard Java web-server API, implemented by Tomcat and Jetty). Almost every Boot service exposing an HTTP/JSON API is built on it. This chapter builds a small **book catalogue API** to explain the request flow, mappings, input, validation, status codes and error bodies, content negotiation, Framework 7 API versioning, CORS (§5.10), uploads, pagination, OpenAPI docs (§5.12) and filters/interceptors (§5.11).

Code: `code/mvc-rest/` (the book API) and `code/api-versioning/` — Spring Boot 4.1.1, Framework 7.0, Java 25. All behaviour is proved with `@WebMvcTest` + `MockMvcTester` tests (only the web layer, fake requests in memory); output is real. Testing in depth is Chapter 12; the reactive alternative (WebFlux) is Chapter 13; calling other APIs is Chapter 6.

The project's single web dependency is the Boot 4 starter:

```
<!-- Spring MVC + embedded Tomcat + Jackson 3 (was spring-boot-starter-web before Boot 4) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

5.1 REST in one page

What it is

REST (Representational State Transfer) is an architectural style for HTTP APIs. In practice a "REST API" means:

- The API exposes **resources** (things) at **URLs**: `/api/books`, `/api/books/3`.
- Clients act on them with standard **HTTP methods**.
- Data travels as **representations** (usually JSON) whose format is agreed with headers (`Content-Type`, `Accept`). An HTTP **header** is a `Name: value` line of metadata sent before the body of a request or response.
- Each request carries everything needed to handle it (**stateless** server — it keeps no memory of earlier requests from that client).
- Results are signalled with **HTTP status codes**.

Methods

Method	Meaning	Safe? (no change)	Idempotent? (repeat = same effect)	Typical response
`GET /books`	list/search	yes	yes	`200` + body
`GET /books/3`	read one	yes	yes	`200`, or `404`
`POST /books`	create (server picks the id)	no	**no** — two POSTs create two books	`201 Created` + `Location` header
`PUT /books/3`	replace the whole resource	no	yes	`200` + body, or `204`
`PATCH /books/3`	partial update	no	not necessarily	`200` / `204`
`DELETE /books/3`	delete	no	yes	`204 No Content`, then `404`

Idempotency matters for retries: a client may safely retry a timed-out `PUT` or `DELETE`, but not a `POST` without an idempotency key (a unique id sent with the request so the server can spot and ignore a duplicate; see System Design Fundamentals) (see Chapter 6).

Status codes you will actually use

Code	Name	When
200	OK	success with a body
201	Created	a POST created something; add `Location`
204	No Content	success, nothing to return (DELETE)
400	Bad Request	malformed JSON, invalid fields, bad parameter types
401	Unauthorized	not authenticated (Chapter 10)
403	Forbidden	authenticated but not allowed; also a rejected CORS request
404	Not Found	no such resource / URL
405	Method Not Allowed	URL exists, method doesn't (response has `Allow`)
406	Not Acceptable	cannot produce any format the client accepts
409	Conflict	state conflict, e.g. duplicate or version clash
413	Content Too Large	upload too big
415	Unsupported Media Type	request body format not supported
422	Unprocessable Content	some teams use it for semantic validation errors (Spring uses 400 by default)
500	Internal Server Error	unexpected bug
503	Service Unavailable	overloaded / dependency down

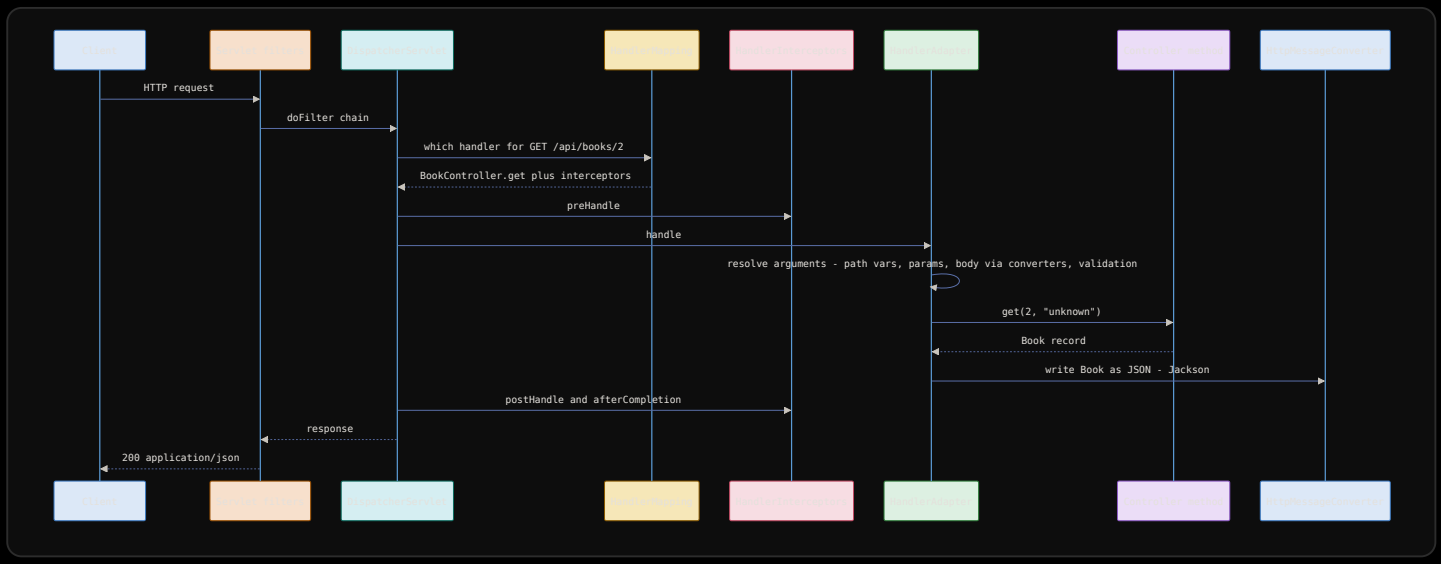
5.2 How a request flows: the `DispatcherServlet`

What it is

Spring MVC is built around one servlet (a class the web server calls to handle HTTP requests), the `DispatcherServlet` — a **front controller**: every request enters through it, and it delegates each step to pluggable components.

Boot registers the `DispatcherServlet` at `/` automatically and starts an embedded **Tomcat 11** (Jakarta Servlet 6.1).

How it works



1. **Servlet filters** run first (security, request ids, compression). They are outside Spring MVC and don't know which controller method will run.

2. The `DispatcherServlet` asks its **`HandlerMapping`s** which handler matches. For annotated controllers, `RequestMappingHandlerMapping` compares path, HTTP method, `consumes`/`produces`, params, headers and — in Framework 7 — the **API version**.
3. **`HandlerInterceptor.preHandle`** runs for the chosen handler.
4. A **`HandlerAdapter`** invokes the method. Before that, **`HandlerMethodArgumentResolver`s** build each argument: `@PathVariable` from the URL, `@RequestParam` from the query string (the `?page=1&size=2` part of a URL), `@RequestBody` via an **`HttpMessageConverter`** (Jackson for JSON), plus validation.
5. The return value is handled by a **`HandlerMethodReturnValueHandler`**: for `@ResponseBody`/`@RestController` it picks an `HttpMessageConverter` based on the `Accept` header (content negotiation) and writes the body.
6. If anything throws, **`HandlerExceptionResolver`s** turn the exception into a response (`@ExceptionHandler` methods, `@ResponseStatus`, Spring's defaults).
7. `postHandle` / `afterCompletion` interceptor callbacks, then filters unwind.

For server-rendered HTML (Thymeleaf, a template engine that fills HTML templates with data), step 5 instead returns a view name that a `ViewResolver` renders; this book focuses on JSON APIs.

5.3 Controllers and request mappings

What it is

A **controller** is a bean whose methods handle requests. `@RestController` = `@Controller` + `@ResponseBody`: every method's return value is written straight into the response body (as JSON), not treated as a view name.

```
@RestController                          // = @Controller + @ResponseBody on every method
@RequestMapping("/api/books")            // common path prefix for the class
public class BookController {

    private final BookRepository repo;

    public BookController(BookRepository repo) { this.repo = repo; }

    @GetMapping
    public PageResponse<Book> list(@RequestParam(defaultValue = "0") @Min(0) int page,
                                   @RequestParam(defaultValue = "2") @Min(1) @Max(50) int size,
                                   @RequestParam(required = false) String author) { ... }

    @GetMapping("/{id:\\d+}")      // regex: only digits, so "/export" never lands here
    public Book get(@PathVariable long id,
                    @RequestHeader(name = "X-Client", defaultValue = "unknown") String client) {
        return repo.findById(id).orElseThrow(() -> new BookNotFoundException(id));
    }

    @PostMapping
    public ResponseEntity<Book> create(
            @Validated({BookRequest.OnCreate.class, Default.class}) @RequestBody BookRequest request) { ... }

    @PutMapping("/{id:\\d+}")
    public Book update(@PathVariable long id, @Validated @RequestBody BookRequest request) { ... }

    @DeleteMapping("/{id:\\d+}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable long id) { ... }
}
```

Mapping annotations

Annotation	Same as
`@GetMapping`	`@RequestMapping(method = GET)`
`@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`	the other methods
`@RequestMapping` on the class	prefix + shared settings for all methods

Useful attributes (all mapping annotations):

Attribute	Narrows the match to requests ...	Example
`path` / `value`	with this URL pattern	`"/{id}"`
`consumes`	whose `Content-Type` matches	`consumes = "application/json"`
`produces`	whose `Accept` allows this type	`produces = "text/csv"`
`params`	having / lacking a parameter	`params = "format=short"`
`headers`	having a header value	`headers = "X-Beta=true"`
`version` (Framework 7)	with a matching API version	`version = "1.1+"`

Path patterns

Spring MVC matches with `PathPattern` (the default since Framework 6 / Boot 3):

Pattern	Matches
`/books/{id}`	one segment captured as `id`
`/books/{id:\d+}`	only if the segment matches the regex (`\d+` = one or more digits; in Java source `"{id:\\d+}"`)
`/files/**`	anything below `/files/`
`/files/{*path}`	the rest of the path, captured (must be at the end)
`/books/*.csv`	`*` = any characters within one segment

Two version-specific facts:

- **Trailing slashes** no longer match by default (since Framework 6.0): `/api/books/` is *not* `/api/books`.
- When several patterns match, the **most specific** wins (fewer wildcards — `*` / `**` placeholders that match anything — or variables, longer literal text).

A real pitfall caught by the tests

The first version of this controller mapped `@GetMapping("/{id}")`. A request `GET /api/books/export` with `Accept: application/xml` could not be served by the two `/export` methods (they produce JSON and CSV only), so Spring fell back to the next matching pattern — `/{id}` with `id = "export"` — and answered **`400 "Failed to convert 'id'"`** instead of **`406 Not Acceptable`**. Constraining the variable with a regex, `/{id:\d+}`, removed the ambiguity. Use regex constraints (or distinct paths) when literal sub-paths share a level with variables.

5.4 Reading input and writing output

Method arguments

Annotation / type	Source	Notes
`@PathVariable long id`	URL segment `{id}`	type conversion is automatic; failure → 400
`@RequestParam int page`	query string / form field	`required` defaults to `true` (missing → 400 `MissingServletRequestParameterException`); `defaultValue` makes it optional; `Optional<T>` also works
`@RequestParam Map<String,String>`	all query params	
`@RequestHeader("X-Client") String c`	request header	`defaultValue`, `required` as above
`@CookieValue`	cookie (small named value the browser stores and sends back on every request)	
`@RequestBody BookRequest r`	body, via `HttpMessageConverter`	JSON → record/class with Jackson
`@RequestPart MultipartFile f`	part of a `multipart/form-data` (the body format browsers use to upload files) body	section 5.10
`@ModelAttribute` / no annotation on a class	query params / form fields bound to an object's properties	handy for search filters
`HttpServletRequest`, `Principal` (the logged-in user), `Locale` (the client's language/region), `UriComponentsBuilder`	framework-provided	

Names: with `@PathVariable long id` Spring uses the parameter name `id`. That needs the `-parameters` compiler flag, which `spring-boot-starter-parent` enables; outside Boot's parent, write `@PathVariable("id")`.

Return values

Return type	Result
any object / record / `List`	`200` + body serialised (turned into JSON text) by a message converter
`void` + `@ResponseStatus(NO_CONTENT)`	`204`, no body
`ResponseEntity<T>`	full control: status, headers, body
`ProblemDetail` / `ErrorResponse`	RFC 9457 error body (section 5.7)
`String` in a `@RestController`	written as text (`text/plain` or what `produces` says)
`CompletableFuture<T>`, `Callable<T>`, `DeferredResult<T>`, `SseEmitter`, `StreamingResponseBody`	async/streaming responses (SSE — Server-Sent Events, a one-way stream of messages from server to browser — is covered in Chapter 13)

`ResponseEntity` for 201 + `Location` — `ResponseEntity.created(uri).body(saved)`

```
@PostMapping
public ResponseEntity<Book> create(
        @Validated({BookRequest.OnCreate.class, Default.class}) @RequestBody BookRequest request) {
    Book saved = repo.save(request);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
            .path("/{id}").buildAndExpand(saved.id()).toUri();
    return ResponseEntity.created(location).body(saved);
}
```

`ServletUriComponentsBuilder.fromCurrentRequest()` starts from the URL the client used, so the `Location` is correct behind different hosts (if you run behind a proxy — e.g. a load balancer that receives the client's request and forwards it to your app — set `server.forward-headers-strategy=framework` so `X-Forwarded-*` headers, which carry the client's original host and protocol, are honoured).

The tests

The tests use `MockMvcTester` (Framework 6.2+), the AssertJ-style front end for MockMvc: it calls the `DispatcherServlet` in-process with mock request/response objects — no network, no Tomcat.

```
@WebMvcTest                         // web slice: controllers, advice, filters, WebMvcConfigurers, Jackson
@Import(BookRepository.class)       // @Repository beans are not part of the slice, so add ours
class BookApiTest {
    @Autowired MockMvcTester mvc;

    @Test
    void createReturns201WithLocation() throws Exception {
        MvcTestResult r = mvc.post().uri("/api/books")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                        {"title":"Release It! 2nd ed","author":"Michael T. Nygard",
                         "isbn":"9781680502398","year":2018}""")
                .exchange();
        show("POST /api/books", r);
        System.out.println("  Location = " + r.getResponse().getHeader("Location"));
        assertThat(r).hasStatus(201).hasHeader("Location", "http://localhost/api/books/6");
    }
    // list, get, update, delete tests ...
}
```

Output (all CRUD — Create, Read, Update, Delete — tests):

```
GET /api/books?page=1&size=2 -> 200 {"content":[{"id":3,"title":"Refactoring","author":"Martin
Fowler","isbn":"9780134757599","published_year":2018,"addedOn":"2026-01-15"},{"id":4,"title":"Domain-Driven
Design","author":"Eric Evans","isbn":"9780321125217","published_year":2003,"addedOn":"2026-01-
15"}],"page":1,"size":2,"totalElements":5,"totalPages":3}
GET /api/books/2 -> 200 {"id":2,"title":"Effective Java","author":"Joshua
Bloch","isbn":"9780134685991","published_year":2018,"addedOn":"2026-01-15"}
  X-Request-Id = req-15
  X-Handler    = BookController#get
POST /api/books -> 201 {"id":6,"title":"Release It! 2nd ed","author":"Michael T.
Nygard","isbn":"9781680502398","published_year":2018,"addedOn":"2026-01-15"}
  Location = http://localhost/api/books/6
PUT /api/books/1 -> 200 {"id":1,"title":"Clean Code (2nd ed)","author":"Robert C.
Martin","isbn":"9780132350884","published_year":2025,"addedOn":"2026-01-15"}
DELETE /api/books/1 -> 204
GET /api/books/1 after delete -> 404
```

(`X-Request-Id` and `X-Handler` come from the filter and interceptor in section 5.11. The number in `req-15` is a counter shared by all tests in the cached context, so it depends on test order and may differ in your run.)

Pagination

`list` returns one **page** plus the numbers a client needs to build page links:

```
public record PageResponse<T>(List<T> content, int page, int size, long totalElements, int totalPages) {}
```

- Use **zero-based** (first page is `0`) `page` and a bounded `size` (`@Max(50)`) so nobody can request a million rows.

- Return a **stable DTO** (Data Transfer Object: a simple class/record shaped for the API) like this one rather than serialising Spring Data's `Page` implementation directly — its JSON shape is not a stable API (Spring Data now warns about it and offers `@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)`). With Spring Data you would accept a `Pageable` argument (`?page=1&size=2&sort=title,asc`) — see Chapter 8.
- For very large or fast-changing data, **keyset (cursor) pagination** (`?after=lastSeenId` — "give me the rows after the last one I saw") is faster and more stable than `OFFSET` (SQL's "skip the first N rows", which still has to read them) — see the SQL book, Indexes and query performance.

5.5 JSON with Jackson 3

What it is

Jackson is the JSON library behind Spring MVC's JSON converter. **Spring Boot 4 uses Jackson 3** by default (Boot 4.1.1 manages 3.1.x) — a new major version with new Maven coordinates, so Jackson 2 code and configuration need small changes.

What changed from Jackson 2

Topic	Jackson 2 (Boot 3)	Jackson 3 (Boot 4)
Maven group / Java package	`com.fasterxml.jackson.*`	`tools.jackson.*`
Annotations (`@JsonProperty`, `@JsonIgnore`, ...)	`com.fasterxml.jackson.annotation`	**unchanged**: still `com.fasterxml.jackson.annotation`
Java baseline	Java 8	Java 17
Main mapper	`ObjectMapper` (mutable)	`JsonMapper` (a format-specific `ObjectMapper`), **immutable**, built with `JsonMapper.builder()`
Exceptions	checked (must be declared or caught) `JsonProcessingException`	unchecked (extends `RuntimeException`) `JacksonException` (`DatabindException`, `StreamReadException`, ...)
`java.time`, `Optional`, parameter names	separate modules (`jsr310`, `jdk8`, ...)	built in
Dates	timestamps by default (Boot turned this off)	ISO strings by default (`WRITE_DATES_AS_TIMESTAMPS` disabled)
Unknown JSON properties	fail by default (Boot turned this off)	ignored by default
Property order	declaration order	alphabetical by default, but creator (constructor/record) properties first
Renamed types	`JsonSerializer`, `JsonDeserializer`, `SerializerProvider`, `Module`	`ValueSerializer`, `ValueDeserializer`, `SerializationContext`, `JacksonModule`

Spring Boot 4 changes that follow from it:

Boot 3	Boot 4
`Jackson2ObjectMapperBuilderCustomizer`	`JsonMapperBuilderCustomizer`
`@JsonComponent`, `@JsonMixin`	`@JacksonComponent`, `@JacksonMixin`
`spring.jackson.read.*` / `spring.jackson.write.*`	`spring.jackson.json.read.*` / `spring.jackson.json.write.*`
define an `ObjectMapper` bean to customise	define/customise a `JsonMapper` bean
—	deprecated `spring-boot-jackson2` module to keep Jackson 2 while migrating
`HttpMessageConverters` bean	deprecated; use `ServerHttpMessageConvertersCustomizer` / `ClientHttpMessageConvertersCustomizer`

Example

The API record uses an annotation from the unchanged annotations package:

```
import com.fasterxml.jackson.annotation.JsonProperty;
import java.time.LocalDate;

public record Book(
        Long id,
        String title,
        String author,
        String isbn,
        @JsonProperty("published_year") int year,   // rename one JSON field
        LocalDate addedOn) {
}
```

The test injects Boot's `JsonMapper`:

```
@SpringBootTest
class JacksonTest {
    @Autowired JsonMapper mapper;   // Jackson 3 (package tools.jackson), configured by Boot 4

    @Test
    void serializeAndDeserializeARecord() {
        Book b = new Book(7L, "Effective Java", "Joshua Bloch", "9780134685991", 2018, LocalDate.of(2026, 1, 15));
        String json = mapper.writeValueAsString(b);          // no checked exception in Jackson 3
        System.out.println("mapper class: " + mapper.getClass().getName());
        System.out.println("JSON: " + json);

        Book back = mapper.readValue("""
                {"id":8,"title":"T","author":"A","isbn":"1","published_year":2001,
                 "addedOn":"2026-02-01","somethingNew":true}""", Book.class);
        System.out.println("read back (unknown field ignored): " + back);
    }

    @Test
    void jacksonExceptionsAreUnchecked() {
        assertThatThrownBy(() -> mapper.readValue("{not json", Book.class))
                .isInstanceOf(JacksonException.class)
                .isInstanceOf(RuntimeException.class) ... ;
    }
}
```

Output:

```
mapper class: tools.jackson.databind.json.JsonMapper
JSON: {"id":7,"title":"Effective Java","author":"Joshua
Bloch","isbn":"9780134685991","published_year":2018,"addedOn":"2026-01-15"}
read back (unknown field ignored): Book[id=8, title=T, author=A, isbn=1, year=2001, addedOn=2026-02-01]
bad JSON -> StreamReadException (a RuntimeException: true)
```

Notice: `LocalDate` became `"2026-01-15"` with no extra module; the record kept its declaration order (record components are creator properties, which are sorted first); the unknown `somethingNew` was ignored. In the

error responses later in this chapter, `ProblemDetail`'s fields *do* appear alphabetically (`detail, instance, status, title, type`) — that is Jackson 3's new default ordering for regular properties.

Customising

```
@Bean
JsonMapperBuilderCustomizer jsonCustomizer() {
    return builder -> builder.enable(SerializationFeature.INDENT_OUTPUT);   // tools.jackson.databind
}
```

or with properties, e.g. `spring.jackson.serialization.indent-output=true`, `spring.jackson.default-property-inclusion=non_null`. (`spring.jackson.json.read.*` / `json.write.*` hold the JSON-format-specific `JsonReadFeature` / `JsonWriteFeature` switches.)

Common mistakes

- Mixing Jackson 2 and 3 imports (`com.fasterxml.jackson.databind.ObjectMapper` vs `tools.jackson.databind...`): Spring MVC uses the Jackson 3 mapper, so a Jackson 2 `ObjectMapper` bean you define is simply not used for HTTP. (Some libraries, e.g. springdoc's swagger-core, still bring Jackson 2 for their own use — that is fine.)
- Exposing JPA entities directly as JSON → lazy-loading errors (related data loaded on first access, after the session closed), infinite recursion (two objects pointing at each other), leaking columns. Use DTOs/records (Chapter 8).
- Relying on field order in clients — JSON objects are unordered by definition.

5.6 Validation

What it is

Jakarta Bean Validation (the standard Java API for validation rules; Hibernate Validator is the implementation; add `spring-boot-starter-validation`) lets you declare rules as annotations on fields/record components and parameters. Spring MVC runs them before your method executes, so the controller only ever sees valid input.

```
public record BookRequest(
        @NotBlank @Size(max = 100) String title,
        @NotBlank String author,
        // required only when creating (group OnCreate); an update may leave it out
        @NotBlank(groups = OnCreate.class)
        @Pattern(regexp = "\\d{13}", message = "must be 13 digits") String isbn,
        @Min(1450) @Max(2100) int year) {

    /** Validation group marker: rules that apply only on create. */
    public interface OnCreate {}
}
```

Common constraints: `@NotNull`, `@NotBlank` (not null and not only whitespace), `@NotEmpty`, `@Size`, `@Min` / `@Max`, `@Positive`, `@Email`, `@Pattern`, `@Past` / `@Future`, `@Valid` (cascade into a nested object or list elements). Custom constraints: an annotation + a `ConstraintValidator`.

Three ways validation is triggered

Where	How to trigger	Failure exception	Default status
`@RequestBody` / `@ModelAttribute` object	`@Valid` or `@Validated` on the parameter	`MethodArgumentNotValidException`	400
Constraints directly on `@RequestParam`, `@PathVariable`, `@RequestHeader` (e.g. `@Max(50) int size`)	automatic — **built-in method validation** since Framework 6.1, no `@Validated` needed	`HandlerMethodValidationException`	400
Any Spring bean method (services)	`@Validated` on the **class** + constraints on parameters	`ConstraintViolationException` (AOP proxy, see Chapter 4)	500 unless you handle it

Do **not** put `@Validated` on the controller class in Framework 6.1+: it switches that controller to the AOP-based validation of the third row, and you get `ConstraintViolationException` instead of the web-aware exceptions.

Validation groups

A **group** is a marker interface (an empty interface used as a label) that labels constraints so different subsets can be validated in different situations. Constraints without `groups` belong to `jakarta.validation.groups.Default`. `@Valid` (Jakarta) has no groups attribute; Spring's `@Validated({...})` does:

```
@PostMapping                    // create: Default rules + OnCreate rules (isbn required)
public ResponseEntity<Book> create(
       @Validated({BookRequest.OnCreate.class, Default.class}) @RequestBody BookRequest request) { ... }

@PutMapping("/{id:\\d+}")       // update: only Default rules (isbn optional)
public Book update(@PathVariable long id, @Validated @RequestBody BookRequest request) { ... }
```

Test output (the same body without `isbn`):

```
POST without isbn -> 400 errors={"isbn":"must not be blank"}
PUT  without isbn -> 200
```

Invalid body and invalid query parameter:

```
POST invalid book -> 400 application/problem+json
   {"detail":"Request body has 3 invalid field(s)","instance":"/api/books","status":400,"title":"Bad Request","errors":
{"isbn":"must be 13 digits","title":"must not be blank","year":"must be greater than or equal to 1450"}}
GET /api/books?size=500 -> 400 application/problem+json
   {"detail":"Validation failure","instance":"/api/books","status":400,"title":"Bad Request","errors":{"size":"must be
less than or equal to 50"}}
```

The `errors` maps are added by the exception handler in the next section.

Common mistakes

- Forgetting `@Valid`/`@Validated` on `@RequestBody` → annotations on the record are ignored.
- Forgetting `@Valid` on a nested field (`@Valid Address address`) → nested object not validated.
- Missing `spring-boot-starter-validation` → no validator on the classpath, constraints silently ignored.
- `@NotNull` on a primitive `int` is pointless (never null); use `Integer` if "missing" must be detected.
- Validating only in the UI → every API must validate its own input.

5.7 Error handling and Problem Details (RFC 9457)

The problem it solves

Without a plan, errors leak as stack traces, HTML error pages, or a different JSON shape per endpoint. Clients need **one predictable error format**. **RFC 9457 "Problem Details for HTTP APIs"** (July 2023, replacing RFC 7807) defines it: media type (the format label used in `Content-Type`) `application/problem+json` with members

Member	Meaning
`type`	URI (a string that identifies a resource; URLs are one kind) identifying the problem type (default `about:blank`)
`title`	short human summary of the type
`status`	HTTP status code
`detail`	explanation of this occurrence
`instance`	URI of this occurrence (Spring fills the request path)
extensions	any extra members, e.g. `bookId`, `errors`

Spring's model: `ProblemDetail` (the body), `ErrorResponse` (an exception that carries a `ProblemDetail`; all Spring MVC exceptions implement it), `ErrorResponseException` (a base class for your own).

How it works

1. Controller code throws.
2. `ExceptionHandlerExceptionResolver` looks for an `@ExceptionHandler` for that type — first in the controller, then in `@ControllerAdvice` / `@RestControllerAdvice` beans (global handlers).
3. Otherwise `ResponseStatusExceptionResolver` (for `@ResponseStatus` or `ResponseStatusException`) and `DefaultHandlerExceptionResolver` (Spring MVC's own exceptions) set a status.
4. Anything still unhandled goes to the servlet container's (the web server, here Tomcat) error page, which Boot maps to `/error` → `BasicErrorController`, producing Boot's JSON (`timestamp`, `status`, `error`, `path`).

The cleanest setup: one `@RestControllerAdvice` that **`extends`** `ResponseEntityExceptionHandler`. The base class already maps every Spring MVC exception (400, 404, 405, 406, 413, 415, 500 ...) to a `ProblemDetail`; you add handlers for your own exceptions and override the hooks you want to enrich.

```
@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(BookNotFoundException.class)
    public ProblemDetail bookNotFound(BookNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Book not found");
        pd.setType(URI.create("https://api.example.com/problems/book-not-found"));
        pd.setProperty("bookId", ex.bookId());          // extra member in the JSON
        return pd;
    }

    // Customise the built-in 400 for @Valid/@Validated @RequestBody failures: add field errors
    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers, HttpStatusCode status, WebRequest request) {
        ProblemDetail pd = ex.getBody();                // already a ProblemDetail (status 400)
        Map<String, String> errors = new TreeMap<>();
        ex.getBindingResult().getFieldErrors()
                .forEach(fe -> errors.put(fe.getField(), fe.getDefaultMessage()));
        pd.setDetail("Request body has " + errors.size() + " invalid field(s)");
        pd.setProperty("errors", errors);
        return ResponseEntity.status(status).headers(headers).body(pd);
    }

    // Customise the built-in 400 for constraint violations on @RequestParam/@PathVariable/... (Framework 6.1+)
    @Override
    protected ResponseEntity<Object> handleHandlerMethodValidationException(
            HandlerMethodValidationException ex, HttpHeaders headers, HttpStatusCode status, WebRequest request) {
        ProblemDetail pd = ex.getBody();
        Map<String, String> errors = new TreeMap<>();
        ex.getParameterValidationResults().forEach(result -> result.getResolvableErrors().forEach(err ->
                errors.put(result.getMethodParameter().getParameterName(), err.getDefaultMessage())));
        pd.setProperty("errors", errors);
        return ResponseEntity.status(status).headers(headers).body(pd);
    }
}
```

Test output — our own exception and several framework errors, all in the same format:

```
GET /api/books/99 -> 404 application/problem+json
   {"detail":"Book 99 does not exist","instance":"/api/books/99","status":404,"title":"Book not
found","type":"https://api.example.com/problems/book-not-found","bookId":99}
GET /api/books?page=abc -> 400 application/problem+json
   {"detail":"Failed to convert 'page' with value: 'abc'","instance":"/api/books","status":400,"title":"Bad Request"}
PATCH /api/books/1 -> 405 application/problem+json
   {"detail":"Method 'PATCH' is not supported.","instance":"/api/books/1","status":405,"title":"Method Not Allowed"}
POST text/plain -> 415 application/problem+json
   {"detail":"Content-Type 'text/plain;charset=UTF-8' is not
supported.","instance":"/api/books","status":415,"title":"Unsupported Media Type"}
POST broken JSON -> 400 application/problem+json
   {"detail":"Failed to read request","instance":"/api/books","status":400,"title":"Bad Request"}
```

The 405 response also carries an `Allow` header listing the supported methods, and the 415 an `Accept` header listing supported body types. Note the generic `detail` for broken JSON: Spring deliberately does not echo parser internals back to clients.

Other options

Option	Use when
`spring.mvc.problemdetails.enabled=true`	you have **no** advice of your own and just want Problem Details for Spring MVC's exceptions (Boot registers a `ResponseEntityExceptionHandler` for you; default `false`)
`@ResponseStatus(HttpStatus.NOT_FOUND)` on an exception class	quick and simple, but no custom body
`throw new ResponseStatusException(HttpStatus.CONFLICT, "...")`	one-off errors without a custom class
extend `ErrorResponseException`	your exception carries its own `ProblemDetail` (status, type, extra properties)
`@ExceptionHandler` inside one controller	an error specific to that controller
`@RestControllerAdvice(basePackages = ...)` / `assignableTypes`	different handlers for different API areas

Common mistakes

- **Catching exceptions in controllers** and returning `200` with an `"error"` field — breaks HTTP semantics, caching and client retries.
- **Leaking internals** (`ex.getMessage()` from a SQL exception, stack traces) in `detail`. Log them server-side; return a safe message.
- **A generic `@ExceptionHandler(Exception.class)` returning 400** — bugs are 500s. Keep a catch-all that returns 500 and logs.
- Two advices handling the same exception without `@Order` — which wins is undefined.

In the interview

30-second answer: "A single `@RestControllerAdvice` extending `ResponseEntityExceptionHandler`, so every Spring MVC exception already produces an RFC 9457 `ProblemDetail` (`application/problem+json`). I add `@ExceptionHandler`s for domain exceptions — e.g. `NotFound` → 404 with a `type` URI and extra properties — and override `handleMethodArgumentNotValid` to include field errors. Unexpected exceptions become 500s with a safe message, logged with a correlation id (an id on every log line of one request)."

- *What's RFC 9457?* → Problem Details for HTTP APIs; it obsoletes RFC 7807; fields `type`, `title`, `status`, `detail`, `instance` plus extensions.
- *Order of lookup?* → `@ExceptionHandler` in the controller, then in `@ControllerAdvice` beans, then `@ResponseStatus`/`ResponseStatusException`, then Spring defaults, finally Boot's `/error`.

5.8 Content negotiation

What it is

Content negotiation is how client and server agree on a representation. The client sends `Accept: text/csv` (what it wants back) and `Content-Type` (what it is sending). Spring matches `Accept` against each method's `produces` and the available `HttpMessageConverter`s; `Content-Type` against `consumes`.

```
@GetMapping(value = "/export", produces = MediaType.APPLICATION_JSON_VALUE)
public List<Book> exportJson() { return repo.findAll(null); }

@GetMapping(value = "/export", produces = "text/csv")
public String exportCsv() {
    return repo.findAll(null).stream()
            .map(b -> b.id() + "," + b.title() + "," + b.year())
            .collect(Collectors.joining("\n", "id,title,year\n", "\n"));
}
```

Output:

```
Accept: text/csv -> text/csv;charset=UTF-8
   id,title,year
   1,Clean Code,2008
   2,Effective Java,2018
Accept: application/json -> application/json starts [{"id":1,"title":"Clean Code","author":"...
Accept: application/xml -> 406
```

- No `Accept` header (or `*/*`) → the first producible type wins.
- XML would need a converter: add `tools.jackson.dataformat:jackson-dataformat-xml` (Jackson 3) and Boot registers it automatically.
- Query-parameter negotiation (`?format=csv`) can be turned on with `spring.mvc.contentnegotiation.favor-parameter=true` (parameter name `format`, changeable with `spring.mvc.contentnegotiation.parameter-name`). Negotiation by path extension (`/books.json`) was deprecated in Framework 5.3 and is not available with the modern `PathPattern` matching.

5.9 API versioning (Framework 7)

What it is

API versioning lets old clients keep the old contract while new clients use a new one. Before Framework 7 every team invented its own scheme (`/v1/...` prefixes, custom header conditions); **Spring Framework 7.0 made it first-class**: a version is resolved from each request, parsed, validated, and used as one more mapping condition, with support on the client side (`RestClient`, `WebClient`, HTTP interfaces — Chapter 6) and in tests.

How it works

1. A **resolver** reads the version from the request: header, query parameter, path segment, or media-type parameter.
2. A **parser** turns it into a comparable version (default: semantic versioning `major.minor.patch` — major for breaking changes, minor for new features, patch for fixes — so `1.1` = `1.1.0`).
3. The version is **validated**: if missing and no default → 400 `MissingApiVersionException`; if not supported → 400 `InvalidApiVersionException`.
4. Mappings with a `version` attribute are matched: `"1.0"` = exactly that version; `"1.1+"` = a **baseline** — 1.1 and higher, until a more specific mapping for a higher version exists. Mappings without `version` match any version.
5. An optional **deprecation handler** adds standard `Deprecation` (RFC 9745), `Sunset` (RFC 8594) and `Link` headers to responses for deprecated versions.

Configuration

In Boot 4 via properties (or `WebMvcConfigurer.configureApiVersioning(ApiVersionConfigurer)` in plain Spring):

```
# Framework 7 API versioning, configured through Boot 4 properties
spring.mvc.apiversion.use.header=X-API-Version
spring.mvc.apiversion.default=1.0
# versions used in @GetMapping(version=...) are detected automatically; list extra ones here
spring.mvc.apiversion.supported=1.2
```

Property	Meaning
`spring.mvc.apiversion.use.header` / `.use.query-parameter` / `.use.path-segment` / `.use.media-type-parameter`	where the version comes from
`spring.mvc.apiversion.default`	version assumed when a request has none
`spring.mvc.apiversion.required`	reject requests without a version
`spring.mvc.apiversion.supported`	extra supported versions
`spring.mvc.apiversion.detect-supported`	detect supported versions from mappings

Beans of type `ApiVersionResolver`, `ApiVersionParser` and `ApiVersionDeprecationHandler` are picked up by Boot's MVC auto-configuration for full control. Our deprecation handler:

```
@Bean
ApiVersionDeprecationHandler deprecationHandler() {
    var handler = new StandardApiVersionDeprecationHandler();
    handler.configureVersion("1.0")
            .setDeprecationDate(ZonedDateTime.of(2026, 6, 1, 0, 0, 0, 0, ZoneOffset.UTC))
            .setSunsetDate(ZonedDateTime.of(2027, 1, 1, 0, 0, 0, 0, ZoneOffset.UTC))
            .setDeprecationLink(URI.create("https://api.example.com/docs/migrate-to-v2"));
    return handler;
}
```

The controller:

```
@RestController
@RequestMapping("/api/greeting")
public class GreetingController {
    public record GreetingV1(String message) {}
    public record GreetingV1_1(String message, String language) {}
    public record GreetingV2(String text, String lang) {}

    @GetMapping(version = "1.0")            // exactly version 1.0
    public GreetingV1 v1()     { return new GreetingV1("Hello"); }

    @GetMapping(version = "1.1+")           // baseline: 1.1 and every later version...
    public GreetingV1_1 v1_1() { return new GreetingV1_1("Hello", "en"); }

    @GetMapping(version = "2.0")            // ...until a more specific mapping takes over
    public GreetingV2 v2()     { return new GreetingV2("Hello", "en"); }
}
```

Output of `ApiVersioningTest`:

```
X-API-Version: none -> 200 {"message":"Hello"}
X-API-Version: 1.0  -> 200 {"message":"Hello"}
X-API-Version: 1.1  -> 200 {"message":"Hello","language":"en"}
X-API-Version: 1.2  -> 200 {"message":"Hello","language":"en"}
X-API-Version: 2.0  -> 200 {"text":"Hello","lang":"en"}
X-API-Version: 3.0  -> 400
Deprecation: @1780272000
Sunset: Fri, 1 Jan 2027 00:00:00 GMT
Link: <https://api.example.com/docs/migrate-to-v2>; rel="deprecation"; type="text/html"
```

- No header → default `1.0`. `1.2` has no exact mapping, so the `1.1+` baseline serves it (1.2 is supported because we listed it). `3.0` is unsupported → 400.

- `Deprecation: @1780272000` is the RFC 9745 format: `@` + Unix seconds (seconds since 1 Jan 1970 UTC; here 2026-06-01T00:00Z).

Trade-offs: where to put the version

Strategy	Example	Pros	Cons
Path segment	`/api/v2/books`	visible, easy to test in a browser, cache-friendly	"the same resource" has several URLs
Header	`X-API-Version: 2.0`	clean URLs	invisible in logs/links unless you add it; needs `Vary` (a header telling caches which request headers change the response) for caches
Query parameter	`?version=2.0`	easy to try	mixes with filters, easy to drop
Media type parameter	`Accept: application/json;version=2.0`	"pure" HTTP content negotiation	least discoverable, harder tooling

Whatever you choose, prefer **additive, backwards-compatible changes** (new optional fields that old clients can ignore) and bump the version only for breaking changes (changes that make existing clients fail).

5.10 CORS and file upload

CORS

What it is

Browsers enforce the **same-origin policy** (an *origin* is scheme + host + port, e.g. `https://shop.example.com`): JavaScript on `https://shop.example.com` may not read responses from `https://api.example.com` unless the API opts in with **CORS** (Cross-Origin Resource Sharing) headers. For "non-simple" requests (e.g. JSON `POST`, custom headers) the browser first sends a **preflight** `OPTIONS` request asking permission.

CORS protects **users' browsers**, not your server: `curl` and server-to-server calls ignore it. It is not authentication.

How to configure

Globally (preferred):

```
@Override
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")
            .allowedOrigins("https://shop.example.com")   // the browser app's origin
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .maxAge(3600);                                // preflight may be cached 1 h
}
```

or per controller/method with `@CrossOrigin(origins = "...")`. With Spring Security, also enable `http.cors(...)` so the security filter chain lets preflights through (Chapter 10).

Test output (preflights from an allowed and an unknown origin):

```
preflight from shop.example.com -> 200 Allow-Origin=https://shop.example.com Allow-Methods=GET,POST,PUT,DELETE Max-Age=3600
preflight from evil.example.net -> 403 Allow-Origin=null
```

Common mistakes

- `allowedOrigins("*")` together with `allowCredentials(true)` — not allowed by the CORS spec (Spring rejects it); use explicit origins or `allowedOriginPatterns`.
- Configuring CORS in MVC but not in Spring Security → preflight gets 401/403 from security first.

File upload

Multipart (`multipart/form-data`) requests carry files plus form fields. Boot configures multipart support; limits:

```
spring.servlet.multipart.max-file-size=1MB       # default 1MB
spring.servlet.multipart.max-request-size=2MB    # default 10MB
```

```
@PostMapping("/api/uploads")
public UploadResult upload(@RequestPart("file") MultipartFile file,
                           @RequestParam(defaultValue = "") String note) throws IOException {
    String text = new String(file.getBytes(), StandardCharsets.UTF_8);
    return new UploadResult(file.getOriginalFilename(), file.getSize(),
            file.getContentType(), (int) text.lines().count(), note);
}
```

Output:

```
POST /api/uploads -> 200 {"fileName":"books.csv","bytes":36,"contentType":"text/csv","lines":3,"note":"weekly import"}
```

- Exceeding the limit in a running server raises `MaxUploadSizeExceededException`, which `ResponseEntityExceptionHandler` maps to **413**. (MockMvc does not parse real multipart streams, so the limit is not exercised by this test.)
- **Security:** never trust `getOriginalFilename()` as a path (path traversal: `../../etc/passwd`); generate your own storage names, check type and size, and stream large files (`file.getInputStream()` / `transferTo`) rather than loading them into memory. Large files usually go straight to object storage (a file store such as Amazon S3) via pre-signed URLs (temporary upload links signed by your server) instead of through your API.

5.11 Filters vs interceptors (vs AOP)

What it is

Three places to put code that runs around requests:

	Servlet `Filter`	**Spring MVC `HandlerInterceptor`**	**AOP aspect (Ch. 4)**
Runs	before/after the `DispatcherServlet`, for every request (also static files, errors)	inside the `DispatcherServlet`, only for requests mapped to a handler	around bean method calls
Knows the controller method?	no	yes (`HandlerMethod`)	yes (join point)
Can wrap/replace request & response objects	yes	no	no
Typical use	security (Spring Security is a filter chain), request ids/MDC (Mapped Diagnostic Context: per-thread values added to every log line), compression, CORS	per-endpoint auth checks, locale, audit, timing by handler	business-level cross-cutting concerns
Register	`@Component` implementing `Filter` (often `OncePerRequestFilter`) or `FilterRegistrationBean`	`WebMvcConfigurer.addInterceptors`	`@Aspect` `@Component`

The chapter's filter adds a correlation id to every response:

```java
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class RequestIdFilter extends OncePerRequestFilter {
    private final AtomicLong counter = new AtomicLong();

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String id = req.getHeader("X-Request-Id");
        if (id == null) id = "req-" + counter.incrementAndGet();
        res.setHeader("X-Request-Id", id);   // echo it so clients and logs can correlate
        chain.doFilter(req, res);            // continue to the next filter / the DispatcherServlet
    }
}
```

The interceptor records which controller method was chosen — something a filter cannot know:

```java
public class HandlerNameInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
        if (handler instanceof HandlerMethod hm) {
            res.setHeader("X-Handler", hm.getBeanType().getSimpleName() + "#" + hm.getMethod().getName());
        }
        return true;                         // false would stop the request here
    }
}
```

registered with `registry.addInterceptor(new HandlerNameInterceptor()).addPathPatterns("/api/**")`. The earlier output showed both headers: `X-Request-Id = req-15`, `X-Handler = BookController#get`.

Interceptor callbacks: `preHandle` (before the method; return `false` to stop), `postHandle` (after the method, **before** the response is written for views — for `@ResponseBody` methods the body is already written, so headers added here are too late), `afterCompletion` (always, even on exceptions — use for cleanup/metrics).

`@WebMvcTest` includes `Filter` beans, `WebMvcConfigurer`s and interceptors, which is why the tests see both headers.

5.12 Documenting the API: OpenAPI with springdoc

What it is

OpenAPI (formerly Swagger) is a machine-readable description of an HTTP API: paths, parameters, bodies, responses. Tools generate client SDKs (ready-made client libraries), contract tests (tests that check the real service matches the published description) and interactive docs from it. **springdoc-openapi** generates the document from your Spring MVC controllers at runtime. Its **3.x** line targets **Spring Boot 4** (2.x is for Boot 3); this project uses 3.1.1.

```xml
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-api</artifactId>   <!-- -ui adds Swagger UI -->
  <version>3.1.1</version>
</dependency>
```

- `/v3/api-docs` serves the JSON document (`/v3/api-docs.yaml` for YAML).
- With `springdoc-openapi-starter-webmvc-ui`, Swagger UI appears at `/swagger-ui.html`.
- It reads your mappings, parameter annotations and Bean Validation constraints (`@Max(50)` becomes `maximum: 50`). Add detail with `io.swagger.v3.oas.annotations` (`@Operation`, `@ApiResponse`, `@Schema`).

The test fetches the document:

```java
@SpringBootTest                // full context: springdoc is not part of the @WebMvcTest slice
@AutoConfigureMockMvc          // Boot 4: @SpringBootTest no longer adds MockMvc on its own
class OpenApiTest { ... mvc.get().uri("/v3/api-docs") ... }
```

Output:

```
openapi: 3.1.0
  /api/books/{id} [get, put, delete]
  /api/uploads [post]
  /api/books [get, post]
  /api/books/export [get]
```

springdoc logs a warning that `/v3/api-docs` is enabled by default — in production either secure it or set `springdoc.api-docs.enabled=false` (and `springdoc.swagger-ui.enabled=false`).

Code-first vs design-first: springdoc is *code-first* (document generated from code). Many API-platform teams go *design-first*: write the OpenAPI file, review it, then generate server interfaces/clients with `openapi-generator`. Both are fine; the key is that the published spec is tested against the real service.

HATEOAS in brief

HATEOAS (Hypermedia as the Engine of Application State) means responses include **links** to related actions (`"_links": {"self": ..., "next": ...}`), so clients navigate the API like a website instead of hard-coding URLs. **Spring HATEOAS** (`spring-boot-starter-hateoas`) provides `EntityModel`, `CollectionModel`, `PagedModel` and `linkTo(methodOn(...))` builders, rendering HAL JSON (a common JSON convention for embedding links). It is valuable for long-lived public APIs; most internal JSON APIs skip it and rely on OpenAPI documentation instead.

5.13 Putting it together: design checklist and mistakes

Area	Do
URLs	plural nouns (`/books`, `/books/{id}`), no verbs; nested only for true ownership (`/orders/{id}/items`)
Methods & codes	follow section 5.1; `201 + Location` on create, `204` on delete
DTOs	separate request/response records; never expose entities
Validation	Bean Validation on every input; groups for create vs update
Errors	one `@RestControllerAdvice`, RFC 9457 `ProblemDetail`, no internals
Lists	always paginate, cap `size`, stable JSON shape
Versioning	pick one strategy; Framework 7 `version` attribute; deprecate with headers
Security	authentication/authorization (Ch. 10), CORS explicit origins, upload limits
Docs	OpenAPI published and tested
Observability (logs, metrics, traces)	request id filter, metrics per endpoint (Ch. 15)
Threads	Spring MVC is thread-per-request; with `spring.threads.virtual.enabled=true` Tomcat uses virtual threads (cheap JVM-managed threads), giving high concurrency without reactive code (Ch. 13)

Functional endpoints. Besides annotated controllers, Spring MVC offers `RouterFunction` / `HandlerFunction` ("WebMvc.fn"): routes are declared as code (`route().GET("/api/books/{id}", handler::get).build()`). Same `DispatcherServlet`, same converters; choose it if you prefer explicit routing over annotations. It also supports API versioning.

5.14 In the interview

Q: Walk me through what happens when a request hits a Spring MVC app. Filters → `DispatcherServlet` → `HandlerMapping` finds the controller method (path, method, consumes/produces, version) → interceptors' `preHandle` → `HandlerAdapter` resolves arguments (message converters for bodies, conversion, validation) → method runs → return value written by a message converter chosen by `Accept` → exceptions go to `HandlerExceptionResolver`s (`@ExceptionHandler`/advice) → `afterCompletion` → filters.

- *Filter vs interceptor?* → Filter: servlet level, every request, can wrap request/response, doesn't know the handler. Interceptor: MVC level, knows the `HandlerMethod`, only mapped requests.
- *`@Controller` vs `@RestController`?* → The latter adds `@ResponseBody`.

Q: How do you validate input and report errors? → Bean Validation annotations on DTOs + `@Valid` / `@Validated` (groups via `@Validated`); constraints on parameters are validated automatically since 6.1; a global `@RestControllerAdvice` extending `ResponseEntityExceptionHandler` returns RFC 9457 `ProblemDetail`s with field errors.

Q: PUT vs PATCH vs POST? Which are idempotent? → POST creates (not idempotent); PUT replaces (idempotent); PATCH partially updates (not necessarily idempotent); GET/DELETE idempotent.

Q: How would you version an API in Spring Boot 4? → Framework 7 API versioning: configure a resolver (`spring.mvc.apiversion.use.header=X-API-Version` or path segment), set default/supported versions, use `@GetMapping(version = "1.1+")` baselines, add a deprecation handler that sends `Deprecation` / `Sunset` / `Link` headers; clients use `ApiVersionInserter`.

Q: What changed with JSON in Boot 4? → Jackson 3: `tools.jackson` packages (annotations unchanged), immutable `JsonMapper`, unchecked exceptions, java.time built in, ISO dates by default; Boot's customizer is `JsonMapperBuilderCustomizer`, `@JsonComponent` → `@JacksonComponent`, JSON read/write properties moved under `spring.jackson.json.*`.

Q: What is CORS and does it secure my API? → A browser mechanism that lets a page from one origin read responses from another if the server allows it via headers and preflight. It doesn't secure the API against non-browser clients — you still need authentication and authorization.

5.15 Chapter TL;DR

- `spring-boot-starter-webmvc` (Boot 4 name) gives Spring MVC on embedded Tomcat with Jackson 3.
- Request flow: **filters → `DispatcherServlet` → HandlerMapping → interceptors → argument resolution (converters + validation) → controller → message converter (by `Accept`) → exception resolvers**.
- `@RestController` + `@GetMapping/@PostMapping/...`; `@PathVariable`, `@RequestParam`, `@RequestHeader`, `@RequestBody`; `ResponseEntity.created(uri)` for 201; `@ResponseStatus(NO_CONTENT)` for 204. Constrain path variables (`{id:\d+}`) to avoid ambiguous matches.
- **Jackson 3**: `tools.jackson.*`, annotations unchanged, immutable `JsonMapper`, unchecked `JacksonException`, java.time built in.
- **Validation**: `@Valid` / `@Validated` on bodies (groups via `@Validated`), parameter constraints validated automatically (6.1+), don't put `@Validated` on controllers.
- **Errors**: one `@RestControllerAdvice extends ResponseEntityExceptionHandler`, returning **RFC 9457 `ProblemDetail`** (`application/problem+json`).
- **Content negotiation** via `Accept` + `produces`; 406 if impossible. Paginate with a bounded `size` and a stable DTO.
- **API versioning** (Framework 7): `spring.mvc.apiversion.*`, `version = "1.0"` / `"1.1+"`, 400 for unsupported versions, `Deprecation` / `Sunset` headers.
- **CORS** in `addCorsMappings` (+ Spring Security); **uploads** with `MultipartFile` and size limits; **filters** vs **interceptors**; **springdoc 3.x** for OpenAPI on Boot 4.
- Next: Chapter 6 — REST clients and HTTP interfaces.

Chapter 6 — REST Clients and HTTP Interfaces

Chapter 5 built an API. Real services also **call** APIs: a payment provider, another team's microservice (a small, independently deployed service that owns one business area), a search engine. Calling over the network brings problems a local call never has — slowness, timeouts, partial failures (some calls fail while others work), error bodies, versioning, retries. This chapter covers Spring's HTTP clients and how to use them safely.

Code: `code/rest-clients/` — Spring Boot 4.1.1, Framework 7.0, Java 25. To make the tests hit a *real* HTTP server without the internet, the project also contains a small "remote" catalog API (`CatalogServerController`) that the test starts on a free local port; imagine it belongs to another team. All output shown is real.

```
<!-- RestClient.Builder, RestTemplateBuilder, HTTP service client auto-configuration (Boot 4 starter) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-restclient</artifactId>
</dependency>
```

(An app that already has `spring-boot-starter-webmvc` for its own endpoints still adds `spring-boot-starter-restclient` for the client side — Boot 4's starters are modular.)

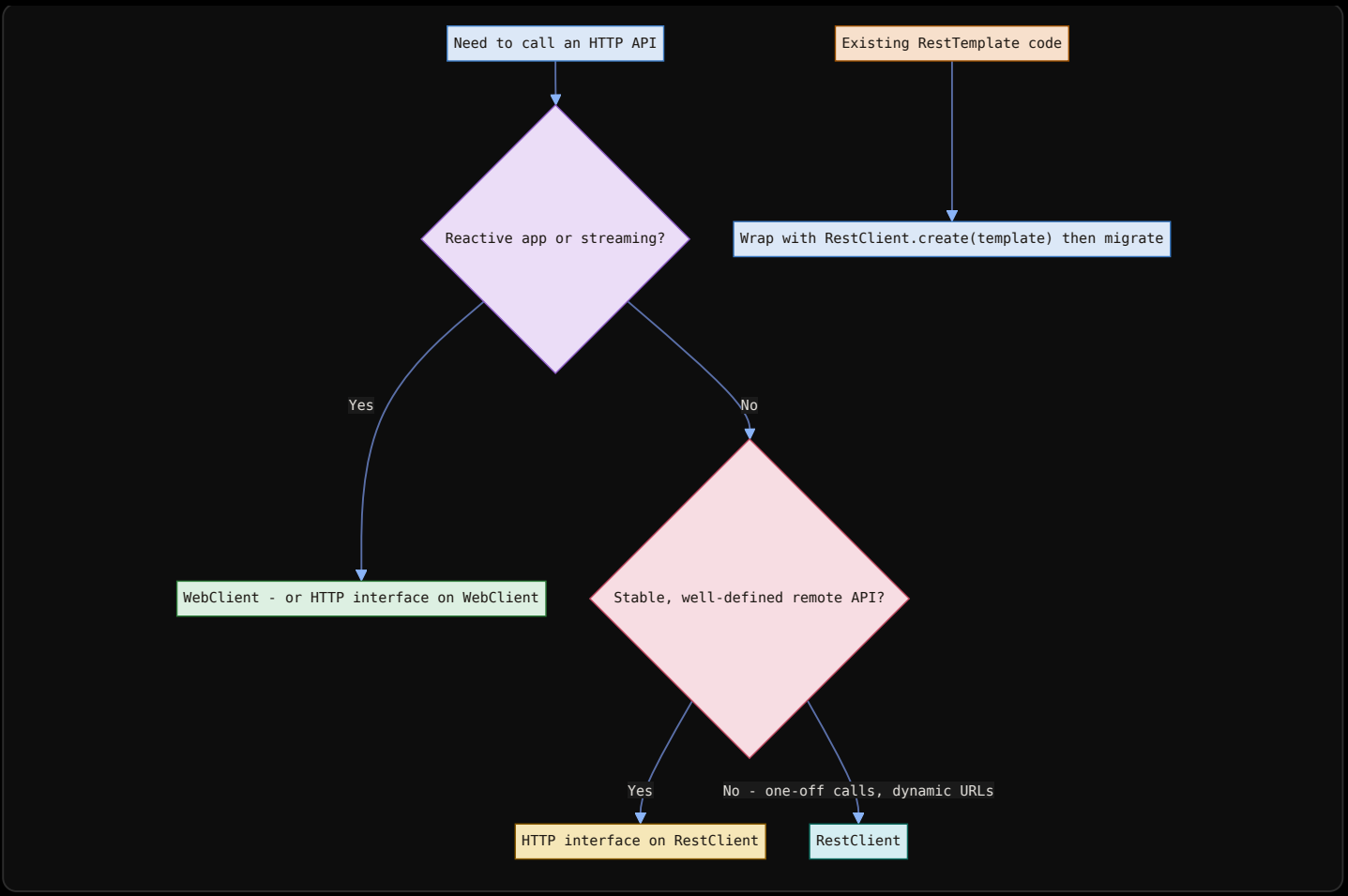
6.1 Choosing a client

What they are

Client	Style	Since	Status (Framework 7)	Use for
`RestClient`	synchronous (the calling thread waits for the response), **fluent** (calls chained with dots: `get().uri(...).retrieve()`) API	Framework 6.1 (2023)	the recommended blocking client	new code in Spring MVC / non-reactive apps
`RestTemplate`	synchronous, template methods (`getForObject`, `exchange`, ...)	Spring 3.0 (2009)	**deprecation announced**: intent to deprecate in 7.0, planned `@Deprecated` in 7.1 (Nov 2026), removal in 8.0	existing code only; migrate to `RestClient`
`WebClient`	non-blocking, reactive (the thread is freed while waiting; results arrive later as `Mono` = 0–1 value or `Flux` = 0–N values)	Framework 5.0	recommended reactive client	WebFlux apps, streaming, reactive pipelines (Chapter 13)
HTTP interface (`@HttpExchange`)	**declarative** (you describe *what* to call, not *how*): an annotated Java interface, Spring generates the implementation	Framework 6.0; Boot auto-config in 4.0	recommended for calling well-defined APIs	typed clients for other services; runs on `RestClient` or `WebClient`

Why `RestClient` replaced `RestTemplate`: the same underlying machinery (request factories, message converters, interceptors) but a modern fluent API instead of dozens of overloaded methods (methods with the same name but different parameter lists), and one place for all new features (API versioning, `requiredBody`, status handlers). The Spring team's plan (spring.io blog, "The state of HTTP clients in Spring", September 2025) keeps `RestTemplate` working through the 7.x line — so no panic, but no new code on it.

Do you need `WebClient` in a Spring MVC app? Not any more. Before `RestClient`, people pulled in WebFlux just to use `WebClient` and called `.block()`. With `RestClient` — and **virtual threads** (lightweight threads managed by the JVM, so millions can wait at once) making blocking calls cheap (Java book 15.4) — the blocking client is the simpler choice for MVC apps.



6.2 `RestClient` basics

How it works

1. Get a **builder**. In Boot, inject `RestClient.Builder` — a **prototype** bean (fresh per injection point, so configuring one doesn't affect others) pre-configured with Boot's message converters (Jackson 3), the chosen HTTP library, timeouts from `spring.http.clients.*`, and (with Micrometer) observability — metrics and traces per call. (Outside Boot: `RestClient.builder()` or `RestClient.create("https://...")`.)
2. Set shared settings (`baseUrl`, `defaultHeader`, `defaultStatusHandler`, `requestInterceptor`, `requestFactory`) and `build()`. A `RestClient` is **immutable and thread-safe** — build it once (e.g. in a constructor) and reuse it.
3. Per call: choose the method (`get()`, `post()`, `put()`, `patch()`, `delete()`, `method(...)`), the `uri(...)`, headers, body.
4. Finish with **`retrieve()`** (then `body(...)`, `toEntity(...)`, `toBodilessEntity()`) or **`exchange(...)`**.

Example: body, entity and generic lists

```
RestClient rest = builder.baseUrl(baseUrl()).build();

BookDto book = rest.get().uri("/catalog/books/{id}", 1).retrieve().body(BookDto.class);

ResponseEntity<List<BookDto>> entity = rest.get()
        .uri(b -> b.path("/catalog/books").queryParam("author", "Martin Fowler").build())
        .retrieve()
        .toEntity(new ParameterizedTypeReference<List<BookDto>>() {});
```

Output:

```
body(BookDto.class) -> BookDto[id=1, title=Release It!, author=Michael T. Nygard]
toEntity(List) -> 200 OK 2 books
```

What each part does:

- `uri("/catalog/books/{id}", 1)` — a **URI template**; variables are URL-encoded for you (spaces → `%20` etc.). Never build URLs by string concatenation with user input.
- `uri(b -> b.path(...).queryParam("author", "Martin Fowler").build())` — the `UriBuilder` form for query parameters (the space is encoded).
- `body(BookDto.class)` — deserialises the JSON body into your record. Your DTO only needs the fields you use; unknown fields are ignored (Jackson 3 default).
- `ParameterizedTypeReference<List<BookDto>>` — Java erases generics at runtime, so `List.class` would give a list of maps; the anonymous subclass (`{}`) captures the full type. Same with `body(new ParameterizedTypeReference<List<BookDto>>() {})`.
- `toEntity(...)` returns `ResponseEntity<T>`: status + headers + body. `toBodilessEntity()` when you only care about the status.
- `requiredBody(...)` (Framework 7.0.4+) is like `body(...)` but throws if the body is empty instead of returning `null`.

A hand-written client class

Wrap the calls in a small class so the rest of your code talks in domain terms (books, orders — not HTTP) (`CatalogClient.java`):

```
@Component
public class CatalogClient {

    private final RestClient rest;

    public CatalogClient(RestClient.Builder builder, @Value("${catalog.base-url}") String baseUrl) {
        this.rest = builder                                    // Boot's pre-configured, prototype builder
                .baseUrl(baseUrl)
                .defaultHeader("X-Client", "catalog-client")
                .defaultStatusHandler(status -> status.is5xxServerError(), (request, response) -> {
                    throw new CatalogUnavailableException(
                            "catalog answered " + response.getStatusCode().value() + " for " +
request.getURI().getPath());
                })
                .build();
    }

    /** 404 becomes Optional.empty(); 5xx becomes CatalogUnavailableException (default status handler). */
    public Optional<BookDto> findBook(long id) {
        try {
            return Optional.ofNullable(rest.get()
                    .uri("/catalog/books/{id}", id)           // URI template: id is encoded for us
                    .retrieve()                               // status handlers apply to retrieve()
                    .body(BookDto.class));
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();
        }
    }

    /** exchange(): full control over the raw response - status handlers are NOT applied. */
    public String describe(long id) {
        return rest.get()
                .uri("/catalog/books/{id}", id)
                .exchange((request, response) -> {
                    if (response.getStatusCode().isError()) {
                        return "status " + response.getStatusCode().value();
                    }
                    BookDto b = response.bodyTo(BookDto.class);
                    return b.title() + " by " + b.author();
                });
    }

    public ResponseEntity<BookDto> create(String title, String author) {
        return rest.post()
                .uri("/catalog/books")
                .body(new BookDto(null, title, author))      // serialised to JSON by Jackson
                .retrieve()
                .toEntity(BookDto.class);                     // status + headers + body
    }
}
```

Output:

```
findBook(2)   -> Optional[BookDto[id=2, title=Refactoring, author=Martin Fowler]]
findBook(999) -> Optional.empty
describe(1)   -> Release It! by Michael T. Nygard
describe(999) -> status 404
create        -> 201 CREATED Location=/catalog/books/102 body=BookDto[id=102, title=Designing Data-Intensive
Applications, author=Martin Kleppmann]
```

`retrieve()` vs `exchange()`

	`retrieve()`	`exchange((req, res) -> ...)`
Error statuses	turned into exceptions (default or your status handlers)	**you** inspect the status; **status handlers are not applied**
Body handling	`body`, `toEntity`, `toBodilessEntity`	`res.bodyTo(...)`, `res.getHeaders()`, anything
Use when	almost always	different body types per status, streaming, very custom handling

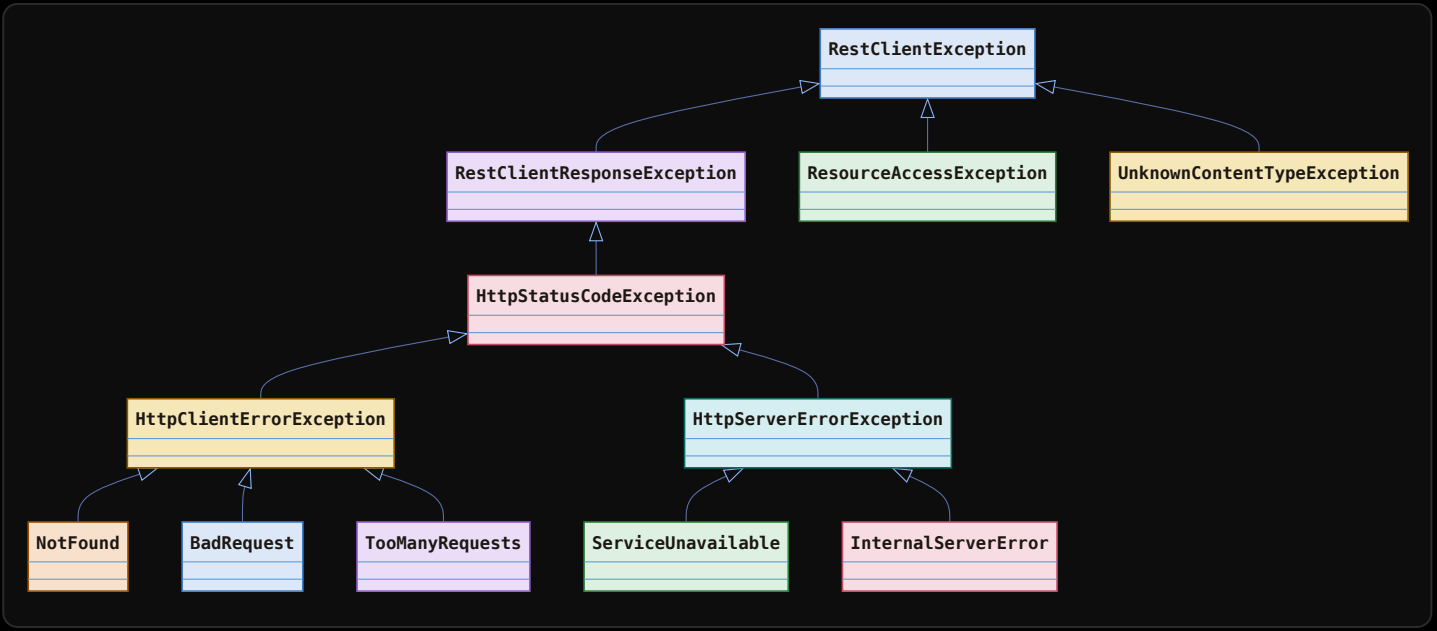
A lesson from writing this chapter: the first version of `findBook` used `exchange()` and treated 404 specially, assuming the 5xx `defaultStatusHandler` would still fire. It does not — `exchange()` bypasses status handlers — so a 503 silently became `Optional.empty()`. The `@RestClientTest` in section 6.8 caught it. If you use `exchange()`, handle **every** status yourself.

More request features (quick reference)

Need	`RestClient` call
Header on one request	`.header("X-Tenant", "acme")`, `.headers(h -> h.setBearerAuth(token))`
Accept / content type	`.accept(MediaType.APPLICATION_JSON)`, `.contentType(MediaType.APPLICATION_JSON)`
Form post (`application/x-www-form-urlencoded`)	`.contentType(APPLICATION_FORM_URLENCODED).body(multiValueMap)`
Multipart upload	`.body(new MultipartBodyBuilder().part("file", resource).build())` with `contentType(MULTIPART_FORM_DATA)`
Download a large file	`.exchange((req, res) -> { res.getBody().transferTo(out); return null; })` — stream, don't buffer
Plain text	`.retrieve().body(String.class)`
Per-request attributes for interceptors/observations	`.attribute("name", value)`

6.3 Error handling

Spring's client exception hierarchy



Exception	Meaning	Retry?
`HttpClientErrorException` (4xx) with nested types `.NotFound`, `.BadRequest`, `.Conflict`, `.TooManyRequests`, ...	the **request** was wrong or not allowed	no (except 429 "too many requests" after waiting the `Retry-After` header's delay, and 408 "request timeout")
`HttpServerErrorException` (5xx) with `.InternalServerError`, `.BadGateway`, `.ServiceUnavailable`, `.GatewayTimeout`	the server failed	often yes (transient), if the operation is idempotent (doing it twice = doing it once — see System Design Fundamentals)
`ResourceAccessException`	network I/O problem: connection refused, **timeout** (cause e.g. `HttpTimeoutException` for the JDK client), DNS	often yes
`UnknownContentTypeException`	no converter for the response type	no — a bug or contract change

All are **unchecked** (`RestClientException extends RuntimeException`). `RestClientResponseException` gives you `getStatusCode()`, `getResponseHeaders()`, `getResponseBodyAsString()`, and **`getResponseBodyAs(Class)`** — which can decode an RFC 9457 error body directly:

```
assertThatThrownBy(() -> rest.get().uri("/catalog/books/{id}", 999).retrieve().body(BookDto.class))
        .isInstanceOf(HttpClientErrorException.NotFound.class)
        .satisfies(e -> {
            var ex = (HttpClientErrorException) e;
            ProblemDetail pd = ex.getResponseBodyAs(ProblemDetail.class);   // read the error body
            System.out.println("404 -> " + ex.getClass().getSimpleName()
                    + ", problem title='" + pd.getTitle() + "', detail='" + pd.getDetail() + "'");
        });
```

Output:

```
404 -> NotFound, problem title='Book not found', detail='No book 999'
```

Mapping to your own exceptions

Callers of `CatalogClient` should not need to know it uses HTTP. Translate at the edge:

- **Per request:** `.retrieve().onStatus(HttpStatusCode::is4xxClientError, (req, res) -> { throw new ... })`.

- **For every request of a client:** `builder.defaultStatusHandler(predicate, handler)` — used above to turn any 5xx into `CatalogUnavailableException`:

```
503 -> CatalogUnavailableException: catalog answered 503 for /catalog/flaky
```

- **Expected "not found":** return `Optional.empty()` (as `findBook` does) instead of throwing.

Common mistakes

- Catching `Exception` around a client call and returning `null` — hides outages; log and translate instead.
- Logging full response bodies of errors — may contain personal data or secrets.
- Treating every 4xx as retryable — a 400 will fail the same way every time.
- Forgetting that **no timeout = wait forever**-ish (next section).

6.4 Timeouts and the underlying HTTP library

What it is

`RestClient` does not open sockets itself; it delegates to a **`ClientHttpRequestFactory`** wrapping a real HTTP library. Two timeouts matter:

- **Connect timeout** — how long to wait for the TCP + TLS (`https` encryption) connection to be established.
- **Read timeout** — how long to wait for response data once connected.

Why it matters: without timeouts, a slow downstream service makes your request threads pile up waiting until your service also stops answering — a **cascading failure** (one failure spreading to the services that depend on it). Every remote call needs a timeout shorter than your own callers' timeout. See also Circuit breaker and Bulkhead.

Which library does Boot pick?

Boot 4 detects, in this order, the first one on the classpath:

Priority	Library	Factory	Add dependency
1	Apache HttpClient 5	`HttpComponentsClientHttpRequestFactory`	`org.apache.httpcomponents.client5:httpclient5`
2	Jetty HttpClient	`JettyClientHttpRequestFactory`	`org.eclipse.jetty:jetty-client`
3	Reactor Netty	`ReactorClientHttpRequestFactory`	`io.projectreactor.netty:reactor-netty-http`
4	JDK `java.net.http.HttpClient`	`JdkClientHttpRequestFactory`	nothing (built in)
5	`HttpURLConnection`	`SimpleClientHttpRequestFactory`	nothing

This project has none of the first three, so it uses the **JDK HttpClient**. Force a choice with `spring.http.clients.imperative.factory=jdk|http-components|jetty|reactor|simple`. (For `WebClient` the equivalent is `spring.http.clients.reactive.connector`.)

Configuring timeouts

Globally, for every auto-configured client (Boot 4 property names — the Boot 3.x `spring.http.client.*` names are deprecated):

```
spring.http.clients.connect-timeout=1s
spring.http.clients.read-timeout=3s
spring.http.clients.redirects=dont-follow     # or follow / follow-when-possible
```

Per client, in code, with Boot's `ClientHttpRequestFactoryBuilder` and `HttpClientSettings`:

```
var factory = ClientHttpRequestFactoryBuilder.jdk()
        .build(HttpClientSettings.defaults()
                .withConnectTimeout(Duration.ofSeconds(1))
                .withReadTimeout(Duration.ofMillis(300)));
RestClient rest = builder.baseUrl(baseUrl()).requestFactory(factory).build();
rest.get().uri("/catalog/slow").retrieve().body(String.class);   // server needs 1.5 s
```

Output:

```
slow endpoint -> ResourceAccessException caused by HttpTimeoutException after ~300 ms
```

For HTTP service groups (section 6.6) the timeouts can be set per group: `spring.http.serviceclient.<group>.read-timeout=2s`.

Choosing values: measure the downstream's latency (p99 = the time 99% of calls stay under) and set the read timeout a bit above it; keep the total (timeout × attempts + back-off waits) below what *your* caller will wait. Connection **pooling** (reusing open connections; Apache/Jetty/Netty pool by default, the JDK client keeps connections alive) avoids TCP/TLS setup on every call.

6.5 Interceptors

A `ClientHttpRequestInterceptor` wraps every request a client sends — the client-side twin of a servlet filter. Uses: auth headers (tokens), correlation ids (one id carried through all calls of a request so logs can be joined), logging, metrics.

```
RestClient rest = builder.baseUrl(baseUrl())
        .requestInterceptor((request, body, execution) -> {
            request.getHeaders().add("X-Trace-Id", "trace-42");          // add a header
            var response = execution.execute(request, body);            // do the call
            log.add(request.getMethod() + " " + request.getURI().getPath() + " -> " + response.getStatusCode().value());
            return response;
        })
        .build();
```

Output:

```
interceptor: GET /catalog/books/1 -> 200
interceptor: GET /catalog/books/3 -> 200
```

- You rarely need interceptors for **tracing**: when Micrometer observability is set up (Actuator, Chapter 15), Boot instruments the auto-configured builder to record `http.client.requests` metrics and propagate trace headers to the next service. This only works if you build from the injected builder — `RestClient.create()` skips it.
- For OAuth2 bearer tokens (`Authorization: Bearer ...`), Spring Security provides `OAuth2ClientHttpRequestInterceptor` (Chapter 11).
- Don't read the response body in an interceptor unless you buffer it (`BufferingClientHttpRequestFactory`); a body can normally be read only once.
- To customise **all** builders Boot creates, declare a `RestClientCustomizer` bean.

6.6 HTTP service interfaces (`@HttpExchange`)

What it is

An **HTTP service interface** is a Java interface whose methods are annotated with the HTTP method, path and parameters. Spring generates a **proxy** that implements it by turning each call into an HTTP request (the same proxy idea as Chapter 4). It is Spring's built-in answer to Feign-style declarative clients (OpenFeign is a popular library doing the same) — no extra library.

```java
@HttpExchange("/catalog")
public interface CatalogApi {

    @GetExchange("/books/{id}")
    BookDto get(@PathVariable long id);

    @GetExchange("/books/{id}")
    ResponseEntity<BookDto> getEntity(@PathVariable long id);   // status + headers too

    @GetExchange("/books")
    List<BookDto> byAuthor(@RequestParam String author);

    @PostExchange("/books")
    ResponseEntity<BookDto> create(@RequestBody BookDto book);

    @GetExchange(url = "/greeting", version = "2.0")   // Framework 7: API version per method
    String greetingV2();
}
```

Annotation	Purpose
`@HttpExchange` (type)	shared URL prefix, content type, accept, headers, version
`@GetExchange`, `@PostExchange`, `@PutExchange`, `@PatchExchange`, `@DeleteExchange`	method + path
`@PathVariable`, `@RequestParam`, `@RequestHeader`, `@RequestBody`, `@RequestPart`, `@CookieValue`	the same annotations as server controllers, now describing the request to send
`URI` parameter	override the full URL dynamically

Return types: a body type (`BookDto`, `List<BookDto>`), `ResponseEntity<T>`, `ResponseEntity<Void>`, `HttpHeaders`, `void`; on `WebClient`, also `Mono<T>` / `Flux<T>`. Errors surface as the same `HttpClientErrorException` / `HttpServerErrorException` as `RestClient`.

Boot 4 auto-configuration: `@ImportHttpServices`

Before Framework 7 / Boot 4 you created each proxy yourself. Now you declare **groups** of interfaces; each group gets its own `RestClient` configured from properties:

```java
@SpringBootApplication
@EnableResilientMethods                                  // for @Retryable (Framework 7)
@ImportHttpServices(group = "catalog", types = CatalogApi.class)   // creates a CatalogApi bean
public class ClientsApplication { ... }
```

```properties
# Boot 4: HTTP service client group "catalog" (the @HttpExchange interface)
spring.http.serviceclient.catalog.base-url=http://localhost:8080
spring.http.serviceclient.catalog.read-timeout=2s
spring.http.serviceclient.catalog.default-header.X-Client=handbook-ch06
# the group's RestClient sends the API version (from @GetExchange(version=...)) in this header
spring.http.serviceclient.catalog.apiversion.insert.header=X-API-Version
```

- `types = ...` lists interfaces; `basePackages` / `basePackageClasses` scans a package instead. Repeat the annotation for several groups (one per remote service).
- `clientType = WEB_CLIENT` makes a group use `WebClient`.
- For configuration beyond properties, declare a `RestClientHttpServiceGroupConfigurer` bean: `groups -> groups.filterByName("catalog").forEachClient((group, builder) -> builder.requestInterceptor(...))`.

Then inject and call it like any bean (`HttpInterfaceTest.java`):

```java
@Autowired CatalogApi catalogApi;     // generated by @ImportHttpServices + Boot auto-config

BookDto one = catalogApi.get(1);
ResponseEntity<BookDto> e = catalogApi.getEntity(2);
ResponseEntity<BookDto> created = catalogApi.create(new BookDto(null, "Accelerate", "Nicole Forsgren"));
String v2 = catalogApi.greetingV2();
```

Output:

```
bean class     -> $ProxyN
get(1)         -> BookDto[id=1, title=Release It!, author=Michael T. Nygard]
byAuthor(...)  -> [BookDto[id=2, title=Refactoring, author=Martin Fowler], BookDto[id=3, title=Patterns of Enterprise
Application Architecture, author=Martin Fowler]]
getEntity(2)   -> 200 OK server saw X-Client=handbook-ch06
create(...)    -> 201 CREATED BookDto[id=101, title=Accelerate, author=Nicole Forsgren]
greetingV2()   -> v2: hello
get(999)       -> NotFound
```

The bean is a JDK dynamic proxy (`$Proxy...`). `X-Client=handbook-ch06` shows the per-group `default-header` property reached the server, and `greetingV2()` shows the method's `version = "2.0"` was sent in the `X-API-Version` header configured for the group.

What Boot does for you, by hand

```
RestClient restClient = RestClient.builder().baseUrl(baseUrl()).build();
HttpServiceProxyFactory factory = HttpServiceProxyFactory
        .builderFor(RestClientAdapter.create(restClient)).build();
CatalogApi manual = factory.createClient(CatalogApi.class);
```

```
manual proxy get(3) -> Patterns of Enterprise Application Architecture
```

(`WebClientAdapter` does the same on `WebClient`.) Useful in plain Spring, in tests, or in libraries.

Trade-offs

	HTTP interface	Hand-written `RestClient` class	OpenFeign (Spring Cloud)
Boilerplate	minimal	more	minimal
Custom per-call logic (fallbacks, `Optional` on 404)	needs a wrapper or default methods	natural	via fallbacks (alternative answers on failure)
Dependencies	none beyond Spring	none	Spring Cloud OpenFeign (maintenance mode — bug fixes only; Spring recommends HTTP interfaces)
Config per remote service	`spring.http.serviceclient.<group>.*` (Boot 4)	your own properties	`spring.cloud.openfeign.client.config.*`
Can share the interface with the server?	technically, but couples client and server release cycles	—	same caveat

Common approach: an HTTP interface for the raw calls + a thin service class on top for domain translation (`Optional`, own exceptions, retries).

6.7 Client-side API versioning and retries

API versioning (Framework 7)

When the server versions its API (Chapter 5, section 5.9), the client must send the version. Framework 7 adds this to every client: an `ApiVersionInserter` decides *how* (header, query param, path segment, media-type parameter), `defaultApiVersion` sets the usual version, and each request can override it with `.apiVersion(...)`:

```
RestClient rest = builder.baseUrl(baseUrl())
        .apiVersionInserter(ApiVersionInserter.useHeader("X-API-Version"))  // how to send it
        .defaultApiVersion("1.0")                                           // unless overridden
        .build();
String v1 = rest.get().uri("/catalog/greeting").retrieve().body(String.class);
String v2 = rest.get().uri("/catalog/greeting").apiVersion("2.0").retrieve().body(String.class);
```

Output:

```
default version -> v1: hello
apiVersion(2.0) -> v2: hello
```

HTTP interfaces put the version on the method (`@GetExchange(version = "2.0")`) and configure the inserter per group (`spring.http.serviceclient.<group>.apiversion.insert.header=...`), as shown in 6.6. Server-side version properties (`spring.mvc.apiversion.*`) are **not** applied to clients; configure them separately.

Retries

What it is

A **retry** repeats a failed call hoping the failure was transient. Framework 7's `@Retryable` (Chapter 4, section 4.7) makes this one annotation:

```
@Service
public class StockService {
    private final RestClient rest;

    public StockService(RestClient.Builder builder, @Value("${catalog.base-url}") String baseUrl) {
        this.rest = builder.baseUrl(baseUrl).build();
    }

    @Retryable(includes = org.springframework.web.client.HttpServerErrorException.class,
               maxRetries = 3, delay = 20, multiplier = 2, jitter = 5)
    public String checkStock() {
        return rest.get().uri("/catalog/flaky").retrieve().body(String.class);
    }
}
```

With `maxRetries = 3` there are at most 4 HTTP calls. The `/catalog/flaky` endpoint answers 503 twice, then 200:

```
checkStock() -> ok on call 3
```

Rules for safe retries

1. **Only transient failures:** 5xx (especially 502/503/504), timeouts/`ResourceAccessException`, 429 (respect `Retry-After`). Never 400/401/403/404/409/422.
2. **Only idempotent operations** — GET, PUT, DELETE — or POSTs protected by an **idempotency key** header that the server uses to de-duplicate (the way payment APIs such as Stripe do).
3. **Exponential back-off + jitter** (growing waits plus a small random amount) so thousands of clients don't hammer a recovering service in sync.
4. **Bound the total time**: attempts × read timeout + delays must fit your own deadline (`@Retryable(timeout = ...)` since 7.0.2 caps the total).
5. **Retry in one layer only**, and combine with a **circuit breaker** (stops calling a failing service for a while; e.g. Resilience4j) so a dead dependency is not retried forever — see Chapter 16 and Retry pattern.
6. `@Retryable` is proxy-based: it must be on a Spring bean method called from **another** bean.

6.8 Testing clients

Option 1 — `@RestClientTest` + `MockRestServiceServer` (fast, no network)

`@RestClientTest` is a **test slice**: it starts only the listed components, Jackson and the `RestClient.Builder`/`RestTemplateBuilder` auto-configuration, and binds a `MockRestServiceServer` to the builder. Requests never leave the JVM; the mock checks each request against expectations and returns canned responses; `server.verify()` checks every expected request was made. Boot 4 dependency: `spring-boot-starter-restclient-test`.

```java
@RestClientTest(components = CatalogClient.class, properties = "catalog.base-url=http://catalog.test")
class CatalogClientMockServerTest {

    @Autowired CatalogClient client;
    @Autowired MockRestServiceServer server;   // intercepts requests; no network at all

    @Test
    void foundNotFoundAndUnavailable() {
        server.expect(requestTo("http://catalog.test/catalog/books/1"))
              .andExpect(method(HttpMethod.GET))
              .andExpect(header("X-Client", "catalog-client"))
              .andRespond(withSuccess("""
                      {"id":1,"title":"Release It!","author":"Michael T. Nygard","pages":376}""",
                      MediaType.APPLICATION_JSON));
        server.expect(requestTo("http://catalog.test/catalog/books/2"))
              .andRespond(withStatus(HttpStatus.NOT_FOUND));
        server.expect(requestTo("http://catalog.test/catalog/books/3"))
              .andRespond(withStatus(HttpStatus.SERVICE_UNAVAILABLE));

        System.out.println("mock 200 -> " + client.findBook(1));
        System.out.println("mock 404 -> " + client.findBook(2));
        assertThatThrownBy(() -> client.findBook(3))
                .isInstanceOf(CatalogUnavailableException.class)
                .satisfies(e -> System.out.println("mock 503 -> " + e.getMessage()));
        server.verify();                        // every expected request really happened
    }
}
```

Output:

```
mock 200 -> Optional[BookDto[id=1, title=Release It!, author=Michael T. Nygard]]
mock 404 -> Optional.empty
mock 503 -> catalog answered 503 for /catalog/books/3
```

The unknown `pages` field was ignored. By default expectations must occur **in order**; `MockRestServiceServer.bindTo(builder).ignoreExpectOrder(true)` relaxes that. `withException(new SocketTimeoutException())` simulates I/O failures.

Option 2 — a real HTTP server

Mocks can't catch problems in the real HTTP layer (timeouts, headers, encoding, the version header). This chapter's other tests start the app on a real port with a local "remote" controller:

```java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT)
abstract class LiveServerTest {

    static final int PORT = freePort();

    @DynamicPropertySource
    static void urls(DynamicPropertyRegistry registry) {
        String base = "http://localhost:" + PORT;
        registry.add("server.port", () -> PORT);
        registry.add("catalog.base-url", () -> base);
        registry.add("spring.http.serviceclient.catalog.base-url", () -> base);
    }
    ...
}
```

The port is picked **before** the context starts so the client base URLs can be set as properties (with `RANDOM_PORT` the port is only known after the clients were created). In real projects the "remote" side is usually a **WireMock** stub server (often via Testcontainers) or a Spring Cloud Contract stub (generated from an agreed API contract) — see Chapter 12.

For testing **your own** endpoints over HTTP, Boot 4 offers **`RestTestClient`** (`@AutoConfigureRestTestClient`), the `RestClient`-based successor of `TestRestTemplate` (which now needs `@AutoConfigureTestRestTemplate` and the `spring-boot-resttestclient` module).

6.9 `RestTemplate` and `WebClient` in brief

`RestTemplate` (legacy)

```
RestTemplate template = restTemplateBuilder.rootUri(baseUrl()).build();
BookDto viaTemplate = template.getForObject("/catalog/books/{id}", BookDto.class, 2);

RestClient wrapped = RestClient.create(template);     // migration step: reuse its config
BookDto viaWrapped = wrapped.get().uri("/catalog/books/{id}", 2).retrieve().body(BookDto.class);
```

```
RestTemplate  -> BookDto[id=2, title=Refactoring, author=Martin Fowler]
RestClient.create(template) -> BookDto[id=2, title=Refactoring, author=Martin Fowler]
```

Boot auto-configures a `RestTemplateBuilder` (not a `RestTemplate`). Migration map:

`RestTemplate`	`RestClient`
`getForObject(url, T.class, vars)`	`get().uri(url, vars).retrieve().body(T.class)`
`getForEntity(...)`	`get().uri(...).retrieve().toEntity(T.class)`
`postForEntity(url, body, T.class)`	`post().uri(url).body(body).retrieve().toEntity(T.class)`
`exchange(url, method, entity, T.class)`	`method(m).uri(url).headers(...).body(...).retrieve().toEntity(T.class)`
`setErrorHandler(...)`	`defaultStatusHandler(...)`
interceptors / request factory	reused as-is (`requestInterceptor` , `requestFactory`)

Steps: (1) wrap existing templates with `RestClient.create(restTemplate)` or `RestClient.builder(restTemplate)` so configuration is reused; (2) rewrite call sites; (3) move shared config to a `RestClientCustomizer` / builder; (4) remove the templates.

`WebClient` (reactive) — not run here

```
WebClient web = webClientBuilder.baseUrl("https://catalog.example.com").build();
Mono<BookDto> book = web.get().uri("/catalog/books/{id}", 1)
        .retrieve()
        .bodyToMono(BookDto.class)
        .timeout(Duration.ofSeconds(2));
```

Same shape as `RestClient`, but returns `Mono` / `Flux` and never blocks a thread. Use it in WebFlux apps or for streaming; don't call `.block()` inside a reactive pipeline. Full coverage in Chapter 13.

6.10 Common mistakes, real-world examples, interview

Common mistakes

Mistake	Consequence	Fix
New `RestClient` per request	loses connection pooling, slower, more sockets	build once, reuse
`RestClient.create()` in a Boot app	no Boot converters/timeouts/observability	inject `RestClient.Builder`
No timeouts	thread pile-up, cascading failure	`spring.http.clients.*-timeout` or per-group/per-client settings
String-concatenated URLs	broken encoding, injection (attacker text changing the URL's meaning)	URI templates / `UriBuilder`
Using `exchange()` and expecting status handlers	errors silently treated as success	handle statuses yourself or use `retrieve()`
Retrying POSTs / 4xx	duplicate orders, wasted calls	retry only idempotent + transient
Leaking `HttpClientErrorException` into business code	HTTP details spread everywhere	translate at the client edge
Mock-only tests	real HTTP issues missed	add a test against a real/stub server

Real-world examples

- **Payment integration**: a `PaymentsApi` HTTP interface for the provider, read timeout ~5 s, retries only for GETs and for POSTs carrying an `Idempotency-Key`, errors mapped to `PaymentDeclined` vs `PaymentProviderUnavailable`.
- **Microservice calls** inside a company: one `@ImportHttpServices` group per downstream service (`inventory`, `pricing`, `shipping`) each with its own base URL and timeouts from config, tracing headers propagated automatically.
- **Third-party APIs with versions** (e.g. APIs that version by header or date): a default API version on the client, bumped deliberately after testing.

In the interview

Q: Which HTTP client would you use in a new Spring Boot 4 MVC service, and why? 30-second answer: "`RestClient` — or an `@HttpExchange` interface backed by it. It's the modern synchronous client: fluent, uses Boot's auto-configured converters, HTTP library, timeouts and observability. `RestTemplate` is being deprecated (formally in 7.1, removal in 8.0), and `WebClient` is for reactive stacks. With virtual threads, blocking calls are cheap enough that I don't need reactive just for concurrency."

- *How do HTTP interfaces get implemented?* → `HttpServiceProxyFactory` creates a JDK dynamic proxy; in Boot 4 `@ImportHttpServices` registers groups and `spring.http.serviceclient.<group>.*` configures each group's client.
- *How do you handle errors?* → `retrieve()` throws `HttpClientErrorException` / `HttpServerErrorException` (with nested per-status types) and `ResourceAccessException` for I/O/timeouts; I map them to domain exceptions with `onStatus` / `defaultStatusHandler` and read `ProblemDetail` bodies with `getResponseBodyAs`.
- *What timeouts do you set?* → Connect and read timeouts on every client (`spring.http.clients.connect-timeout/read-timeout` or per group), sized from downstream p99 and my own SLA (the response time/availability promised to my callers).

Q: How do you make remote calls resilient? → Timeouts, retries with exponential back-off + jitter only for idempotent/transient failures (`@Retryable` in Framework 7), circuit breaker + bulkhead (Resilience4j / `@ConcurrencyLimit`), fallbacks where acceptable, and idempotency keys for writes.

Q: How do you test a REST client? → `@RestClientTest` + `MockRestServiceServer` for fast request/response contract checks; plus an integration test against a real stub server (WireMock) or the provider's contract stubs for the HTTP layer.

6.11 Chapter TL;DR

- **Clients:** `RestClient` (sync, recommended) · HTTP interfaces `@HttpExchange` (declarative, on `RestClient` or `WebClient`) · `WebClient` (reactive) · `RestTemplate` (legacy; deprecated from 7.1, removed in 8.0).
- **Boot 4:** add `spring-boot-starter-restclient`; inject the **prototype** `RestClient.Builder`; build once, reuse (thread-safe).
- **Calls:** `get().uri(template, vars).retrieve().body(T)` / `toEntity(T)` / `toBodilessEntity()`; `ParameterizedTypeReference` for generics; `exchange()` for raw control (**status handlers not applied**).
- **Errors:** 4xx → `HttpClientErrorException.*`, 5xx → `HttpServerErrorException.*`, I/O/timeouts → `ResourceAccessException`; `getResponseBodyAs(ProblemDetail.class)`; translate with `onStatus` / `defaultStatusHandler`.
- **Timeouts:** `spring.http.clients.connect-timeout` / `read-timeout` (Boot 4 names), per group `spring.http.serviceclient.<g>.read-timeout`, or `ClientHttpRequestFactoryBuilder` + `HttpClientSettings`. Library order: Apache → Jetty → Reactor Netty → JDK → simple.
- **HTTP interfaces in Boot 4:** `@ImportHttpServices(group = "...", types = ...)` + `spring.http.serviceclient.<group>.base-url/default-header/apiversion.*`; customise with `RestClientHttpServiceGroupConfigurer`.
- **Versioning (Framework 7):** `ApiVersionInserter.useHeader(...)` + `defaultApiVersion(...)`, per-request `.apiVersion(...)`, `@GetExchange(version = ...)`.
- **Retries:** `@Retryable` for transient + idempotent calls only, with back-off, jitter and a total budget; add a circuit breaker.
- **Tests:** `@RestClientTest` + `MockRestServiceServer`; real/stub server tests for the HTTP layer; `RestTestClient` for your own endpoints.
- Next: Chapter 7 — Data access with JDBC and transactions.

Chapter 7 — Data Access with JDBC and Transactions

Almost every Spring application stores data in a relational database (tables of rows queried with SQL, e.g. PostgreSQL). This chapter covers the plumbing: how Boot creates a **connection pool**, running SQL with `JdbcClient` and `JdbcTemplate`, and translating database errors into one exception family. Most of it is about **transactions** (changes saved together or not at all): what `@Transactional` really does, the seven propagation types (what happens when one transactional method calls another), rollback rules, isolation (how much concurrent transactions see of each other), read-only and timeouts, and the pitfalls that make `@Transactional` silently do nothing.

This is not a SQL course. Transactions at the database level (ACID — Atomicity, Consistency, Isolation, Durability; isolation anomalies; MVCC — multi-version concurrency control, readers see a snapshot instead of waiting for writers; locks) are taught in the SQL book: SQL ch13 — Transactions and concurrency. Here we look at the Spring side: who opens the transaction, on which connection, and when it commits (makes its changes permanent).

The code (Spring Boot 4.1.1, Spring Framework 7.0, Java 25):

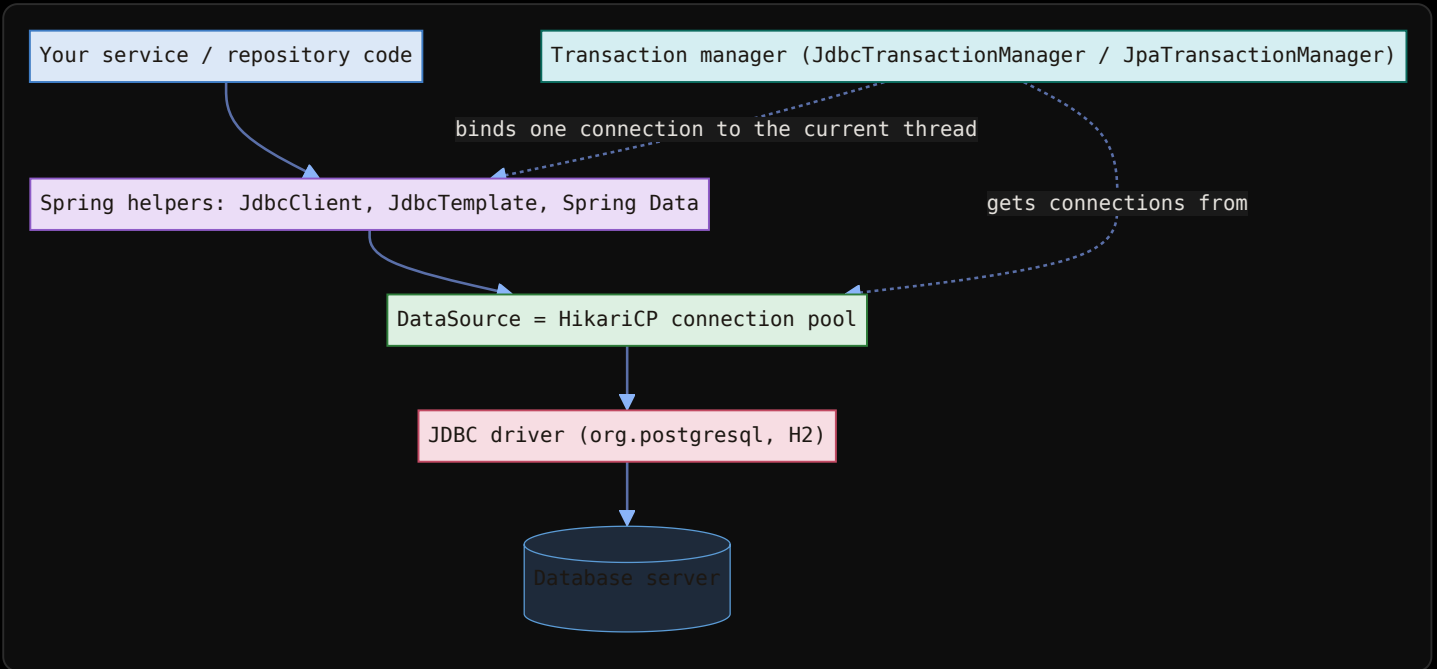
Project	Database	What it proves
`code/jdbc-basics`	H2 in memory	HikariCP settings, `JdbcClient` / `JdbcTemplate`, batch, exception translation, `@JdbcTest`
`code/tx-demos`	the book's private PostgreSQL 14 (port 5544)	propagation, rollback rules, pitfalls, isolation, read-only, timeout, events, `TransactionTemplate`

The transaction demos use real PostgreSQL; H2 (a small in-memory Java database) would hide how read-only transactions and serialization failures (the database refusing a transaction because a concurrent one changed the same data) act. The `tx-demos` test run creates a database called `spring_ch07_tx` before the tests and drops it afterwards (see 7.7). Every result below was printed by a test and checked by an assertion. Run everything with `bash tools_run_spring.sh spring '07-*'` from the repository root. Start PostgreSQL first with `bash tools_pg.sh start`.

7.1 The data-access stack

What it is

A `SELECT` passes through several layers, each with one job:



Layer	Job	Spring Boot gives you
JDBC (Java Database Connectivity)	The standard Java API for SQL: `Connection`, `PreparedStatement`, `ResultSet`	the driver from your dependencies
DataSource	A factory for connections. In practice it is a **pool** that keeps connections open and lends them out	`HikariDataSource`, set up from `spring.datasource.*`
Spring JDBC helpers	Take away the boilerplate (repetitive setup code): open/close, loops over `ResultSet`, turning `SQLException` into something useful	`JdbcTemplate`, `NamedParameterJdbcTemplate`, `JdbcClient` beans
Transaction manager	Starts, commits and rolls back transactions. Keeps the current connection on the current thread (each request usually runs on its own thread) so every helper call in one transaction uses the same connection	`JdbcTransactionManager` (JDBC) or `JpaTransactionManager` (JPA, ch08)

Plain JDBC (without Spring) is covered in the SQL book: SQL 14.11 — Using SQL from Java (JDBC). Compared with that code, Spring adds three things: a pool, much less boilerplate, and declarative transactions (you mark a method with an annotation instead of writing begin/commit code).

Choosing an API

Need	Use
Hand-written SQL, full control, reporting queries	`JdbcClient` (or `JdbcTemplate`) — this chapter
Simple aggregates (a root object plus the objects it owns, saved as one unit), no ORM (Object-Relational Mapper, which maps objects to tables and writes SQL for you), predictable SQL	Spring Data JDBC — ch09
Rich domain model with relationships, change tracking (the library notices which objects you modified)	JPA / Hibernate + Spring Data JPA — ch08

You can mix them in one application. A `JdbcClient` query and a JPA repository call can even share one transaction when the `JpaTransactionManager` is in charge.

7.2 DataSource and HikariCP

What it is

A **connection pool** keeps a set of open database connections and lends one to each piece of work, because opening a connection (TCP handshake, authentication, session setup) is expensive. **HikariCP** is the pool Spring Boot picks first whenever it is on the classpath. `spring-boot-starter-jdbc` and `spring-boot-starter-data-jpa` both bring it in.

How Boot builds it

1. You add a starter plus a driver (`org.postgresql:postgresql`, `com.h2database:h2`).
2. Boot's `DataSourceAutoConfiguration` reads `spring.datasource.url`, `username` and `password`.
3. If you give **no URL** and an embedded database (runs inside your Java process: H2, HSQLDB, Derby) is on the classpath, Boot creates an in-memory database whose name is unique per application context (`spring.datasource.generate-unique-name=true` by default), so test contexts never see each other's tables.
4. Pool settings go under `spring.datasource.hikari.*`.

```
# jdbc-basics/src/main/resources/application.properties
spring.datasource.url=jdbc:h2:mem:shop;DB_CLOSE_DELAY=-1
spring.datasource.username=sa
spring.datasource.hikari.maximum-pool-size=5
spring.datasource.hikari.pool-name=shop-pool
spring.datasource.hikari.connection-timeout=2000
```

The test reads the settings back from the real bean:

```
HikariDataSource hikari = (HikariDataSource) dataSource;
System.out.println("DataSource class = " + hikari.getClass().getSimpleName()
        + ", pool = " + hikari.getPoolName()
        + ", maxPoolSize = " + hikari.getMaximumPoolSize()
        + ", connectionTimeout = " + hikari.getConnectionTimeout() + " ms");
```

Output:

```
DataSource class = HikariDataSource, pool = shop-pool, maxPoolSize = 5, connectionTimeout = 2000 ms
```

The settings that matter

Property (`spring.datasource.hikari.`)	HikariCP default	Meaning
`maximum-pool-size`	10	Most connections the pool will ever open
`minimum-idle`	same as max	Idle (open but unused) connections kept ready. Leaving it unset gives a fixed-size pool, which HikariCP recommends — it never has to open connections during a traffic spike
`connection-timeout`	30 000 ms	How long a thread waits for a free connection; then `SQLTransientConnectionException`, which Spring wraps as `CannotGetJdbcConnectionException`
`idle-timeout`	600 000 ms	Idle connections above `minimum-idle` are closed after this
`max-lifetime`	1 800 000 ms	Connections are retired after this. Keep it shorter than any database or firewall idle limit
`keepalive-time`	120 000 ms	Pings idle connections so network gear does not drop them
`leak-detection-threshold`	0 (off)	Logs a stack trace when a connection is held longer than this. Good for finding leaks (connections never returned) in staging

Boot 4.1 added `spring.datasource.connection-fetch=lazy`. It wraps the pool in Spring's `LazyConnectionDataSourceProxy`, so a transaction takes a physical connection only when the first SQL statement runs. A `@Transactional` method that returns early without touching the database never borrows a connection.

Pool sizing — the counter-intuitive part

A bigger pool is not faster. The database has a fixed number of CPU cores and disks. Hundreds of connections all running at once make it switch between them and fight over locks. HikariCP's own guidance starts from a small pool, roughly *cores* × *2* + *effective disks*, and then measures. Two rules follow from this:

- **Sum across instances.** 20 application pods (running copies) × pool 10 = 200 connections, and the default PostgreSQL `max_connections` is 100.
- **Hold connections briefly.** A transaction that calls a slow HTTP API keeps its connection the whole time (see 7.13).

More on pooling in general: SQL 14.12 — Connection pooling.

> **Common mistakes**

- Setting `maximum-pool-size=100` "to be safe", then running out of database connections as soon as you scale out.
- Leaving `connection-timeout` at 30 s in a user-facing API. Requests hang for 30 s when the pool is exhausted, when failing fast would be better.
- Forgetting the pool when you add virtual threads (cheap JVM-managed threads) (ch15). Thousands of virtual threads still share, say, 10 connections, and the pool becomes the limit.

7.3 JdbcClient and JdbcTemplate

What it is

Both run SQL you write yourself. They open and close the connection, bind parameters, loop over rows, map each row to an object, and translate errors.

- **`JdbcTemplate`** (since Spring 1.x): the classic API with many overloaded methods, `?` placeholders (filled with values safely), and a `RowMapper` (a function turning one result row into an object).
- **`NamedParameterJdbcTemplate`**: the same, with `:name` placeholders.
- **`JdbcClient`** (Spring Framework 6.1+): one fluent API that covers both styles. Boot auto-configures it from the `NamedParameterJdbcTemplate`. It is the recommended default for new code.

Querying with JdbcClient

```
@Repository
public class ProductRepository {

    private final JdbcClient jdbc;

    public ProductRepository(JdbcClient jdbc) { this.jdbc = jdbc; }

    public List<Product> findAll() {
        return jdbc.sql("SELECT id, sku, name, price, in_stock FROM product ORDER BY id")
                   .query(Product.class)          // columns -> record components by name
                   .list();
    }

    public Optional<Product> findBySku(String sku) {
        return jdbc.sql("SELECT id, sku, name, price, in_stock FROM product WHERE sku = :sku")
                   .param("sku", sku)             // named parameter
                   .query(Product.class)
                   .optional();                   // 0 rows -> empty, 2+ rows -> exception
    }

    public List<String> namesCheaperThan(BigDecimal max) {
        return jdbc.sql("SELECT name FROM product WHERE price < ? ORDER BY price")
                   .param(max)                     // positional parameter
                   .query(String.class)            // one column -> simple type
                   .list();
    }
    // ...
}
```

`Product` is a record: `record Product(Long id, String sku, String name, BigDecimal price, int inStock)`. `query(Product.class)` maps columns to constructor parameters by name. It understands `in_stock` → `inStock` because it ignores underscores and case. For a single column, `query(String.class)` (or `Long.class`, ...) returns plain values.

Output:

```
row: Product[id=1, sku=KB-01, name=Mechanical keyboard, price=89.00, inStock=12]
row: Product[id=2, sku=MS-01, name=Wireless mouse, price=25.50, inStock=40]
row: Product[id=3, sku=MN-27, name=27-inch monitor, price=249.99, inStock=3]
findBySku(MS-01) = Optional[Product[id=2, sku=MS-01, name=Wireless mouse, price=25.50, inStock=40]]
findBySku(NOPE)  = Optional.empty
cheaper than 100 = [Wireless mouse, Mechanical keyboard]
```

The terminal methods after `query(...)`:

Method	Returns	When
`.list()`	`List<T>`	any number of rows
`.optional()`	`Optional<T>`	0 or 1 row. More than one throws `IncorrectResultSizeDataAccessException`
`.single()`	`T`	exactly 1 row. Zero rows throws `EmptyResultDataAccessException`
`.set()` / `.stream()`	`Set<T>` / `Stream<T>`	a set, or streaming. Close the stream (try-with-resources)
`.query(rowMapper)` / `.query(rowCallbackHandler)`	custom	full control over mapping

Writing: generated keys and row counts

```
public long insert(String sku, String name, BigDecimal price, int stock) {
    KeyHolder keys = new GeneratedKeyHolder();
    jdbc.sql("INSERT INTO product (sku, name, price, in_stock) VALUES (:sku, :name, :price, :stock)")
        .param("sku", sku).param("name", name).param("price", price).param("stock", stock)
        .update(keys, "id");                  // ask the driver to return the generated "id"
    return keys.getKeyAs(Long.class);
}

public int changeStock(String sku, int delta) {
    return jdbc.sql("UPDATE product SET in_stock = in_stock + :d WHERE sku = :sku")
               .param("d", delta).param("sku", sku)
               .update();                     // number of rows changed
}
```

Output:

```
generated id = 6, rows updated = 1, now = Product[id=6, sku=HD-01, name=Headset, price=59.90, inStock=5]
```

Why 6, with only three rows? Earlier tests inserted rows and were rolled back, but **identity and sequence values (auto-increment id counters) are never rolled back** — the database hands out numbers without waiting for the commit, so gaps are normal. Never use ids to count rows or assume gap-free ordering.

The same with JdbcTemplate, plus a batch

```
private static final RowMapper<Product> MAPPER = (rs, rowNum) -> new Product(
        rs.getLong("id"), rs.getString("sku"), rs.getString("name"),
        rs.getBigDecimal("price"), rs.getInt("in_stock"));

public List<Product> inStockAtLeast(int min) {
    return jdbc.query("SELECT * FROM product WHERE in_stock >= ? ORDER BY sku", MAPPER, min);
}

/** One JDBC batch: one round trip per batch instead of one per row. */
public int[] restock(List<String> skus, int amount) {
    return jdbc.batchUpdate("UPDATE product SET in_stock = in_stock + ? WHERE sku = ?",
            skus.stream().map(s -> new Object[] {amount, s}).toList());
}
```

Output:

```
batch update counts = [1, 1, 0]
in stock >= 13: [KB-01, MN-27, MS-01]
```

The batch returns one count per parameter set (`NOPE` matched no row → 0). It sends the statements in one network round trip — for 10 000 rows, a handful of round trips instead of 10 000.

Trade-offs

	`JdbcTemplate`	`NamedParameterJdbcTemplate`	`JdbcClient`
Placeholders	`?`	`:name`	both
Style	overloaded methods	overloaded methods	fluent chain
Record/bean mapping	`DataClassRowMapper` by hand	same	automatic with `query(Type.class)`
Batch updates	`batchUpdate`	`batchUpdate`	not in the fluent API. Use `JdbcTemplate`
Stored procedures	`SimpleJdbcCall`	—	—

Common mistakes

- **String concatenation in SQL** (`"... WHERE sku = '" + sku + "'"`) → SQL injection (attacker input runs as SQL). Always bind parameters. See SQL 14.9 — SQL injection.
- Using `.single()` when "no row" is a normal case. Use `.optional()`.
- `SELECT *` into a record. A column added later breaks nothing, but a renamed one fails at runtime. List the columns.
- Loading a million rows with `.list()`. Stream them, or page with keyset pagination (SQL 12.14).

In the interview

> **30-second answer:** "`JdbcTemplate` and `JdbcClient` run hand-written SQL and handle connections, parameter binding, row mapping and exception translation. `JdbcClient` is the fluent API from Spring 6.1 and supports named and positional parameters and automatic mapping to records. Both take their connection from the `DataSource` pool. If a transaction is active, they use the connection bound to the current thread, so they take part in `@Transactional` automatically."

- *How does `JdbcTemplate` know about the current transaction?* → It gets its connection through `DataSourceUtils.getConnection`, which first checks `TransactionSynchronizationManager` for a connection already bound to this thread.
- *Is `JdbcTemplate` thread-safe?* → Yes. After configuration it holds no per-call state, so one shared bean is fine.

7.4 Schema and data initialization

Spring Boot can run SQL scripts at startup:

File	Runs	Purpose
`schema.sql` (and `schema-<platform>.sql`)	first	`CREATE TABLE ...`
`data.sql` (and `data-<platform>.sql`)	after `schema.sql`	seed rows

Property	Default	Meaning
`spring.sql.init.mode`	`embedded`	`embedded` = only for H2/HSQLDB/Derby, `always` = any database, `never` = off
`spring.sql.init.platform`	`all`	picks `schema-${platform}.sql`
`spring.sql.init.continue-on-error`	`false`	stop startup on a failing statement
`spring.jpa.defer-datasource-initialization`	`false`	with JPA: run the scripts *after* Hibernate has created the tables

`jdbc-basics` uses H2, so the scripts would run anyway. The project sets `mode=always` to make that visible. `tx-demos` points at PostgreSQL, which is not embedded, so there `mode=always` is required. Its `schema.sql` begins with `DROP TABLE IF EXISTS` so it can run again on every startup.

```sql
-- tx-demos/src/main/resources/schema.sql
CREATE TABLE account (
    owner   TEXT PRIMARY KEY,
    balance BIGINT NOT NULL CHECK (balance >= 0)
);
CREATE TABLE audit_log (id BIGSERIAL PRIMARY KEY, message TEXT NOT NULL);
```

When not to use it

In production the schema changes over time, and a script that only ever creates tables cannot handle that. Use a **migration tool** (Flyway or Liquibase: tools that keep track of which schema changes a database has already received) that applies numbered, versioned changes exactly once. See 8.16. `schema.sql` is fine for demos, tests and prototypes.

7.5 Exception translation: DataAccessException

The problem it solves

JDBC reports every failure as a checked `SQLException` carrying a vendor error code and an SQLState (a standard five-character code). Telling "duplicate key" from "deadlock" (two transactions each waiting for the other's lock) would mean checking vendor codes like PostgreSQL's `23505` or MySQL's `1062`. Spring translates them into one **unchecked** hierarchy, the same for every database and data-access technology (JDBC, JPA, R2DBC — a non-blocking database API, ...).

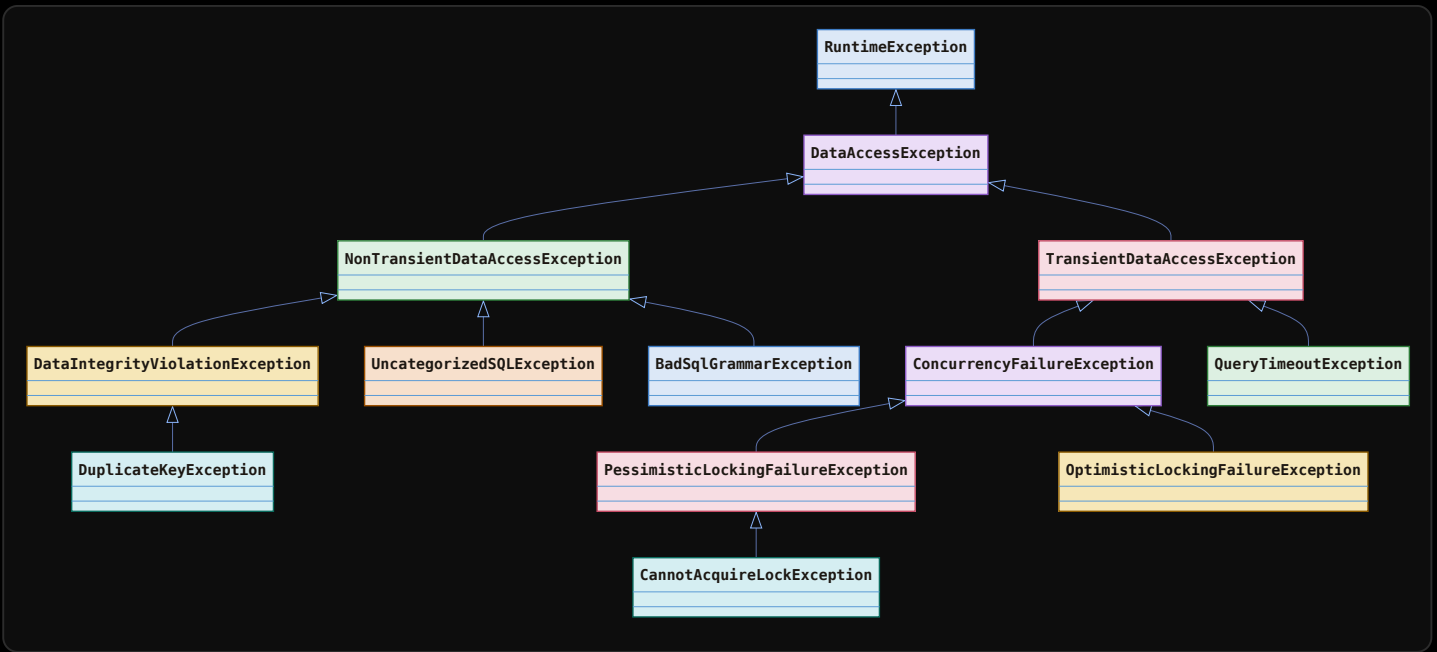
```
assertThatThrownBy(() -> repo.insert("KB-01", "Duplicate", BigDecimal.ONE, 1))
        .isInstanceOf(DuplicateKeyException.class)
        .isInstanceOf(DataAccessException.class);
assertThatThrownBy(() -> repo.insert("NEG-1", "Negative", new BigDecimal("-1"), 1))
        .isInstanceOf(DataIntegrityViolationException.class);
```

Output:

```
duplicate sku -> DuplicateKeyException
CHECK violated -> DataIntegrityViolationException
```

The part of the hierarchy you will meet

The diagram is simplified: a few intermediate classes are left out. For example, `BadSqlGrammarException` really extends `InvalidDataAccessResourceUsageException`.



Exception	Typical cause	Retry?
`DuplicateKeyException`	unique/primary key violated	no — fix the data
`DataIntegrityViolationException`	NOT NULL, CHECK, foreign key	no
`BadSqlGrammarException`	typo in SQL, missing table	no — bug
`CannotAcquireLockException`, `PessimisticLockingFailureException`	lock timeout, deadlock, PostgreSQL serialization failure	**yes**, the whole transaction
`OptimisticLockingFailureException`	`@Version` conflict (someone else saved the row since you read it) (8.14)	yes, after reloading
`QueryTimeoutException`	statement cancelled by a timeout	maybe
`UncategorizedSQLException`	an error Spring has no specific class for	inspect `getCause()`

The **transient** branch means "the same operation might succeed if you try again". That is exactly the test to use when deciding whether to retry (ch04 covers `@Retryable`).

For JPA repositories the same translation is applied by a proxy around `@Repository` beans. In Boot 4 the switch is `spring.persistence.exceptiontranslation.enabled`. It was `spring.dao.exceptiontranslation.enabled` before Boot 4.

7.6 Testing JDBC code with @JdbcTest

`@JdbcTest` is a **test slice**: it starts only the JDBC parts of the application (`DataSource`, `JdbcTemplate`, `JdbcClient`, the transaction manager, SQL init) and none of your services or controllers. Two defaults matter:

1. It replaces your `DataSource` with a fresh embedded database. `@AutoConfigureTestDatabase(replace = Replace.NONE)` keeps yours, which is how the Hikari settings above could be checked.
2. **Every test method runs in a transaction that is rolled back afterwards**, so tests do not see each other's changes.

```
@JdbcTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Import({ProductRepository.class, ProductTemplateDao.class})   // our beans are not in the slice
class JdbcClientTest {
    @Test
    void eachTestRunsInsideARolledBackTransaction() {
        System.out.println("test method in a transaction? "
                + TransactionSynchronizationManager.isActualTransactionActive());
        repo.changeStock("KB-01", -12);          // undone after the test
    }
}
```

Output:

```
test method in a transaction? true
```

In Boot 4 the slice annotations moved with the modules: `org.springframework.boot.jdbc.test.autoconfigure.JdbcTest`, from `spring-boot-starter-jdbc-test`. The rolled-back test transaction is convenient, but it can **hide transaction bugs**. Code that only works because the test opened a transaction will pass in the test and fail in production. That is why every demo in `tx-demos` is a non-transactional `@SpringBootTest`. More in ch12.

7.7 How a Spring transaction works

What it is

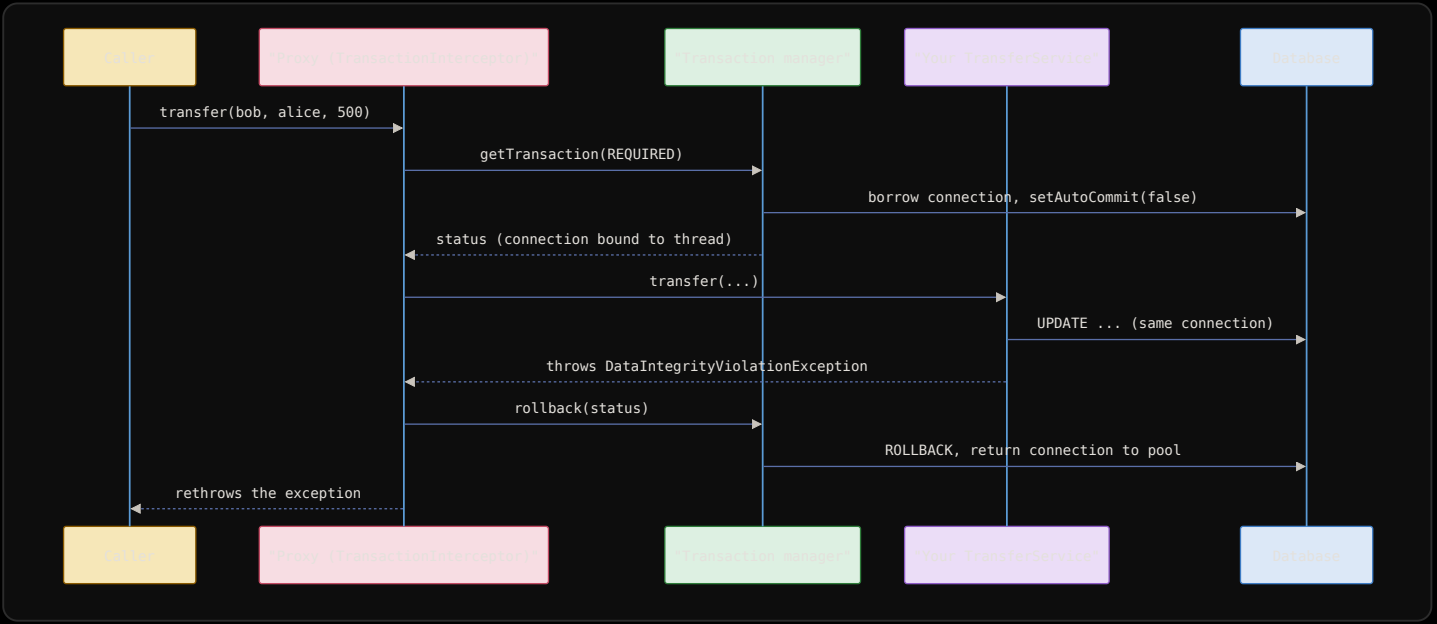
A **database transaction** groups statements so they all commit or all roll back (see SQL 13.1). On one JDBC connection that means `setAutoCommit(false)`, the statements, then `commit()` or `rollback()`. A **Spring transaction**

does that work for you and adds three things:

1. **One connection per transaction, bound to the thread.** Every `JdbcClient`, `JdbcTemplate` or JPA call inside the transaction, in any class, finds that connection through `TransactionSynchronizationManager`, which works like a `ThreadLocal` (per-thread variable) map.
2. **Declarative boundaries.** `@Transactional` on a method means "wrap this method in a transaction".
3. **Rules** for what to do when a transactional method calls another one (propagation), when to roll back, which isolation level to use, and more.

How @Transactional works, step by step

Spring does not change your class. It puts a **proxy** in front of the bean (AOP — see ch04). Other beans are injected with the proxy, not your object.



1. The caller calls the proxy.
2. The `TransactionInterceptor` reads the `@Transactional` attributes and asks the `PlatformTransactionManager` for a transaction.
3. The manager takes a connection from the pool, switches off auto-commit, applies isolation/read-only/timeout, and **binds the connection to the current thread**.
4. Your method runs. Every data-access call finds the bound connection.
5. If the method returns normally, the manager commits. If it throws, the rollback rules decide between commit and rollback (7.10).
6. The connection is unbound and returned to the pool.

Which transaction manager?

You use	Boot auto-configures	Notes
JDBC only (`spring-boot-starter-jdbc`)	`JdbcTransactionManager`	a `DataSourceTransactionManager` that also translates commit errors to `DataAccessException`
JPA (`spring-boot-starter-data-jpa`)	`JpaTransactionManager`	also manages the `EntityManager`. JDBC calls on the same `DataSource` join its transactions
Several resources (JMS — Java's messaging API — + DB)	none by default	a JTA (Java Transaction API) manager, or better, avoid distributed transactions (slow and fragile; see the outbox idea in ch14)
Reactive (R2DBC)	`R2dbcTransactionManager`	`ReactiveTransactionManager`, context-bound instead of thread-bound (ch13)

Seeing it

The demos print a small "where am I" string: whether a Spring transaction is active, the transaction's name (the method that started it), and the PostgreSQL **backend pid**. Every PostgreSQL connection is served by its

own server process, so the same pid means the same connection.

```
public String where() {
    int pid = jdbc.sql("SELECT pg_backend_pid()").query(Integer.class).single();
    return "txActive=" + TransactionSynchronizationManager.isActualTransactionActive()
            + " name=" + TransactionSynchronizationManager.getCurrentTransactionName()
            + " pid=" + pid;
}
```

Test plumbing for the PostgreSQL demos

`tx-demos` needs its own database. A JUnit `LauncherSessionListener` (plain JUnit, not Spring, registered in `META-INF/services`) runs before the first test and after the last one. It connects to the admin database and creates or drops `spring_ch07_tx`:

```
public class DemoDatabase implements LauncherSessionListener {
    public void launcherSessionOpened(LauncherSession s) {
        run("DROP DATABASE IF EXISTS spring_ch07_tx WITH (FORCE)");
        run("CREATE DATABASE spring_ch07_tx");
    }
    public void launcherSessionClosed(LauncherSession s) {
        run("DROP DATABASE IF EXISTS spring_ch07_tx WITH (FORCE)"); // pool may still hold connections
    }
    // run(sql) = DriverManager.getConnection("jdbc:postgresql://127.0.0.1:5544/postgres", ...) + execute
}
```

The tests extend a base class that is **not** `@Transactional`. Commits and rollbacks really happen, and `@BeforeEach` resets the data to `alice = 100, bob = 50`.

7.8 @Transactional attributes

Attribute	Default	What it does
`propagation`	`REQUIRED`	what to do if a transaction already exists (7.9)
`isolation`	`DEFAULT` (= the database's default, READ COMMITTED on PostgreSQL)	7.11
`readOnly`	`false`	hint, and on some drivers a rule, that nothing is written (7.12)
`timeout` / `timeoutString`	none	seconds before statements are cancelled
`rollbackFor` / `rollbackForClassName`	—	also roll back for these (checked) exceptions
`noRollbackFor` / `noRollbackForClassName`	—	commit even though this exception was thrown
`transactionManager` (alias `value`)	the primary one	choose among several managers
`label`	—	free-form labels that a custom manager can read

Where to put it. On a class, it applies to every method the proxy can intercept. On a method, it overrides the class. The Spring team recommends annotating **concrete classes**, not interfaces. A common pattern is `@Transactional(readOnly = true)` on the class with `@Transactional` on the write methods. Spring Data's `SimpleJpaRepository` is built exactly that way.

7.9 Propagation

What it is

Propagation answers: *this method is transactional, and a transaction may already be running. What now?*

Propagation	Transaction already running	No transaction running	Typical use
`REQUIRED` (default)	**join** it	start one	almost everything
`REQUIRES_NEW`	**suspend** it, start an independent one (new connection)	start one	audit logs that must survive a rollback, id generators
`NESTED`	create a **savepoint** (a marker you can roll back to without undoing everything) inside it	start one	"try this part, undo just this part on failure" (JDBC only)
`MANDATORY`	join it	**throw** `IllegalTransactionStateException`	methods that must never run on their own
`SUPPORTS`	join it	run **without**	read helpers that do not care
`NOT_SUPPORTED`	suspend it, run without	run without	slow non-transactional work inside a transactional flow
`NEVER`	**throw** `IllegalTransactionStateException`	run without	guard: "must not be called inside a transaction"

Demo 1 — one outer transaction calling each type

`OuterService.callEachPropagation()` is `@Transactional`. It calls methods on a **different** bean (`AuditService`, so the calls go through the proxy), and each method has a different propagation:

```
@Transactional
public List<String> callEachPropagation() {
    List<String> seen = new ArrayList<>();
    seen.add("outer        : " + db.where());
    seen.add("REQUIRED     : " + audit.required("required"));
    seen.add("REQUIRES_NEW : " + audit.requiresNew("requires-new"));
    seen.add("NESTED       : " + audit.nestedAndMaybeFail("nested", false));
    seen.add("MANDATORY    : " + audit.mandatory("mandatory"));
    seen.add("SUPPORTS     : " + audit.supports());
    seen.add("NOT_SUPPORTED: " + audit.notSupported());
    return seen;
}
```

The test raises the transaction manager's log level to DEBUG so you can read its decisions. Output (logger name and connection hashes shortened, a few "Initiating commit" and "Releasing connection" lines left out):

```
DEBUG JdbcTransactionManager - Creating new transaction with name [ch07.tx.OuterService.callEachPropagation]:
PROPAGATION_REQUIRED,ISOLATION_DEFAULT
DEBUG JdbcTransactionManager - Acquired Connection [HikariProxyConnection@... wrapping PgConnection@2b6262bc] for JDBC
transaction
DEBUG JdbcTransactionManager - Switching JDBC Connection [...PgConnection@2b6262bc] to manual commit
DEBUG JdbcTransactionManager - Participating in existing transaction
DEBUG JdbcTransactionManager - Suspending current transaction, creating new transaction with name
[ch07.tx.AuditService.requiresNew]
DEBUG JdbcTransactionManager - Acquired Connection [...PgConnection@3e28dc96] for JDBC transaction
DEBUG JdbcTransactionManager - Committing JDBC transaction on Connection [...PgConnection@3e28dc96]
DEBUG JdbcTransactionManager - Resuming suspended transaction after completion of inner transaction
DEBUG JdbcTransactionManager - Creating nested transaction with name [ch07.tx.AuditService.nestedAndMaybeFail]
DEBUG JdbcTransactionManager - Releasing transaction savepoint
DEBUG JdbcTransactionManager - Participating in existing transaction
DEBUG JdbcTransactionManager - Participating in existing transaction
DEBUG JdbcTransactionManager - Suspending current transaction
DEBUG JdbcTransactionManager - Resuming suspended transaction after completion of inner transaction
DEBUG JdbcTransactionManager - Committing JDBC transaction on Connection [...PgConnection@2b6262bc]
outer        : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
REQUIRED     : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
REQUIRES_NEW : txActive=true name=ch07.tx.AuditService.requiresNew pid=92827
NESTED       : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
MANDATORY    : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
SUPPORTS     : txActive=true name=ch07.tx.OuterService.callEachPropagation pid=92825
NOT_SUPPORTED: txActive=false name=ch07.tx.AuditService.notSupported pid=92827
```

Reading it (the pids and connection hashes change from run to run; what matters is which lines share a pid):

- **REQUIRED, MANDATORY, SUPPORTS** joined: same name, same pid 92825 ("Participating in existing transaction").
- **REQUIRES_NEW** took a **second connection** (pid 92827) and committed on its own before the outer transaction continued.
- **NESTED** stayed on the same connection. The log shows a savepoint being created and then released.
- **NOT_SUPPORTED** suspended the outer transaction and ran with no transaction (`txActive=false`). It still needed a connection, so it borrowed one from the pool, here the one REQUIRES_NEW had just returned.

A subtle cost: while REQUIRES_NEW runs, **one thread holds two connections**. If every request does that, a pool of 10 can deadlock with only 5 concurrent requests, each holding one connection and waiting for a second.

Demo 2 — without an outer transaction

```
SUPPORTS      : txActive=false name=ch07.tx.AuditService.supports pid=92825
MANDATORY     : No existing transaction found for transaction marked with propagation 'mandatory'
NEVER in tx   : Existing transaction found for transaction marked with propagation 'never'
```

SUPPORTS simply ran without a transaction. MANDATORY and NEVER threw `IllegalTransactionStateException`.

Demo 3 — REQUIRES_NEW survives the outer rollback

```java
@Transactional
public void outerFailsAfterRequiresNew() {
    db.audit("outer work");
    audit.requiresNew("audit written in REQUIRES_NEW");
    throw new IllegalStateException("outer failed");
}
```

```
=== REQUIRES_NEW commits even though the outer transaction rolls back ===
audit_log = [audit written in REQUIRES_NEW]
```

"outer work" is gone and the audit row stayed. This is the standard way to record *"someone tried to do X and it failed"*.

Demo 4 — NESTED rolls back to the savepoint only

```
@Transactional
public void outerSurvivesNestedFailure() {
    db.audit("outer work");
    try {
        audit.nestedAndMaybeFail("nested work", true);
    } catch (IllegalStateException e) {
        db.audit("outer caught: " + e.getMessage());
    }
}
```

```
DEBUG JdbcTransactionManager - Creating nested transaction with name [ch07.tx.AuditService.nestedAndMaybeFail]
DEBUG JdbcTransactionManager - Rolling back transaction to savepoint
DEBUG JdbcTransactionManager - Committing JDBC transaction on Connection [...]
audit_log = [outer work, outer caught: inner NESTED failed]
```

"nested work" was undone with `ROLLBACK TO SAVEPOINT`, and the rest committed. NESTED is supported by the JDBC transaction managers. With JPA, `JpaTransactionManager` can create JDBC savepoints, but Hibernate's in-memory state (the persistence context) is not rolled back with them. Avoid NESTED with JPA.

Demo 5 — catching an inner REQUIRED failure does not save you

This surprises almost everyone:

```
@Transactional
public void outerCatchesRequiredFailure() {
    db.audit("outer work");
    try {
        audit.requiredAndFail("inner work");   // REQUIRED -> joins, then throws
    } catch (IllegalStateException e) {
        db.audit("outer caught: " + e.getMessage() + " and carries on");
    }
}
```

```
DEBUG JdbcTransactionManager - Participating in existing transaction
DEBUG JdbcTransactionManager - Participating transaction failed - marking existing transaction as rollback-only
DEBUG JdbcTransactionManager - Global transaction is marked as rollback-only but transactional code requested commit
DEBUG JdbcTransactionManager - Rolling back JDBC transaction on Connection [...]
UnexpectedRollbackException: Transaction rolled back because it has been marked as rollback-only
audit_log = []
```

The inner method **shared** the outer transaction. When the exception left the inner proxy, that proxy could not roll back only its own part, because there is only one physical transaction. So it marked the whole transaction **rollback-only**. The outer method caught the exception and returned normally. At commit time Spring found the mark, rolled everything back, and threw `UnexpectedRollbackException` so the failure is not silent.

Fixes: let the exception propagate. Or make the inner call `REQUIRES_NEW` / `NESTED` if its failure really should be isolated. Or check for the error condition before calling instead of catching the exception.

Common mistakes

- REQUIRES_NEW everywhere "for safety" doubles connection use and breaks atomicity, because parts commit independently.
- Expecting propagation to apply to calls inside the same class. It does not (7.13).
- Expecting a transaction to follow work into another thread (`@Async`, `CompletableFuture`, parallel streams). Transactions are bound to one thread, so the new thread has none.

In the interview

30-second answer: "Propagation decides what a transactional method does when a transaction already exists. REQUIRED, the default, joins it or starts one. REQUIRES_NEW suspends the caller's transaction and runs an independent one on a new connection. NESTED uses a savepoint. MANDATORY and NEVER throw if a transaction is missing or present. SUPPORTS and NOT_SUPPORTED run with or without one."

- *Inner REQUIRED method throws, outer catches it. What happens?* → The shared transaction is marked rollback-only. The outer commit becomes a rollback and throws `UnexpectedRollbackException`.
- *REQUIRES_NEW vs NESTED?* → REQUIRES_NEW is a separate physical transaction that commits independently, even if the outer one rolls back later. NESTED is a savepoint in the same transaction, so if the outer transaction rolls back, the nested work is lost too.
- *Risk of REQUIRES_NEW?* → Two connections per thread, which can exhaust the pool. It can also deadlock if the inner transaction waits for a row lock the outer one holds.

7.10 Rollback rules

What it is

When an exception leaves a `@Transactional` method, Spring chooses rollback or commit. The default copies EJB (Enterprise JavaBeans, the older Java EE component model):

- **`RuntimeException` and `Error` → rollback.**
- **Checked exceptions (`Exception` subclasses that are not runtime exceptions) → commit.**

The idea was that checked exceptions are "business outcomes" the caller is expected to handle. In practice this causes bugs. (Checked vs unchecked is explained in the Java book: Java 8.2.)

The demo

```
public class FraudCheckException extends Exception { ... }  // CHECKED

@Transactional
public void transferThenFailChecked(String from, String to, long amount) throws FraudCheckException {
    transferSteps(from, to, amount);
    throw new FraudCheckException("fraud check failed (checked)");
}

@Transactional(rollbackFor = FraudCheckException.class)
public void transferThenFailCheckedWithRollbackFor(...) throws FraudCheckException { /* same */ }

@Transactional(noRollbackFor = IllegalArgumentException.class)
public void transferWithNoRollbackFor(...) { transferSteps(...); throw new IllegalArgumentException(...); }
```

Output (start: `alice=100 bob=50`, transfer 30 from alice to bob):

```
CHECK violation      -> rolled back, alice=100 bob=50
RuntimeException     -> rolled back, alice=100 bob=50
checked exception    -> COMMITTED (!), alice=70 bob=80
checked + rollbackFor -> rolled back, alice=100 bob=50
unchecked + noRollbackFor -> committed, alice=70 bob=80
```

The third line is the trap. The method **threw**, the caller saw an exception, and the money moved anyway.

Changing the default for the whole application (Framework 6.2+)

```
@Configuration
@EnableTransactionManagement(proxyTargetClass = true, rollbackOn = RollbackOn.ALL_EXCEPTIONS)
public class RollbackOnAllExceptionsConfig { }
```

With this configuration active (a separate test context with the profile `rollback-all`), the *same* method rolls back:

```
rollbackOn=ALL_EXCEPTIONS, checked exception -> rolled back, alice=100 bob=50
```

The Spring reference docs recommend `ALL_EXCEPTIONS` unless you really rely on "commit on checked exception". `rollbackFor` / `noRollbackFor` on a method still override the global default. When you declare your own `@EnableTransactionManagement`, Boot's version backs off. Keep `proxyTargetClass = true` so beans still get class-based proxies, as with Boot's default.

Rules summary

Thrown	Default	`rollbackFor = X`	`noRollbackFor = X`	`rollbackOn = ALL_EXCEPTIONS`
`RuntimeException` / `Error`	rollback	rollback	commit if it is an X	rollback
checked exception	**commit**	rollback if it is an X	commit	rollback
exception caught inside the method	nothing to decide — commits	same	same	same

Rules match **subclasses** too: `rollbackFor = Exception.class` covers every checked exception. The `...ClassName` variants match by name pattern, which is powerful but easy to over-match.

Common mistakes

- Catching an exception, logging it, and not rethrowing. The method ends normally and **commits**. If you must swallow it, call `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`, or better, use `TransactionTemplate`.
- Wrapping a runtime exception into a checked one on the way out, which silently switches the outcome to commit.
- Your own JDBC code throwing raw `SQLException` (checked) → commit; Spring's helpers throw unchecked `DataAccessException`s, which do roll back.
- Assuming `@Transactional(rollbackFor = ...)` on an interface method is honoured by every proxy type. Annotate the class.

In the interview

> **30-second answer:** "By default Spring rolls back on unchecked exceptions and errors and commits on checked exceptions. You change it per method with `rollbackFor` / `noRollbackFor`, or since Spring 6.2 globally with `@EnableTransactionManagement(rollbackOn = ALL_EXCEPTIONS)`. A caught exception never triggers a rollback, because the proxy never sees it."

- *My method threw an `IOException` but the data was saved. Why?* → `IOException` is checked, so the default rule commits. Add `rollbackFor` or switch the global default.

7.11 Isolation levels

What it is

Isolation controls what one transaction can see of other transactions that run at the same time. The anomalies (dirty read, non-repeatable read, phantom, lost update, write skew) and how PostgreSQL implements each level with MVCC are taught in SQL 13.4 — Isolation and SQL 13.7 — Lost updates. In Spring you only choose the level:

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
```

Spring calls `Connection.setTransactionIsolation(...)` when it starts the transaction and resets it afterwards. `Isolation.DEFAULT` leaves the database default in place: READ COMMITTED on PostgreSQL, Oracle and SQL Server, REPEATABLE READ on MySQL InnoDB. Isolation only applies when a **new** transaction starts. A method that joins an existing transaction inherits the outer transaction's level.

Demo — two reads, and another user commits in between

"Another user" is simulated with a `REQUIRES_NEW` method on a second bean. It uses a second connection and commits immediately, which makes the demo run on one thread with a predictable result.

```java
@Transactional(isolation = Isolation.READ_COMMITTED)
public List<Long> readTwiceReadCommitted() {
    long first = db.balance("alice");
    other.deposit("alice", 50);          // REQUIRES_NEW: commits on another connection
    long second = db.balance("alice");
    return List.of(first, second);
}
// readTwiceRepeatableRead(): identical, with Isolation.REPEATABLE_READ
```

```
READ_COMMITTED  reads alice twice: [100, 150]  (non-repeatable read)
REPEATABLE_READ reads alice twice: [100, 100]  (same snapshot)
```

PostgreSQL's REPEATABLE READ takes one snapshot at the first query; every later read uses it.

Demo — the lost update

Read a value into Java, change it, write it back. This is the most common concurrency bug in business code:

```java
@Transactional(isolation = Isolation.READ_COMMITTED)
public void addTenReadCommitted() {
    long read = db.balance("alice");     // 100
    other.deposit("alice", 50);          // another user: 150, committed
    db.setBalance("alice", read + 10);   // writes 110
}
```

```
READ_COMMITTED  read 100, other +50, write read+10 -> alice=110  (the +50 is LOST)
REPEATABLE_READ same steps -> CannotAcquireLockException: ERROR: could not serialize access due to concurrent update
alice=150 (the deposit survived; retry the whole transaction)
```

Under REPEATABLE READ, PostgreSQL notices that the row changed after this transaction's snapshot was taken and refuses the update (SQLState `40001`). Spring translates that into `CannotAcquireLockException`, a `ConcurrencyFailureException`, so it is in the "transient, retry" branch of 7.5. **The whole transaction must be retried from the start**, not just the failed statement.

Ways to prevent lost updates

Technique	How	Where
Do the math in SQL	`UPDATE account SET balance = balance + 10`	simplest — no read-modify-write in Java
Pessimistic lock	`SELECT ... FOR UPDATE`, then update	SQL 13.12, JPA `@Lock` in 8.14
Optimistic lock	version column, `UPDATE ... WHERE version = ?`	JPA `@Version` in 8.14
Stronger isolation	REPEATABLE READ / SERIALIZABLE + retry on `40001`	this section

Common mistakes

- Setting `isolation` on a method that is always called inside another transaction. It is ignored, because only the method that starts the transaction sets the level.
- Using SERIALIZABLE without a retry loop. Serialization failures are *expected* at that level.
- Relying on H2 to test isolation. Its behaviour differs from PostgreSQL's, which is why these demos use a real server.

7.12 readOnly and timeout

readOnly

`@Transactional(readOnly = true)` does different things depending on the stack:

Stack	Effect
JDBC (`JdbcTransactionManager`)	calls `Connection.setReadOnly(true)`. The PostgreSQL driver then starts the transaction as `READ ONLY`, so writes **fail**
JPA / Hibernate	the session becomes read-only and flush mode goes to manual (flushing = sending pending changes to the database). No snapshots for dirty checking, no automatic flush (faster reads, 8.4)
Routing `DataSource`	can send read-only transactions to a replica (a read-only synced copy; see System Design Fundamentals) (a custom `AbstractRoutingDataSource` that checks `TransactionSynchronizationManager.isCurrentTransactionReadOnly()`)

On PostgreSQL:

```
@Transactional(readOnly = true)
public void writeInReadOnly() { db.audit("should not be allowed"); }
```

```
readOnly=true + INSERT -> UncategorizedSQLException: ERROR: cannot execute INSERT in a read-only transaction
```

Other drivers may treat read-only only as a hint. Do not rely on it for security. Use database permissions for that (SQL 14.8).

timeout

`timeout` (seconds) is a deadline for the whole transaction. Spring's JDBC helpers turn the *remaining* time into a JDBC query timeout on each statement. When the deadline has already passed, the next statement fails before it runs.

```
@Transactional(timeout = 1)
public void slowQuery() { db.sleepSeconds(3); }   // SELECT pg_sleep(3)
```

```
timeout=1 + pg_sleep(3) -> QueryTimeoutException: ERROR: canceling statement due to user request
gave up after ~1000 ms
```

The driver asked PostgreSQL to cancel the statement after one second. Keep in mind: a timeout cancels **database statements**. Plain Java code, or a slow HTTP call made inside the transaction, is not interrupted. The deadline is only checked at the next statement.

7.13 Transaction pitfalls

All of these pitfalls come from one fact: `@Transactional` **works only when the call goes through the Spring proxy.** Proxies and AOP are covered in ch04. Here you see the effect on real data.

```
System.out.println("injected class = " + imports.getClass().getSimpleName()
        + ", CGLIB proxy? " + AopUtils.isCglibProxy(imports));
```

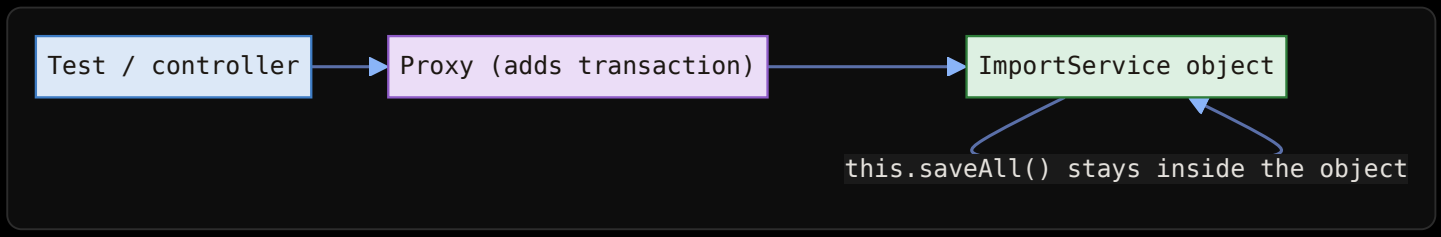
```
injected class = ImportService$$SpringCGLIB$$0, CGLIB proxy? true
```

Pitfall 1 — self-invocation

```
@Service
public class ImportService {
    public void importAll(List<String> lines) {   // NOT transactional
        saveAll(lines);                            // this.saveAll(): bypasses the proxy
    }

    @Transactional
    public void saveAll(List<String> lines) {
        for (String line : lines) {
            if (line.equals("BAD")) throw new IllegalArgumentException("bad line");
            db.audit(line);
        }
    }
}
```

```
via this.saveAll()  -> audit_log = [line-1, line-2]   (partial import!)
via proxy.saveAll() -> audit_log = []   (rolled back)
```



The `@Transactional` on `saveAll` was ignored because the call never left the object. Each `INSERT` ran in auto-commit mode.

Fixes, in order of preference:

1. Put the annotation on the method that is called from outside (`importAll`), or on both.
2. Move `saveAll` to another bean, so the call goes through that bean's proxy.
3. Use `TransactionTemplate` inside the method (7.14). No proxy is needed.
4. Inject the bean into itself (`@Lazy` self-reference), or use AspectJ weaving (`@EnableTransactionManagement(mode = AdviceMode.ASPECTJ)`). Both work, and both are rarely worth the complexity.

Pitfall 2 — `final` and `private` methods

A CGLIB proxy is a generated **subclass** that overrides your methods. It cannot override:

- `private` **methods**, which are invisible to subclasses. `@Transactional` on them is silently ignored. They can only be called from inside the class anyway, which is self-invocation.
- `final` **methods**. The call runs directly on the *proxy instance*. The proxy object is created without running your constructor, so its fields are `null`:

```
@Transactional
public final String finalMethod() { return db.where(); }
```

```
final @Transactional method -> NullPointerException (runs on the proxy, whose 'db' field is null)
```

Why this happens, with a proof that does not involve transactions, is in 4.5.

Protected and package-private methods *are* transactional with class-based proxies since Spring Framework 6.0:

```
package-private @Transactional -> txActive=true name=ch07.tx.ImportService.packagePrivate pid=92825
```

With JDK interface proxies, only methods of the interface are intercepted. Spring Boot uses CGLIB class proxies by default (`spring.aop.proxy-target-class=true`).

Pitfall 3 — the transaction on the wrong layer

A transaction should cover one **use case**, "transfer money", not each individual SQL call. Here every DAO (Data Access Object) method is its own transaction, which is what you get by default with Spring Data

repository methods:

```
@Repository
public class AccountDao {
    @Transactional public void add(String owner, long delta) { db.addToBalance(owner, delta); }
}

@Service
public class NoTxTransferService {           // no @Transactional on the use case
    public void transfer(String from, String to, long amount) {
        dao.add(to, amount);    // commits immediately
        dao.add(from, -amount); // fails (CHECK balance >= 0) -> credit above stays
    }
}
```

```
tx per DAO call     -> alice=600 bob=50   (500 created from nothing)
tx on service method -> alice=100 bob=50
```

Rule of thumb: `@Transactional` **goes on service methods (the use case)**. Not on controllers, which would hold a connection during serialization and HTTP work. And not only on repositories, whose transactions are too small.

Pitfall 4 — long transactions and remote calls

```
@Transactional
public void placeOrder(Order o) {
    orders.save(o);
    paymentClient.charge(o);   // 2 s HTTP call while holding a DB connection and row locks
}
```

The connection is held for the whole HTTP call. If the payment succeeds and the commit then fails, the customer has been charged and no order exists. Keep transactions short and database-only. Make remote calls before or after the transaction, or publish an event after commit (7.15), or use the transactional outbox (write the outgoing message to a table in the same transaction; a separate process sends it) (ch14, event-driven patterns).

Pitfall 5 — other threads

A transaction is bound to the thread that started it. Work handed to `@Async` methods, executors, `CompletableFuture.supplyAsync` or parallel streams runs **outside** the transaction, on other connections, and cannot see uncommitted data.

Checklist: "why is my @Transactional not working?"

Symptom	Likely cause
Partial writes after an exception	self-invocation, `private` / `final` method, bean not managed by Spring (`new MyService()`)
Data committed despite an exception	checked exception without `rollbackFor`, or the exception was caught inside
`UnexpectedRollbackException`	an inner REQUIRED method failed and someone caught its exception
`LazyInitializationException`	JPA entity used after the transaction ended (8.6)
Works in tests, fails in production	the test itself was `@Transactional` and supplied the missing transaction
Pool exhausted under load	REQUIRES_NEW nesting, or slow remote calls inside transactions

In the interview

> **30-second answer:** "`@Transactional` is implemented by a proxy, so it only applies to calls that come in through the proxy. Calls from one method to another in the same class skip it. So do `private` methods, and `final` methods break outright. Put transactions on service methods that represent a use case, keep them short, and never make remote calls inside them."

- *How do you fix self-invocation?* → Move the method to another bean, annotate the entry method, or use `TransactionTemplate`.
- *Why not put `@Transactional` on the controller?* → It widens the transaction to include request parsing, serialization and remote calls, holding connections and locks longer. The service layer is the use-case boundary.

7.14 Programmatic transactions: TransactionTemplate

What it is

`TransactionTemplate` starts a transaction around a lambda. It uses the same manager and the same rules, with **no proxy**, so self-invocation cannot affect it. Boot auto-configures one bean with default settings. For other settings, create your own from the `PlatformTransactionManager`.

```
@Service
public class ProgrammaticService {
    private final TransactionTemplate tx;           // auto-configured
    private final TransactionTemplate serializable;

    public ProgrammaticService(Db db, TransactionTemplate tx, PlatformTransactionManager txManager) {
        this.tx = tx;
        this.serializable = new TransactionTemplate(txManager);
        this.serializable.setIsolationLevel(TransactionDefinition.ISOLATION_SERIALIZABLE);
        this.serializable.setTimeout(5);
    }

    public long transferAndReturnNewBalance(String from, String to, long amount) {
        return tx.execute(status -> {               // only the lambda is transactional
            db.addToBalance(to, amount);
            db.addToBalance(from, -amount);
            return db.balance(to);
        });
    }

    public String tryThenUndo() {
        return tx.execute(status -> {
            db.audit("temporary");
            String seen = "inside: " + db.auditMessages();
            status.setRollbackOnly();               // roll back without throwing
            return seen;
        });
    }

    public String isolationOfCustomTemplate() {
        return serializable.execute(status -> db.showIsolation());   // SHOW transaction_isolation
    }
}
```

```
TransactionTemplate transfer -> bob now 70
setRollbackOnly: inside: [temporary], after: []
custom template isolation = serializable
```

When to use it

`@Transactional`	`TransactionTemplate`
whole method = one transaction	transaction around *part* of a method
clean business code	explicit and visible
proxy pitfalls apply	no proxy — works for self-calls
rollback via exceptions/rules	also `status.setRollbackOnly()`

Good fits for `TransactionTemplate`: batch jobs that commit every N items, loops where each item gets its own transaction, retry loops (the retry must be *outside* the transaction), and places where the proxy pitfalls

bite. Even lower level, `PlatformTransactionManager.getTransaction/commit/rollback` exists, but the template handles the error cases for you.

7.15 @TransactionalEventListener

The problem it solves

"After an order is saved, send a confirmation e-mail." If the e-mail goes out inside the transaction and the transaction then rolls back, the customer gets an e-mail for an order that does not exist. You want **"only if the commit succeeded"**.

How it works

1. Inside the transaction, publish an application event (`ApplicationEventPublisher.publishEvent`).
2. A plain `@EventListener` runs **immediately**, in the same thread and the same transaction.
3. A `@TransactionalEventListener` is **registered with the transaction** and runs at a chosen phase:

Phase	Runs when	Typical use
`BEFORE_COMMIT`	just before commit (can still write, can still fail the commit)	last validations, flushing extra data
`AFTER_COMMIT` (default)	after a successful commit	e-mails, messages, cache eviction
`AFTER_ROLLBACK`	after a rollback	compensation (undoing side effects), alerts
`AFTER_COMPLETION`	after commit **or** rollback	cleanup

```
@Transactional
public void place(String item, boolean failAfterPublish) {
    db.audit("order " + item);
    events.publishEvent(new OrderPlaced(item));
    if (failAfterPublish) throw new IllegalStateException("payment declined");
}

@Component
public class OrderListeners {
    @EventListener                        void immediately(OrderPlaced e)    { ... }
    @TransactionalEventListener(phase = TransactionPhase.BEFORE_COMMIT)    void beforeCommit(OrderPlaced e) { ... }
    @TransactionalEventListener           void afterCommit(OrderPlaced e)    { ... }
    @TransactionalEventListener(phase = TransactionPhase.AFTER_ROLLBACK)   void afterRollback(OrderPlaced e) { ... }
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMPLETION) void afterCompletion(OrderPlaced e) { ... }
}
```

```
commit   -> [@EventListener book, BEFORE_COMMIT book, AFTER_COMMIT book, AFTER_COMPLETION book]
rollback -> [@EventListener lamp, AFTER_ROLLBACK lamp, AFTER_COMPLETION lamp], audit_log=[]
no tx    -> [@EventListener pen]
```

In the last line the event was published with no transaction active, so the transactional listeners were **skipped**. Set `@TransactionalEventListener(fallbackExecution = true)` if they should run in that case too.

Common mistakes

- **Writing to the database in an AFTER_COMMIT listener without a new transaction.** The original transaction has finished, but its resources are still bound during cleanup. Since Spring 6.1, a `@TransactionalEventListener` method may be `@Transactional` only with `REQUIRES_NEW` (or `NOT_SUPPORTED`, allowed since 6.1.3). Anything else fails at startup.
- **Assuming AFTER_COMMIT is reliable delivery.** If the process crashes right after the commit, the e-mail is never sent, because the event lived only in memory. For guaranteed delivery, write an "outbox" row in the same transaction and have a separate process send it (ch14).
- Doing slow work in the listener on the request thread. Combine it with `@Async` to move the work off the request thread.

In the interview

> **30-second answer:** "`@TransactionalEventListener` defers an event listener to a transaction phase, by default after commit. So side effects like e-mails or messages only happen when the data is really saved. If no transaction is active the listener is skipped unless `fallbackExecution` is true. If the listener must write to the database it needs `REQUIRES_NEW`. For guaranteed delivery you still need an outbox, because the event is only in memory."

7.16 Interview questions

1. **What does Spring add on top of JDBC?** → Connection pooling (HikariCP via Boot), boilerplate removal (`JdbcClient` / `JdbcTemplate`), exception translation to `DataAccessException`, and declarative transactions.
2. **`JdbcClient` vs `JdbcTemplate`?** → Same engine. `JdbcClient` (6.1+) is a fluent API with named/positional parameters and automatic record mapping. `JdbcTemplate` stays for batches and older code.
3. **How does `@Transactional` work internally?** → A proxy's `TransactionInterceptor` asks the `PlatformTransactionManager` to begin a transaction, which binds a connection to the thread. The method runs. Then the interceptor commits, or rolls back according to the rollback rules.
4. **Default propagation and isolation?** → REQUIRED, and the database default (READ COMMITTED on PostgreSQL).
5. **Checked exception thrown from `@Transactional` — commit or rollback?** → Commit, unless `rollbackFor` or `rollbackOn = ALL_EXCEPTIONS` (6.2+) says otherwise.
6. **Why is my transaction ignored when I call the method from the same class?** → Self-invocation bypasses the proxy.
7. **Explain `UnexpectedRollbackException`.** → A participating (joined) method failed, which marked the shared transaction rollback-only. The outer code caught the exception and tried to commit.
8. **REQUIRES_NEW vs NESTED?** → A separate physical transaction on a new connection vs a savepoint in the same transaction.
9. **What does `readOnly = true` do?** → With JDBC it makes the connection read-only (PostgreSQL rejects writes). With Hibernate it disables dirty checking and flushing. It can also route to replicas.
10. **How do you run code only after a commit?** → `@TransactionalEventListener` (AFTER_COMMIT) or `TransactionSynchronization.afterCommit`. For guaranteed delivery, an outbox.
11. **How do you handle a serialization failure?** → Catch `ConcurrencyFailureException` / `TransientDataAccessException` and retry the *whole* transaction, with the retry outside the transactional method.
12. **Does `@Transactional` work with `@Async`?** → The async method runs on another thread with no inherited transaction. It can have its own `@Transactional`, but it is a separate transaction.

Chapter TL;DR

- Boot builds a **HikariCP** pool from `spring.datasource.*`. Keep it small, keep connections short, and remember the total across instances.
- `JdbcClient` (6.1+) is the modern way to run SQL: `sql().param().query(Type.class).list()`, `update()`. Use `JdbcTemplate.batchUpdate` for batches. Always bind parameters.
- Spring translates `SQLException` into the unchecked `DataAccessException` hierarchy. The *transient* branch means "retry may work".
- `@Transactional` = proxy + transaction manager + a connection bound to the thread. Defaults: REQUIRED, database isolation, read-write, rollback on unchecked exceptions only.
- **Propagation**: REQUIRED joins, REQUIRES_NEW runs separately on a second connection, NESTED uses a savepoint, MANDATORY/NEVER guard, SUPPORTS/NOT_SUPPORTED are optional/none. A failure in a joined transaction dooms it (`UnexpectedRollbackException`).
- **Checked exceptions commit** by default. Use `rollbackFor` or `rollbackOn = ALL_EXCEPTIONS`.
- **Isolation** is the database's job (SQL ch13). Spring sets the level. Retry on serialization failures.
- **Pitfalls**: self-invocation, `private` / `final` methods, the wrong layer, remote calls inside transactions, other threads, `@Transactional` tests hiding bugs.
- `TransactionTemplate` for precise, proxy-free boundaries. `@TransactionalEventListener` for after-commit side effects.

Next: Chapter 8 — Spring Data JPA and Hibernate moves from SQL you write to entities and repositories.

Chapter 8 — Spring Data JPA and Hibernate

Chapter 7 wrote SQL by hand. This chapter lets an **ORM** (object-relational mapper) write it. You describe Java classes (**entities** — objects stored as table rows) and their relationships, and Hibernate (the most widely used Java ORM) generates the SQL, tracks changes to your objects, and loads related objects on demand. Spring Data JPA adds **repositories**: interfaces whose methods become queries (derived from method names, or `@Query` in JPQL — JPA's object-oriented query language — or native SQL) without implementation code. (JPA, the Jakarta Persistence API, is the standard API that ORMs such as Hibernate implement.)

This is productive, and also where many Spring performance problems come from, so every important behaviour below is *measured*: the tests count and print the SQL Hibernate really sends. You will see the N+1 problem (one query for a list, then one per item — §8.7) appear and disappear with three fixes, a `LazyInitializationException` (related data loaded after the session closed), dirty checking (Hibernate writing changed objects back by itself) producing an `UPDATE` without `save()`, and two users colliding on an optimistic lock (a version check that detects a conflicting save instead of blocking). Also covered: auditing (who changed what, when), pessimistic locking (locking the row so others wait), and schema migrations (versioned scripts evolving the tables) with Flyway.

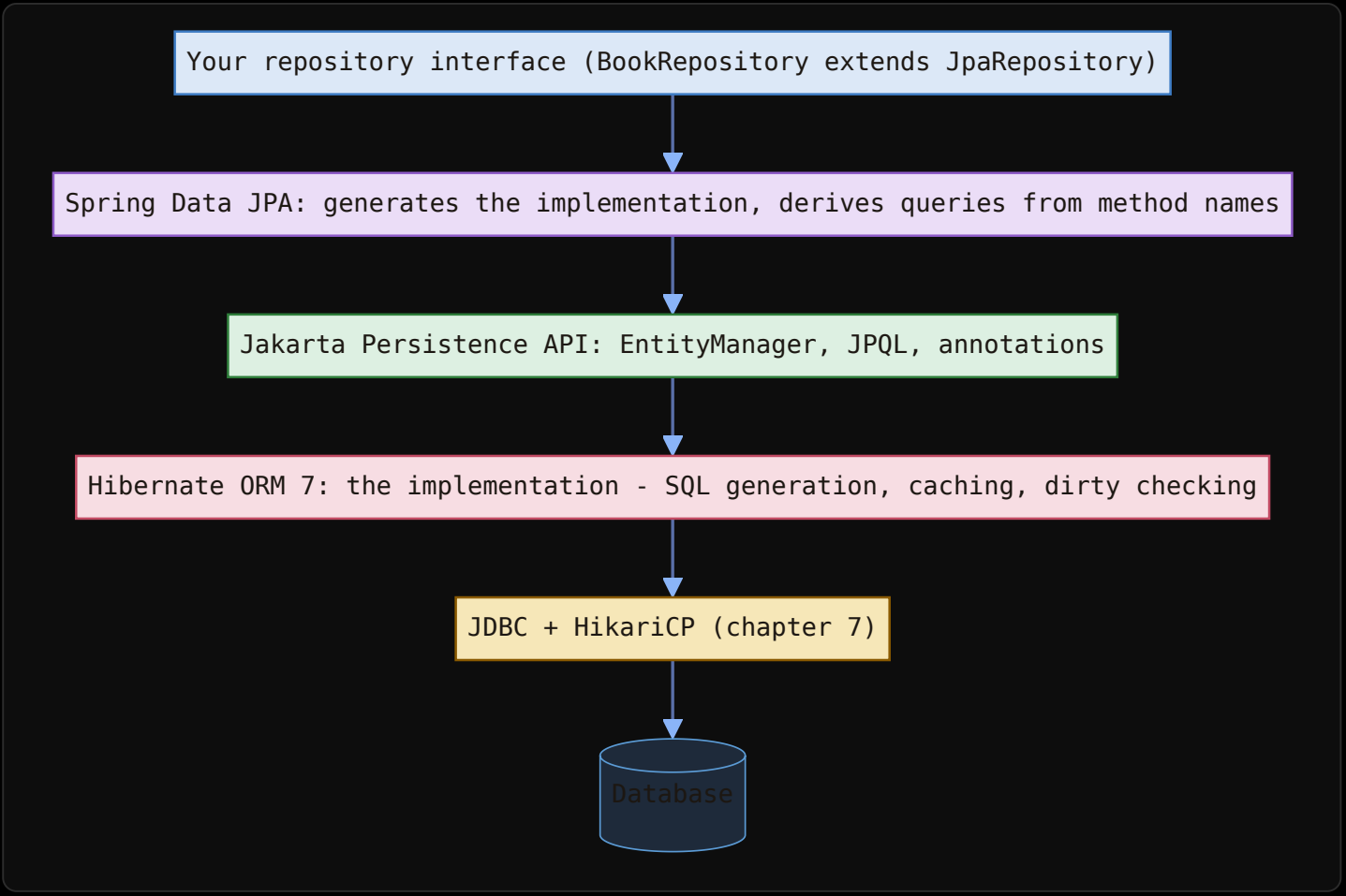
The code (Spring Boot 4.1.1 → Hibernate ORM 7.4, Spring Data 2026.0, H2 in memory):

Project	What it proves
`code/jpa-demos`	persistence context, relationships, lazy loading, N+1 with statement counts, repositories, projections, auditing, locking, open-in-view
`code/flyway-migrations`	versioned migrations + `ddl-auto=validate`

Run everything with `bash tools_run_spring.sh spring '08-*'`. The sample data is a small library: 4 authors, 8 books, 3 tags, 6 reviews (`src/main/resources/data.sql`).

8.1 JPA, Hibernate and Spring Data — who does what

What it is



Layer	What it is	Version with Boot 4.1
Jakarta Persistence (JPA)	a *specification*: the `jakarta.persistence` annotations, `EntityManager`, the JPQL query language	3.2
Hibernate ORM	the most used *implementation* of JPA. It does the real work	7.4 (Boot 4.1 upgraded to 7.4; Hibernate 7.0 came out in 2025)
Spring Data JPA	a Spring layer on top of JPA: repositories, derived queries, paging, auditing	part of the Spring Data 2026.0 release train (modules released together)
Spring ORM	the glue in Spring Framework: `JpaTransactionManager`, the shared `EntityManager` proxy, exception translation to `DataAccessException`	Framework 7.0

A few Hibernate 7 facts worth knowing: it implements **Jakarta Persistence 3.2** and **Jakarta Data 1.0**, it needs Java 17 or later, and it is the first Hibernate version licensed under **Apache License 2.0** (permissive; earlier versions were LGPL, with more distribution conditions). Spring Data 2025.1 (the Boot 4.0 generation) moved to JSpecify null annotations and added **AOT repositories** (ahead-of-time), which generate repository code at build time for faster startup and native images. Spring Data 2026.0 (Boot 4.1) added **type-safe property paths**, for example `Sort.by(Book::getPrice)` instead of `Sort.by("price")`.

When not to use JPA

JPA fits a rich domain model: many related entities, loaded, changed and saved as a unit of work (one business operation whose changes are saved together). It fits poorly for:

- reporting and analytics (big joins, aggregations, window functions): use SQL through `JdbcClient` (7.3) or jOOQ (a library for writing type-safe SQL in Java)
- bulk data processing (millions of rows): use JDBC batches or database-side SQL
- teams who want to see every statement: Spring Data JDBC is simpler and has no lazy loading and no persistence context (ch09)

The general ORM vs raw SQL debate is covered in SQL 14.13.

8.2 Setting up

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>   <!-- Hibernate + Spring Data JPA + HikariCP -->
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>                                                <!-- @DataJpaTest -->
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
```

```
# jpa-demos/src/main/resources/application.properties
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.defer-datasource-initialization=true          # run data.sql after Hibernate created tables
spring.jpa.properties.hibernate.generate_statistics=true # count statements
spring.jpa.properties.hibernate.session_factory.statement_inspector=ch08.jpa.support.SqlRecorder
```

Anything under `spring.jpa.properties.*` is passed to Hibernate unchanged. That is how you set any Hibernate option Boot has no property for.

`ddl-auto`: who creates the tables?

Value	Hibernate at startup	Use for
`none`	nothing	production with a migration tool
`validate`	checks that tables and columns match the entities, and fails startup if not	production with Flyway/Liquibase (see 8.16)
`update`	adds missing tables and columns, never drops	quick prototypes only. Unreliable for real schema changes
`create`	drops and creates the schema	tests
`create-drop`	like `create`, and drops again at shutdown	tests, demos (this project)

Boot's default is `create-drop` for an embedded database when no migration tool is present, and `none` otherwise.

Seeing the SQL

Option	Shows
`spring.jpa.show-sql=true`	every statement on stdout, prefixed `Hibernate:` (`@DataJpaTest` turns this on by default)
`logging.level.org.hibernate.SQL=DEBUG`	the same through the logger (better)
`logging.level.org.hibernate.orm.jdbc.bind=TRACE`	the bound parameter values too
`hibernate.generate_statistics=true`	counters: statements, entity loads, collection fetches, flushes
a `StatementInspector`	your own code sees every SQL string, which is used here to print and count

The tests use a small helper built on Hibernate's `Statistics` and the `SqlRecorder` inspector:

```
final class SqlCount {
    private final Statistics stats;
    SqlCount(EntityManagerFactory emf) { stats = emf.unwrap(SessionFactory.class).getStatistics(); }
    void reset()      { stats.clear(); SqlRecorder.clear(); }
    long statements() { return stats.getPrepareStatementCount(); }   // JDBC statements prepared
    void print(String label) { /* label + count + each recorded SQL string */ }
}
```

In your own projects you can get the same effect in tests with the *datasource-proxy* library or Hibernate statistics, and fail a test when a request uses more statements than expected. That turns N+1 bugs into test failures.

8.3 Entities and ids

What it is

An **entity** is a class whose instances are rows of a table. The JPA rules: annotate it with `@Entity`, give it an `@Id`, give it a no-argument constructor (it may be `protected`), and do not make the class or its persistent methods `final`, because Hibernate subclasses entities to build lazy proxies (stand-ins that load the real data on first use). **Records cannot be entities**, since they are final and immutable. Records work well as `@Embeddable` value types (small objects without their own id, stored as columns of the owning table; Hibernate 6.2+) and as projections (8.12).

```java
@Entity
@EntityListeners(AuditingEntityListener.class)
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;                // column "title" - names are mapped automatically
    private BigDecimal price;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "author_id")
    private Author author;

    @ManyToMany
    @JoinTable(name = "book_tag",
               joinColumns = @JoinColumn(name = "book_id"),
               inverseJoinColumns = @JoinColumn(name = "tag_id"))
    private Set<Tag> tags = new HashSet<>();

    @Version private Long version;       // optimistic locking, 8.14
    @CreatedDate private Instant createdAt;       // auditing, 8.13
    @LastModifiedDate private Instant updatedAt;
    @CreatedBy private String createdBy;

    protected Book() {}                  // for JPA
    public Book(String title, BigDecimal price) { this.title = title; this.price = price; }
    // getters, a few setters ...
}
```

The chapter's tests switch `show-sql` off, but running one of them with `-Dlogging.level.org.hibernate.SQL=DEBUG` shows the table Hibernate generated at startup. Boot's default naming strategy turns `createdAt` into `created_at`:

```
create table book (price numeric(38,2), author_id bigint not null, created_at timestamp(6) with time zone,
  id bigint generated by default as identity, updated_at timestamp(6) with time zone, version bigint not null,
  created_by varchar(255), title varchar(255), primary key (id))
```

Useful column annotations: `@Column(nullable = false, length = 100, unique = true)`, `@Column(precision = 10, scale = 2)` (note the default `numeric(38,2)` above), `@Enumerated(EnumType.STRING)` (always STRING: ORDINAL stores the position number, which breaks when someone reorders the enum), `@Transient` (not stored), and `@Embedded` for value objects such as `Address`.

Id generation strategies

Strategy	How the id is produced	Trade-off
`IDENTITY`	database auto-increment column	simple, but Hibernate must `INSERT` **immediately** on `persist` to learn the id, which also rules out JDBC insert batching
`SEQUENCE`	database sequence (an object handing out increasing numbers). Hibernate fetches blocks of ids (`allocationSize`, default 50)	best for PostgreSQL/Oracle: inserts can wait until flush and be batched
`AUTO` (default)	Hibernate picks, which in Hibernate 6+ is usually a sequence per entity, `book_seq`	know what you get
`UUID` (JPA 3.1+)	`java.util.UUID` generated in Java	no database round trip, safe to generate anywhere. 16 bytes, and random UUIDs spread inserts all over the index
assigned (no `@GeneratedValue`)	you set it	natural keys (real-world ids, like an ISBN). `save()` must then use `merge`, see 8.8

This project uses IDENTITY so `data.sql` can insert rows without ids. You can see the immediate inserts in 8.5.

equals and hashCode

Entities live in `Set`s (which use `hashCode` and `equals`) and are compared across persistence contexts, so these methods matter:

- The **default** (`Object` identity) is correct inside one persistence context, because Hibernate guarantees one instance per row there (8.4). This project relies on that.
- **Never** base `hashCode` on a generated id that is `null` before persist. The hash changes after saving, and the object gets lost inside a `HashSet`.
- Good options: a natural business key (ISBN), or `equals` on id with a **constant** `hashCode` (`getClass().hashCode()`).
- Never include lazy associations in `equals`, `hashCode` or `toString`. Doing so triggers loading, or a `LazyInitializationException`.

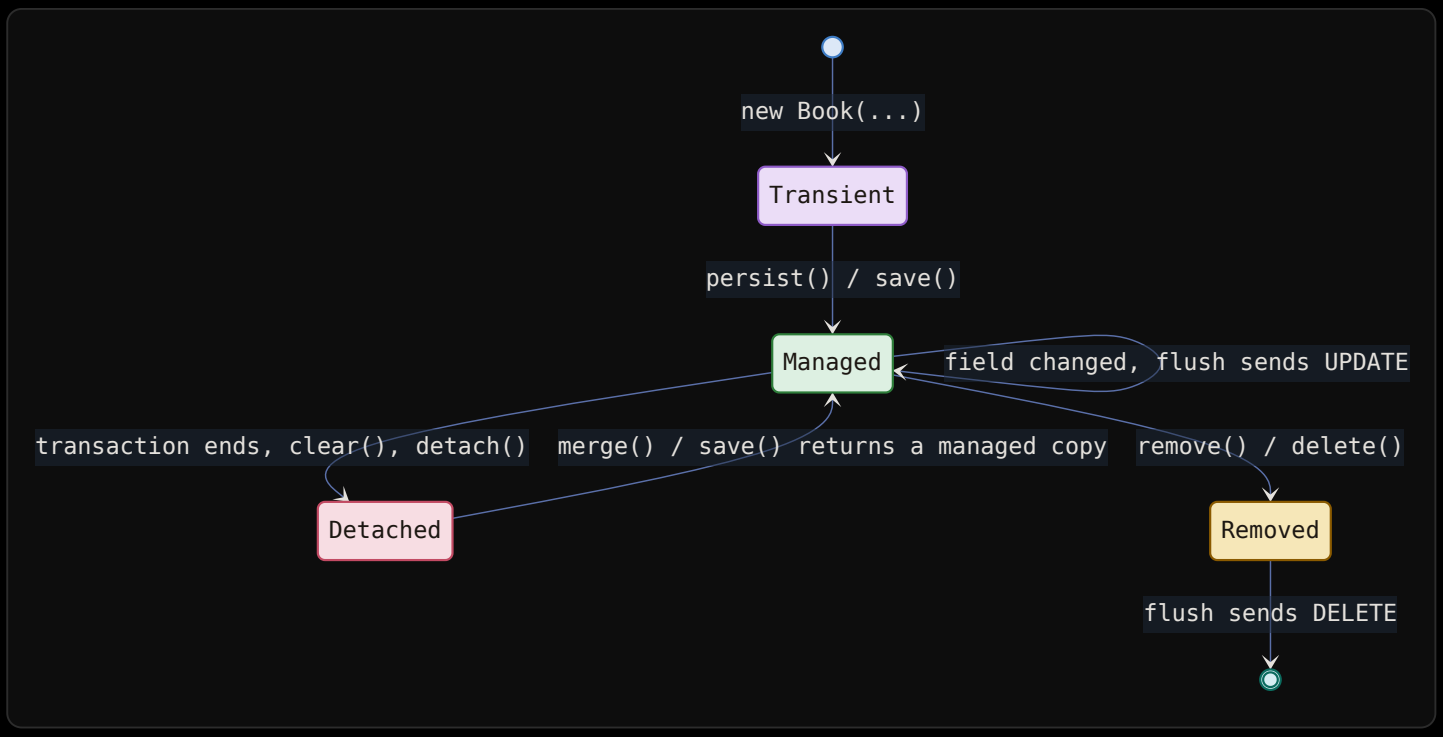
8.4 The persistence context and dirty checking

What it is

The **persistence context** is Hibernate's in-memory workspace for one unit of work. It is the `EntityManager` (Hibernate calls it `Session`). In Spring it usually lives exactly as long as the transaction. It does three jobs:

1. **Identity map / first-level cache**: each row is loaded at most once and always gives the *same* Java object.
2. **Change tracking (dirty checking)**: at load time it keeps a snapshot (copy of the loaded values) of each entity. At **flush** time it compares every managed entity with its snapshot and writes an `UPDATE` for the ones that changed. You never call `update()`.
3. **Write-behind**: inserts, updates and deletes are queued and sent at flush. Flush happens before commit, before a query that might read the changed tables, or when you call `flush()`.

Entity states



State	Tracked?	Example
Transient	no — Hibernate does not know it	`new Book(...)`
Managed	yes — changes are flushed automatically	anything returned by `find`, a query, or `persist` inside the current transaction
Detached	no — it has an id but no persistence context	an entity returned by a repository call *after* its transaction ended
Removed	scheduled for `DELETE` at flush	after `remove()`

Demo — one row, one object, one SELECT

```
Book first = books.findById(1L).orElseThrow();
Book second = books.findById(1L).orElseThrow();
System.out.println("same Java object? " + (first == second));
```

```
same Java object? true
findById(1) twice -> 1 SQL statement(s)
    select b1_0.id,b1_0.author_id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version
from book b1_0 where b1_0.id=?
```

The second `findById` came from the persistence context without any SQL. (`@DataJpaTest` wraps each test in one transaction, and so in one persistence context.)

Demo — dirty checking: an UPDATE without save()

```
Book book = books.findById(1L).orElseThrow();
book.setPrice(new BigDecimal("14.99"));   // just change the Java object...
em.flush();                               // ...Hibernate compares with its snapshot and writes SQL
```

```
setPrice() + flush, no save() -> 2 SQL statement(s)
    select b1_0.id,b1_0.author_id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version
from book b1_0 where b1_0.id=?
    update book set author_id=?,created_at=?,created_by=?,price=?,title=?,updated_at=?,version=? where id=? and
version=?
version is now 1
```

In real code the flush happens at commit. `LibraryService.changePrice` is exactly this, and it is tested in 8.13:

```
@Transactional
public void changePrice(Long bookId, BigDecimal newPrice) {
    books.findById(bookId).orElseThrow().setPrice(newPrice);   // no save()
}
```

Notice that the `UPDATE` lists **all** columns. By default Hibernate uses one cached prepared `UPDATE` per entity type. `@DynamicUpdate` on the entity makes it send only the changed columns instead.

Demo — detached entities are not tracked

```
Book book = books.findById(1L).orElseThrow();
em.clear();                               // persistence context emptied: book is DETACHED
book.setPrice(new BigDecimal("1.00"));
em.flush();
```

```
change a detached entity + flush -> 1 SQL statement(s)
    select ... from book b1_0 where b1_0.id=?
```

No `UPDATE` was sent: the change is simply lost. To store changes to a detached entity you call `save()` (which calls `merge`). Merge loads the current row, copies your state onto the managed copy, and returns that copy. Use the return value.

readOnly transactions and JPA

`@Transactional(readOnly = true)` tells Spring to put the Hibernate session in read-only mode with manual flushing. Hibernate then keeps no snapshots and never runs dirty checking, which saves memory and CPU when you load many entities just to display them. Spring Data's `SimpleJpaRepository` is annotated `@Transactional(readOnly = true)` at class level and plain `@Transactional` on its write methods.

Common mistakes

- Calling `save()` on an entity you just loaded in the same transaction. It is harmless but unnecessary, because the entity is managed already.
- Ignoring the result of `save()`/`merge()` for a detached entity, then modifying the old detached object.
- Loading 100 000 entities in one transaction. The persistence context keeps all of them *and* their snapshots. Process in pages and call `clear()`, or use a read-only transaction, projections or JDBC.

In the interview

> **30-second answer:** "The persistence context is the `EntityManager`'s first-level cache and unit of work. It returns the same object for the same row, keeps a snapshot of every managed entity, and at flush it writes `UPDATE`s for entities that changed. That is dirty checking, and it is why you don't need `save()` for loaded entities. Entities are transient, managed, detached or removed. Only managed ones are tracked."

- *When does Hibernate flush?* → Before commit, before a query that touches affected tables (AUTO flush mode), and on an explicit `flush()`.
- *`persist` vs `merge`?* → `persist` makes a transient object managed (the same instance). `merge` copies a detached object's state onto a managed copy and returns the copy.

8.5 Relationships

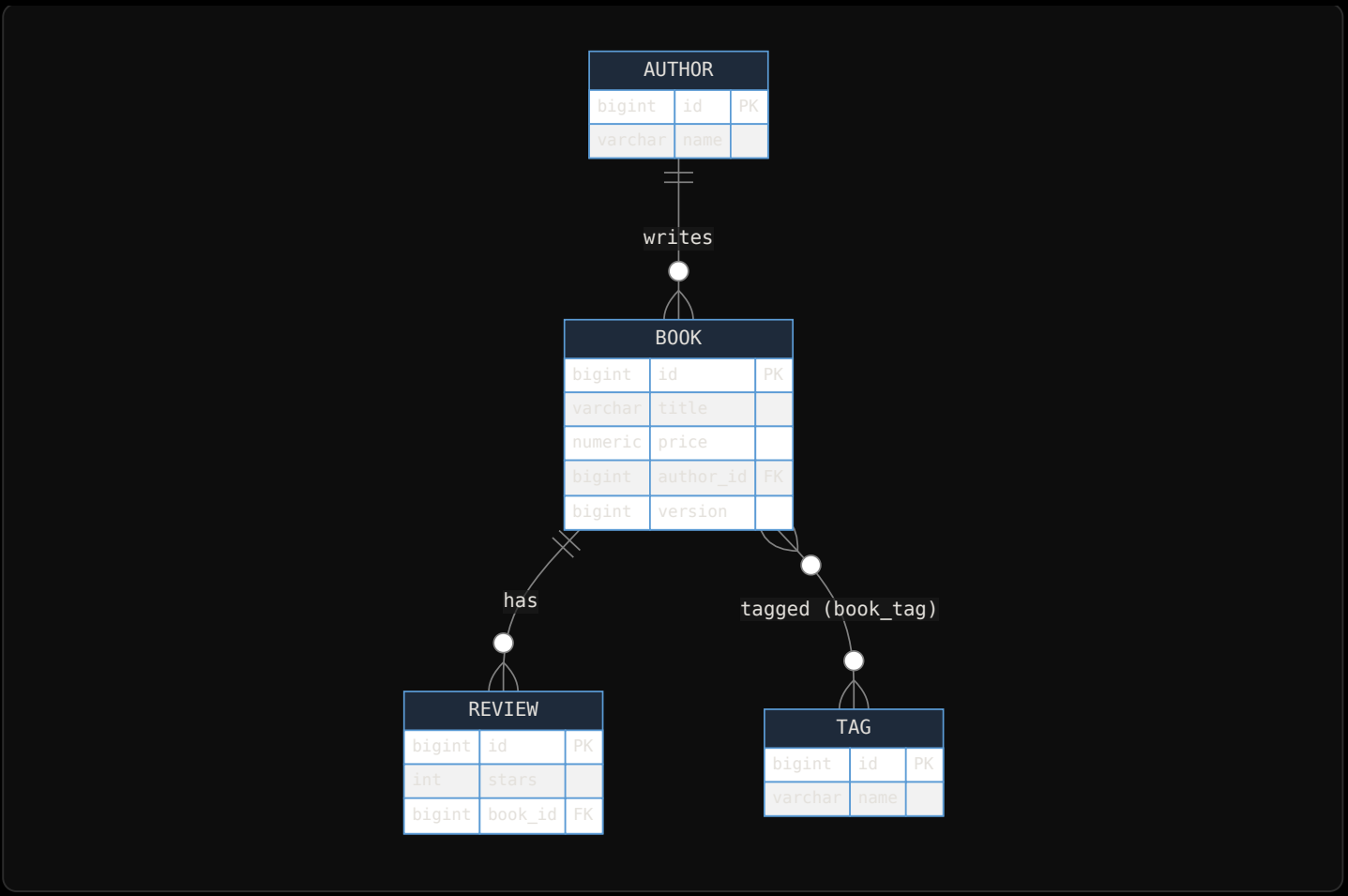
What it is

Annotation	Meaning	Default fetch	Example
`@ManyToOne`	many of these point to one of those	**EAGER**	`Book.author`
`@OneToMany`	one of these has a collection of those	LAZY	`Author.books`
`@OneToOne`	exactly one on each side	**EAGER**	`User.profile`
`@ManyToMany`	join table between the two	LAZY	`Book.tags`

The owning side and mappedBy

In the database a relationship is **one foreign-key column** (holding the id of a row in another table) or a join table (rows linking two tables). In Java it may be navigable from both sides, but only one side may control that column. That side is the **owning side**:

- For `@ManyToOne`/`@OneToMany` the owner is the **many** side (`Book.author` → `book.author_id`).
- The other side says `mappedBy = "author"`, meaning "I am just a mirror of `Book.author`". This is the **inverse side**.
- JPA **only reads the owning side** when it writes SQL. Adding a book to `author.getBooks()` without setting `book.author` stores nothing.



That is why `Author` has helper methods that keep both sides in sync:

```
@Entity
public class Author {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    private String name;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Book> books = new ArrayList<>();

    public void addBook(Book book)    { books.add(book);    book.setAuthor(this); }
    public void removeBook(Book book) { books.remove(book); book.setAuthor(null); }
    // ...
}
```

Cascades

A **cascade** passes an operation from parent to children:

CascadeType	Passes on
`PERSIST`	saving a new parent also saves new children
`MERGE`	merging the parent merges the children
`REMOVE`	deleting the parent deletes the children
`REFRESH`, `DETACH`	reload / detach together
`ALL`	all of the above

Use cascades only from an **aggregate root** (the main object of a group always saved together) to the parts it owns (author → books in this demo, order → order lines in real life). Never cascade `REMOVE` on `@ManyToMany` or towards shared entities. Deleting a book must not delete the "sci-fi" tag that other books use.

```
Author author = new Author("N. K. Jemisin");
author.addBook(new Book("The Fifth Season", new BigDecimal("11.00")));
author.addBook(new Book("The Obelisk Gate", new BigDecimal("11.00")));
authors.save(author);                       // persist cascades to both books
```

```
save(new author with 2 new books) -> 3 SQL statement(s)
    insert into author (name,id) values (?,default)
    insert into book (author_id,created_at,created_by,price,title,updated_at,version,id) values (?,?,?,?,?,?,?,default)
    insert into book (author_id,created_at,created_by,price,title,updated_at,version,id) values (?,?,?,?,?,?,?,default)
ids assigned: author 5, books [9, 10]
```

The inserts ran **during** `save()`, before any flush. That is the IDENTITY strategy at work (8.3).

orphanRemoval

`orphanRemoval = true` means *"a child removed from the collection should be deleted"*. `CascadeType.REMOVE` only deletes children when the *parent* is deleted.

```
Author pratchett = authors.findById(2L).orElseThrow();
Book smallGods = pratchett.getBooks().stream()
        .filter(b -> b.getTitle().equals("Small Gods")).findFirst().orElseThrow();
pratchett.removeBook(smallGods);            // no repository.delete() call
em.flush();
```

```
remove 'Small Gods' from author.books + flush -> 4 SQL statement(s)
    select a1_0.id,a1_0.name from author a1_0 where a1_0.id=?
    select b1_0.author_id,b1_0.id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version
from book b1_0 where b1_0.author_id=?
    delete from book_tag where book_id=?
    delete from book where id=? and version=?
```

Hibernate first removed the book's rows from the `book_tag` join table (Book owns that many-to-many), then deleted the book. The first version of this demo removed a book that had **reviews**, and the database rejected the delete with a foreign-key violation (`REVIEW FOREIGN KEY(BOOK_ID) REFERENCES BOOK(ID)`). Hibernate only cleans up relationships it knows it owns. `Review.book` points at the book from the other direction, so you must deal with those rows yourself.

Many-to-many advice

A plain `@ManyToMany` suits a pure link with no data of its own. As soon as the link needs data ("tagged by whom, when?"), turn the join table into an **entity** (`BookTag` with two `@ManyToOne`s). Use `Set`, not `List`, for many-to-many. Hibernate handles `List` without an order column (a "bag") inefficiently: removing one element deletes all the link rows and re-inserts the rest.

Common mistakes

- Setting only the inverse side (`author.getBooks().add(book)`), so nothing is saved.
- `cascade = ALL` everywhere, which deletes shared data.
- Bidirectional mappings everywhere. Map the reverse side only when you really navigate it.
- `@OneToMany` without `mappedBy`. Hibernate then creates an extra join table.

In the interview

30-second answer: "The owning side is the one holding the foreign key, in a one-to-many that is the `@ManyToOne` side, and it is the only side JPA reads when writing. The other side uses `mappedBy`. Keep both sides in sync with helper methods. Cascade from aggregate roots only. `orphanRemoval` deletes children removed from the collection, while `CascadeType.REMOVE` deletes children when the parent is deleted."

8.6 Fetch types and LazyInitializationException

What it is

- **LAZY**: the related data is loaded only when you first touch it. For a collection, Hibernate puts a special `PersistentBag` / `PersistentSet` in the field. For a single entity it puts a generated **proxy** subclass there.
- **EAGER**: the related data is loaded together with the owner, always.

Make everything LAZY and decide per use case what to load (see 8.7). The JPA defaults are EAGER for `@ManyToOne` and `@OneToOne`, so change them explicitly: `@ManyToOne(fetch = FetchType.LAZY)`. EAGER cannot be switched off for a single query, while LAZY can always be fetched eagerly when you need it.

LazyInitializationException

A lazy collection can only load itself while its persistence context is open. Outside a transaction, a repository call opens and closes its own short transaction. The entity it returns is already **detached**:

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.NONE)   // no test transaction
class LazyLoadingTest {
    @Test
    void lazyCollectionOutsideTheTransaction() {
        Author a = authors.findById(2L).orElseThrow();   // transaction opens and closes inside findById
        assertThatThrownBy(() -> a.getBooks().size())
                .isInstanceOf(LazyInitializationException.class);
    }
}
```

```
LazyInitializationException: Cannot lazily initialize collection of role 'ch08.jpa.Author.books' with key '2' (no
session)
```

"No session" means no open persistence context. The fixes:

```
@EntityGraph(attributePaths = "books")
Optional<Author> findWithBooksById(Long id);       // load the books in the same query

@Transactional(readOnly = true)                     // or: do the work inside a transaction
public AuthorSummary summary(Long authorId) {
    Author a = authors.findById(authorId).orElseThrow();
    return new AuthorSummary(a.getName(), a.getBooks().size());   // return a DTO, not the entity
}
```

```
fetched with @EntityGraph -> 3 books
inside @Transactional service -> AuthorSummary[name=Terry Pratchett, books=3]
```

Fix	Good?
fetch what the use case needs (JOIN FETCH / `@EntityGraph`)	✅ best
do the work in a `@Transactional` service and return a DTO	✅ best
a DTO projection query (8.12)	✅ for read-only views
`spring.jpa.open-in-view=true`	⚠️ hides the problem, see 8.15
switch the mapping to EAGER	❌ every query pays for it
`hibernate.enable_lazy_load_no_trans=true`	❌ each lazy load opens its own mini-transaction: N+1 with no consistency

---

8.7 The N+1 problem

What it is

You run **1** query to load N parents. Then, for each parent, touching a lazy association fires **1 more** query. With N = 1000 that is 1001 round trips for data that one query could return. It is the most common JPA performance bug, and a functional test cannot see it: the result is correct, only slow.

Demo — watch it happen

```
List<Author> all = authors.findAll();
all.forEach(a -> System.out.println("  " + a.getName() + ": " + a.getBooks().size() + " books"));
```

```
  Ursula K. Le Guin: 2 books
  Terry Pratchett: 3 books
  Octavia E. Butler: 1 books
  Isaac Asimov: 2 books
findAll() then a.getBooks().size() -> 5 SQL statement(s)
    select a1_0.id,a1_0.name from author a1_0
    select b1_0.author_id,b1_0.id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version
from book b1_0 where b1_0.author_id=?
    select b1_0.author_id,b1_0.id,... from book b1_0 where b1_0.author_id=?
    select b1_0.author_id,b1_0.id,... from book b1_0 where b1_0.author_id=?
    select b1_0.author_id,b1_0.id,... from book b1_0 where b1_0.author_id=?
```

1 + 4 = 5 statements. The test asserts `statements() == 1 + all.size()`.

EAGER does not fix it — demo

`Review.book` was left at the JPA default (`@ManyToOne` = EAGER). We never touch `review.getBook()`:

```
var all = reviews.findAll();
```

```
reviews.findAll() with EAGER @ManyToOne Review.book (6 reviews) -> 6 SQL statement(s)
    select r1_0.id,r1_0.book_id,r1_0.stars from review r1_0
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
    select b1_0.id,b1_0.author_id,... from book b1_0 where b1_0.id=?
```

1 query for the reviews, then 1 per distinct book (5), **even though the code never used the books**. EAGER tells Hibernate that the association must be loaded. It does not say *how*. For JPQL and derived queries Hibernate runs the query as written, then loads each eager association separately.

Fix 1 — JOIN FETCH

```
@Query("select a from Author a join fetch a.books order by a.id")
List<Author> findAllWithBooks();
```

```
JOIN FETCH -> 1 SQL statement(s)
    select a1_0.id,b1_0.author_id,b1_0.id,...,a1_0.name from author a1_0 join book b1_0 on a1_0.id=b1_0.author_id order
by a1_0.id
```

Hibernate 6+ removes the duplicate parent rows that a join produces on its own, so `distinct` is no longer needed. Note that it is an **inner** join: authors with no books disappear. Use `left join fetch` (keeps every left row, even without a match) to keep them.

Fix 2 — @EntityGraph

```
@EntityGraph(attributePaths = "books")
@Query("select a from Author a order by a.id")
List<Author> findAllWithBooksGraph();
```

```
@EntityGraph(attributePaths = "books") -> 1 SQL statement(s)
    select a1_0.id,b1_0.author_id,b1_0.id,...,a1_0.name from author a1_0 left join book b1_0 on a1_0.id=b1_0.author_id
order by a1_0.id
```

The effect is the same, but it is a **left** join and needs no JPQL. It also works on derived query methods (`findWithBooksById` above). Entity graphs can also be declared on the entity with `@NamedEntityGraph`.

Fix 3 — batch fetching

Same code as the N+1 demo, one property added:

```
@DataJpaTest(showSql = false, properties = "spring.jpa.properties.hibernate.default_batch_fetch_size=16")
```

```
findAll() + books with default_batch_fetch_size=16 -> 2 SQL statement(s)
    select a1_0.id,a1_0.name from author a1_0
    select b1_0.author_id,b1_0.id,... from book b1_0 where b1_0.author_id in (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)
```

When the first lazy collection is touched, Hibernate loads the collections of up to 16 authors in the persistence context at once. N+1 becomes 1 + ⌈N/16⌉. It needs no query changes, which makes it a good **global safety net**. Per association, use `@BatchSize(size = 16)`.

Paging a JOIN FETCH (Hibernate 7.4)

Paging and fetching a collection in the same query was a classic trap. `LIMIT 2` on a joined result limits *rows*, not authors, so older Hibernate versions loaded **everything** and paged in memory, with the warning `HHH90003004: firstResult/maxResults specified with collection fetch; applying in memory`. Hibernate 7.4 pages the parents in a subquery instead. Our test, run on Boot 4.1.1:

```
@Query(value = "select a from Author a join fetch a.books",
       countQuery = "select count(a) from Author a")
Page<Author> pageWithBooks(Pageable pageable);

authors.pageWithBooks(PageRequest.of(0, 2, Sort.by("id")));
```

```
JOIN FETCH + Pageable(page 0, size 2) -> 2 SQL statement(s)
    select a1_0.id,b1_0.author_id,...,a1_0.name from (select a1_0.id,a1_0.name from author a1_0 where exists(select 1
from book b1_0 where a1_0.id=b1_0.author_id) order by a1_0.id offset ? rows fetch first ? rows only) a1_0(id,name) join
book b1_0 on a1_0.id=b1_0.author_id order by a1_0.id
    select count(a1_0.id) from author a1_0
```

The inner subquery picks 2 authors with `offset/fetch first`, and the outer query joins their books. On older Hibernate versions, set `hibernate.query.fail_on_pagination_over_collection_fetch=true` so the in-memory case fails loudly. Then page the ids first and fetch the collections in a second query (or rely on batch fetching).

Choosing a fix

Fix	Queries	Good for	Watch out
`JOIN FETCH`	1	one known use case	inner join by default. Fetching **two** `List` collections at once → `MultipleBagFetchException` (use `Set`, or two queries)
`@EntityGraph`	1	the same, declared on a repository method	same cartesian-product (rows multiplied) risk with several collections
batch size	1 + N/size	global default, several collections	still more than one query
DTO projection	1	read-only screens and APIs	no entities, so no dirty checking (usually what you want)

Fetching two collections in one query multiplies rows: 10 books × 10 tags = 100 rows per author. For several collections, prefer batch fetching or separate queries.

Common mistakes

- Returning entities from a REST controller. Jackson walks every getter, triggering lazy loads (N+1) or `LazyInitializationException`, and exposes your schema. Return DTOs.

- `toString()`/Lombok `@Data` (generated getters, `equals`, `toString`) on entities with associations. Printing loads them.
- Testing with 3 rows. N+1 only hurts with real volumes, so assert statement counts in tests instead.

In the interview

30-second answer: "N+1 is one query for the parents plus one per parent when a lazy association is touched, or even untouched with EAGER to-one mappings. I find it by counting statements with Hibernate statistics or a datasource proxy (a `DataSource` wrapper that sees every statement). I fix it per use case with `JOIN FETCH` or an `@EntityGraph`, globally reduce it with `default_batch_fetch_size`, or avoid entities with a DTO projection. EAGER is not a fix, it makes the problem permanent."

- *Why doesn't EAGER prevent N+1?* → EAGER says *whether* to load, not *how*. For queries Hibernate loads the associations with separate selects after the main query.
- *JOIN FETCH with pagination?* → Historically in-memory pagination (HHH90003004). Hibernate 7.4 pages parents in a subquery. Otherwise page ids first, then fetch.
- *`MultipleBagFetchException`?* → Two `List` collections fetched in one query. Use `Set`, or fetch them in separate queries.

8.8 Repositories

What it is

A **repository** is an interface. At startup Spring Data creates a proxy that implements it, using `SimpleJpaRepository` for the standard methods and generated query code for your own ones.

```
public interface BookRepository extends JpaRepository<Book, Long> { ... }
```

Interface	Adds
`Repository<T, ID>`	marker only. Declare exactly the methods you want
`CrudRepository` / `ListCrudRepository`	`save`, `findById`, `findAll`, `count`, `delete...` (the List variant returns `List` instead of `Iterable`)
`PagingAndSortingRepository` / `ListPagingAndSortingRepository`	`findAll(Sort)`, `findAll(Pageable)`
`JpaRepository`	all of the above + `flush`, `saveAndFlush`, `deleteAllInBatch`, `getReferenceById`, ...
`JpaSpecificationExecutor<T>`	dynamic `Specification` (conditions built in code) queries

Expose only what callers should use. A `Repository` with a handful of methods is often better than handing out `deleteAll()`.

How save() decides between INSERT and UPDATE

`save(entity)` asks "is this entity new?". If yes, it calls `persist` (the same instance becomes managed). If not, it calls `merge` (it returns a **different**, managed instance). An entity counts as new when its id is `null`, or when a wrapper-type `@Version` field is `null`. With **assigned ids** every entity has an id, so `save` runs `merge`, which first `SELECT`s the row. Implement `Persistable<ID>` to tell Spring Data yourself.

Transactions in repositories

`SimpleJpaRepository` methods are transactional: read methods `readOnly = true`, write methods read-write. Each call **outside** a service transaction is its own tiny transaction, which leads straight to the "wrong layer" bug of 7.13. Put `@Transactional` on the service method so that all repository calls join one transaction.

8.9 Derived queries

The method name *is* the query. Spring Data parses it at startup, so a typo fails startup, not a request:

```
List<Book> findByTitleContainingIgnoreCase(String part);
List<Book> findByPriceLessThanOrderByPriceAsc(BigDecimal max);
List<Book> findByAuthorName(String authorName);          // traverses book.author.name (a join)
long countByAuthorName(String authorName);
boolean existsByTitle(String title);
List<Book> findTop2ByOrderByPriceDesc();
List<Book> findByTagsName(String tag, Sort sort);        // many-to-many join + dynamic sort
```

```
containing 'go'      : [Small Gods, Going Postal]
price < 10 asc       : [I, Robot, Foundation, Guards! Guards!, A Wizard of Earthsea]
by author name       : [A Wizard of Earthsea, The Dispossessed]
count by author      : 3
exists 'Kindred'     : true
top 2 by price       : [Kindred, The Dispossessed]
tag sci-fi, title asc: [Foundation, I, Robot, Kindred, The Dispossessed]
tag fantasy, by price: [Guards! Guards!, A Wizard of Earthsea, Small Gods]
```

The last line used the Spring Data 2026.0 type-safe form `Sort.by(Book::getPrice)`. A renamed getter is then a compile error rather than a runtime one. The SQL Spring Data generated for some of them:

```
containing / count / exists -> 3 SQL statement(s)
    select b1_0.id,...,b1_0.version from book b1_0 where upper(b1_0.title) like upper(?) escape '\'
    select count(b1_0.id) from book b1_0 join author a1_0 on a1_0.id=b1_0.author_id where a1_0.name=?
    select b1_0.id from book b1_0 where b1_0.title=? fetch first ? rows only
```

`existsBy` fetches just one id. Note also that `upper(title) like` cannot use a normal index on `title` (SQL 12.13).

Keywords

Keyword	Example	SQL idea
`And`, `Or`	`findByTitleAndPrice`	`title = ? and price = ?`
`Is`, `Equals`, `Not`	`findByTitleNot`	`title <> ?`
`LessThan`, `GreaterThanEqual`, `Between`	`findByPriceBetween(a, b)`	`price between ? and ?`
`IsNull`, `IsNotNull`	`findByAuthorIsNull`	`author_id is null`
`Like`, `StartingWith`, `EndingWith`, `Containing`	`findByTitleStartingWith`	`like 'x%'`
`IgnoreCase`	`findByTitleIgnoreCase`	`upper(title) = upper(?)`
`In`, `NotIn`	`findByIdIn(List)`	`id in (...)`
`True`, `False`	`findByActiveTrue`	`active = true`
`OrderBy...Asc/Desc`	`findByAuthorNameOrderByTitleDesc`	`order by`
`First`, `Top`, `TopN`, `Distinct`	`findFirst3ByOrderByPriceAsc`	`fetch first 3 rows`
nested property	`findByAuthorName`	join to `author`
prefixes	`count...By`, `exists...By`, `delete...By`	count / exists / load-then-delete

Special parameters: `Sort`, `Pageable`, `Limit` (Spring Data 3.2+), `ScrollPosition` (keyset scrolling, 3.1+). Return types: `List`, `Optional`, `Stream` (close it), `Page`, `Slice`, `Window`, a single entity, or a projection.

When not to use derived queries: when the name grows past two or three conditions (`findByAuthorNameAndPriceLessThanAndTagsNameOrderBy...`). Switch to `@Query`.

8.10 @Query: JPQL, native SQL and @Modifying

JPQL

JPQL (Jakarta Persistence Query Language) looks like SQL but talks about **entities and fields**, not tables and columns: `b.author.name` instead of a join written by hand.

```
@Query("select b from Book b where b.author.name = :name and b.price <= :max order by b.price")
List<Book> byAuthorUpTo(@Param("name") String name, @Param("max") BigDecimal max);
```

```
JPQL  Pratchett <= 11 : [Guards! Guards!, Small Gods]
```

(`@Param` can be omitted when you compile with `-parameters`, which the Boot parent POM enables.)

Native SQL

```
@Query(value = "SELECT title FROM book WHERE price > ?1 ORDER BY title", nativeQuery = true)
List<String> titlesAbove(BigDecimal min);
```

```
native titles > 11    : [Going Postal, Kindred, The Dispossessed]
```

Use native queries for database features JPQL lacks (window functions, `jsonb` operators, CTEs — `WITH` clauses, `ON CONFLICT` upserts). The price: the query is tied to one database and to table/column names.

Bulk updates with @Modifying

```
@Modifying(clearAutomatically = true) // bulk UPDATE bypasses the persistence context -> clear it
@Query("update Book b set b.price = b.price * :factor where b.author.id = :authorId")
int reprice(@Param("authorId") Long authorId, @Param("factor") BigDecimal factor);
```

```
@Modifying bulk update -> 1 SQL statement(s)
    update book b1_0 set price=(b1_0.price*cast(? as numeric(38,2))) where b1_0.author_id=?
bulk update rows      : 2 -> Asimov now [BookRow[title=Foundation, price=15.98], BookRow[title=I, Robot, price=13.98]]
```

A bulk statement goes straight to the database. Entities already loaded in the persistence context keep their **old** values, and `@Version` is **not** incremented. `clearAutomatically = true` empties the context afterwards, and `flushAutomatically = true` flushes pending changes before the statement runs. `@Modifying` methods need a read-write transaction.

Dynamic queries

For search screens with optional filters, use `Specification`s (`JpaSpecificationExecutor`), Query by Example (a sample object's non-null fields become filters), or Querydsl (type-safe queries in Java) instead of dozens of derived methods. All three build the query at runtime from the filters present.

Common mistakes

- Building JPQL with string concatenation. That is injection again: always use parameters.
- Forgetting that JPQL uses **entity** names (`Book`, `b.author.name`), not table names.
- Bulk updates followed by reading "stale" entities from the same persistence context.

8.11 Pagination and sorting

```
Page<Book> findByPriceGreaterThan(BigDecimal min, Pageable pageable);   // also runs a COUNT query
Slice<Book> findByAuthorIdIn(List<Long> authorIds, Pageable pageable);  // no COUNT, fetches size+1 rows

Page<Book> page = books.findByPriceGreaterThan(new BigDecimal("8"),
        PageRequest.of(1, 2, Sort.by("price").descending()));     // page index 1 = the second page
Slice<Book> slice = books.findByAuthorIdIn(List.of(1L, 2L), PageRequest.of(0, 3, Sort.by("id")));
```

```
Page 1 of 3 (6 rows total): [Going Postal, Small Gods]
Slice 0: [A Wizard of Earthsea, The Dispossessed, Guards! Guards!], hasNext=true
Page + Slice -> 3 SQL statement(s)
    select b1_0.id,...,b1_0.version from book b1_0 where b1_0.price>? order by b1_0.price desc offset ? rows fetch first
? rows only
    select count(*) from book b1_0 where b1_0.price>?
    select b1_0.id,...,b1_0.version from book b1_0 where b1_0.author_id in (?,?) order by b1_0.id fetch first ? rows
only
```

(Column lists shortened with `...` here and in a few other listings.) The `Slice` query asks for `size + 1` rows: if the extra row arrives, `hasNext()` is true.

Return type	Extra query	Knows	Use for
`Page<T>`	`COUNT(*)`	total elements and pages	UIs with page numbers
`Slice<T>`	none (fetches one extra row)	whether there is a next slice	"load more" / infinite scroll
`Window<T>` + `ScrollPosition`	none	next position	keyset (cursor) scrolling over big tables
`List<T>` + `Limit`	none	nothing	"top N"

Pages are numbered from **0**. `Page` and `Slice` use `OFFSET`, which gets slower the deeper you page, because the database still reads and throws away the skipped rows. For large tables use keyset pagination (`Window` with `ScrollPosition.keyset()`). The reasoning is in SQL 12.14 — OFFSET vs keyset pagination. Always sort by something unique, such as `id` as a tie-breaker, or rows can repeat or vanish between pages.

In Spring MVC, a `Pageable` controller parameter is filled from `?page=0&size=20&sort=price,desc` (ch05).

8.12 Projections

What it is

A **projection** returns only the fields you need, as an interface or a record, not the whole entity. There is no persistence context overhead, no dirty checking and no lazy loading surprises, and the SELECT names fewer columns.

```
public interface BookSummary {          // interface projection (closed: only plain getters)
    String getTitle();
    BigDecimal getPrice();
}
public record BookRow(String title, BigDecimal price) {}   // class-based (DTO) projection

List<BookSummary> findByAuthorNameOrderByTitle(String authorName);      // interface
List<BookRow> findByPriceLessThan(BigDecimal max);                       // record
@Query("select new ch08.jpa.BookRow(b.title, b.price) from Book b where b.author.id = :id order by b.title")
List<BookRow> rowsForAuthor(@Param("id") Long authorId);                 // JPQL constructor expression
<T> List<T> findByAuthorId(Long authorId, Class<T> type);               // dynamic: caller picks the type
```

```
interface projection SQL -> 1 SQL statement(s)
    select b1_0.title,b1_0.price from book b1_0 join author a1_0 on a1_0.id=b1_0.author_id where a1_0.name=? order by
b1_0.title
interface projection: Going Postal 11.50
interface projection: Guards! Guards! 8.99
interface projection: Small Gods 10.99
record projection     : [BookRow[title=Foundation, price=7.99], BookRow[title=I, Robot, price=6.99]]
constructor expression: [BookRow[title=A Wizard of Earthsea, price=9.99], BookRow[title=The Dispossessed, price=12.50]]
dynamic (BookSummary) : Kindred
```

Only `title` and `price` were selected, compared with eight columns for the entity.

Kind	How	Notes
closed interface	getters that match properties	optimized SELECT of just those columns. Nested interfaces for associations
open interface	`@Value("#{target.title + ' by ' + target.author.name}")`	loads the full entity, so no SELECT optimization
class / record (DTO)	constructor parameters match properties	derived queries select only those columns. With `@Query` use `select new ...`
dynamic	`<T> List<T> find...(..., Class<T> type)`	one method, several shapes
native query	interface projection with column aliases	`select title as title, ...`

Rule of thumb: load entities when you are going to *change* them, and use projections when you are going to *show* them.

8.13 Auditing

What it is

Auditing fills "who and when" columns automatically: `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`.

```
@SpringBootApplication
@EnableJpaAuditing                      // switch it on
public class JpaDemosApplication {
    @Bean
    AuditorAware<String> auditor() {     // "who is the current user?"
        return () -> Optional.of("system");   // real app: read it from Spring Security's context
    }
}

@Entity
@EntityListeners(AuditingEntityListener.class)   // on each audited entity
public class Book { @CreatedDate Instant createdAt; @LastModifiedDate Instant updatedAt; @CreatedBy String createdBy;
... }
```

This test runs without a test transaction, so each step commits:

```
Book b = books.findById(bookId).orElseThrow();
library.changePrice(bookId, new BigDecimal("10.50"));   // @Transactional, dirty checking, no save()
Book after = books.findById(bookId).orElseThrow();
```

```
created: by=system at=true version=0
after changePrice: price=10.50 version=1 updatedAt moved? true
```

The auditing listener runs on the JPA `@PrePersist`/`@PreUpdate` callbacks, so it only sees changes that go through the persistence context. Bulk `@Modifying` updates bypass it. For a full history use Hibernate Envers (`@Audited`, revision tables holding every past version of a row) or database-level change data capture (streaming the database's own change log).

8.14 Locking: optimistic and pessimistic

Optimistic locking with @Version

The problem: Alice and Bob both open the edit page for the same book, which is version 0. Alice saves. Bob saves a few seconds later and silently overwrites Alice's change. This is the *lost update* from 7.11 and SQL 13.7, now spread over two HTTP requests, where no database lock can help.

How it works:

1. Add a `@Version` field (`Long` / `Integer`, or a timestamp).
2. Every `UPDATE` becomes `... SET ..., version = version + 1 WHERE id = ? AND version = ?`, using the version the entity was loaded with.
3. If another transaction changed the row in the meantime, **0 rows** match. Hibernate reports it, and Spring throws `ObjectOptimisticLockingFailureException` (an `OptimisticLockingFailureException`).

```
Book aliceCopy = books.findById(bookId).orElseThrow();   // two users load version 0
Book bobCopy = books.findById(bookId).orElseThrow();

aliceCopy.setPrice(new BigDecimal("8.00"));
books.save(aliceCopy);                                   // merge -> UPDATE ... WHERE id=? AND version=0

bobCopy.setPrice(new BigDecimal("12.00"));
books.save(bobCopy);                                     // still version 0 -> 0 rows -> exception
```

```
both loaded version 0 and 0
Alice saves -> 2 SQL statement(s)
    select b1_0.id,b1_0.author_id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version
from book b1_0 where b1_0.id=?
    update book set author_id=?,created_at=?,created_by=?,price=?,title=?,updated_at=?,version=? where id=? and
version=?
Bob saves -> ObjectOptimisticLockingFailureException: Row was already updated or deleted by another transaction for
entity [ch08.jpa.Book with id '10']
price in DB = 8.00 (Alice's)
```

Alice's `save` is a `merge` of a detached entity, so it first `SELECT`s the current row, then updates with the version check. For Bob the merge found version 1 in the database, while his object said 0, so it failed.

Handling it: tell the user "someone else changed this, reload", which a REST API reports as `409 Conflict`. Or reload and retry automatically when the change is safe to reapply. In REST APIs, send the version to the client as an `ETag` (a header identifying one version of a resource) and require it back in `If-Match` ("only apply if the current version is still this one") (ch05).

Pessimistic locking

Lock the row in the database for the rest of the transaction:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Book> findWithLockById(Long id);
```

```
@Lock(PESSIMISTIC_WRITE) -> 1 SQL statement(s)
    select b1_0.id,b1_0.author_id,b1_0.created_at,b1_0.created_by,b1_0.price,b1_0.title,b1_0.updated_at,b1_0.version
from book b1_0 where b1_0.id=? for update
```

Other transactions that want the same row wait until this one commits. Set a lock timeout (the `jakarta.persistence.lock.timeout` query hint) so they do not wait forever. How `FOR UPDATE`, `NOWAIT` and `SKIP LOCKED` behave is covered in SQL 13.12.

	Optimistic (`@Version`)	Pessimistic (`@Lock`)
Conflict detected	at write time (0 rows updated)	prevented. Others wait
Cost	none until a conflict	locks held, possible deadlocks and waiting
Works across requests / user think time	✅	❌ (a lock lives only as long as its transaction)
Best when	conflicts are rare (most web apps)	conflicts are frequent and short (counters, stock, job queues)

In the interview

> **30-second answer:** "Optimistic locking adds a version column. Updates include `WHERE version = ?`, and if another transaction got there first, zero rows match and Spring throws `ObjectOptimisticLockingFailureException`, which I turn into a 409 or a retry. It holds no locks, so it works across user think time. Pessimistic locking uses `SELECT ... FOR UPDATE` through `@Lock(PESSIMISTIC_WRITE)` and makes others wait. I use it for short, high-contention updates."

8.15 Open Session in View

What it is

Open Session in View (OSIV) keeps the persistence context open for the **whole web request**, including controller code and JSON serialization, not just during the service transaction. Boot enables it by default for web applications (`spring.jpa.open-in-view=true`) and logs a warning at startup:

```
WARN  JpaBaseConfiguration$JpaWebConfiguration - spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
```

The demo controller touches a lazy collection outside any `@Transactional` method:

```
@GetMapping("/authors/{id}/summary")
String summary(@PathVariable Long id) {
    Author a = authors.findById(id).orElseThrow();
    return a.getName() + " has " + a.getBooks().size() + " books";   // lazy load in the controller
}
```

```
open-in-view=true  -> 200 Terry Pratchett has 3 books
open-in-view=false -> Cannot lazily initialize collection of role 'ch08.jpa.Author.books' with key '2' (no session)
```

Trade-offs

OSIV on	OSIV off
lazy loading "just works" anywhere in the request	lazy loading only inside transactions
a database connection may be held for the whole request, including slow serialization	connections held only during transactions
lazy loads during serialization run in auto-commit mode, one query each (hidden N+1)	forces you to fetch explicitly per use case
convenient for prototypes	predictable and recommended for APIs

Recommendation: set `spring.jpa.open-in-view=false` in new applications, fetch what each use case needs inside the service, and return DTOs. The failing test above is exactly what you want to see during development rather than in production under load.

8.16 Schema migrations with Flyway

What it is

In production you never let Hibernate change the schema. A **migration tool** applies small, numbered SQL scripts in order, **exactly once per database**, and records each one in a history table. The schema's history then lives in version control, like code. **Flyway** (SQL files) and **Liquibase** (XML/YAML/SQL changelogs) are the two Boot supports.

Boot 4 change: Flyway is only auto-configured when you use `spring-boot-starter-flyway`. `flyway-core` on its own is no longer enough, because of Boot 4's modular auto-configuration. For PostgreSQL also add `org.flywaydb:flyway-database-postgresql` (database support has been split into separate modules since Flyway 10).

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-flyway</artifactId>
</dependency>
```

```
src/main/resources/db/migration/
  V1__create_customer.sql        CREATE TABLE customer (id ... PRIMARY KEY, name VARCHAR(100) NOT NULL);
  V2__add_customer_email.sql     ALTER TABLE customer ADD COLUMN email VARCHAR(200); CREATE UNIQUE INDEX ...
  V3__seed_customers.sql         INSERT INTO customer (name, email) VALUES ('Asha', ...), ('Ben', ...);
```

```
spring.jpa.hibernate.ddl-auto=validate   # Hibernate only checks the entities against Flyway's schema
```

```
@DataJpaTest(showSql = false)            // the JPA slice includes Flyway
class MigrationTest {
    @Test
    void migrationsRanInOrderAndHibernateValidatedTheSchema() {
        for (MigrationInfo m : flyway.info().all())
            System.out.printf("V%s  %-25s %s%n", m.getVersion(), m.getDescription(), m.getState());
        // + print flyway_schema_history and the customers
    }
}
```

```
V1  create customer           SUCCESS
V2  add customer email        SUCCESS
V3  seed customers            SUCCESS
history: -1 | null | << Flyway Schema History table created >> | TABLE
history: 1 | 1 | create customer | SQL
history: 2 | 2 | add customer email | SQL
history: 3 | 3 | seed customers | SQL
customer: Asha <asha@example.com>
customer: Ben <ben@example.com>
```

(Flyway 12.4 also printed a warning that H2 2.4 is newer than the H2 version it was verified with, which is harmless here.)

How it works

1. At startup, before JPA initializes, Boot runs Flyway against the `DataSource`.
2. Flyway reads `flyway_schema_history`, finds the scripts not yet applied, runs them in version order (each in a transaction where the database allows), and records each with a **checksum** (a fingerprint of the file's contents, so any edit is detected).
3. Hibernate starts with `ddl-auto=validate`. If an entity has a column the schema lacks, for example if V2 were missing, startup fails. That is a useful safety net.

Rules

Rule	Why
Never edit a migration that has been applied anywhere	Flyway's checksum check fails (`validate` error). Fix forward with a new version
Naming: `V<version>__<description>.sql` (two underscores), `R__<name>.sql` for repeatable scripts (views, functions)	how Flyway orders and classifies
Make changes backward-compatible (expand → migrate → contract: add new columns, move data and code, only later remove old ones)	during a rolling deploy (servers updated one at a time), old and new app versions share one schema
Big data changes in separate, batched migrations	one huge transaction locks tables
Test migrations against the real database type	H2 accepts SQL that PostgreSQL rejects, and the other way round. Tests with PostgreSQL via Testcontainers (real databases in Docker): ch12

8.17 Testing JPA code

`@DataJpaTest` (package `org.springframework.boot.data.jpa.test.autoconfigure` in Boot 4, from `spring-boot-starter-data-jpa-test`) starts only JPA: entities, repositories, `EntityManager`, `TestEntityManager`, an embedded database, and Flyway/Liquibase if present. Like `@JdbcTest`, it wraps each test in a rolled-back transaction and turns on `show-sql` (set `showSql = false` to silence it).

Remember what that transaction hides. Inside it, lazy loading always works and nothing is ever committed. That is why this chapter's `LazyLoadingTest`, `AuditingAndLockingTest` and `OpenInViewTest` use `@SpringBootTest` **without** a test transaction. Call `em.flush()` and `em.clear()` in `@DataJpaTest` tests when you want to see the SQL and read back from the database, not from the cache. More in ch12.

8.18 Performance checklist

Check	How
All `@ManyToOne` / `@OneToOne` LAZY	`fetch = FetchType.LAZY`
No N+1 on hot paths	assert statement counts in tests. Use JOIN FETCH / `@EntityGraph`
Safety net	`spring.jpa.properties.hibernate.default_batch_fetch_size=16` (or similar)
Read-only screens	projections / DTOs, `@Transactional(readOnly = true)`
Batched inserts	SEQUENCE ids + `hibernate.jdbc.batch_size=50` + `hibernate.order_inserts=true` (IDENTITY disables insert batching)
OSIV off	`spring.jpa.open-in-view=false`
No unbounded queries	paging (keyset for deep pages), `Limit`
Big batch jobs	chunk + `flush()` / `clear()`, or JDBC
Indexes for your query patterns	foreign keys are not indexed automatically in PostgreSQL. See SQL ch12

8.19 Interview questions

1. **JPA vs Hibernate vs Spring Data JPA?** → Specification vs implementation vs Spring's repository layer on top.
2. **What is the persistence context?** → The `EntityManager`'s first-level cache and unit of work. One instance per row, snapshots, dirty checking, write-behind at flush.

3. **Why don't I need `save()` after changing a loaded entity?** → Dirty checking at flush, inside a transaction.
4. **Entity states?** → Transient, managed, detached, removed.
5. **What is the owning side?** → The side with the foreign key (`@ManyToOne`). JPA only reads it. The other side uses `mappedBy`.
6. **`orphanRemoval` vs `CascadeType.REMOVE`?** → Removal from the collection deletes the child vs deleting the parent deletes the children.
7. **Default fetch types?** → To-one: EAGER. To-many: LAZY. Make everything LAZY.
8. **What causes `LazyInitializationException` and how do you fix it?** → Touching a lazy association after the persistence context closed. Fetch it in the query, work inside a transaction and return DTOs. Not EAGER, and preferably not OSIV.
9. **Explain N+1 and three fixes.** → 1 query + 1 per parent. JOIN FETCH, `@EntityGraph`, batch fetching (and DTO projections).
10. **JOIN FETCH vs `@EntityGraph`?** → The same idea. JOIN FETCH is written in JPQL (inner join unless `left`). An entity graph is declared on the method and uses a left join.
11. **How does `save()` choose between persist and merge?** → `isNew`: null id, or null wrapper version. Otherwise merge.
12. **Derived query vs `@Query` vs native?** → Short conditions vs readable JPQL vs database-specific SQL.
13. **`Page` vs `Slice`?** → `Page` runs an extra COUNT and knows totals. `Slice` only knows whether there is a next one.
14. **Interface vs DTO projections?** → Both select fewer columns. Records are plain values, interfaces are proxies. Open interface projections load the entity.
15. **How does optimistic locking work?** → A `@Version` column, `WHERE version = ?`, 0 rows → `ObjectOptimisticLockingFailureException`.
16. **What is OSIV and should it be on?** → The persistence context stays open for the whole web request. It is on by default with a warning. Turn it off for APIs.
17. **`ddl-auto` in production?** → `validate` or `none`, with Flyway/Liquibase owning the schema.
18. **What changed for Flyway in Boot 4?** → Use `spring-boot-starter-flyway`. `flyway-core` alone is not auto-configured.

Chapter TL;DR

- **JPA** is the spec, **Hibernate 7** the implementation, **Spring Data JPA** the repository layer. Boot 4.1 ships Hibernate 7.4 and Spring Data 2026.0.
- The **persistence context** gives one object per row, tracks changes, and flushes `UPDATE`s automatically (**dirty checking**). Detached entities are not tracked.
- **Relationships**: the owning side holds the foreign key. Use `mappedBy` on the other side and keep both in sync. Cascade from aggregate roots only. `orphanRemoval` deletes removed children.
- **LAZY everything.** `LazyInitializationException` means you touched it outside a transaction. Fetch per use case instead.
- **N+1** is 1 + N queries, measured here as 5 statements for 4 authors. Fix it with `JOIN FETCH` (1), `@EntityGraph` (1), batch size (2), or projections. EAGER makes it worse, not better.
- **Repositories**: derived queries, `@Query` (JPQL/native), `@Modifying` (clears nothing unless asked), `Page` / `Slice` / `Window`, projections for reads.
- **Auditing** fills created/modified columns. `@Version` turns lost updates into `ObjectOptimisticLockingFailureException`. `@Lock(PESSIMISTIC_WRITE)` gives `FOR UPDATE`.
- Turn **open-in-view off**, and let **Flyway** (`spring-boot-starter-flyway` in Boot 4) own the schema with `ddl-auto=validate`.

Next: Chapter 9 — Spring Data beyond JPA, and caching covers Spring Data JDBC, MongoDB, Redis and the cache abstraction.

Chapter 9 — Spring Data beyond JPA, and caching

Chapter 8 used JPA, but it is not the only option, nor always the best. This chapter covers the other Spring Data modules you are most likely to meet: **Spring Data JDBC**, a simpler relational mapper built around *aggregates* (groups of objects saved and loaded as one unit, §9.2); **Spring Data MongoDB** for document databases (each record a JSON-like document instead of table rows); **Spring Data Redis** for the in-memory key-value store; and **Spring Data REST**, which turns repositories into a hypermedia REST API. It then compares the schema-migration tools **Flyway and Liquibase** (Java and repeatable migrations, checksum validation).

The second half is **caching**: remembering an expensive call's result so the next identical call is fast. Spring's cache abstraction (`@Cacheable`, `@CachePut`, `@CacheEvict`, `@Caching`, `@CacheConfig`, keys, `condition`/`unless`, TTL — time to live, how long an entry stays valid — and `sync`) adds caching with one annotation and is easy to get wrong, so every behaviour is measured: the tests count real method calls and cache hits/misses, move a fake clock to show expiry, and start eight threads at once to show a cache stampede (many requests recomputing the same missing entry) and its fix. Caffeine (a fast Java in-memory cache) is the local cache; Redis the shared one.

The code (Spring Boot 4.1.1 → Spring Framework 7.0, Spring Data 2026.0, Flyway 12.4, Caffeine 3.2, H2 in memory):

Project	What it proves	Runs here?
`code/data-jdbc`	Spring Data JDBC aggregates with SQL printed, optimistic locking, queries, assigned-id trap; Flyway SQL + Java + repeatable migrations and checksum validation; Spring Data REST	yes
`code/caching`	`@Cacheable`/`@CachePut`/`@CacheEvict` with Caffeine, hit/miss counts, key generation, TTL with a fake clock, mutable-object pitfall, stampede with and without `sync`	yes
`code/nosql`	MongoDB and Redis repositories, `MongoTemplate` aggregation, `StringRedisTemplate`, Redis cache configuration, Testcontainers test	compiled only (needs Docker, not run)

Run everything with `bash tools_run_spring.sh spring '09-*'`. The `nosql` project has a `.tools_run_skip` file, so the runner skips it.

9.1 The Spring Data family

What it is

Spring Data is an umbrella project with one module per kind of data store. Each module gives you the same programming model:

1. You write an interface that extends a repository interface, such as `CrudRepository<T, ID>`.
2. Spring creates the implementation at startup.
3. Method names like `findByStatus` become queries (**derived queries**), and `@Query` lets you write a query yourself.

What differs is the **mapping** (how an object becomes a row, a document or a hash) and the extra features each store has.

Module	Store	Boot starter	Maps objects to	Template class for hand-written work
Spring Data JPA	relational, via Hibernate	`spring-boot-starter-data-jpa`	rows, with a persistence context	`EntityManager`
Spring Data JDBC	relational, plain JDBC	`spring-boot-starter-data-jdbc`	rows, **aggregate by aggregate**	`JdbcAggregateTemplate`
Spring Data R2DBC	relational, reactive	`spring-boot-starter-data-r2dbc`	rows (no relationships)	`R2dbcEntityTemplate` (ch13)
Spring Data MongoDB	document database	`spring-boot-starter-data-mongodb`	BSON (binary JSON) documents	`MongoTemplate`
Spring Data Redis	in-memory key-value	`spring-boot-starter-data-redis`	Redis hashes (a key holding a small field → value map)	`RedisTemplate`, `StringRedisTemplate`
Spring Data REST	(any of the above)	`spring-boot-starter-data-rest`	repositories → HTTP endpoints	—

Other modules exist (Elasticsearch, Cassandra, Neo4j, Couchbase, LDAP), and they follow the same pattern.

All Spring Data modules are released together as a **release train**. The train is named by year: Spring Boot 4.1.1 manages **Spring Data 2026.0**, which contains Spring Data JDBC 4.1, MongoDB 5.1 and Redis 4.1.

What is shared, and what is not

Shared by all modules (from `spring-data-commons`)	Store-specific
`CrudRepository`, `ListCrudRepository`, `PagingAndSortingRepository`	`JpaRepository`, `MongoRepository`, … with extra methods
Derived query parsing (`findByNameAndAgeGreaterThan`)	What a query *is*: SQL, a Mongo JSON filter, a Redis index lookup
`Pageable`, `Sort`, `Page`, `Slice` (8.11)	Transactions (full in JDBC/JPA; limited in Mongo; very different in Redis)
Auditing (`@CreatedDate`, `@LastModifiedBy`), `@Version`, `@Id`, `@Transient` from `org.springframework.data.annotation`	Mapping annotations: `@Table` / `@Column` (relational), `@Document` / `@Field` (Mongo), `@RedisHash` (Redis)
Projections, query by example, domain events (published when an aggregate changes)	Store features: aggregations, TTLs, geo queries

Because the model is shared, the repository chapter of ch08 (sections 8.8–8.12) applies here too. This chapter shows only what is different.

9.2 Spring Data JDBC and aggregates

What it is

Spring Data JDBC maps Java objects to relational tables **without JPA**. It has no Hibernate, no persistence context, no lazy loading, no dirty checking and no first-level cache. When you call `save`, it runs SQL right away. When you call `findById`, it runs SQL and gives you a plain object that is not connected to anything.

Its design comes from **Domain-Driven Design (DDD)**, where an **aggregate** is a cluster of objects always loaded, changed and saved as one unit, entered only through its **aggregate root** — e.g. an order and its lines; rules like "you cannot add lines to a paid order" live in the root.

Spring Data JDBC builds on that idea:

- **One repository per aggregate root.** There is a `PurchaseOrderRepository` but no `OrderLineRepository`.
- **Everything the root references by object is part of the aggregate.** It is saved and deleted with the root.
- **References to other aggregates are ids, not objects.** An order holds the *id* of its customer, never a `Customer` object.

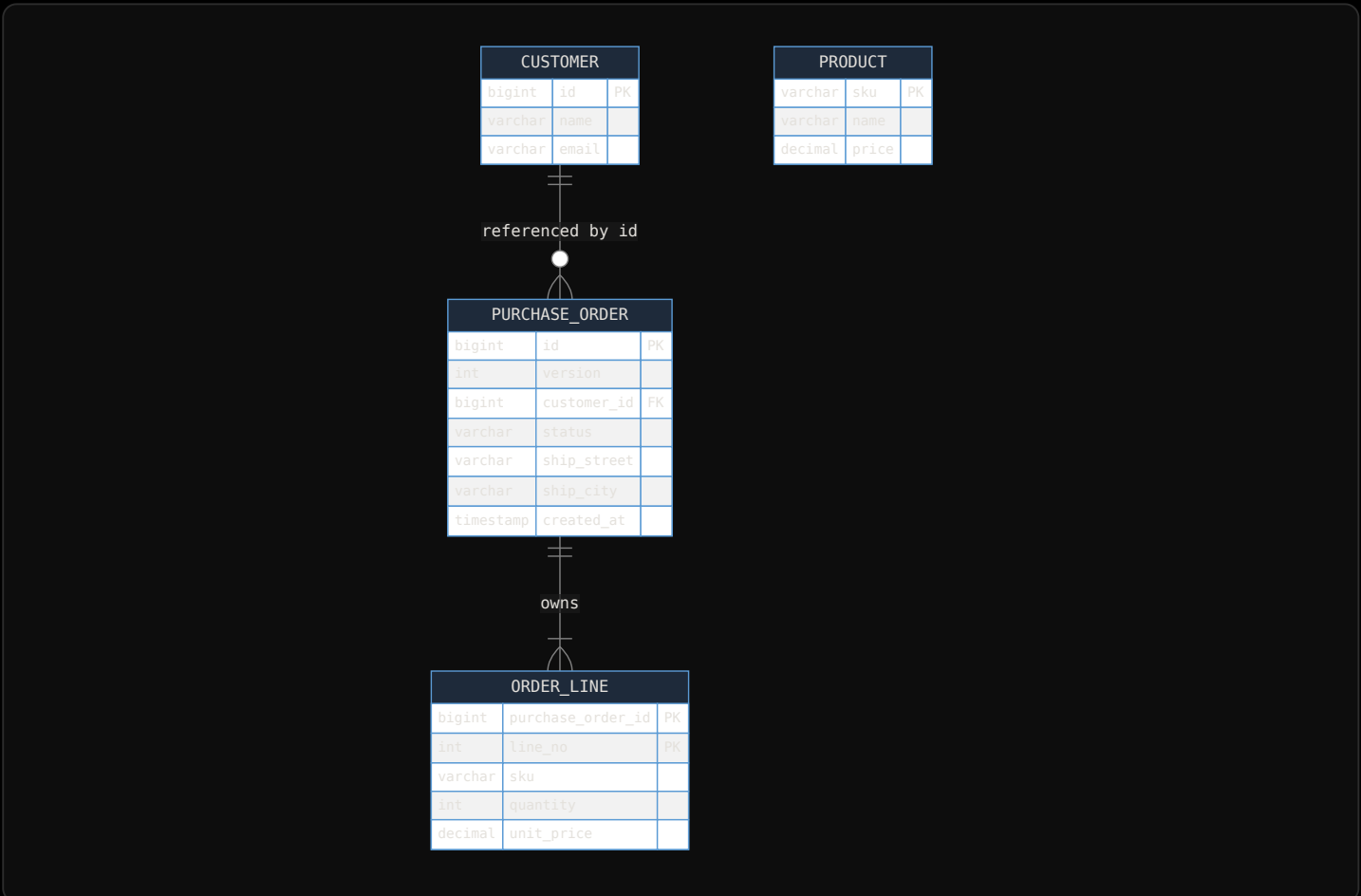
Why it matters

JPA is powerful but complex. Most of the surprises in ch08 (`LazyInitializationException`, N+1 queries, updates without a `save`, detached entities) come from the persistence context and lazy loading. Spring Data JDBC removes both. You get fewer features, and in exchange every call does exactly what it says.

	Spring Data JPA (Hibernate)	Spring Data JDBC
Loading	lazy or eager, proxies	everything in the aggregate, eagerly; nothing outside it
Change tracking	dirty checking: changes are flushed automatically	none: only `save()` writes
Caching	first-level cache per persistence context (+ optional shared second level)	none
Relationships	any direction, many-to-many, bidirectional	one direction, root → children; other aggregates by id
Schema generation	`ddl-auto` can create tables	never; you write the DDL (`CREATE TABLE` etc.; use Flyway)
SQL you can predict	not always	yes
Updates of child collections	only the changed rows	**delete all children, insert them again**
Best for	rich object graphs, existing JPA knowledge	clear aggregates, teams that want to see the SQL

The example aggregate

An online shop. `Customer` and `Product` are aggregates on their own. `PurchaseOrder` is an aggregate whose root holds a list of `OrderLine`s, an embedded shipping `Address`, and the *id* of its customer.



The schema comes from a Flyway migration (`V1__create_shop.sql`), because Spring Data JDBC never creates tables. The aggregate root:

```java
public class PurchaseOrder {

    @Id Long id;
    @Version Integer version;                                   // optimistic locking; null = new
    AggregateReference<Customer, Long> customerId;             // another aggregate: id only, never loaded
    Status status = Status.NEW;
    @Embedded.Nullable(prefix = "ship_") Address shipTo;      // value object in the same row
    @CreatedDate Instant createdAt;                             // filled by auditing

    // Names written in annotations are used exactly as given AND quoted. Upper case matches H2's
    // unquoted DDL (and Oracle/DB2); on PostgreSQL/MySQL write them in lower case.
    @MappedCollection(idColumn = "PURCHASE_ORDER_ID", keyColumn = "LINE_NO")
    List<OrderLine> lines = new ArrayList<>();                 // owned children (table order_line)

    PurchaseOrder() {}                                          // for Spring Data

    public PurchaseOrder(long customerId, Address shipTo) {
        this.customerId = AggregateReference.to(customerId);
        this.shipTo = shipTo;
    }

    /** Business rules live in the root: all changes to lines go through it. */
    public void addLine(String sku, int quantity, BigDecimal unitPrice) {
        if (status != Status.NEW) throw new IllegalStateException("order is " + status);
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be positive");
        lines.add(new OrderLine(sku, quantity, unitPrice));
    }
    // total(), getters ...
}

public record OrderLine(String sku, int quantity, BigDecimal unitPrice) {}   // no @Id: part of the order
public record Address(String street, String city) {}                          // value object
public record Customer(@Id Long id, String name, String email) {}             // its own aggregate
```

The mapping rules

Java	Table/column	Notes
class `PurchaseOrder`	table `purchase_order`	default naming: camelCase (`purchaseOrder`) → snake_case (`purchase_order`). Override with `@Table("...")`
field `createdAt`	column `created_at`	override with `@Column("...")`
`@Id Long id`	primary key	`null` means "new": the database generates the id (identity column)
`@Version Integer version`	`version`	optimistic locking, as in 8.14
`List<OrderLine>` + `@MappedCollection`	child table `order_line`	`idColumn` = foreign key to the root, `keyColumn` = the list index. `Set` needs no key column; `Map<K, V>` uses the map key
`AggregateReference<Customer, Long>`	`customer_id`	just the id (a `Customer customer` field would make the customer part of the order aggregate). You cannot navigate to it; load it with `CustomerRepository` if needed
`@Embedded.Nullable(prefix = "ship_")`	`ship_street`, `ship_city` in the same row	`Nullable`: all columns `NULL` → field `null`. `@Embedded.Empty` gives an empty object instead
records	supported	Spring Data creates a new instance through the constructor when it must set a generated id

Spring Data JDBC quotes every identifier it generates. With H2 that means upper case: `"PURCHASE_ORDER"`. A name you write in an annotation is quoted **as you wrote it**, so `idColumn = "purchase_order_id"` became `"purchase_order_id"` and H2 answered `Column "line_no" not found`. That is why the example uses upper case. On PostgreSQL, unquoted names are folded to lower case, so there you write lower case.

How saving and loading work — measured

The tests capture the SQL that Spring's `JdbcTemplate` logs at `DEBUG` (helper `SqlLog.java`, which also removes the quotes to make it readable). `@DataJdbcTest` is the slice for Spring Data JDBC. It starts the repositories, JDBC, Flyway and an embedded database, and rolls back each test.

```
@DataJdbcTest                                // JDBC + Spring Data JDBC + Flyway, each test rolled back
class AggregateTests {
    @Autowired PurchaseOrderRepository orders;
    final SqlLog sql = new SqlLog();

    PurchaseOrder newOrder() {
        var order = new PurchaseOrder(1L, new Address("1 Main St", "Pune"));
        order.addLine("BOOK-1", 1, new BigDecimal("40.00"));
        order.addLine("PEN-7", 4, new BigDecimal("2.50"));
        return order;
    }

    @Test
    void savingTheRootSavesItsLinesAndLoadingReadsThemBack() {
        sql.clear();
        PurchaseOrder saved = orders.save(newOrder());
        sql.print("save(new order with 2 lines)");

        sql.clear();
        PurchaseOrder loaded = orders.findById(saved.getId()).orElseThrow();
        sql.print("findById");
        // print id, version, customer id, lines, total, address, createdAt ...
    }
}
```

Output:

```
-- save(new order with 2 lines)
   INSERT INTO PURCHASE_ORDER (CREATED_AT, CUSTOMER_ID, SHIP_CITY, SHIP_STREET, STATUS, VERSION) VALUES (?, ?, ?, ?, ?,
?)
   [batch] INSERT INTO ORDER_LINE (LINE_NO, PURCHASE_ORDER_ID, QUANTITY, SKU, UNIT_PRICE) VALUES (?, ?, ?, ?, ?)
-- findById
   SELECT ID, STATUS, SHIP_STREET, SHIP_CITY, VERSION, CREATED_AT, CUSTOMER_ID FROM PURCHASE_ORDER WHERE
PURCHASE_ORDER.ID = ?
   SELECT SKU, QUANTITY, UNIT_PRICE, LINE_NO FROM ORDER_LINE WHERE ORDER_LINE.PURCHASE_ORDER_ID = ? ORDER BY LINE_NO
loaded: id=3 version=0 customer=1 lines=2 total=50.00 shipTo=Address[street=1 Main St, city=Pune] createdAt set=true
```

What happened, step by step:

1. `save` saw `id == null`, so the order is **new**. It inserted the root row and got the generated id back.
2. It inserted both lines in **one JDBC batch**, with `PURCHASE_ORDER_ID` set to the new id and `LINE_NO` set to the list index (0 and 1).
3. Auditing filled `createdAt`. `@EnableJdbcAuditing` sits on the application class, and the test slice picks it up from there. Adding `@EnableJdbcAuditing` a second time in a test configuration fails with a duplicate bean definition error.
4. `findById` ran **two** queries: one for the root, one for its lines. There is no join, no lazy proxy, and the customer was **not** loaded. The order only knows `customer=1`.
5. The version of a new row starts at 0.

Loading the root plus one query per collection is fine for one aggregate. When you load many aggregates, it becomes an N+1 pattern (one lines query per order). Spring Data JDBC has an opt-in **single query loading** mode (`JdbcMappingContext.setSingleQueryLoadingEnabled(true)`) that loads aggregates with one SQL statement using window functions. The reference docs call it experimental. It works only for `find…` methods of `CrudRepository`, for aggregates without nested collections, embedded values or `AggregateReference`s, and not on H2. So in practice, keep aggregates small and read lists of data with a `@Query` DTO (shown below).

Updating: all children are deleted and inserted again

```
@Test
void updatingTheRootDeletesAndReinsertsAllLines() {
    PurchaseOrder order = orders.save(newOrder());
    order.addLine("PEN-7", 1, new BigDecimal("2.50"));     // change ONE thing: add a third line

    sql.clear();
    orders.save(order);
    sql.print("save(order with 1 added line)");
}
```

```
-- save(order with 1 added line)
   UPDATE PURCHASE_ORDER SET VERSION = ?, STATUS = ?, SHIP_STREET = ?, SHIP_CITY = ?, CREATED_AT = ?, CUSTOMER_ID = ?
WHERE PURCHASE_ORDER.ID = ? AND PURCHASE_ORDER.VERSION = ?
   DELETE FROM ORDER_LINE WHERE ORDER_LINE.PURCHASE_ORDER_ID = ?
   [batch] INSERT INTO ORDER_LINE (LINE_NO, PURCHASE_ORDER_ID, QUANTITY, SKU, UNIT_PRICE) VALUES (?, ?, ?, ?, ?)
```

Spring Data JDBC does not remember what it loaded, so it cannot know *which* line changed. It updates the root (checking the version), deletes **all** lines, and inserts all three again. This is simple and always correct, and it is the main cost of the approach:

- Fine for aggregates with a few children (an order with 1–50 lines).
- Bad for an aggregate with thousands of children. That is usually a sign the aggregate is too big: make the children a separate aggregate that references the parent by id.
- Child rows get new physical rows on every save, so do not point foreign keys from *other* tables at child rows.

Optimistic locking

The `UPDATE` above ends with `AND PURCHASE_ORDER.VERSION = ?`. If another transaction changed the row in between, no row matches:

```
Long id = orders.save(newOrder()).getId();
PurchaseOrder alice = orders.findById(id).orElseThrow();   // both read version 0
PurchaseOrder bob = orders.findById(id).orElseThrow();

alice.setStatus(Status.PAID);
orders.save(alice);                                        // version 0 -> 1

bob.setStatus(Status.SHIPPED);
assertThatThrownBy(() -> orders.save(bob))                 // still has version 0
        .isInstanceOf(OptimisticLockingFailureException.class);
```

```
alice saved, version now 1
bob: OptimisticLockingFailureException: Failed to update versioned entity with id '1' (version '0') in table
["PURCHASE_ORDER"]; Was the entity updated or deleted concurrently?
```

It is the same idea as JPA's `@Version` (8.14). Because the whole aggregate is saved through the root, **one** version column protects the root and all its lines together. That is exactly what you want: two users cannot both add lines to the same order and silently overwrite each other.

Queries

```
public interface PurchaseOrderRepository extends ListCrudRepository<PurchaseOrder, Long> {

    List<PurchaseOrder> findByStatus(Status status);              // derived query

    List<PurchaseOrder> findByCustomerId(Long customerId);        // derived query on the reference column

    @Query("SELECT order_id, customer, total FROM order_totals ORDER BY order_id")
    List<OrderTotal> totals();                                    // hand-written SQL into a DTO

    @Modifying
    @Query("UPDATE purchase_order SET status = :status WHERE id = :id")
    boolean updateStatus(Long id, String status);                 // bulk-style update, bypasses the aggregate
}

public record OrderTotal(Long orderId, String customer, BigDecimal total) {}
```

```
findByCustomerId(2): 1 order(s)
shipTo of order without address: null
updateStatus -> true, findByStatus(PAID): 1
total: OrderTotal[orderId=4, customer=Asha, total=50.00]
total: OrderTotal[orderId=5, customer=Ben, total=80.00]
```

- **Derived queries** work as in JPA, but they may only use properties of the root row (and embedded values). You cannot write `findByLinesSku`, because that would need a join into the aggregate. Use `@Query` for that.
- `@Query` takes plain SQL, not JPQL. Parameters are named (`:id`) and bound by parameter name. The result can be the entity, a DTO record (columns mapped by name, `order_id` → `orderId`) or a simple value.
- `@Modifying` marks `UPDATE`/`DELETE` queries. It returns the row count as `int`, or `boolean` ("at least one row changed"). It bypasses the aggregate: no version check, no callbacks, no business rules. Use it for bulk or administrative changes only.
- The view `order_totals` is a read model (shaped just for queries) built by a repeatable migration (section 9.4). Reading reports with SQL and DTOs, and writing through aggregates, is a common split.

Pitfall: "new or existing?" with assigned ids

Spring Data must decide between `INSERT` and `UPDATE`. The rule (Spring Data JDBC reference):

- the entity is **new** if its id is `null` (or `0` for a primitive), **or** if it has a `@Version` property that is `null` (or `0` for a primitive);
- otherwise it is **existing**.

`Product` uses its SKU as the id, a value *we* assign. So a brand-new product already has a non-null id:

```
public record Product(@Id String sku, String name, BigDecimal price) {}

Product mug = new Product("MUG-3", "Coffee mug", new BigDecimal("9.00"));
products.save(mug);                                   // no exception...
template.insert(mug);                                 // JdbcAggregateTemplate: "this is new"
```

```
-- save(new Product("MUG-3", ...))
   UPDATE PRODUCT SET NAME = ?, PRICE = ? WHERE PRODUCT.SKU = ?
rows after save():   2   <- nothing was inserted!
rows after insert(): 3   Product[sku=MUG-3, name=Coffee mug, price=9.00]
```

`save` ran an `UPDATE` that matched zero rows, and **no exception was thrown**. The product was silently lost. This is one of the most common Spring Data JDBC bugs. The fixes:

Fix	How
Add a `@Version` field	a new object has version `null` / `0` → `INSERT`. Also gives you optimistic locking. Usually the best fix
Call `JdbcAggregateTemplate.insert(entity)` / `update(entity)`	you state the intent explicitly (shown above)
Implement `Persistable<ID>`	you write `isNew()` yourself, e.g. with a `@Transient boolean isNew` flag
Let the database generate ids	`null` id → `INSERT`. Keep the natural key as a unique column

The same `isNew` rule exists in Spring Data JPA (`save` then calls `merge`, which runs an extra `SELECT` rather than losing data).

Callbacks, events and auditing

Spring Data JDBC calls beans at fixed points of the lifecycle. **Callbacks** can change the entity: `BeforeConvertCallback` (set an id before insert), `BeforeSaveCallback`, `AfterSaveCallback`, `AfterConvertCallback` (after loading). **Events** such as `BeforeSaveEvent`, `AfterSaveEvent` and `AfterDeleteEvent` are published as application events (ch02) and are for observing only. Auditing (`@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`) is switched on with `@EnableJdbcAuditing`. For the "by" fields you also provide an `AuditorAware` bean.

Common mistakes

Mistake	Result	Fix
Expecting lazy loading or dirty checking	changes are lost unless you call `save`	load, change, **save** — explicitly
Mapping a child as an object reference to *another* aggregate	Spring Data treats it as part of your aggregate and deletes/inserts it on save	reference other aggregates with `AggregateReference` or a plain id
Huge child collections	every save deletes and re-inserts thousands of rows	smaller aggregates
Assigned ids without `@Version` or `Persistable`	`save` silently updates 0 rows	see the table above
Lower-case names in annotations on H2 (upper-case on PostgreSQL)	`column not found`	match the database's folding rules
Expecting `ddl-auto`	there is none; tables do not exist	Flyway/Liquibase
Relying on `@DataJdbcTest` rollback when testing concurrency	two "users" share one test transaction	`@SpringBootTest` without a test transaction, as in ch08

In the interview

30-second answer: "Spring Data JDBC is a simpler alternative to JPA. It maps aggregates, in the DDD sense: a root plus the entities it owns, loaded and saved as one unit. There is no persistence context, so no lazy loading, no dirty checking and no caching. `save` writes immediately, loading reads the whole aggregate, and references to other aggregates are just ids (`AggregateReference`). On update it rewrites the root and deletes and re-inserts all children, which is why aggregates must stay small. I pick it when I want predictable SQL and my model has clear aggregates."

- *"How does Spring Data decide between INSERT and UPDATE?"* → id `null` / `0` or `@Version` `null` / `0` → new. With assigned ids you need `@Version`, `Persistable`, or `JdbcAggregateTemplate.insert`, otherwise `save` issues an `UPDATE` that changes nothing.
- *"How do you model many-to-many?"* → As a separate small entity inside one aggregate that holds the *id* of the other aggregate (for example `Set<BookAuthor>` in `Book`, where `BookAuthor` has an `AggregateReference<Author, Long>`).
- *"Can you mix JPA and JDBC in one app?"* → Yes, on the same `DataSource`, but give them different entities and packages, and remember that JDBC writes are not seen by an open Hibernate persistence context.

9.3 Spring Data REST

What it is

Spring Data REST looks at your repositories and publishes each one as a REST resource, with no controller code. It uses **HAL** (Hypertext Application Language), a JSON format where every resource has a `_links` section pointing to related resources. That style is called **HATEOAS** (Hypermedia as the Engine of Application State; more in ch05).

The `data-jdbc` project adds `spring-boot-starter-data-rest` and one property:

```
spring.data.rest.base-path=/api
```

```
@RepositoryRestResource(path = "customers")
public interface CustomerRepository extends ListCrudRepository<Customer, Long> {
    List<Customer> findByName(String name);   // -> GET /api/customers/search/findByName?name=Ben
}

@RepositoryRestResource(exported = false)     // not every repository should become a public API
public interface ProductRepository extends ListCrudRepository<Product, String> {}
```

The test (`DataRestTests.java`) calls it with `MockMvcTester`:

```
GET /api/customers/2 -> 200
{
  "_links" : {
    "self" : {
      "href" : "http://localhost/api/customers/2"
    },
    "customer" : {
      "href" : "http://localhost/api/customers/2"
    }
  },
  "name" : "Ben",
  "email" : "ben@example.com"
}
GET /api/customers/search/findByName?name=Asha -> 200
GET /api/products -> 404 (exported = false)
```

For each exported repository you get `GET` (collection, with paging and sorting if the repository supports it), `GET /{id}`, `POST`, `PUT`, `PATCH`, `DELETE`, and a `/search` resource listing the query methods.

Trade-offs

Good for	Not good for
Internal admin tools, prototypes, simple CRUD services	public APIs with a designed contract
Clients that follow links (HAL browsers, hypermedia clients)	APIs with business operations ("pay order") rather than CRUD
	Hiding your table structure: the API *is* your domain model, so a field rename breaks clients

Common mistakes

- **Exposing every repository by accident.** Every public repository is exported by default. Use `exported = false`, or set `spring.data.rest.detection-strategy=annotated` so that only `@RepositoryRestResource` repositories are exported.
- **Forgetting security.** `DELETE /api/customers/1` works for anyone until you add Spring Security rules (ch10).
- **Business rules in the database layer.** Validation and side effects then have to go into `@RepositoryEventHandler` callbacks, which are easy to miss. When the API needs real behaviour, write controllers (ch05).

9.4 Schema migrations: Flyway beyond the basics, and Flyway vs Liquibase

Section 8.16 introduced Flyway: numbered SQL scripts in `db/migration`, run once each, recorded in `flyway_schema_history`, and `spring-boot-starter-flyway` in Boot 4. Spring Data JDBC depends on migrations even more than JPA, because it has no `ddl-auto` at all. This section adds three Flyway features and compares Flyway with Liquibase.

Three kinds of Flyway migration

The `data-jdbc` project has all three:

```
src/main/resources/db/migration/
  V1__create_shop.sql            versioned SQL: tables
  V2__seed_customers.sql         versioned SQL: data
  R__order_totals_view.sql       REPEATABLE: CREATE OR REPLACE VIEW order_totals ...
src/main/java/db/migration/
  V3__normalise_emails.java      versioned JAVA migration
```

```java
package db.migration;

/** A Java-based migration: for data changes that are awkward in SQL. Found by the class name V3__... */
public class V3__normalise_emails extends BaseJavaMigration {
    @Override
    public void migrate(Context context) throws Exception {
        var conn = context.getConnection();              // same connection and transaction as Flyway
        try (Statement select = conn.createStatement();
             ResultSet rs = select.executeQuery("SELECT id, email FROM customer");
             PreparedStatement update = conn.prepareStatement("UPDATE customer SET email = ? WHERE id = ?")) {
            while (rs.next()) {
                update.setString(1, rs.getString("email").trim().toLowerCase());
                update.setLong(2, rs.getLong("id"));
                update.addBatch();
            }
            update.executeBatch();
        }
    }
}
```

Kind	Name	Runs	Use for
Versioned SQL	`V<version>__<description>.sql`	once, in version order	tables, columns, indexes, data fixes
Versioned Java	class `V<version>__<description>` extending `BaseJavaMigration`, in a scanned location (e.g. package `db.migration`), or a Spring bean implementing `JavaMigration` (Boot registers such beans)	once, in version order with the SQL ones	data changes needing logic, calling code, big batched updates
Repeatable	`R__<description>.sql`	after all versioned ones, **again whenever its checksum changes**	views, functions, stored procedures, grants — objects you can always re-create

The test prints what Flyway recorded:

```java
for (MigrationInfo m : flyway.info().all()) {
    System.out.printf("%-4s %-22s %-5s %s%n",
            m.getVersion() == null ? "R" : "V" + m.getVersion(), m.getDescription(), m.getType(), m.getState());
}
customers.findAll().forEach(c -> System.out.println("customer: " + c));
```

```
V1   create shop            SQL   SUCCESS
V2   seed customers         SQL   SUCCESS
V3   normalise emails       JDBC  SUCCESS
R    order totals view      SQL   SUCCESS
customer: Customer[id=1, name=Asha, email=asha@example.com]
customer: Customer[id=2, name=Ben, email=ben@example.com]
```

V2 inserted `ASHA@Example.com` and `ben@EXAMPLE.com`, and the Java migration V3 normalised them. The Java migration shows up with type `JDBC`.

Checksums: never edit an applied migration — proved

Flyway stores a **checksum** of each applied migration's content. On startup it validates that the files still match. The second test points a new Flyway instance at the same database but at a folder `db/tampered` that holds copies of V1, V2 and the repeatable script, with V1 edited:

```
Flyway tampered = Flyway.configure()
        .dataSource(dataSource)
        .locations("classpath:db/tampered")                // same files, but V1 was edited
        .load();
ValidateResult result = tampered.validateWithResult();
```

```
validation successful: false
  1: Migration checksum mismatch for migration version 1
-> Applied to database : 1770847798
-> Resolved locally    : 415358952
Either revert the changes to the migration, or run repair to update the schema history.
```

Widening the `name` columns in V1 from `VARCHAR(100)` to `VARCHAR(150)` was enough: the file no longer matches the recorded checksum. In an application this validation runs at startup, and the app **refuses to start**. The right reaction is almost never `repair`. Undo the edit and add a new migration (`V4__widen_customer_name.sql`) instead.

Useful Flyway properties

Property	Default	Meaning
`spring.flyway.locations`	`classpath:db/migration`	where scripts are. Use `classpath:db/migration/{vendor}` for per-database folders
`spring.flyway.baseline-on-migrate`	`false`	adopting Flyway on an **existing** database: mark the current state as the baseline instead of failing on a non-empty schema
`spring.flyway.clean-disabled`	`true`	`clean` drops everything in the schema. Keep it disabled everywhere except throwaway databases
`spring.flyway.enabled`	`true`	switch off, e.g. when a separate deploy pipeline runs migrations
`spring.flyway.url` / `user` / `password`	the app's `DataSource`	migrate with a more powerful user than the app uses

Liquibase

Liquibase solves the same problem with a different model. Migrations are **changesets** inside **changelog** files (YAML, XML, JSON or SQL). Each changeset is identified by `id` + `author` + file name, and applied changesets are recorded in the `DATABASECHANGELOG` table (with a lock table `DATABASECHANGELOGLOCK`). Changesets can be written as database-independent *change types* (`createTable`, `addColumn`), which Liquibase turns into SQL for each database.

In Boot 4, add `spring-boot-starter-liquibase`. The default master changelog is `classpath:/db/changelog/db.changelog-master.yaml` (`spring.liquibase.change-log`). A snippet (not run):

```yaml
# src/main/resources/db/changelog/db.changelog-master.yaml
databaseChangeLog:
  - changeSet:
      id: 1
      author: asha
      changes:
        - createTable:
            tableName: customer
            columns:
              - column: { name: id, type: BIGINT, autoIncrement: true, constraints: { primaryKey: true } }
              - column: { name: name, type: VARCHAR(100), constraints: { nullable: false } }
  - changeSet:
      id: 2
      author: asha
      changes:
        - addColumn:
            tableName: customer
            columns:
              - column: { name: email, type: VARCHAR(200) }
      rollback:
        - dropColumn: { tableName: customer, columnName: email }
```

Flyway vs Liquibase

	Flyway	Liquibase
Migration format	plain SQL files (+ Java)	changelogs in YAML/XML/JSON/SQL; database-independent change types
Ordering	version number in the file name	order of changesets in the changelog
Identity of a migration	version	`id` + `author` + file path
History table	`flyway_schema_history`	`DATABASECHANGELOG` (+ lock table)
Rollback / undo	undo migrations only in Redgate's paid editions; in practice you fix forward	`rollback` blocks, and automatic rollback for many change types, in the free edition
Conditional logic	little (placeholders, callbacks)	preconditions, contexts, labels (e.g. "only in test")
Multi-database products	per-vendor folders	one changelog can target several databases
Learning curve	very low: you already know SQL	higher
Boot 4 starter	`spring-boot-starter-flyway`	`spring-boot-starter-liquibase`
Version in Boot 4.1.1	Flyway 12.4 (Community edition is Apache 2.0)	Liquibase 5.0. **Since 5.0 the Community edition uses the Functional Source License (FSL)**, which turns into Apache 2.0 two years after each release. Free to use, but check it with your legal team

Rule of thumb: a team that writes SQL and targets one database picks **Flyway**. A product that must run on several databases, or a team that needs rollback scripts and conditional changesets, picks **Liquibase**. Do not use both in one application.

Migration rules that apply to both

- Fix forward: never edit or delete an applied migration.
- Make each change **backward-compatible** during rolling deploys (expand → migrate data → contract). See 8.16.
- Test migrations against the **real database type** (Testcontainers, ch12). The H2 in this chapter accepts SQL that PostgreSQL rejects, and the other way round.
- Long data migrations lock tables. Run them in batches, outside the startup path if needed.

In the interview

- *"Flyway or Liquibase?"* → "Flyway for SQL-first teams on one database: simple versioned SQL files and a checksum history. Liquibase when I need database-independent changelogs, rollbacks or conditional changes. Either way the tool owns the schema, and Hibernate's `ddl-auto` is `validate` or `none`."
- *"Someone edited V5 after it ran in production. What happens?"* → Startup fails with a checksum mismatch. Revert the edit and ship the change as V6. `repair` only rewrites the history table's checksums; it does not change the schema.
- *"What is a repeatable migration?"* → `R__` script re-applied whenever its content changes, after all versioned ones. For views and functions.

9.5 Spring Data MongoDB (not run — needs Docker)

What it is

MongoDB is a **document database**. It stores JSON-like documents (the binary format is called BSON) in **collections**, with no fixed schema. A document can hold nested objects and arrays, so data that is read together is stored together. What would be three joined tables in SQL (a review, its tags, its product reference) can be one document. The concepts are covered in SQL book ch16 and DDIA ch2. Here we look at how Spring talks to it.

Setup (Boot 4)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

```
spring.mongodb.uri=mongodb://localhost:27017/shop
spring.data.mongodb.auto-index-creation=true
```

Boot 4 change: Boot 4 split MongoDB support into a driver module (`spring-boot-mongodb`, properties `spring.mongodb.*`) and a Spring Data module (`spring-boot-data-mongodb`, properties `spring.data.mongodb.*` for repository and mapping settings). So the connection is now `spring.mongodb.uri`. The old `spring.data.mongodb.uri` (and `.host`, `.port`, `.database`, ...) is marked deprecated at **error** level in Boot 4.1's configuration metadata: it is no longer bound, and `spring-boot-properties-migrator` reports it.

Documents and repositories

```
/** One MongoDB document. Nested data (tags) is stored inside it - no join table. */
@Document("reviews")
public record Review(
        @Id String id,                  // Mongo's _id; null -> an ObjectId is generated on insert
        @Indexed String productSku,     // index is created only if spring.data.mongodb.auto-index-creation=true
        int stars,
        String text,
        List<String> tags,
        Instant createdAt) {}

public interface ReviewRepository extends MongoRepository<Review, String> {

    // derived queries become Mongo filters: { productSku: ?0 } sorted by createdAt desc
    List<Review> findByProductSkuOrderByCreatedAtDesc(String productSku);

    List<Review> findByStarsGreaterThanEqual(int stars);   // { stars: { $gte: ?0 } }

    @Query("{ 'tags': ?0 }")                               // a JSON filter written by hand
    List<Review> withTag(String tag);
}
```

A saved review looks like this in MongoDB (Spring Data adds a `_class` field with the Java type, used to read back polymorphic data — several subclasses in one collection):

```
{ "_id": ObjectId("..."), "productSku": "BOOK-1", "stars": 5, "text": "Great",
  "tags": ["clear"], "createdAt": ISODate("..."), "_class": "ch09.nosql.mongo.Review" }
```

`MongoTemplate` for aggregations and partial updates

Repositories cover simple reads and writes. For grouping, and for changing one field without rewriting the whole document, use `MongoTemplate`:

```
public record AverageRating(String id, double average, long count) {}   // id = the product sku

public List<AverageRating> averageByProduct() {
    var pipeline = newAggregation(
            group("productSku").avg("stars").as("average").count().as("count"),
            sort(Sort.Direction.DESC, "average"));
    return mongo.aggregate(pipeline, "reviews", AverageRating.class).getMappedResults();
}

/** Update one field in place ($push) without reading and rewriting the whole document. */
public long addTag(String reviewId, String tag) {
    return mongo.updateFirst(
            Query.query(Criteria.where("id").is(reviewId)),
            new Update().push("tags", tag),
            Review.class).getModifiedCount();
}
```

An **aggregation pipeline** is a list of stages that documents flow through (`$match`, `$group`, `$sort`, `$project`, `$lookup`), roughly the document-database version of `WHERE` / `GROUP BY` / `ORDER BY`.

Things that work differently from JPA

Topic	MongoDB with Spring Data
Relationships	embed data that belongs together. Reference other documents by id. `@DBRef` and `@DocumentReference` exist but cost extra queries, like lazy loading
Schema changes	no DDL. Old documents keep their old shape. Code must accept both, or you run a data migration (tools such as Mongock exist)
Transactions	single-document writes are always atomic. Multi-document transactions need a **replica set** (several MongoDB servers holding copies of the data) and a `MongoTransactionManager` bean, which Boot does **not** create for you
Optimistic locking	`@Version` works (the version is part of the update filter)
Indexes	`@Indexed`, `@CompoundIndex`, created only if `auto-index-creation=true`. In production, many teams create indexes in a migration instead
Reactive	`spring-boot-starter-data-mongodb-reactive` + `ReactiveMongoRepository` (ch13)

When to use it (and when not)

- **Good fit:** data that is read as a whole and has a flexible shape (product catalogs with different attributes per category, content, event logs, user profiles).
- **Poor fit:** data with many relationships queried from many directions, or which needs multi-row transactions and strong constraints (accounting, inventory). A relational database with a `jsonb` column is often enough for "a bit of flexible data".

9.6 Spring Data Redis (not run — needs Docker)

What it is

Redis is an **in-memory data structure server**. Keys map to strings, hashes, lists, sets, sorted sets, streams and more, and each command is atomic on the server. Data lives in memory, so reads and writes take well under a millisecond. It is used as a cache (section 9.8), a session store, for rate limiting, counters, leaderboards, distributed locks (shared by many app instances) and simple queues.

`spring-boot-starter-data-redis` brings **Lettuce** as the default client library (Jedis is the alternative). Connection properties keep the `spring.data.redis.*` prefix in Boot 4:

```
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

Templates: the usual way to use Redis

```
@Component
public class PageViewCounter {
    private final StringRedisTemplate redis;

    public PageViewCounter(StringRedisTemplate redis) { this.redis = redis; }

    public long hit(String page) {
        return redis.opsForValue().increment("views:" + page);   // INCR is atomic on the Redis server
    }
}
```

`opsForValue()`, `opsForHash()`, `opsForList()`, `opsForSet()`, `opsForZSet()` match the Redis data types. `StringRedisTemplate` stores keys and values as plain strings. `RedisTemplate<K, V>` can use other **serializers** (objects ↔ bytes; JDK serialization by default, JSON if configured). Choose them deliberately, because anything written with one serializer can only be read back with the same one.

Redis repositories

```
/** Stored as a Redis hash under key "carts:<id>"; removed by Redis after 1 hour (timeToLive, seconds). */
@RedisHash(value = "carts", timeToLive = 3600)
public record Cart(@Id String id, @Indexed String userId, List<CartItem> items) {
    public record CartItem(String sku, int quantity) {}
}

public interface CartRepository extends CrudRepository<Cart, String> {
    List<Cart> findByUserId(String userId);   // works only because userId is @Indexed (a secondary index set)
}
```

How it is stored: the cart becomes a Redis **hash** at key `carts:c1`, the key is added to a set `carts` (for `findAll`), and `@Indexed` adds it to a set `carts:userId:u42`. So derived queries work only on `@Indexed` properties (each such set is a secondary index) and only for equality. There are no ranges and no sorting.

A trap: when Redis expires `carts:c1` because of the TTL, the index sets still contain its id. Spring Data cleans them only if you enable keyspace events (Redis notifications when keys change or expire) (`@EnableRedisRepositories(enableKeyspaceEvents = RedisKeyValueAdapter.EnableKeyspaceEvents.ON_STARTUP)`). Most teams use templates directly and use repositories only for simple cases.

9.7 Testing MongoDB and Redis with Testcontainers (not run)

Testcontainers starts real databases in Docker containers for the duration of a test. Boot's `@ServiceConnection` reads the container's host and port and configures the connection. You do not need `@DynamicPropertySource` or any properties. The full treatment of Testcontainers is in ch12. Here is the setup for this chapter's NoSQL code. It compiles, but it is not run here because Docker is not available:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-junit-jupiter</artifactId>   <!-- Testcontainers 2.x artifact names -->
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-mongodb</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>com.redis</groupId>
  <artifactId>testcontainers-redis</artifactId>
  <scope>test</scope>
</dependency>
```

```
@SpringBootTest
@Testcontainers                       // starts/stops the @Container fields (needs a Docker daemon)
class NoSqlTests {

    @Container @ServiceConnection     // Boot points spring.mongodb.* at this container
    static MongoDBContainer mongo = new MongoDBContainer("mongo:8.0");   // org.testcontainers.mongodb

    @Container @ServiceConnection     // ...and spring.data.redis.* at this one
    static RedisContainer redis = new RedisContainer("redis:8.2");       // com.redis.testcontainers

    @Autowired ReviewRepository reviews;
    @Autowired RatingService ratings;
    @Autowired CartRepository carts;
    @Autowired PageViewCounter views;

    @Test
    void mongoRepositoryAndAggregation() {
        reviews.deleteAll();
        reviews.save(new Review(null, "BOOK-1", 5, "Great", List.of("clear"), Instant.now()));
        reviews.save(new Review(null, "BOOK-1", 3, "Dense", List.of(), Instant.now()));
        reviews.save(new Review(null, "PEN-7", 4, "Smooth", List.of("clear"), Instant.now()));

        assertThat(reviews.findByProductSkuOrderByCreatedAtDesc("BOOK-1")).hasSize(2);
        assertThat(reviews.withTag("clear")).hasSize(2);
        assertThat(ratings.averageByProduct()).first()
                .isEqualTo(new RatingService.AverageRating("PEN-7", 4.0, 1));
    }

    @Test
    void redisRepositoryAndCounter() {
        carts.save(new Cart("c1", "u42", List.of(new Cart.CartItem("PEN-7", 3))));
        assertThat(carts.findByUserId("u42")).extracting(Cart::id).containsExactly("c1");
        assertThat(views.hit("home")).isEqualTo(1);
        assertThat(views.hit("home")).isEqualTo(2);
    }
}
```

Testcontainers 2.0 renamed its modules (`testcontainers-mongodb` instead of `mongodb`) and moved container classes into module packages (`org.testcontainers.mongodb.MongoDBContainer`). Boot 4.1.1 manages Testcontainers 2.0.

For narrower tests there are slices: `@DataMongoTest` (starter `spring-boot-starter-data-mongodb-test`) and `@DataRedisTest` (`spring-boot-starter-data-redis-test`). They start only the store's configuration and repositories. Combine them with the same `@Container @ServiceConnection` fields.

9.8 The caching abstraction

What it is

A **cache** keeps the results of expensive work (a slow query, a remote call, a calculation) so that the next request for the same thing is answered from memory (see also System Design Fundamentals). A **cache hit** means the answer was found there. A **cache miss** means the real work had to be done.

Spring's **cache abstraction** lets you cache the result of a *method* declaratively:

```
@Cacheable("products")
public Product find(String sku) { ... }      // runs only if "products" has no entry for sku
```

It is an *abstraction*: the annotations talk to a `CacheManager` interface, and a **provider** does the storing. That can be a local in-memory cache such as **Caffeine**, or a shared one such as **Redis**. Switching providers is a dependency and configuration change, not a code change.

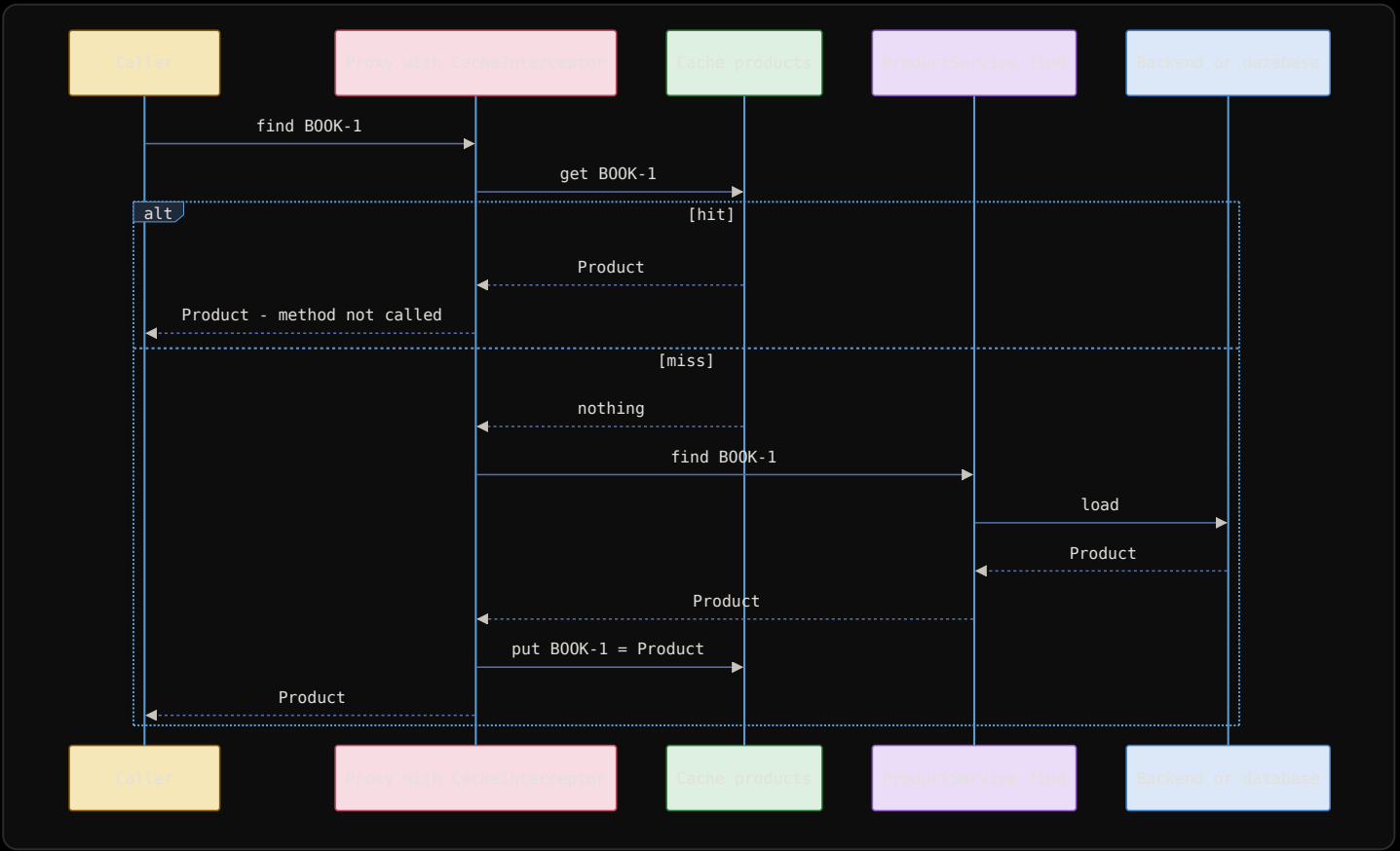
Why it matters

Reads usually far outnumber writes. A product page might be viewed 10,000 times between two price changes. Loading it from the database every time wastes capacity and adds latency; with a cache, one load serves thousands of requests. The price is **staleness**: the cache may show old data until it is refreshed or evicted. Every caching decision is about how stale is acceptable.

How it works

Caching is an AOP feature, built exactly like `@Transactional` (4.6).

1. `@EnableCaching` registers a `CacheInterceptor`. Every bean with cache annotations is wrapped in a **proxy**.
2. A call to `find("BOOK-1")` reaches the proxy first. The interceptor computes a **key** from the arguments (here `"BOOK-1"`).
3. It asks the `CacheManager` for the cache named `products`, and asks that cache for the key.
4. **Hit** → return the cached value. Your method does **not** run.
5. **Miss** → call your method, store the result under the key, return it.



This pattern, "look in the cache, and on a miss load and store", is called **cache-aside**. The system-design caching module compares it with read-through, write-through and write-behind.

Setup with Caffeine

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
<dependency>
  <groupId>com.github.ben-manes.caffeine</groupId>
  <artifactId>caffeine</artifactId>
</dependency>
```

```
@SpringBootApplication
@EnableCaching                // without it every cache annotation is silently ignored
public class CachingApplication { ... }

@Configuration(proxyBeanMethods = false)
public class CacheConfig {

    /** The clock Caffeine uses for expiry. Production: Ticker.systemTicker(). Tests replace it. */
    @Bean
    Ticker cacheTicker() {
        return Ticker.systemTicker();
    }

    /** Boot's CaffeineCacheManager uses this builder for every cache it creates. */
    @Bean
    Caffeine<Object, Object> caffeine(Ticker cacheTicker) {
        return Caffeine.newBuilder()
                .maximumSize(10_000)                        // bound memory: evicts least-useful entries
                .expireAfterWrite(Duration.ofMinutes(10))   // TTL: entries are at most 10 minutes old
                .ticker(cacheTicker)
                .recordStats();                             // hit/miss counters (also used by Actuator metrics)
    }
}
```

```
# Only these caches exist (no caches created on the fly by a typo in a cache name)
spring.cache.cache-names=products,catalog,tags,reports-plain,reports-sync
```

How Boot chooses a provider: if you define no `CacheManager` bean yourself, Boot checks the providers in a fixed order (Generic, JCache, Hazelcast, Infinispan, Couchbase, **Redis**, **Caffeine**, Cache2k, **Simple**) and configures the first one it finds on the classpath. You can force one with `spring.cache.type`. **Simple** is a `ConcurrentHashMap` with no expiry and no size limit, fine for demos but never for production.

For Caffeine, Boot applies the settings from, in order of precedence: `spring.cache.caffeine.spec` (e.g. `maximumSize=500,expireAfterWrite=10m`), a `CaffeineSpec` bean, or a `Caffeine` bean (used here, so that the test can swap the clock). **Caffeine** is the standard local cache for Java. Its eviction policy (W-TinyLFU) keeps the entries that are used most often and most recently, which gives better hit rates than plain LRU (evict whatever was touched longest ago).

With `spring.cache.cache-names` set, the manager is **static**: only those caches exist, and using any other name fails with an error instead of quietly creating a new, unconfigured cache.

`@Cacheable` — hits and misses, measured

```
@Service
@CacheConfig(cacheNames = "products")      // default cache name for the annotations in this class
public class ProductService {
    private final ProductBackend backend;   // stands in for a slow database; counts its calls

    @Cacheable(unless = "#result == null")          // do not cache "not found"
    public Product find(String sku) {
        return backend.load(sku);
    }
    // ...
}
```

```
@Test
void cacheableRunsTheMethodOnlyOnAMiss() {
    CacheStats before = nativeCache("products").stats();          // Caffeine's own counters
    for (int i = 1; i <= 3; i++) {
        Product p = products.find("BOOK-1");
        System.out.println("call " + i + ": " + p.name() + "   backend loads so far = " + backend.loads.get());
    }
    CacheStats delta = nativeCache("products").stats().minus(before);
    // print cache manager class, hits, misses; assert loads = 1, hits = 2, misses = 1
}
```

```
call 1: Spring in Depth   backend loads so far = 1
call 2: Spring in Depth   backend loads so far = 1
call 3: Spring in Depth   backend loads so far = 1
cache manager: CaffeineCacheManager   hits=2 misses=1
```

Three calls, one real load. `nativeCache("products")` unwraps Spring's `CaffeineCache` to the underlying Caffeine cache, whose `recordStats()` counters prove what happened. In production, the same counters are published by Actuator as `cache.gets` metrics tagged `result=hit|miss` (ch15). For Caffeine they exist only when `recordStats()` is on. The `/actuator/caches` endpoint lists caches and can clear them.

`@CachePut` and `@CacheEvict` — keeping the cache correct

```
/** ALWAYS runs the method, then writes the result into the cache (key = the saved product's sku). */
@CachePut(key = "#result.sku()")
public Product update(Product product) {
    return backend.store(product);
}

/** Runs the method, then removes that one key. */
@CacheEvict
public void delete(String sku) {
    backend.remove(sku);
}

/** Empties the whole "products" cache (e.g. after a bulk import). */
@CacheEvict(allEntries = true)
public void reloadAll() {}
```

```
products.find("PEN-7");                                          // miss -> load 1
products.update(new Product("PEN-7", "Gel pen", new BigDecimal("3.00")));   // method runs + cache put
products.find("PEN-7");                                          // hit: the NEW price, no load
products.delete("PEN-7");                                        // method runs + key evicted
products.find("PEN-7");                                          // miss -> load 2 -> null
products.find("PEN-7");                                          // null was not cached -> load 3
products.find("BOOK-1");                                         // load 4
products.reloadAll();                                            // allEntries = true
```

```
after @CachePut:   3.00  loads=1
after @CacheEvict: null  loads=2
asked again:       loads=3  (null results are not cached)
after reloadAll(): entries in 'products' = 0
```

Annotation	Runs the method?	Effect on the cache	Typical use
`@Cacheable`	only on a miss	stores the result	reads
`@CachePut`	**always**	stores the result	updates, keeping the cache fresh
`@CacheEvict`	always	removes the key (or all entries with `allEntries = true`)	deletes; updates when you prefer "evict and reload"
`@Caching`	—	groups several of the above on one method (e.g. put in one cache, evict from another)	a method affecting several caches
`@CacheConfig`	—	class-level defaults: `cacheNames`, `keyGenerator`, `cacheManager`	less repetition

Details that matter:

- `@CacheEvict` runs **after** the method, and only if it returns normally. `beforeInvocation = true` evicts first, even if the method then throws.
- Do not put `@Cacheable` and `@CachePut` on the same method. One skips the method, the other always runs it; the Spring docs strongly discourage it.
- **Evict or put?** Evicting is safer: the next read loads fresh data from the database. `@CachePut` saves that read, but if two updates race, the cache can end up holding the older value.
- The key of `@CachePut` must match the key the readers use. Here readers use `sku`, so the put uses `#result.sku()`.

Keys: how they are generated, and how they collide

By default Spring uses `SimpleKeyGenerator`:

Parameters	Key
none	`SimpleKey.EMPTY`
exactly one	the parameter itself
two or more	`SimpleKey[p1, p2, …]` (uses each parameter's `equals`/`hashCode`)

The key does **not** include the method name. Two methods that share a cache and take the same argument therefore read each other's entries:

```
@Service
public class CatalogService {

    @Cacheable("catalog")                              // key = sku  (default SimpleKeyGenerator)
    public String name(String sku) { return "name of " + sku; }

    @Cacheable("catalog")                              // SAME key = sku -> collides with name()!
    public String description(String sku) { return "description of " + sku; }

    @Cacheable(cacheNames = "catalog", key = "'desc:' + #sku")   // fix: make the key unique
    public String descriptionFixed(String sku) { return "description of " + sku; }

    @Cacheable(cacheNames = "catalog")                 // two parameters -> key = SimpleKey[sku, currency]
    public String price(String sku, String currency) { ... }

    @Cacheable(cacheNames = "catalog", key = "#sku", condition = "#currency == 'EUR'")
    public String priceEurOnly(String sku, String currency) { ... }   // condition: cache only EUR lookups
}
```

The test calls `name("A")`, `description("A")`, `descriptionFixed("A")`, `price("A", "EUR")`, `priceEurOnly("B", "USD")` and `priceEurOnly("C", "EUR")`, then prints the keys stored in the Caffeine cache:

```
name("A")             -> name of A
description("A")      -> name of A   <- WRONG: same cache, same key
descriptionFixed("A") -> description of A
key in 'catalog': A
key in 'catalog': C
key in 'catalog': SimpleKey [A, EUR]
key in 'catalog': desc:A
```

- `description("A")` returned the **name**. If the two methods returned different types, you would get a `ClassCastException` instead. Fix: one cache per method (the usual choice), a key prefix, or a custom `KeyGenerator` bean (referenced with `keyGenerator = "myKeyGen"`) that includes the method name.
- `B` is missing: `condition` was false for USD, so the method ran but nothing was cached.
- `key` is a **SpEL** expression (Spring Expression Language). `#sku` is a parameter by name (Boot compiles with `-parameters`, so names are available), `#root.methodName` is the method name, `#result` is the return value (in `unless` and `@CachePut` keys only).
- Parameters used in keys need proper `equals`/`hashCode`. Records have them. A mutable object used as a key that changes after being cached can never be found again.

`condition` VS `unless`

Attribute	Evaluated	Can use `#result`?	If it says "no"
`condition`	**before** the call	no	cache is bypassed completely: no lookup, no store
`unless`	**after** the call	yes	the result is not stored (the lookup still happened)

`@Cacheable(unless = "#result == null")` is the usual way to avoid caching "not found". Otherwise a lookup for a product that does not exist *yet* keeps returning `null` until the entry expires. For `Optional` return types, Spring caches the contained value (or `null`), and `#result` refers to that value, so write `unless = "#result == null"` there too.

Expiry (TTL) — tested with a fake clock

TTL (time to live) bounds staleness: after it passes, the entry is dropped and the next call reloads. Testing a 10-minute TTL by sleeping 10 minutes is not practical, so the test replaces Caffeine's `Ticker` (its clock) with one it can move:

```java
class FakeTicker implements Ticker {
    private final AtomicLong nanos = new AtomicLong();
    @Override public long read() { return nanos.get(); }
    void advance(Duration d) { nanos.addAndGet(d.toNanos()); }
}

@SpringBootTest
class CachingTests {
    @TestConfiguration
    static class ClockConfig {
        @Bean @Primary FakeTicker fakeTicker() { return new FakeTicker(); }   // replaces the system ticker
    }

    @Test
    void entriesExpireAfterTheTtl() {
        products.find("BOOK-1");                                    // load 1, written at t=0
        ticker.advance(Duration.ofMinutes(9));
        products.find("BOOK-1");
        ticker.advance(Duration.ofMinutes(2));
        products.find("BOOK-1");
        assertThat(backend.loads).hasValue(2);
    }
}
```

```
t=9 min:  loads=1  (still fresh -> hit)
t=11 min: loads=2  (expired after 10 min -> miss -> reload)
```

Caffeine setting	Meaning	Use when
`expireAfterWrite(d)`	entry dies `d` after it was stored	data changes elsewhere; bounds staleness
`expireAfterAccess(d)`	entry dies `d` after its last read or write	keep hot entries, drop unused ones (staleness is **not** bounded)
`refreshAfterWrite(d)`	after `d`, the next read returns the old value and reloads in the background	needs a `CacheLoader` bean; avoids latency spikes
`maximumSize(n)` / `maximumWeight(w)`	size bound, evicts by W-TinyLFU	always: an unbounded cache is a memory leak

The Spring annotations have **no TTL attribute**. TTL is a provider setting, per cache manager or per cache.

Cache stampede and `sync = true` — measured

A **cache stampede** (also called *thundering herd* or *dog-piling*) happens when a popular entry is missing, because it expired or was never loaded, and many requests arrive at the same time. Each one misses, and each one runs the expensive computation. The database gets hit N times at the worst possible moment.

By default `@Cacheable` does nothing to prevent this. `sync = true` asks the provider to let **one** thread compute the value per key while the others wait for it.

```
@Cacheable("reports-plain")                       // every thread that misses computes the report
public String plain(String day) throws InterruptedException { ... }

@Cacheable(cacheNames = "reports-sync", sync = true)   // one thread computes, the others wait for it
public String synced(String day) throws InterruptedException { ... }
```

Both methods count their computations. The "expensive work" waits until all 8 callers are inside the method, or 300 ms at most, so the unsynchronised case really overlaps. The test submits 8 tasks at once to a thread pool:

```
8 concurrent callers, sync=false -> computed 8 times
8 concurrent callers, sync=true  -> computed 1 time, all got "report 2026-09-28"
```

With Caffeine, `sync = true` maps to Caffeine's atomic `get(key, loader)`: the first thread computes, and the others block on that key only. Other keys are not blocked.

Limits of `sync = true`: it cannot be combined with `unless`, it allows only one cache per annotation, it cannot be combined with other cache annotations on the same method, and it protects only **one JVM**. With 10 application instances you can still get 10 computations. Other stampede defences:

Technique	Idea
`sync = true` / `get(key, loader)`	one loader per key per instance
Random TTL jitter	entries written together do not all expire together
Refresh ahead (`refreshAfterWrite`)	reload *before* expiry, while still serving the old value
Distributed lock / single-flight (one caller computes, the rest share its result) in Redis	one loader across all instances (costs complexity)
Pre-warming	load hot keys at startup or on a schedule

More in the system-design caching module (section "Cache Stampede Prevention").

Pitfalls

1. Self-invocation: the cache is never consulted

A call from inside the same bean (`this.find(sku)`) does not pass through the proxy, so `@Cacheable` does nothing. The same applies to `private` methods (proxy mode only works on public methods) and to calls from a constructor or `@PostConstruct`. This was **proved** in 4.5: two internal calls ran the real method twice. Fix: move the cached method to another bean.

Related trap, hit while writing this chapter's test: reading a public **field** of the cached `ReportService` returned `null`, because the bean is a CGLIB proxy whose own fields are never initialised (also shown in 4.5). Access state on proxied beans through methods only.

2. Cached mutable objects are shared

A local cache like Caffeine stores a **reference** to your object. Every caller gets the *same instance*:

```
@Cacheable("tags")
public List<String> tags(String sku) {
    return new ArrayList<>(backend.tags(sku));
}

List<String> first = products.tags("BOOK-1");
first.add("hacked");                                    // caller changes "its" list...
List<String> second = products.tags("BOOK-1");          // ...another caller gets the same object
```

```
second caller sees: [new, sale, hacked]   same instance: true
```

One caller's change corrupted the cache for everyone, and nothing was written to the database. Cache **immutable** values: records, `List.copyOf`, unmodifiable collections. (A Redis cache stores a *serialized copy*, so this does not happen there. That is exactly why code that worked with Caffeine can behave differently after a switch to Redis, and the other way round.)

3. Caching inside a transaction

If `@CachePut` or `@CacheEvict` runs inside a `@Transactional` method that later rolls back, the cache already holds data that the database never committed. Options: wrap the manager in `TransactionAwareCacheManagerProxy` (puts and evicts are applied only after commit), use `RedisCacheManager.builder(...).transactionAware()`, or evict in an `@TransactionalEventListener(phase = AFTER_COMMIT)` (7.15).

4. Local caches in a cluster

With 5 instances, each has its own Caffeine cache. An update evicts the entry on the instance that handled it, and the other 4 keep serving the old value until their TTL expires. Either accept staleness up to the TTL (keep it short), use a shared cache (Redis), or broadcast evictions (e.g. Redis pub/sub messaging). A common design is **two levels**: a small, short-TTL Caffeine cache in front of Redis.

5. Other mistakes

Mistake	Symptom	Fix
No `@EnableCaching`	annotations ignored, no error	add it
Caching everything	memory pressure, stale data everywhere, hard-to-reproduce bugs	cache only measured hot, expensive, rarely-changing reads
No size bound (or the `simple` provider in production)	memory grows until an `OutOfMemoryError`	`maximumSize`, a real provider
Caching per-user data under a key without the user	one user sees another's data	include the user or tenant in the key
Non-serializable values with Redis	`SerializationException` at runtime	JSON serializer, or `Serializable` types
Changing a cached class's shape with Redis	old entries fail to deserialize after deploy	versioned key prefix per release, or short TTL
Tests influenced by cached values from other tests	flaky tests	clear caches in `@BeforeEach` (as here), or disable caching with `spring.cache.type=none` / `@AutoConfigureCache`

Redis as the cache (snippet — not run)

With `spring-boot-starter-data-redis` on the classpath, Boot's detection order picks **Redis before Caffeine**. The annotations stay the same. Basic settings are properties:

```
spring.cache.cache-names=products,prices
spring.cache.redis.time-to-live=10m
spring.cache.redis.cache-null-values=false
# the prefix is followed by the cache name; bump v3 when cached classes change
spring.cache.redis.key-prefix=shop:v3:
```

For more control, define beans (`RedisCacheSetup.java`, compiled):

```
@Configuration(proxyBeanMethods = false)
public class RedisCacheSetup {

    /** Defaults for every Redis cache: JSON values instead of JDK serialization. */
    @Bean
    RedisCacheConfiguration redisCacheConfiguration() {
        var allowedTypes = BasicPolymorphicTypeValidator.builder()   // tools.jackson (Jackson 3)
                .allowIfSubType("ch09.")
                .allowIfSubType("java.")
                .build();
        return RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(10))
                .disableCachingNullValues()
                .serializeValuesWith(SerializationPair.fromSerializer(
                        GenericJacksonJsonRedisSerializer.builder().enableDefaultTyping(allowedTypes).build()));
    }

    /** Per-cache overrides on top of the defaults. */
    @Bean
    RedisCacheManagerBuilderCustomizer perCacheTtl(RedisCacheConfiguration defaults) {
        return builder -> builder
                .withCacheConfiguration("products", defaults.entryTtl(Duration.ofMinutes(30)))
                .withCacheConfiguration("prices", defaults.entryTtl(Duration.ofSeconds(30)));
    }
}
```

- Your own `RedisCacheConfiguration` bean **replaces** the one Boot would build from `spring.cache.redis.*`, so those properties no longer apply. Put everything in the bean.
- `RedisCacheManagerBuilderCustomizer` (package `org.springframework.boot.cache.autoconfigure` in Boot 4) adjusts the auto-configured manager, typically with per-cache TTLs.
- **Serialization:** the default is JDK serialization, which is binary, Java-only, and breaks when classes change. JSON is readable and survives small changes. Spring Data Redis 4 ships Jackson 3 serializers (`GenericJacksonJsonRedisSerializer`, `JacksonJsonRedisSerializer<T>`); the Jackson 2 ones are deprecated. The generic JSON serializer stores each value's class name, and the **type validator** restricts which classes may be created from it. Allowing any class is a known deserialization attack vector (crafted data makes your app instantiate dangerous classes). The builder's `enableUnsafeDefaultTyping()` is named that way for a reason.
- Every hit costs a network round trip plus deserialization (well under a millisecond on a local network, but not free). Cache larger, more expensive results in Redis, not tiny lookups.

Local vs shared cache

	Caffeine (local)	Redis (shared)
Speed	nanoseconds, no serialization	network hop + (de)serialization
Consistency across instances	each instance has its own copy	one copy for all
Survives restarts	no	yes (as far as Redis persistence is configured)
Stores	references (mutable-object trap)	serialized copies
Capacity	limited by the JVM heap	a separate server's memory
Operations cost	none	another system to run and monitor
Typical use	per-instance hot reference data, short TTLs	sessions, data shared between instances, expensive results

In the interview

30-second answer: "Spring's cache abstraction is proxy-based AOP. `@EnableCaching` adds an interceptor. `@Cacheable` checks the cache by a key made from the arguments and only runs the method on a miss. `@CachePut` always runs and stores the result, and `@CacheEvict` removes entries. The storage is a `CacheManager`: Caffeine for a local cache, Redis for a shared one. Boot auto-configures whichever is on the classpath. TTL and size are provider settings. The pitfalls are the proxy rules (no self-invocation, public methods only), key collisions between methods sharing a cache, mutable cached objects, stale data across instances, and stampedes, which `sync = true` mitigates within one JVM."

- *"How do you set a TTL on `@Cacheable`?"* → You don't. The annotation has no TTL. Configure it in the provider: `expireAfterWrite` in Caffeine, `spring.cache.redis.time-to-live` or `RedisCacheConfiguration.entryTtl` in Redis, per cache if needed.
- *"The cache returns stale data after an update. Why?"* → The update path does not evict or put the same key (key mismatch or a different cache name), the update happened on another instance with a local cache, or the eviction happened inside a transaction that rolled back.
- *"What is a cache stampede and how do you prevent it?"* → Many concurrent misses for the same hot key all hit the database. Use `sync = true` (one loader per JVM), TTL jitter, refresh-ahead, or a distributed lock for cluster-wide single-flight.
- *"`condition` vs `unless`?"* → `condition` is checked before the call and can bypass the cache entirely. `unless` is checked after the call, can see `#result`, and only vetoes storing.

Measured in this section: 3 calls → 1 load (2 hits, 1 miss); a 10-min TTL → reload at 11 min; 8 concurrent misses → 8 loads, or 1 with `sync = true` (without it all 8 miss before any stores the value). Default keys omit the method name, so two `@Cacheable("x")` methods taking one `Long id` each return each other's values.

9.9 Choosing a data technology

Need	Choose
Relational data, rich object graph, team knows JPA	Spring Data JPA (ch08)
Relational data, clear aggregates, predictable SQL	**Spring Data JDBC**
Full SQL control, reports, bulk work	`JdbcClient` (ch07), or jOOQ
Non-blocking relational access	R2DBC (ch13)
Flexible, nested documents read as a whole	**MongoDB**
Sub-millisecond counters, sessions, rate limits, shared cache	**Redis**
Quick CRUD API over repositories, internal use	**Spring Data REST**
Faster repeated reads of rarely-changing data	**Caching** (Caffeine locally, Redis shared) — after measuring

---

9.10 Interview questions

Quick round; the section-level "In the interview" blocks above cover `save` on existing aggregates, repeatable and edited migrations, default keys, TTL, `sync`, mutable values and transactions.

1. **What is an aggregate in Spring Data JDBC?** → A root entity plus the entities it owns, loaded and saved as one unit through one repository (9.2).
2. **Spring Data JDBC vs JPA?** → No persistence context, no lazy loading or dirty checking; eager aggregate loading; children rewritten on save; SQL is predictable.
3. **How do you reference another aggregate?** → By id: `AggregateReference<T, ID>` or a plain id field.
4. **Why did saving a `Product` with an assigned id insert nothing?** → Non-null id without `@Version` means "existing" → `UPDATE` of 0 rows. Use `@Version`, `Persistable` or `JdbcAggregateTemplate.insert`.

5. **What does Spring Data REST give you, and what is the risk?** → HAL CRUD endpoints for repositories without controllers. It exposes your domain model as the API and exports everything by default.
6. **Flyway vs Liquibase?** → SQL-file versioning vs changelogs with database-independent changes, rollbacks and preconditions (9.4).
7. **Where does the MongoDB connection go in Boot 4?** → `spring.mongodb.uri` (driver module). `spring.data.mongodb.*` holds Spring Data settings such as `auto-index-creation`.
8. **Does Boot configure MongoDB transactions?** → No. Declare a `MongoTransactionManager`, and run a replica set.
9. **How are Redis repository queries implemented?** → Through secondary-index sets for `@Indexed` properties; equality only.
10. **How do you test against real MongoDB/Redis?** → Testcontainers with `@Container @ServiceConnection` fields (9.7).
11. **How does `@Cacheable` work internally?** → Proxy + `CacheInterceptor` → key → `CacheManager` → `Cache` lookup; the method runs only on a miss.
12. **`@CachePut` vs `@Cacheable`?** → `@CachePut` always runs the method and stores the result; `@Cacheable` skips it on a hit.
13. **Why is my `@Cacheable` ignored?** → No `@EnableCaching`, self-invocation, non-public method, or the bean was created with `new` (ch04).
14. **Caffeine or Redis?** → Caffeine: fastest, per instance, stale across instances. Redis: shared, survives restarts, costs a network hop and serialization.

Chapter TL;DR

- **Spring Data** gives every store the same repository model. Only mapping and store features differ.
- **Spring Data JDBC** maps **aggregates** with no persistence context. Loading is eager, `save` writes immediately and rewrites child rows, other aggregates are referenced by id, and `@Version` on the root protects the whole aggregate. Assigned ids need `@Version` / `Persistable` / `insert()`.
- **Spring Data REST** turns repositories into HAL endpoints. Good for internal CRUD; restrict exports and secure them.
- **Flyway**: versioned SQL, Java and repeatable migrations with checksums (an edited migration stops startup). **Liquibase**: changelogs with rollbacks and preconditions; FSL-licensed since 5.0.
- **MongoDB**: documents, `MongoRepository`, `MongoTemplate` aggregations. Boot 4 uses `spring.mongodb.*` for the connection. **Redis**: templates for data structures, `@RedisHash` repositories for simple cases. Test both with Testcontainers + `@ServiceConnection`.
- **Caching**: `@Cacheable` / `@CachePut` / `@CacheEvict` over a `CacheManager`. Caffeine locally, Redis shared. TTL and size live in the provider. Watch for proxy rules, key collisions, mutable values, transactions, cluster staleness and stampedes (`sync = true` per JVM).

Next: Chapter 10 — Spring Security fundamentals protects the endpoints and data you have built so far.